达梦数据
全栈数据产品与解决方案提供商

股票代码 688692

达梦技术丛书

DM8 数据共享集群



前言

概述

本文档主要介绍 DM 数据共享集群的系统特性、基本概念、关键技术、主要功能，以及如何搭建 DM 数据共享集群并使用等。

读者对象

本文档主要适用于 DM 数据库的：

- 开发工程师
- 测试工程师
- 技术支持工程师
- 数据库管理员

通用约定

在本文档中可能出现下列标志，它们所代表的含义如下：

表 0.1 标志含义

标志	说明
 警告：	表示可能导致系统损坏、数据丢失或不可预知的结果。
 注意：	表示可能导致性能降低、服务不可用。
 小窍门：	可以帮助您解决某个问题或节省您的时间。
 说明：	表示正文的附加信息，是对正文的强调和补充。

在本文档中可能出现下列格式，它们所代表的含义如下：

表 0.2 格式含义

格式	说明
HarmonyOS Sans SC Light	表示正文。
Poppins Light	表示代码或者屏幕显示内容。
粗体	表示命令行中的关键字（命令中保持不变、必须照输的部分）或者正文中强调的内容。标题、警告、注意、小窍门、说明等内容均采用粗体。
< >	语法符号中，表示一个语法对象。
::=	语法符号中，表示定义符，用来定义一个语法对象。定义符左边为语法对象右边为相应的语法描述。
|	语法符号中，表示或者符，限定的语法选项在实际语句中只能出现一个。
{ }	语法符号中，大括号内的语法选项在实际的语句中可以出现 0…N 次(N 为大于 0 的自然数)，但是大括号本身不能出现在语句中。
[ ]	语法符号中，中括号内的语法选项在实际的语句中可以出现 0…1 次，但是中括号本身不能出现在语句中。
关键字	关键字在 DM_SQL 语言中具有特殊意义，在 SQL 语法描述中，关键字以大写形式出现。但在实际书写 SQL 语句时，关键字既可以大写也可以小写。

访问相关文档

如果您安装了 DM 数据库，可在安装目录的“\doc”子目录中找到 DM 数据库的各种手册与技术丛书。

您也可以通过访问我们的网站 www.dameng.com 阅读或下载 DM 的各种相关文档。

联系我们

如果您有任何疑问或是想了解达梦数据库的最新动态消息，请联系我们：

网址：www.dameng.com

技术服务电话：400-991-6599

技术服务邮箱：dmtech@dameng.com

目录

1 引言 1

2 DMDSC 概述 2

2.1 系统特性 3

2.2 基本概念 5

2.3 暂不支持功能 10

3 DMDSC 使用的环境 11

4 DMDSC 关键技术 12

4.1 事务管理 12

4.2 封锁管理 14

4.3 闩管理 15

4.4 缓存交换 15

4.5 重做日志管理 22

4.6 回滚记录管理 24

5 DMCSS 介绍 26

5.1 启动命令 26

5.2 心跳信息 27

5.3 选举 DMCSS 控制节点 27

5.4 选取监控对象控制节点 28

5.5 启动流程管理 28

5.6 状态检测 28

5.7 故障处理.....28
5.8 节点重加入.....29
5.9 集群指令.....30
5.10 控制台命令.....30
5.11 注意事项.....36
6 DMDSC 的启动与退出.....38
6.1 DMDSC 的启动.....38
6.2 DMDSC 的退出.....38
7 DMDSC 故障处理.....51
7.1 故障处理.....51
7.2 故障日志.....53
8 DMDSC 节点重加入.....56
9 DMDSC 配置文件.....57
9.1 DMDCR_CFG.INI.....57
9.2 DMDCR.INI.....63
9.3 DMINIT.INI.....70
9.4 MAL 系统配置文件.....74
9.5 DM.INI.....80
9.6 DMARCH.INI.....82
9.7 DMCSSM.INI.....82
10 DMASM 介绍.....83
10.1 DMASM 概述.....83

10.2 使用 DMASM 的好处 84
10.3 DMASM 术语和基本概念 86
10.4 DMASM 关键技术 90
10.5 DMASM 技术指标 93
10.6 DMASM 主要部件 94
10.7 DMASM 使用说明 123
11 DMASM 镜像介绍 125
11.1 DMASM 镜像概述 125
11.2 DMASM 镜像术语和基本概念 127
11.3 DMASM 镜像关键技术 132
11.4 DMASM 镜像技术指标 143
11.5 DMASM 镜像主要部件 145
11.6 DMASM 镜像配置文件 182
11.7 DMASM 镜像使用说明 190
12 DMDSC 搭建 192
12.1 基于 DMASM 的 DMDSC 192
12.2 基于 DMASM 镜像的 DMDSC 212
12.3 检验是否搭建成功 236
13 巧用服务名 240
13.1 配置服务名 240
13.2 集群的 dm_svc.conf 模板 242
13.3 故障自动切换 244
13.4 只连集群的主控节点 245

13.5 只连集群的第 N 个节点 247

14 动态增删节点 249

14.1 动态增加节点流程 249

14.2 动态删除节点流程 256

15 监控 DMDSC 257

15.1 DMCSSM 监视器 257

15.2 动态视图 264

15.3 DMDSC 日志文件 298

16 备份还原 302

16.1 DMDSC 和单节点差异 302

16.2 远程归档 302

16.3 DMDSC 备份集 305

16.4 非镜像环境下备份还原实例 306

16.5 镜像环境下备份还原实例 308

16.6 使用说明 313

17 DMDSC 使用说明 315

17.1 统一组件版本 315

17.2 提升 DMDSC 性能 315

17.3 心跳说明 316

17.4 重新格式化 DMASM 317

17.5 重新初始化 DMDSC 库 317

17.6 内部网络异常 318

17.7 创建 DBLink 319
17.8 节点硬件故障，如何启动 DMDSC 集群 322
17.9 MOUNT/OPEN 操作 322
17.10 块设备路径变化 322
17.11 滚动升级 323
17.12 正常 HALT 现象 324
17.13 ASM 磁盘版本 324
17.14 表空间负载均衡 324
17.15 暂停 DMDSC 库 325
18 版本升级 327
18.1 日志格式升级 327
18.2 回滚管理段升级 329
18.3 HUGE 表空间升级 330
18.4 数据字典 SYSOBJECTS 升级 334
19 附录 335
19.1 DMASMAPI 接口 335
19.2 DMCSSM 接口 374
19.3 DMASMAPIM 接口 442
DM 技术支持 479
手册版本 481

1 引言

DM 数据共享集群又称为 DM 共享存储集群，英文全称 DM Data Shared Cluster，简称 DMDSC。

DM 数据共享集群，允许多个数据库实例同时访问、操作同一数据库，具有高可用、高性能、负载均衡等特性。DMDSC 支持故障自动切换和故障自动重加入，某一个数据库实例故障后，不会导致数据库服务无法提供。

本文主要介绍 DMDSC 集群的功能、概念、实现原理，并举例说明搭建过程和管理方法。

本手册可以帮助用户：

- 了解 DMDSC/DMCSS/DMASM/DMASM 镜像等集群相关概念
- 了解 DMASM 分布式文件系统、DMDSC 集群，以及基于 DMASM 的 DMDSC 集群配置过程和应用

2 DMDSC 概述

DMDSC 集群是一个多实例、单数据库的系统。多个数据库实例可以同时访问、修改同一个数据库的数据。用户可以登录集群中的任意一个数据库实例，获得完整的数据库服务。数据文件、控制文件在集群系统中只有一份，不论有几个节点，这些节点都平等地使用这些文件，这些文件保存在共享存储上。每个节点有自己独立的联机日志和归档日志，联机日志和归档日志都需要保存在共享存储上。

DMDSC 集群主要由数据库和数据库实例、共享存储、DMASM 或 DMASM 镜像、本地存储、通信网络、集群控制软件 DMCSS、集群监视器 DMCSSM 组成。DMDSC 集群最多支持 8 个数据库实例节点。下图展示了一个两节点的 DMDSC 集群系统结构图。

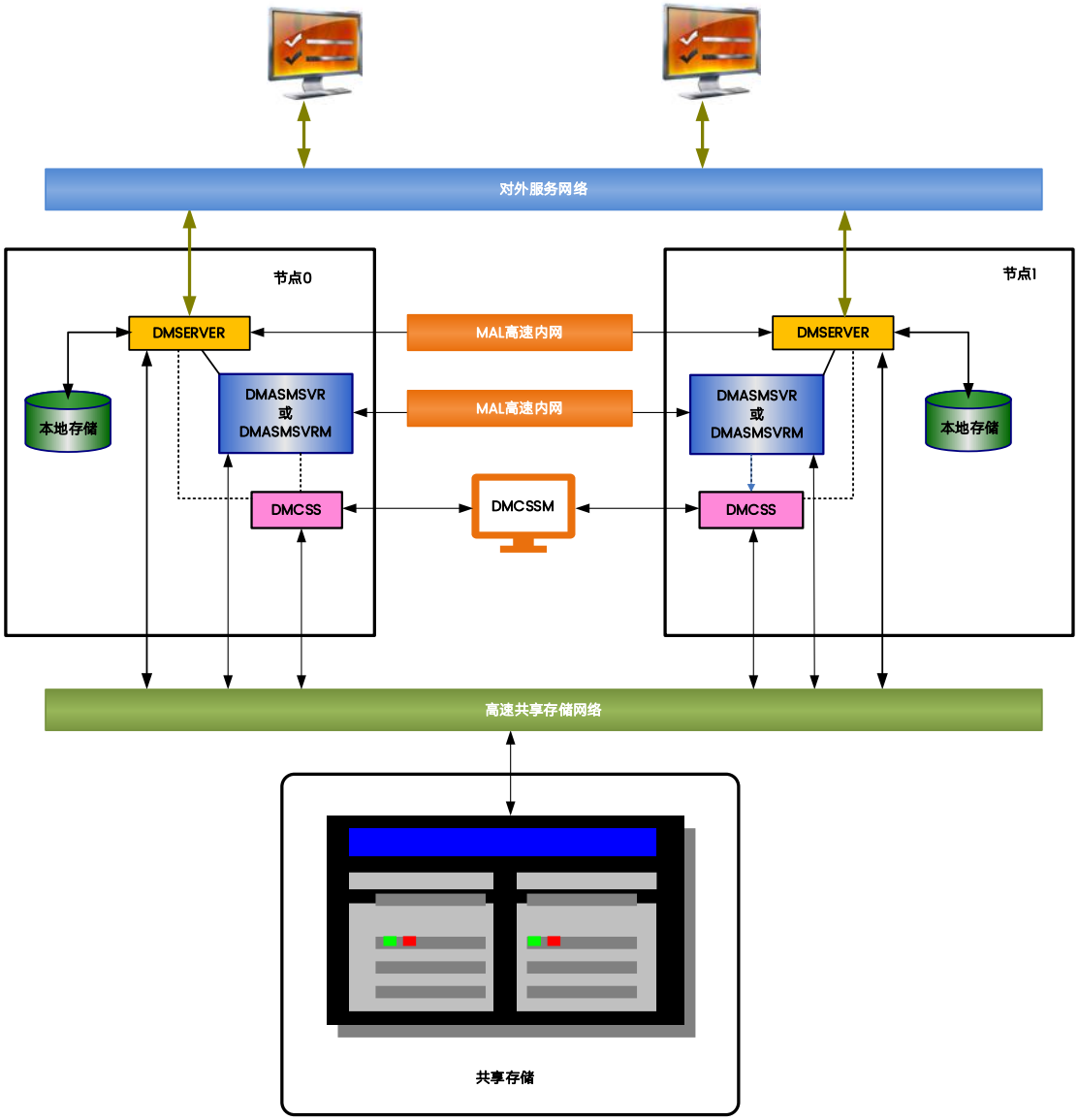


图 2.1 一个 DMDSC 集群结构图

2.1 系统特性

DMDSC 的主要特点包括:

■ 高可用性　只要集群中有一个活动节点，就能正常提供数据库服务。此外，当出现磁盘损坏或数据丢失时，既可以利用其他镜像副本继续提供数据库服务，又可以使用其他镜像副本进行数据恢复。

■ 高吞吐量　多个节点同时提供数据库服务，有效提升集群的整体事务处理能力。

■ 负载均衡　一方面，通过巧用服务名，用户的连接请求被平均分配到集群中的各

个节点，确保连接负载平衡；另一方面，条带化技术可保证写入的数据均匀分布到磁盘组内的不同磁盘中，实现数据负载均衡。

2.1.1 高可用性

DMDSC 集群通过两种方式提供达梦数据库高可用解决方案。

- **一 使用集群控制软件 DMCSS**

当出现系统故障、硬件故障、或人为操作失误时，DMCSS 可检测故障并自动将故障节点踢出集群，保证数据库服务的正常提供。

故障节点的用户连接会自动切换到活动节点，这些连接上的未提交事务将被回滚，已提交事务不受影响；活动节点的用户连接不受影响，正在执行的操作将被挂起一段时间，在故障处理完成后，继续执行。当 DMCSS 检测到故障节点恢复时，自动启动节点重加入流程，将恢复的故障节点重新加入 DMDSC 集群，将集群恢复到正常的运行状态。因此，通过部署 DMDSC 集群，可以在一定程度上避免由软、硬件故障引起的非计划停机，减少这些意外给客户带来的损失。

与同样使用共享存储的双机热备系统相比，DMDSC 具有更快的故障处理速度。双机热备系统故障切换时，需要完整地重做 REDO 日志，所有数据均需重新从磁盘加载；而 DMDSC 故障处理时（由 INI 参数 DSC_CRASH_RECV_POLICY 控制），只要重做故障节点的 REDO 日志，并且大部分数据页已经包含在处理节点的数据缓冲区中，不需要重新从磁盘加载。

- **二 使用 DMASM 镜像的多副本技术**

如果 DMDSC 配置了 DMASM 镜像，镜像功能可提供多副本技术。当出现磁盘损坏或数据丢失时，系统无需人工干预即可利用其他镜像副本继续提供数据库服务，同时又可以自动或手动通过使用其他镜像副本进行数据恢复。

2.1.2 高吞吐量

DMDSC 集群中包含多个数据库实例，数据库实例访问独立的处理器、内存，数据库实例之间通过缓存交换技术提升共享数据的访问速度，每个数据库实例都可以接收并处理用户的各种数据库请求。

与单节点数据库管理系统相比，DMDSC 集群可以充分利用多台物理机器的处理能力，支撑更多的用户连接请求，提供更高的吞吐量。与双机热备系统相比，DMDSC 集群不存在始终保持备用状态的节点，不会造成硬件资源的浪费。

2.1.3 负载均衡

DMDSC 从连接和数据两个层面提供负载均衡特性。

- **一 通过巧用服务名**

通过配置 DM 数据库连接服务名来访问 DMDSC 集群，可以实现节点间的连接自动负载均衡。用户的数据库连接请求会被自动、平均地分配到 DMDSC 集群中的各个节点。并且连接服务名支持 JDBC、DPI、ODBC、DCI、.Net Provider 等各种数据库接口。

- **二 使用镜像的条带化技术**

通过配置 DMASM 镜像，使用镜像的条带化技术可保证写入的数据均匀分布到磁盘组内的不同磁盘中，实现数据负载均衡。

2.2 基本概念

2.2.1 数据库和数据库实例

数据库（Database）是一个文件集合（包括数据文件、临时文件、重做日志文件和控制文件等），保存在物理磁盘或文件系统中。

数据库实例（Dmserver）就是一组操作系统进程（或一个多线程的进程）以及一些内

存。通过数据库实例，可以操作数据库，一般情况下，我们访问、修改数据库都是通过数据库实例来完成的。

2.2.2 共享存储

共享存储是指可被多台主机同时访问到的存储设备。

DMDSC 集群中，为了实现多个实例同时访问、修改数据，要求将数据文件、控制文件、日志文件保存在共享存储上。

DMDSC 使用 DMASM 文件系统管理共享存储设备。一台共享存储上，只能搭建一套 DMASM 文件系统，多套会导致系统启动失败。

2.2.3 本地存储

DMDSC 集群中，本地存储用来保存配置文件（记录数据库实例配置信息的 DM.INI、DMARCH.INI、DMMAL.INI）。

2.2.4 通信网络

DMDSC 集群中，网络分为对外服务网络、MAL 高速内网和高速共享存储网络三部分。

对外服务网络用于对外提供数据库服务，用户使用公共网络地址登录 DMDSC 集群，访问数据库。

MAL 高速内网用于数据库实例之间交换信息和数据。MAL 链路即为 MAL 高速内网。

高速共享存储网络用于数据库实例和共享存储之间的通信。常见的两种方式为通过光纤通道实现或通过网络 SCSI 实现。高速共享存储网络搭建工作一般由存储设备厂商完成，不是本书讨论的内容，具体实施方案请咨询共享存储设备厂商或达梦 DBA。

2.2.5 集群和集群组

集群（Cluster）是由两个或多个节点（服务器）构成的一种松散耦合的计算机节点集合，这个集合在整个网络中表现为一个单一的系统，并通过单一接口进行使用和管理。大多数模式下，集群中的所有计算机都拥有一个相同的名称，集群内任意一个系统都可以被所有的网络用户使用。每个集群节点都是运行其自己进程的独立服务器，因此每个节点都有自己的运算能力。这些进程间彼此通信进行协调，协同起来向用户提供应用程序、系统资源和数据以及计算能力。

本文档中涉及到的集群有三种：DMDSC 集群，DMCSS 集群和 DMASM 集群。一个 DMDSC 集群由多个 DMSERVER 服务器共同构成；一个 DMCSS 集群由多个 DMCSS 服务器共同构成；一个 DMASM 集群由多个 DMASM 服务器共同构成。

集群又称为集群组。集群组的概念在配置 DMDCR_CFG.INI 文件中使用，用于配置一个集群中的组内成员节点的公用和专用信息。DMDSC 集群，DMCSS 集群和 DMASM 集群的集群组类型分别为 DB、CSS 和 ASM。例如：一个 DMDSC 集群中共含有 5 个 DMSERVER 节点，则在 DMDCR_CFG.INI 中，该 DMDSC 集群的组类型为 DB，组中成员的个数为 5。

2.2.6 DMDSC 集群

DMDSC 集群由若干数据库实例组成，这些实例间通过网络（MAL 链路）连接，通过一个特殊的软件（DMCSS，集群同步服务）的协助，共同操作一个数据库。从外部用户视角来看，他们看到的只是一个数据库。数据文件、控制文件等文件在集群中只有一份，所有节点平等地使用这些数据文件。这份数据存放在共享存储上，每个服务器通过高速共享存储网络连接到共享存储上。

2.2.7 DM 自动存储管理器

DM 自动存储管理器（DM Auto Storage Manager，简称 DMASM）是一个专门用来

为块设备管理文件的分布式文件系统。使用 DMASM 文件系统可以灵活地在块设备上创建、删除、扩展、截断文件，不用担心空间不足（可以通过在线增加块设备的磁盘来扩展空间）或空间浪费；不用考虑文件个数限制；可以方便地查看空间使用情况。

DMDSC 支持多个节点同时访问、修改 DMASM 中的数据文件。

DMASM 不是一个通用的文件系统，应用程序只能通过 DMASMAPI 接口访问。理论上通过 DMASMAPI 接口可以存放任何文件，但在 DMDSC 集群中，一般只建议将需要在节点间共享访问的文件存在 DMASM 中，如数据文件、联机 REDO 日志文件、控制文件等。归档 REDO 日志文件、备份集文件也可以考虑保存到 DMASM 中，避免还原、恢复等操作时节点间的文件拷贝，简化备份、还原操作。其他的一些本地配置文件比如 DM.INI 等建议保存在本地磁盘中。

DMASM 有两个版本：一是早期版本，提供基础的磁盘组操作、文件操作等，详细用法请参考 10 DMASM 介绍；二是 ASM 镜像版本，在早期版本的基础上增加了 ASM 文件镜像功能、文件条带化功能、DCRV 多磁盘功能等，详细用法请参考 11 DMASM 镜像介绍。两种版本的 ASM 文件互不兼容。

DMDSC 集群中若配置 DMASM，则要求 DMASM 站点数和 DMCSS 站点数一致，且只能存在一个 DMCSS 组和一个 DMASM 组。

2.2.8 DM 集群同步服务

DM 集群同步服务（DM Cluster Synchronization Services，简称 DMCSS）是一款集群控制软件，是 DMDSC 集群应用的基础。DMCSS 专门负责监控集群中各个节点的运行状态，主要功能包括集群环境中节点的启动、故障处理、节点重加入等操作。

每个 DMDSC 集群或 DMASM 集群节点都必须配置一个 DMCSS 服务。这些 DMCSS 服务又共同构成一个 DMCSS 集群。单节点应用时，可以不配置 DMCSS。

2.2.9 DM 集群监视器

DM 集群监视器（DM Cluster Synchronization Services Monitor，简称 DMCSSM）用来监控整个集群的状态信息。

DMCSSM 与 DMCSS 相互通信，从 DMCSS 处获取整个集群系统的状态信息。DMCSSM 提供一系列管理维护集群的命令。

配置了 DMCSS 的集群中，可配置 0~10 个 DMCSSM。为了防止硬件损坏导致 DMCSSM 和其他服务器同时故障，建议用户将 DMCSSM 和其他服务器分开放置，DMCSSM 可单独放置在一台机器上。

2.2.10 心跳机制

DMCSS 的心跳机制（Heartbeat）是通过 VOTE 磁盘（非镜像环境下）或 DCRV 磁盘（镜像环境下）的 Disk Heartbeat 实现。更多关于 VOTE 磁盘的介绍请参考 10 DMASM 介绍。更多关于 DCRV 磁盘的介绍请参考 11 DMASM 镜像介绍。

心跳机制有最大时延，只有超过最大时延，才认为监测对象故障。

2.2.11 控制节点

DMCSS 集群中负责监控、管理整个 DMASM 集群和 DMDSC 集群的节点称为控制节点（Control Node），其它 DMCSS 节点称为普通节点（Normal Node）。在一套 DMDSC 集群\DMCSS 集群\DMASM 集群中，只有一个节点是控制节点，其他均为普通节点（即非控制节点），普通节点不参与 DMASM 集群和 DMDSC 集群管理。DMCSS 启动后，控制节点通过选举产生。当 DMCSS 控制节点故障时，会从活动的普通节点中重新选取一个 DMCSS 控制节点。控制节点向目标对象的 VOTE 磁盘（非镜像环境下）或 DCRV 磁盘（镜像环境下）指令区写入命令，通知目标对象执行相应命令，并等待执行响应，从而控制各节点的启动、故障处理、故障重加入等操作。

2.2.12 MAL 链路

MAL 系统是达梦数据库基于 TCP 协议实现的一种内部通信机制，具有可靠、灵活、高效的特性。使用 DMASM 文件系统的 DMDSC 集群中存在两套 MAL 系统，DMASM 服务器之间配置一套 MAL 系统，DMSERVER 服务器之间配置一套 MAL 系统。两套 MAL 系统工作原理相同：一旦 MAL 链路出现异常，DMCSS 会进行裁定，并从集群中踢出一个节点，保证集群环境正常运行。

2.2.13 共享内存

共享内存是一种快速、高效的进程间通信手段。所谓共享内存，就是同一块物理内存被映射到多个进程的地址空间，进程 A 可以即时看到进程 B 对共享内存的修改，反之亦然。DMASM 服务器进程和 DMASM 客户端进程之间通过共享内存方式共享 ASM 文件到实际磁盘的映射关系。

2.3 暂不支持功能

目前 DMDSC 在功能上与单机版 DM 相比存在一定限制，具体功能限制如下：

1. 支持定时器，但只有控制节点上配置的定时器生效。只支持脱机配置定时器，不支持联机配置。
2. 支持作业，但只有控制节点上支持执行作业。
3. 不支持外部表。支持 HUGE 表，但只有控制节点支持。
4. 不支持 table 级别的 space limit 功能。
5. 支持全文索引，但仅支持默认词库，需要将默认词库文件 SYSWORD.UTF8.LIB 放到 ASM 文件系统中。
6. 不能与 MPP 集群混合使用。
7. 不支持 SERERR 类型的事件触发器。

3 DMDSC 使用的环境

部署 DMDSC 集群所用到的硬件和软件环境。

- **硬件环境**

  - 主机 N 台，N 为 DMDSC 节点数量和 DMCSSM 节点数量的总和。用于部署数据库实例 DMSERVER、DMCSS、DMCSSM、DMASMSVR。其中，DMCSSM 为可选项，其他为必选项。DMCSSM 单独部署在一台机器上。内存大小要求：至少 2GB。
  - 共享存储。N 台主机可以同时访问到的，可以划分为块设备的磁盘。
  - 网卡。每台主机至少准备 2 块网卡。提供 MAL 高速内网和对外服务网络。

- **软件环境**

  - 操作系统。Linux、Unix、Windows 等。
  - 达梦数据库软件。安装好 DM 数据库软件之后，将拥有配置和管理 DMDSC 所需的所有软件：DMSERVER、DMINIT、DMASMCMD、DMASMSVR、DMASMTOOL、DMCSS、DMCSSM、DMASMCMDM、DMASMSVRM、DMASMTOOLM 等。这些软件位于安装目录/dmdbms/bin 中。

4 DMDSC 关键技术

DMDSC 是一个共享存储的数据库集群系统。多个数据库实例同时访问、修改同一个数据库，必然会带来全局并发问题。DMDSC 集群基于 DM 单节点数据库管理系统，改造了缓冲区、事务系统、封锁系统和日志系统等，以适应数据共享集群节点间的全局并发访问控制要求。同时，引入缓存交换技术，提升数据在节点间的传递效率。

4.1 事务管理

多版本并发控制（MVCC）可以确保数据库的读操作与写操作不会相互阻塞，大幅度提升数据库的并发度以及使用体验，大多数主流商用数据库管理系统都实现了 MVCC。DM 的多版本并发控制实现策略是：数据页中只保留物理记录的最新版本数据，通过回滚记录维护数据的历史版本。

每一条物理记录中包含了两个字段：TID 和 RPTR。TID 保存修改记录的事务号，RPTR 保存回滚段中上一个版本回滚记录的物理地址。插入、删除和更新物理记录时，RPTR 指向操作生成的回滚记录的物理地址。

回滚记录与物理记录一样，也包含了 TID 和 RPTR 这两个字段。TID 保存产生回滚记录时物理记录上的 TID 值（也就是上一个版本的事务号），RPTR 保存回滚段中上一个版本回滚记录的物理地址。

每一条记录（物理记录或回滚记录）代表一个版本。如下图所示：

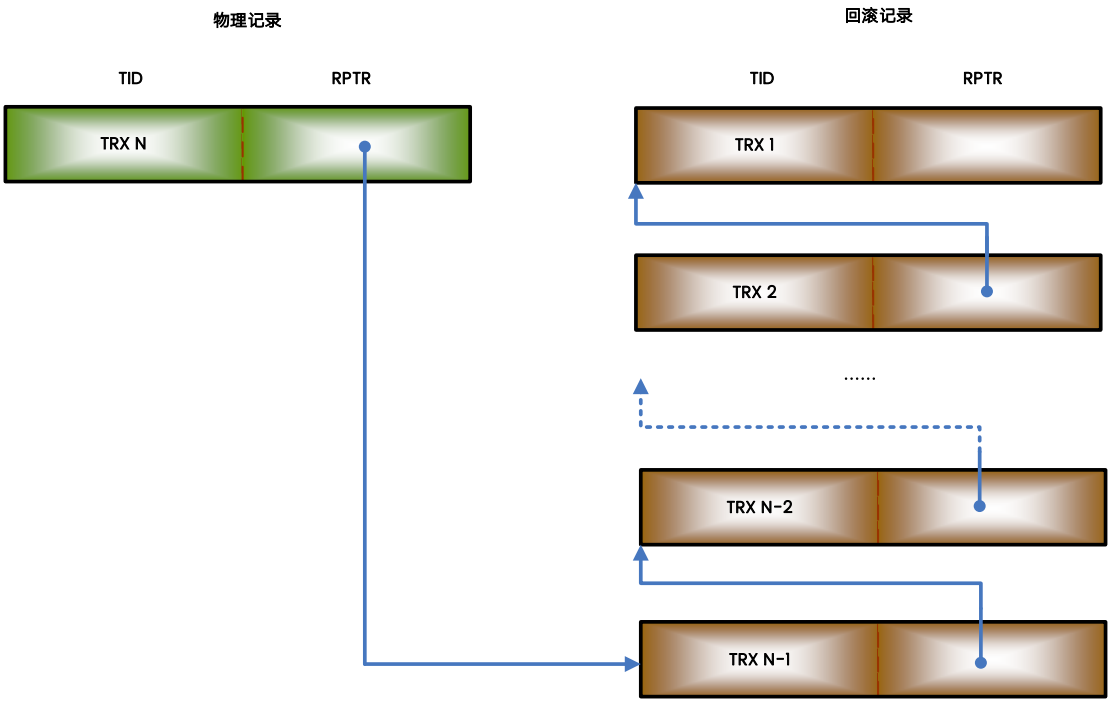


图 4.1 各版本之间的关系

如何找到对当前事务可见的特定版本数据，进行可见性判断，是 DM 实现多版本并发控制的关键。根据事务隔离级别的不同，在事务启动时（串行化），或者语句执行时（读提交），收集这一时刻所有活动事务，并记录系统中即将产生的事务号 NEXT_TID。DM 多版本并发控制可见性原则：

1. 物理记录 TID 等于当前事务号，说明是本事务修改的物理记录，物理记录可见
2. 物理记录 TID 不在活动事务表中，并且 TID 小于 NEXT_TID，物理记录可见
3. 物理记录的 TID 包含在活动事务表中，或者 TID>=NEXT_TID，物理记录不可见

为了在 DMDSC 集群中实现与单节点相同的多版本并发控制（MVCC）策略，每个事务需要知道所有节点当前活动的事务信息，根据事务隔离级的不同，在事务启动时（串行化），或者语句执行时（读提交），收集这一时刻所有节点上的活动事务，以及系统中即将产生的事务号 NEXT_TID，记录到事务的活动事务视图中。DMDSC 集群将事务信息全局化，由控制节点统一管理集群中所有节点的全局事务视图（Global Transaction View，简称 GTV）；与之对应的是每个节点维护一个本地事务视图（Local Transaction View，简称 LTV），在

事务启动、收集活动事务信息时通知全局事务视图，并获取相应的信息。

4.2 封锁管理

数据库管理系统一般采用行锁进行并发访问控制，避免多个用户同时修改相同数据；通过表锁、字典锁控制 DDL 和 DML 操作的并发访问，保证对象定义的有效性和数据访问的正确性。DM 则采用了独特的封锁机制，使用 TID 锁和对象锁进行并发访问控制，有效减少封锁冲突、提升系统并发性能。

TID 锁以事务号为封锁对象，每个事务启动时，自动以独占（X）方式对当前事务号进行封锁，由于事务号是全局唯一的，因此这把 TID 锁不存在冲突，总是可以封锁成功。同时，在 4.1 事务管理中，介绍了物理记录上包含一个 TID 字段，记录了修改数据的事务号。执行 INSERT、DELETE、UPDATE 操作修改物理记录时，设置事务号到 TID 字段的动作，就相当于隐式地对物理记录上了一把 X 方式的 TID 锁。因此，通过事务启动时创建的 TID 锁，以及写入物理记录的 TID 值，DM 中所有修改物理记录的操作都不再需要额外的行锁，避免了大量行锁对系统资源的消耗，有效减少封锁冲突。特别是在 DMDSC 集群中，需要进行全局封锁，封锁的代价比单节点更高，通过 TID 锁可以有效减少封锁引发的性能损失。

对象锁则通过对象 ID 进行封锁，将对数据字典的封锁和表锁合并为对象锁，以达到减少封锁冲突、提升系统并发性能的目的。

与事务管理类似，DMDSC 集群将封锁管理拆分为全局封锁服务（Global Locking Services，简称 GLS）和本地封锁服务（Local Locking Services，简称 LLS）两部分。整个系统中，只有控制节点拥有一个 GLS。控制节点的 GLS 统一处理集群中所有节点的封锁请求、维护全局封锁信息、进行死锁检测，确保事务并发访问的正确性。每个节点都有一个 LLS。各节点的 LLS 负责与 GLS 协调、通讯，完成事务的封锁请求，DMDSC 集群中所有封锁请求都需要通过 LLS 向 GLS 发起，并在获得 GLS 授权后，才能进行后续操作。

4.3 闩管理

闩（Latch）是数据库管理系统的一种内部数据结构，通常用来协调、管理数据缓冲区、字典缓存和数据库文件等资源的并发访问。与锁（Lock）在事务生命周期中一直保持不同，闩（Latch）通常只保持极短的一段时间，比如修改 Buffer 中数据页内容后，马上会释放。闩（Latch）的封锁类型也比较简单，就是共享（Share）和独占（Exclusive）两种类型。

为了适用 DMDSC 集群，我们同样将闩划分为全局闩服务（Global Latch Services，简称 GLS）和本地闩服务（Local Latch Services，简称 LLS）两个部分。但是，为了与全局封锁服务 GLS 和本地封锁服务 LLS 的名字简称区分开来，我们以使用最为频繁的 Buffer 来命名全局闩服务。因此，全局闩服务也称为全局缓冲区服务（Global Buffer Services），简称 GBS；本地闩服务也称为本地缓冲区服务（Local Buffer Services），简称 LBS。

整个系统中，每一个节点上都部署一个 GBS 和一个 LBS。GBS 服务协调节点间的 Latch 封锁请求、以及 Latch 权限回收。GBS 与 GTV/GLS 由控制节点统一管理不同，GBS 不是集中式管理，而是由 DMDSC 集群中的所有节点共同管理，Buffer 对象会根据数据页号（Page No）对数据页进行划分，分给某一个节点的 GBS 服务处理。LBS 服务与 LLS/LTV 一样，部署在每一个节点，LBS 服务根据用户请求，向 GBS 发起 Latch 封锁，或者根据 GBS 请求，回收本地的 Latch 封锁。

为了避免两个或多个节点同时修改同一个数据页导致数据损坏，或者数据页修改过程中别的节点读取到无效内容，DMDSC 集群中数据页的封锁流程做了一些改变。与单节点相比，增加了全局 Latch 封锁、释放两个步骤。同时在获取全局 Latch 授权后，仍然需要进行正常的本地 Latch 封锁，有效避免了节点内的访问冲突。

4.4 缓存交换

根据目前的硬件发展状况来看，网络的传输速度比磁盘的读、写速度更快，因此，DMDSC

集群引入了缓存交换（Buffer Swap）技术，节点间的数据页尽可能通过网络传递，避免通过磁盘的写入、再读出方式在节点间传递数据，从而减少数据库的 IO 等待时间，提升系统的响应速度。

缓存交换的实现基础是 GBS/LBS 服务，在 GBS/LBS 中维护了 Buffer 数据页的相关信息。包括：1. 闩的封锁权限（LATCH）；2. 哪些站点访问过此数据页（Access MAP）；3. 最新数据保存在哪一个节点（Fresh EP）中；4. 以及最新数据页的 LSN 值（Fresh LSN）等信息。这些信息作为 LBS 封锁、GBS 授权和 GBS 权限回收请求的附加信息进行传递，因此并不会带来额外的通讯开销。

下面以两节点 DMDSC 集群（EP0/EP1）访问数据页 P1 为例子。初始页 P1 位于共享存储上，P1 的 GBS 控制结构位于节点 EP1 上。初始页 P1 还没有被任何一个节点访问过，初始页 P1 的 LSN 为 10000。通过几种常见场景分析，逐步深入，解析缓存交换的原理。

- **场景 1**

节点 EP0 访问数据页 P1。

1. 节点 EP0 的本地 LBS 向 EP1 的 GBS 请求数据页 P1 的 S LATCH 权限

2. 节点 EP1 的 GBS 修改 P1 控制结构，记录访问节点 EP0 的封锁模式为 S LATCH（数据分布节点为 EP0），并响应 EP0 的 LBS 请求

3. 节点 EP0 的 LBS 获得 GBS 授权后，记录获得的授权模式是 S_LATCH，P1 数据不在其他节点的 Buffer 中，发起本地 IO 请求，从磁盘读取数据。IO 完成后，修改 LBS 控制结构，记录数据页上的 LSN 信息

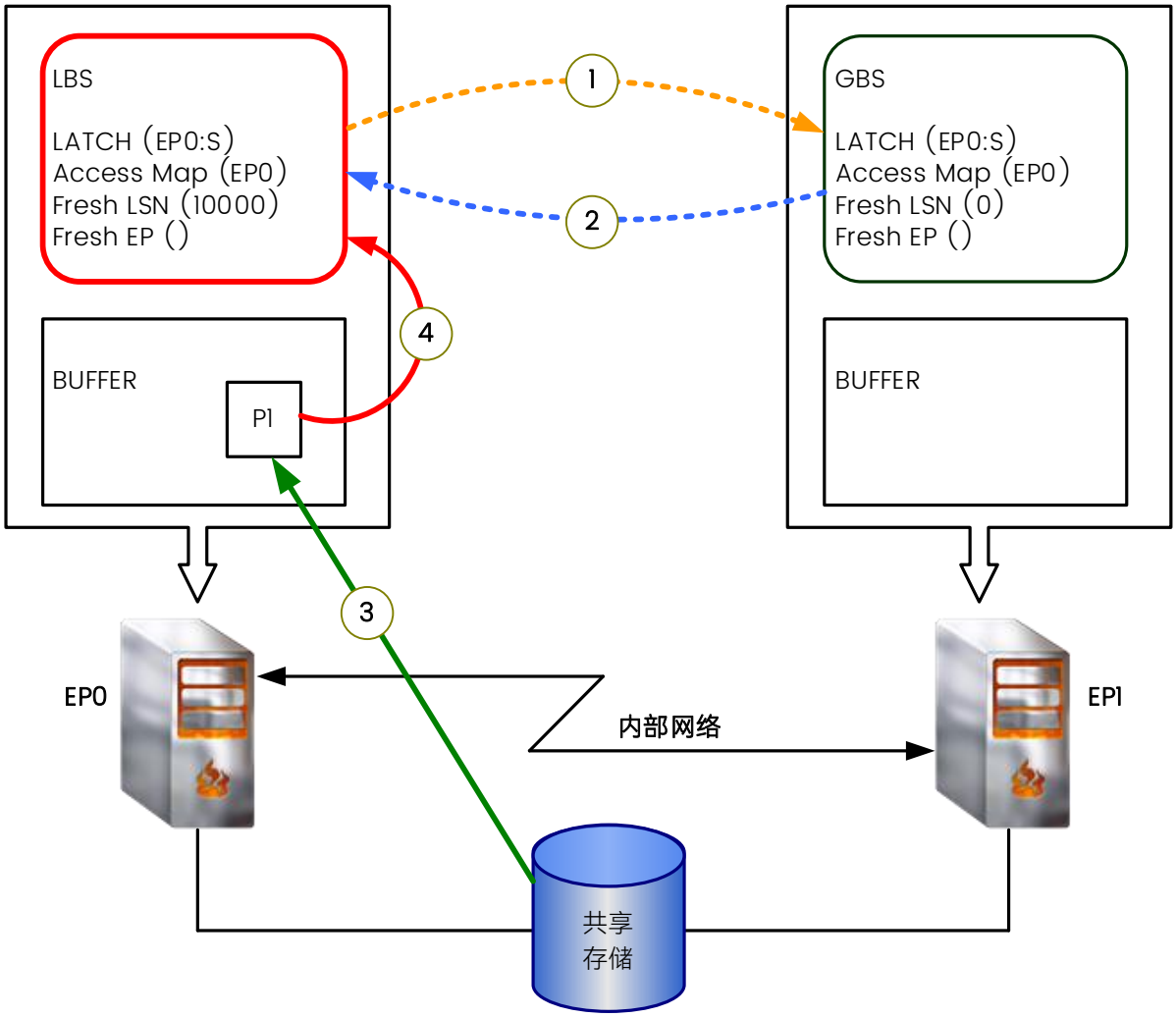


图 4.2 本地 IO

■ **场景 2**

节点 EP1 访问数据页 P1。

1. 节点 EP1 本地 LBS 向 EP1 的 GBS 请求数据页 P1 的 S LATCH 权限

2. 节点 EP1 的 GBS 修改控制结构，记录访问节点 EP1 的封锁模式为 S LATCH（数据分布节点为 EP0/EP1），并响应 EP1 的 LBS 请求

3. 节点 EP1 的 LBS 获得 GBS 授权后，记录获得的授权模式是 S LATCH，根据数据分布情况，EP1 向 EP0 发起 P1 的读请求，通过内部网络从 EP0 获取数据，而不是重新从磁盘

读取 P1 数据

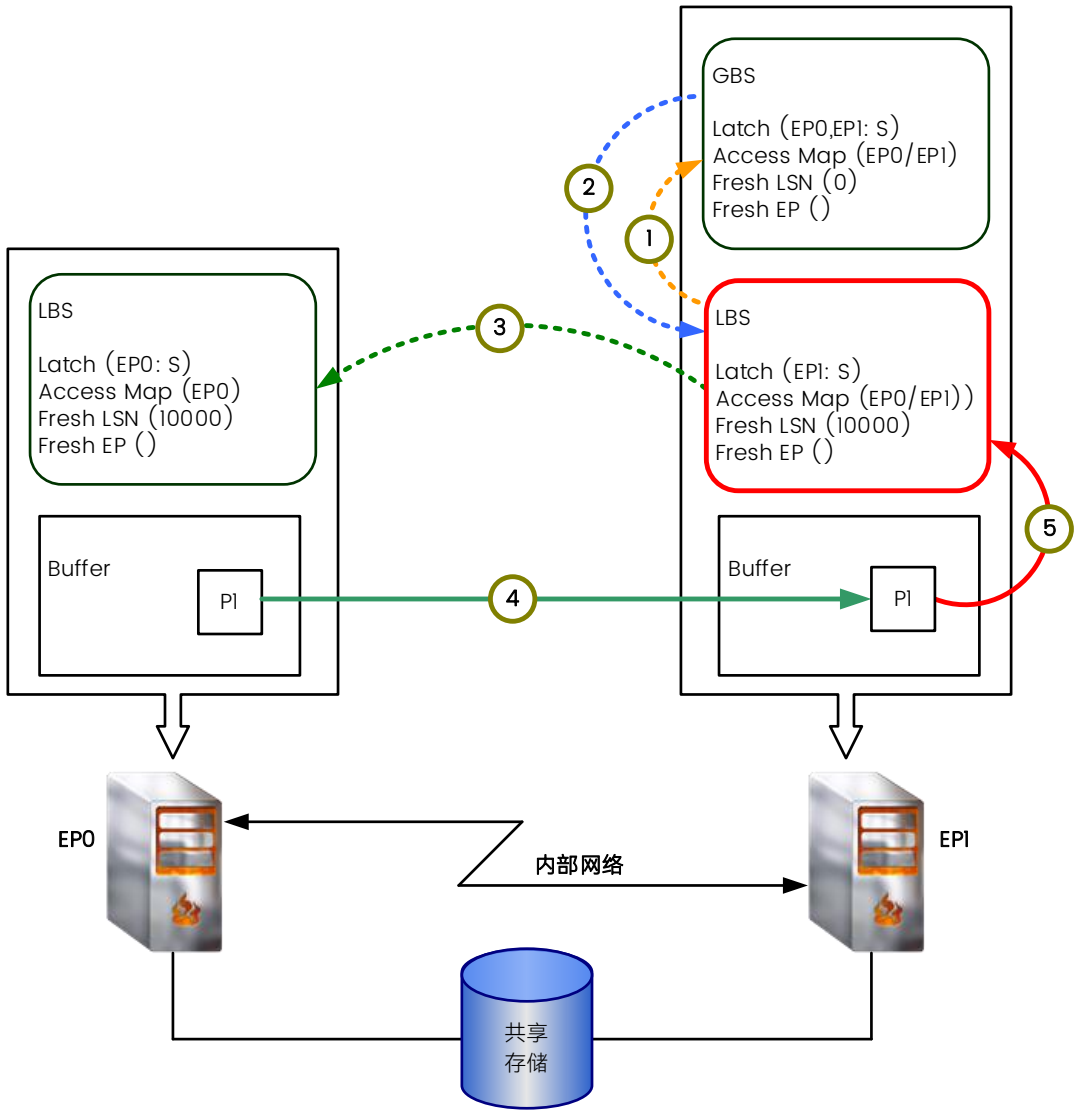


图 4.3 远程 IO

■ **场景 3**

节点 EP0 修改数据页 P1。

1. 节点 EP0 本地 LBS 向 EP1 的 GBS 请求数据页 P1 的 X LATCH 权限(附加 LSN 信息)
2. 节点 EP1 的 GBS 修改控制结构的 LSN 值，从 EP1 的 LBS 回收 P1 的权限
3. 修改访问节点 EP0 的封锁模式为 S + X LATCH，并响应 EP0 的 LBS 请求
4. 节点 EP0 的 LBS 获得 GBS 授权后，记录获得的授权模式是 S + X LATCH
5. 节点 EP0 修改数据页 P1，LSN 修改为 11000

这个过程中，只有全局 Latch 请求，数据页并没有在节点间传递。

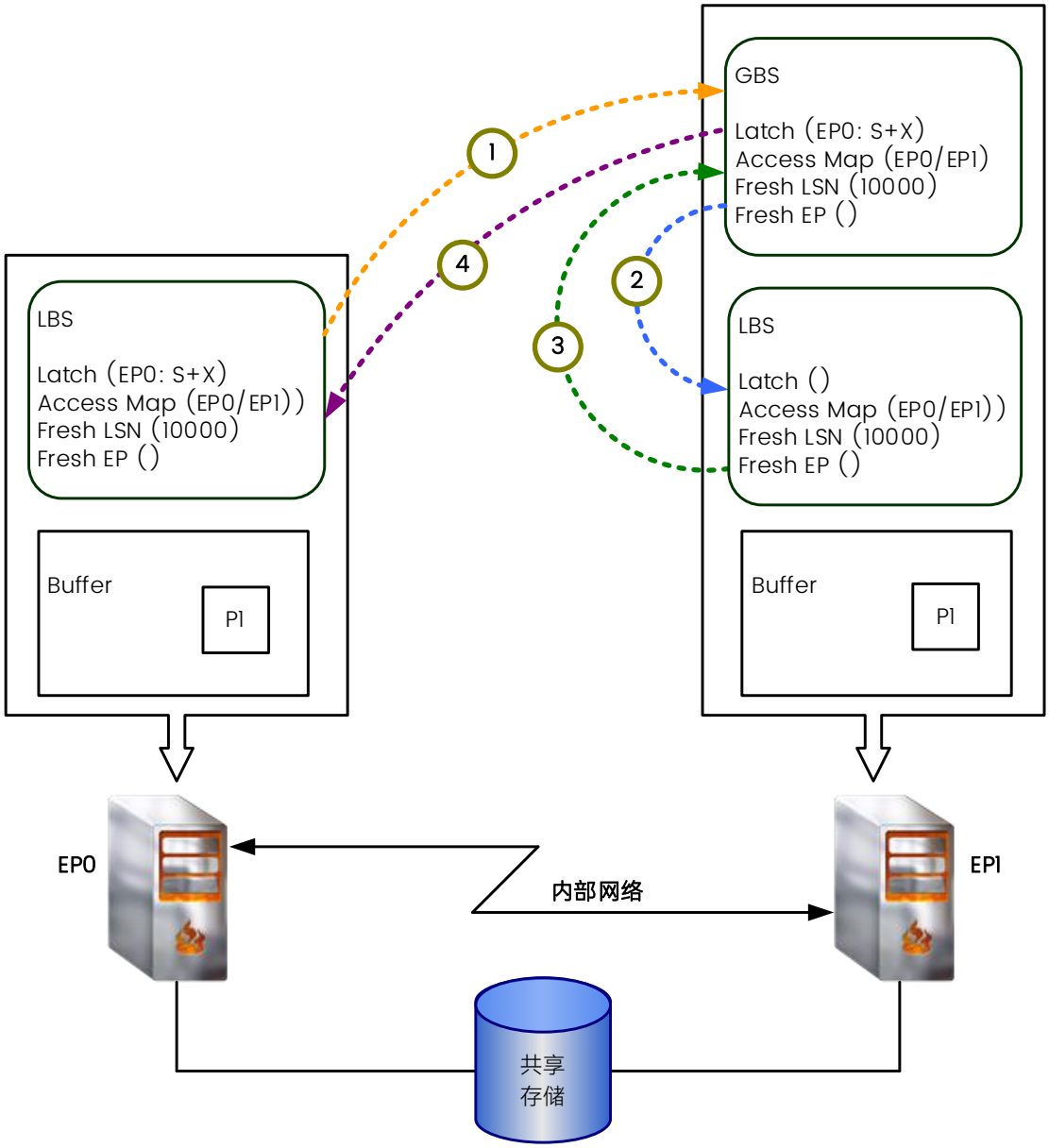


图 4.4 GBS 管理

修改之后，数据页 P1 的 LSN 修改为 11000。如下所示:

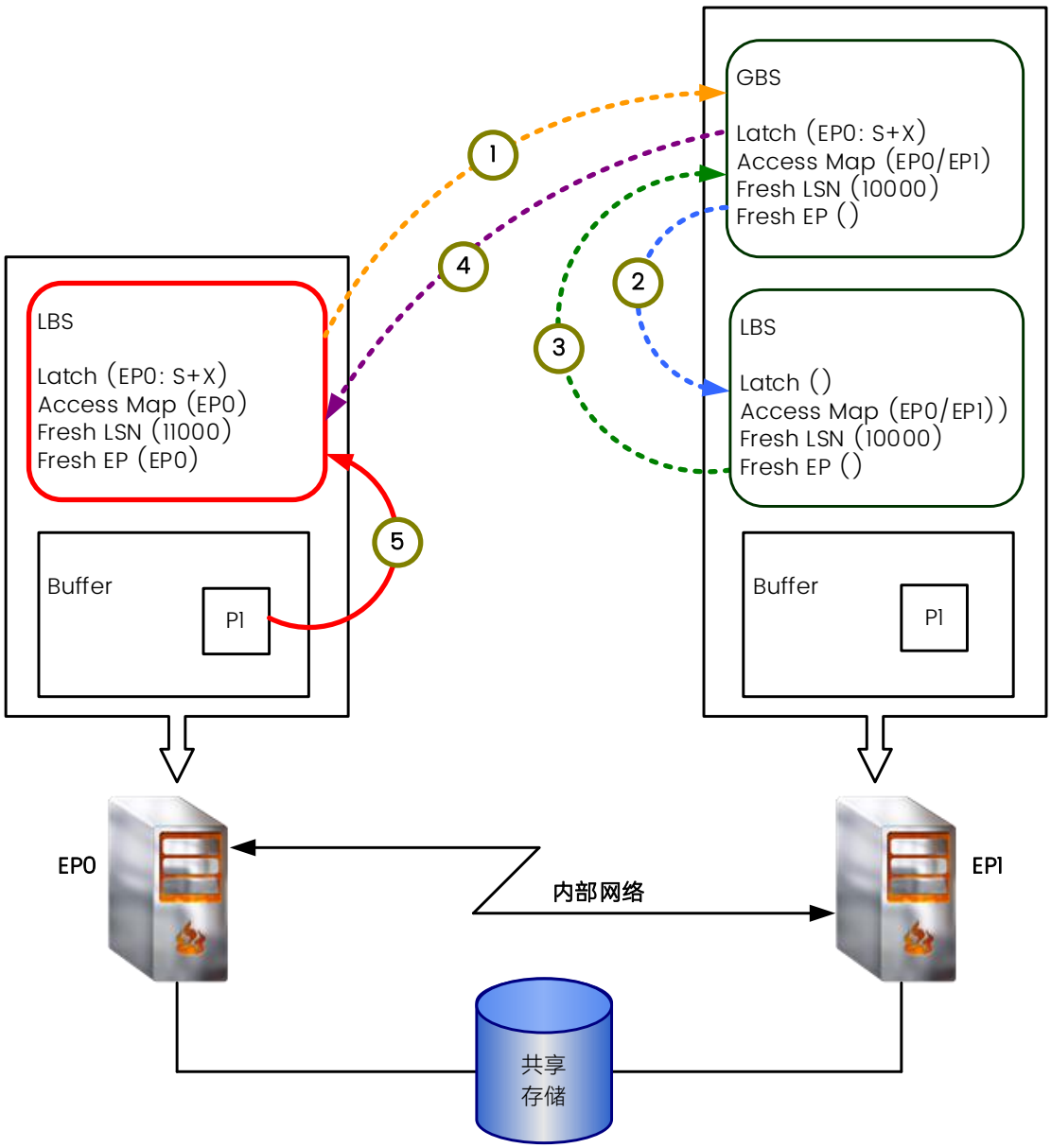


图 4.5 数据修改

■ **场景 4**

节点 EP1 修改数据页 P1。

1.节点 EP1 本地 LBS 向 EP1 的 GBS 请求数据页 P1 的 X LATCH 权限

2.节点 EP1 的 GBS 发现 P1 被 EP0 以 S + X 方式封锁，向 EP0 发起回收 P1 权限的请求

3.节点 EP0 释放 P1 的全局 LATCH，响应 GBS，并且在响应消息中附加了最新的 PAGE

LSN 值

4.节点 EP1 的 GBS 收到 EP0 的响应后，修改 GBS 控制结构，记录最新数据保存在 EP0，最新的 LSN 值信息，记录 EP1 获得的授权模式是 S + X LATCH（此时，数据分布节点仍然是 EP0/EP1），并授权 EP1 的 LBS

5.节点 EP1 的 LBS 收到授权信息后，记录获得的授权模式是 S + X LATCH，并根据数据分布情况，向节点 EP0 发起数据页 P1 的读请求

6.节点 EP1 修改数据页 P1，LSN 修改为 12000

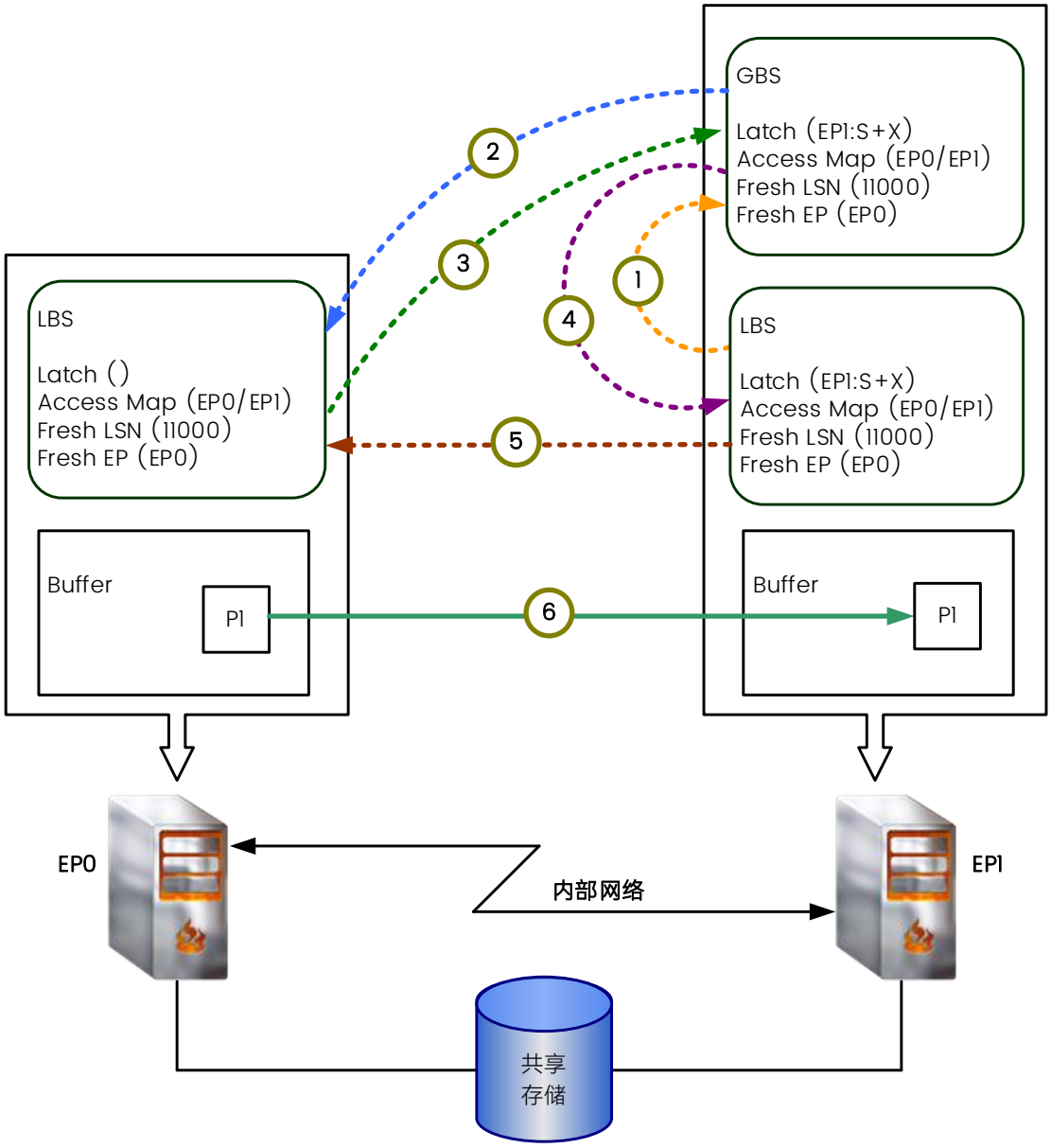


图 4.6 GBS 管理

修改之后，数据页 P1 的 LSN 修改为 12000。如下所示:

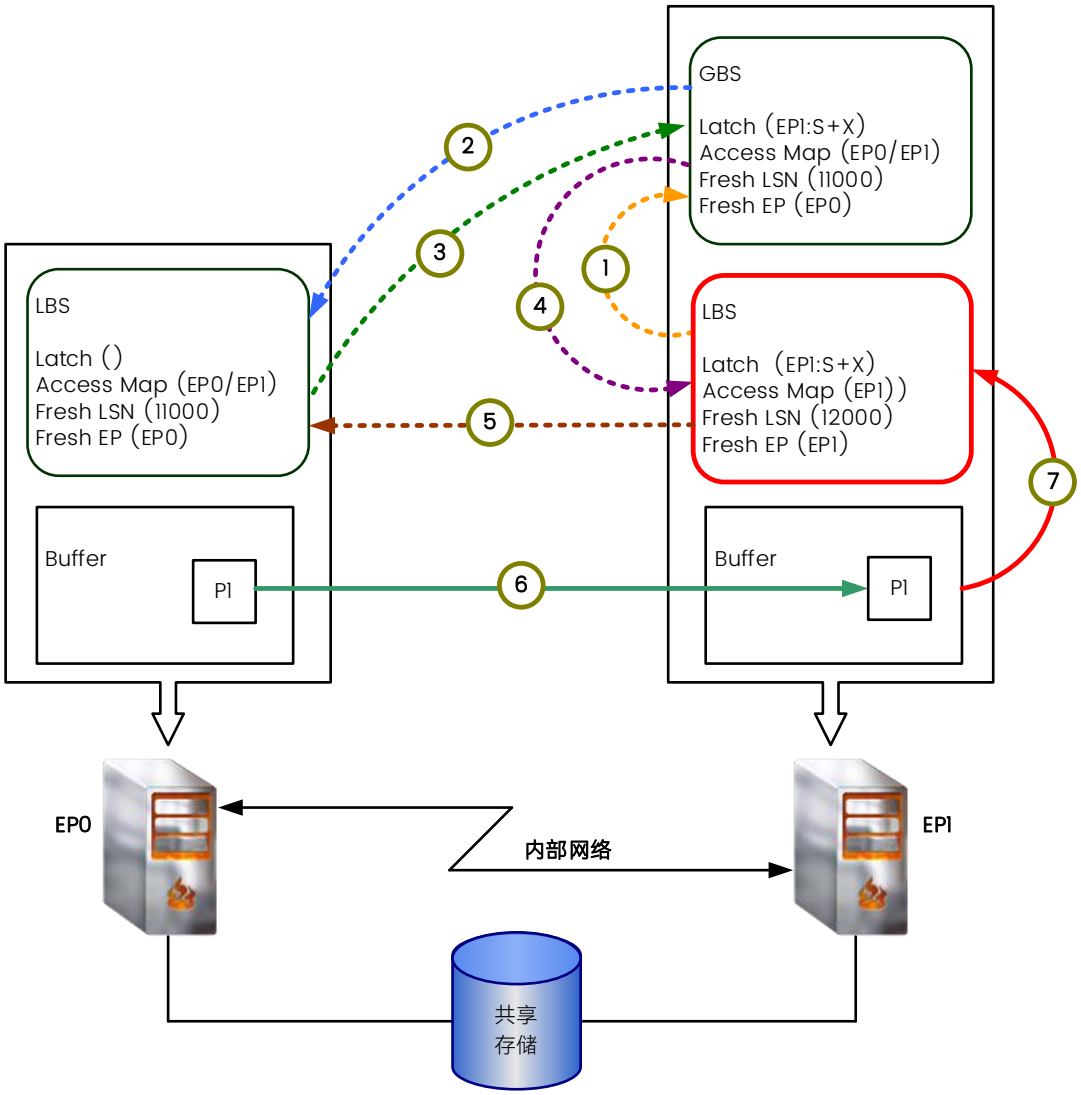


图 4.7 数据修改

这个过程中，数据页 P1 的最新数据从 EP0 传递到了 EP1，但并没有产生磁盘 IO。

4.5 重做日志管理

REDO 日志包含了所有物理数据页的修改内容，Insert/delete/update 等 DML 操作、Create Table 等 DDL 操作，最终都会转化为对物理数据页的修改，这些修改都会反映到

REDO 日志中。一般说来，一条 SQL 语句在系统内部会转化为多个相互独立的物理事务来完成，物理事务提交时产生 REDO 日志，并最终写入联机 REDO 日志文件中。

一个物理事务包含一个或者多个 REDO 记录（Redo Record，简称 RREC），每条 REDO 记录都对应一个修改物理数据页的动作。根据记录内容的不同，RREC 可以分为两类：物理 RREC 和逻辑 RREC。物理 RREC 记录的是数据页的变化情况，内容包括：操作类型、修改数据页地址、页内偏移、数据页上的修改内容、长度信息（变长类型的 REDO 记录才有）。逻辑 RREC 记录的是一些数据库逻辑操作步骤，主要包括：事务启动、事务提交、事务回滚、字典封锁、事务封锁、B 树封锁、字典淘汰等，一般只在配置为 Primary 模式时才产生逻辑 RREC。

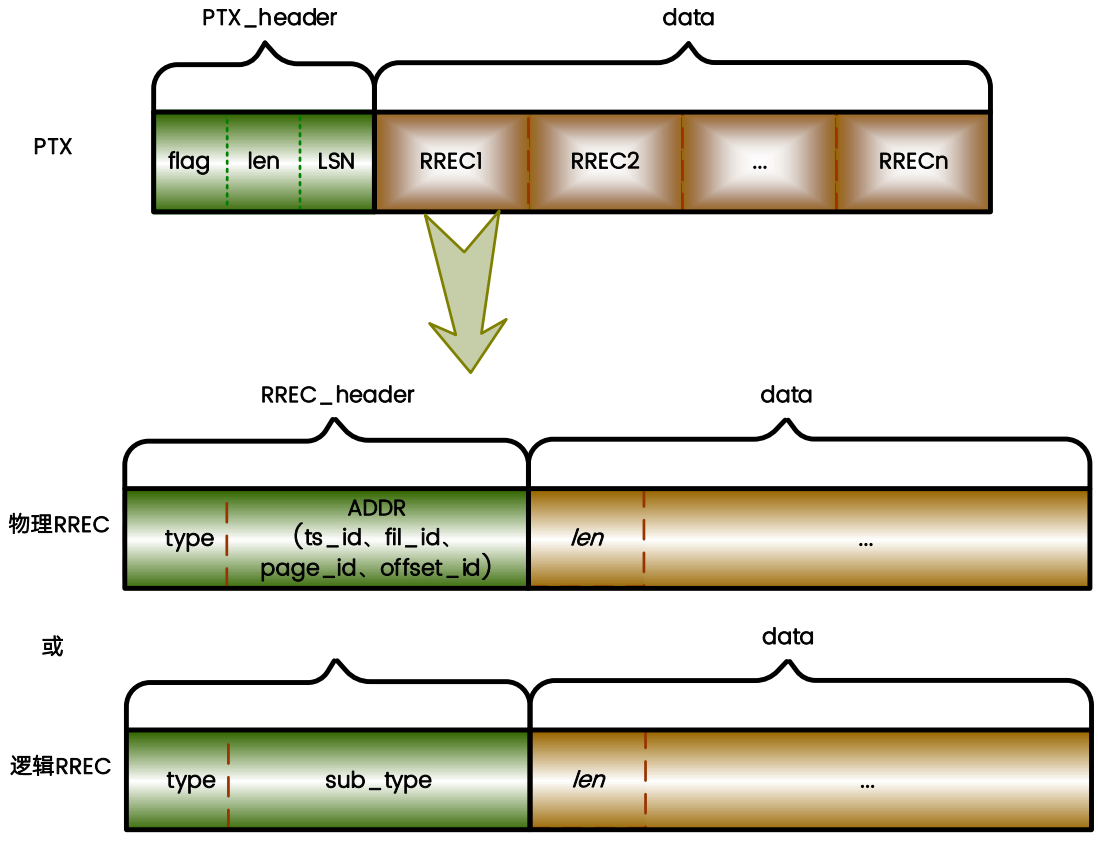


图 4.8 PTX 和 RREC 结构图

DMDSC 集群中，各个节点拥有独立的日志文件，REDO 日志的 LSN 值也是顺序递增的，REDO 日志只会写入当前数据库实例的联机日志文件，与集群系统中的其他数据库实例没有关系。考虑到所有节点都可以修改数据，同一个数据页可能由不同节点先后修改，为了体现

修改的先后顺序，确保故障恢复时能够按照操作的顺序将数据正确恢复。DMDSC 集群要求对同一个数据页的修改，产生的 LSN 值是全局递增的，各个节点对同一数据页的修改在日志系统中是严格有序的。但是，针对不同数据页的修改并不要求 LSN 是全局递增的，也就是说只有多个节点修改相同数据页时，才会产生全局 LSN 同步问题。并且 LSN 全局同步，是在缓存交换时附带完成的，并不会增加系统的额外开销。

与单节点系统相比，DMDSC 的日志系统存在以下差异：

1.本地 REDO 日志系统中，LSN 值保证是递增的，后提交物理事务的 LSN 值一定更大；但顺序提交的两个物理事务产生的 LSN 值，不能保证一定是连续的。

2.全局 REDO 日志系统中，LSN 值不再严格保证唯一性。不同节点可能存在 LSN 值相等的重做日志记录。

3.故障重启时，控制节点需要重做所有节点的 REDO 日志，重做过程中会根据 LSN 排序，从小到大依次重做。

4.联机 REDO 日志文件需要保存在共享存储中。

4.6 回滚记录管理

DMDSC 集群的多版本并发控制（MVCC）实现策略是，通过回滚记录获取数据的历史版本，通过活动事务视图判断事务可见性、确定获取指定版本数据。因此，回滚记录也必须进行全局维护，有可能在节点间进行传递。与单节点一样，DMDSC 集群中只有一个回滚表空间，回滚记录保存在回滚页中，回滚页与保存用户记录的数据页一样，由 Buffer 系统管理，并通过缓存交换机制实现全局数据共享。

为了减少并发冲突，提高系统性能，DMDSC 集群中为每个节点分配了一个单独的回滚段（Segment），虽然这些回滚段位于同一个回滚表空间中，但是各个节点的回滚页申请、释放，并不会产生全局冲突。

与重做日志不同，DMDSC 集群故障重启时，各个活动节点扫描各自的回滚段，控制节

点增加扫描故障节点回滚段，收集未提交事务进行回滚，收集已提交事务进行 Purge 操作。

5 DMCSS 介绍

达梦集群同步服务（Dameng Cluster Synchronization Services，简称 DMCSS）使用 DMASM 集群或 DMDSC 集群都必须配置 DMCSS 服务。在 DMASM 集群或 DMDSC 集群中，每个节点都需要配置一个 DMCSS 服务。这些 DMCSS 服务自身也构成一个集群，DMCSS 集群中负责监控、管理整个 DMASM 集群和 DMDSC 集群的节点称为控制节点（Control Node），其他 DMCSS 节点称为普通节点(Normal Node)。DMCSS 普通节点不参与 DMASM 集群和 DMDSC 集群管理，当 DMCSS 控制节点故障时，会从活动的普通节点中重新选取一个 DMCSS 控制节点。

DMCSS 工作的基本原理是：在 VOTE 磁盘（非镜像环境下）或 DCRV 磁盘（镜像环境下）中，为每个被监控对象（DMASMSVR、DMSERVER、DMCSS）分配一片独立的存储区域，被监控对象定时向 VOTE 或 DCRV 磁盘写入信息（包括时间戳、状态、命令、以及命令执行结果等）；DMCSS 控制节点定时从 VOTE 或 DCRV 磁盘读取信息，检查被监控对象的状态变化，启动相应的处理流程；被监控对象只会被动的接收 DMCSS 控制节点命令，执行并响应。

DMCSS 主要功能包括：写入心跳信息、选举 DMCSS 控制节点、选取 DMASM/DMDSC 控制节点、管理被监控对象的启动流程、集群状态监控、节点故障处理、节点重加入等，DMCSS 还可以接收并执行 DMCSSM 指令。

5.1 启动命令

```
dmcss V8

version: 03134284389-20250411-268655-10000

书写格式: ./dmcss KEYWORD=<value> {KEYWORD=<value>}

        其中{}表示大括号内的关键词可为 0~N 个，N 为正整数，{}本身无需出现在语句中
```

```
使用示例: ./dmcss DCR_INI=/opt/data/DAMENG/dmdcr.ini

------------------------------------------------------------------

关键词（KEYWORD）          说明                    缺省值          是否必选项

------------------------------------------------------------------

DCR_INI                   dmdcr.ini 路径          无              是

-NOCONSOLE                以服务方式启动          无              否

LISTEN_IPV6               是否监听 IPV6           Y               否

HELP                      打印帮助信息            无              否

------------------------------------------------------------------
```

5.2 心跳信息

DMCSS 实例启动后，每间隔 1 秒向 VOTE 磁盘（非镜像环境下）或 DCRV 磁盘（镜像环境下）指定区域写入心跳信息（包括自身的状态、时间戳等），表示 DMCSS 节点处于活动状态。

5.3 选举 DMCSS 控制节点

DMCSS 启动后向 VOTE 磁盘（非镜像环境下）或 DCRV 磁盘（镜像环境下）写入信息，并读取其他 DMCSS 节点的信息，如果 DMCSS 集群中还没有活动的控制节点，则选举 DMCSS 控制节点。DMCSS 选举的原则有两条:

1. 先启动的 DMCSS 作为控制节点。
2. 如果 DMCSS 同时启动，那么则选择节点号小的节点为控制节点。
3. 如果 DMCSS 控制节点挂掉，那么会将先向 VOTE 磁盘（非镜像环境下）或 DCRV 磁盘（镜像环境下）写入心跳信息的节点设置为控制节点；若同时有多个节点先向 VOTE 磁盘（非镜像环境下）或 DCRV 磁盘（镜像环境下）写入心跳信息，那么选择节点号小的节

点为控制节点。

5.4 选取监控对象控制节点

DMCSS 控制节点启动后，会为 DMDSC 集群指定控制节点。DMCSS 选取监控对象控制节点的原则有两条:

1. 只有一个活动节点，则设置活动节点为控制节点。
2. 存在多个活动节点，则选择节点号小的节点为控制节点。

5.5 启动流程管理

DMASM 和 DMDSC 集群中的实例启动后，一直处于 WAITING 状态，等待 DMCSS 的启动命令。DMCSS 控制节点在选取监控对象控制节点后，通知控制节点启动，在控制节点启动完成后，再依次通知其他普通节点启动。

5.6 状态检测

DMCSS 维护集群状态，随着节点活动信息的变化，集群状态也会产生变化，DMCSS 控制节点会通知被监控节点执行不同命令,来控制节点启动、故障处理、故障重加入等操作。

DMCSS 控制节点每秒从 VOTE 磁盘（非镜像环境下）或 DCRV 磁盘（镜像环境下）读取被监控对象的心跳信息。一旦被监控对象的时间戳在 DCR_GRP_DSKCHK_CNT 秒内没有变化，则认为被监控对象出现异常。

DMCSS 普通节点定时读取 DMCSS 控制节点的心跳信息，监控 DMCSS 运行状态。

5.7 故障处理

DMCSS 控制节点检测到实例故障后，首先向故障实例的 VOTE 磁盘（非镜像环境下）或 DCRV 磁盘（镜像环境下）区域写入 Kill 命令（所有实例一旦发现 Kill 命令，无条件自杀），

避免故障实例仍然处于活动状态，引发脑裂，然后启动故障处理流程，不同类型实例的故障处理流程存在一些差异。

- DMCSS 控制节点故障处理流程

1. 活动节点重新选举 DMCSS 控制节点
2. 新的 DMCSS 控制节点通知 DMCSS 故障节点对应的 DMASMSVR、DMSERVER 强制退出

- DMASMSVR 实例故障处理流程

1. 挂起工作线程
2. 更新 DCR（参考 10 DMASM 介绍）故障节点信息
3. 通知故障节点对应 DMSERVER 强制退出
4. dmasmsvr 进行故障恢复
5. 恢复工作线程

- DMSERVER 实例故障处理流程

1. 更新 DCR 故障节点信息
2. 重新选取一个控制节点
3. 通知 DMSERVER 控制节点启动故障处理流程（参考 7 DMDSC 故障处理）
4. 等待 DMSERVER 故障处理结束

5.8 节点重加入

如果检测到故障节点恢复，DMCSS 会通知控制节点启动节点重加入流程。

- 数据库实例重加入

1. 挂起工作线程
2. 修改节点的状态为故障节点重加入状态
3. 执行恢复操作

4. 节点进入 STARTUP 状态，准备启动

5. OPEN 重加入的节点

6. 重启工作线程

7. 执行 OPEN 数据库实例的操作

■ DMASM 实例重加入

1. 挂起工作线程

2. 修改节点的状态为故障节点重加入状态

3. 执行恢复操作

4. 节点进入 STARTUP 状态，准备启动

5. OPEN 重加入的节点

6. 重启工作线程

5.9 集群指令

DMCSS 控制节点通过一系列的集群指令控制被监控对象的启动、故障处理、状态切换等。DMCSS 控制节点向目标对象的 VOTE 磁盘（非镜像环境下）或 DCRV 磁盘（镜像环境下）指令区写入命令，通知目标对象执行相应命令，并等待执行响应。每条指令的功能都比较单一，比如修改状态、设置控制节点、执行一条 SQL 等，复杂的集群流程控制就是由这些简单的指令组合起来完成的。

5.10 控制台命令

1. help

显示帮助信息。

2. show

查看集群状态。DMCSS 控制节点会显示 DMCSS、DMASM、DMDSC 三个集群的信息，

而 DMCSS 普通节点只会显示 DMCSS 集群的信息。

具体显示的内容如下所示。

group[]行显示的内容为：

name: 集群名称。

seq: 集群编号。

type: 集群组类型。包含：CSS、ASM、DB。

control_node: 集群内控制节点。

css 集群组额外显示的内容为：

DCRV_STATUS：dcrv 磁盘状态，仅在 dsc 镜像环境中展示。包括：OK(正常状态)、FAILED(故障状态)、UNUSED(未使用状态)、UNKNOWN(未知状态)。

ep 行显示的内容为：

css_time：取得 ep 信息的 css 当前时间,仅 dmcssm 显示。

inst_name：节点实例名。

seqno：节点编号。

port：实例对外提供服务的端口号。

mode：节点模式。包括：CONTROL（控制节点）、NORMAL（普通节点）。

inst_status：实例系统状态。包括：INSTALL（初始化数据库）、STARTUP（节点启动）、AFTER REDO（启动，REDO 完成）、MOUNT（挂载）、OPEN（正常打开）、SUSPEND（挂起）、SHUTDOWN（关闭）、CRASH_RECV（出现节点故障）、INVALID EP STATUS（集群非 OPEN 状态下，不支持故障处理，可能是集群同步服务 DMCSS 或监视器 DMCSSM 版本过旧导致）。

vtd_status：实例的集群状态。包括：STARTUP（启动）、STARTUP2（启动第二步）、WORKING（工作中）、TO SHUTDOWN（准备关闭）、SHUTDOWN（已经关闭）、SYSHALT（系统崩溃）、STOP（停止）、ADD_DCRV（添加 DCRV 磁盘）、DEL_DCRV

（删除 DCRV 磁盘）。

is_ok：实例在集群内是否正常。OK 是，ERROR 否。ERROR 节点暂时从集群内踢出。

active：实例是否活动。TRUE 是，FALSE 否。

guid：实例的 GUID 值。

pid:实例进程 id。

ts：实例的时间戳。

css 集群组额外显示的内容为：

auto check:CSS 是否允许自动执行故障处理/故障恢复。TRUE 是，FALSE 否。该参数仅 dmcssm 可查看。

auto restart:节点是否能够被自动拉起。TRUE 是，FALSE 否。

DCRV_STATUS：dcrv 磁盘状态，仅在 dsc 镜像环境中展示。包括：OK(正常状态)、FAILED(故障状态)、UNUSED(未使用状态)、UNKNOWN(未知状态)。

asm/db 集群组额外显示的内容为：

n_ok_ep:当前集群中 OK 状态的站点个数。

ok_ep_arr(index,seqno):当前集群中 OK 状态的站点号。index 表示内存中 ok_ep_arr 数组下标，seqno 表示 index 下标对应的 ok 状态站点号。

sta:当前集群组状态。包括：STARTUP（控制节点选举）、OPEN（节点启动）、Control Node STARTUP（控制节点启动）、Normal Node STARTUP（非控制节点启动）、EP_OPEN（节点启动到 OPEN 状态）、PROCESS_CRASH(节点故障处理)、PROCESS_RECOVER（节点故障恢复）、SYS_HALT（系统 halt）、SHUTDOWN（组退出）、LINK_CHECK（链路检查）、STOP（节点单独退出）、INVALID GRP STATUS（无效状态）。

sub_sta:当前集群组子状态。包括：STARTUP（节点启动）、WAIT_CSS_MASTER

（等待控制节点选举）、WAIT_STARTUP（等待节点启动）、WAIT_EP_OPEN（等待节点启动到 open 状态）、WAIT_CRASH_RECV（等待故障恢复结束）、SUB_WAIT_ERROR_EP_ADD（等待故障节点重加入结束）、WAIT_EP_RECOVER（等待节点故障恢复结束）、WAIT_SYS_HALT（等待系统 HALT 结束）、PRE_SUSPEND（准备挂起工作线程）、SUSPEND_WORKER（挂起工作线程）、WAIT_SUSPEND_WORKER（等待工作线程挂起）、WAIT_RESUME_WORKER（等待恢复工作线程）、WAIT_REAL_OPEN（等待节点启动到 REAL_OPEN 状态）、WAIT_REDO（主机 redo 执行完，等待从机启动）、AFTER_REDO（redo 执行完成）、MASTER_CONFIG_VIP（控制节点配置 vip）、SLAVE_CONFIG_VIP（非控制节点配置 vip）、WAIT_SLAVE_CONFIG_VIP（等待非控制节点配置 vip）、WAIT_MASTER_CONFIG_VIP（等待控制节点配置 vip）、EP_CONFIG_VIP（dmserver 销毁 VIP 监听线程）、WAIT_EP_CONFIG_VIP（等待 dmserver 销毁 VIP 监听线程）、EP_CRASH（节点故障处理）、WAIT_EP_CRASH（等待节点故障处理）、STOP_EP_CRASH（单节点故障处理）、WAIT_LINK_CHECK（等待链路检查）、WAIT_DCR_LOAD（等待 dcr 信息加载）、WAIT_ADJUST_MASTER（等待执行 ADJUST_MASTER_EP 命令完成）、CRASH_ROLLBACK（回滚故障处理）、WAIT_CRASH_ROLLBACK（等待回滚故障处理）、WAIT_CMD_CLEAR（等待命令清理）。

break ep:正在进行故障处理的站点。

recover ep:正在进行故障恢复的站点。

例 下面展示一段 show 信息。

```
======== group[name = GRP_CSS, seq = 0, type = CSS, Control Node = 0]=========


[CSS0] auto check = TRUE, global info:
```

```
[ASM0] auto restart = FALSE

[DSC0] auto restart = FALSE

[CSS1] auto check = TRUE, global info:

[ASM1] auto restart = FALSE

[DSC1] auto restart = FALSE


ep:       css_time              inst_name      seqno     port       mode            inst_status
          vtd_status      is_ok     active       guid              pid             ts
          2024-11-18 17:00:48      CSS0            0         9941     Control Node          OPEN
          WORKING         OK        TRUE      892779205       50328       892779531
          2024-11-18 17:00:48       CSS1            1         9942     Normal Node           OPEN
          WORKING         OK        TRUE      893084408       80628       893084733


======== group[name = GRP_ASM, seq = 1, type = ASM, Control Node = 0]==========


n_ok_ep = 1

ok_ep_arr(index, seqno):

(0, 0)


sta = OPEN, sub_sta = STARTUP

break ep = NULL

recover ep = NULL


crash process over flag is TRUE
```

```
ep:       css_time              inst_name   seqno     port        mode           inst_status
          vtd_status      is_ok       active      guid             pid          ts
          2024-11-18 17:00:48      ASM0        0       9949     Control Node          OPEN
          WORKING         OK          TRUE     894305753       151772       894306070
          2024-11-18 17:00:48      ASM1        1       9950     Normal Node    OPEN
          WORKING         ERROR       FALSE    894611051       43264        894611275


======= group[name = GRP_DSC, seq = 2, type = DB, Control Node = 255]=========


n_ok_ep = 2
ok_ep_arr(index, seqno):
(0, 0)
(1, 1)


sta = OPEN, sub_sta = STARTUP
break ep = NULL
recover ep = NULL


crash process over flag is FALSE
ep:       css_time              inst_name     seqno     port        mode           inst_status
          vtd_status     is_ok      active       guid             pid            ts
          2024-11-18 17:00:48      DSC0          0          5236     Normal Node       SHUTDOWN
          SHUTDOWN        OK       FALSE    901786929       126008       901787096
          2024-11-18 17:00:48      DSC1          1          5237     Normal Node       SHUTDOWN
```

```
        SHUTDOWN        OK        FALSE    901940019        79784        901940183



============================================================
```

3. open force group_name

在启动 ASM 或 DB 组时，如果某个节点故障一直无法启动，可借助此命令将 ASM 或 DB 组强制 OPEN。

此命令需要在 DMCSS 控制节点上执行，并且 DMCSS 控制节点的监控需要处于打开状态。

4. extend node

DMDSC 集群联机增加节点时使用。

程序会通知所有实例(DMCSS/DMASMSVR/DMSERVER)更新 dcr 信息。

使用 show 命令能看到新增节点信息，新增 DMASMSVR/DMSERVER 为 ERROR 状态。

5. set master force

该命令用于强制设置当前节点为控制节点，该命令不能在 DSC 集群处于启动、故障处理或故障节点重加入过程中进行。

6. halt css ep_seqno

该命令用于使指定的 DMCSS 节点故障，ep_seqno 为 DMCSS 节点号。控制节点可以将 ep_seqno 设置为任意需要故障的节点号；非控制节点只能将 ep_seqno 设置为自身节点号，如果设置 ep_seqno 为非自身节点号，则会报错。

7. exit

退出 DMCSS。

5.11 注意事项

如果节点 A 的 DMCSS 退出或者故障，活动 DMCSS 会给节点 A 上所监控的 DMASMSVR

和 DMSERVER 发送 halt 命令，确保节点 A 上的 DMASMSVR 和 DMSERVER 自动退出。

单站点 ASM 也支持启动 css，但单站点 css 仅支持状态监控功能，在检测到单站点 ASM/DB 节点故障后，自动拉起故障节点。单站点 css 和其连接的 cssm 也仅支持执行 show 命令和自动拉起相关命令。

6 DMDSC 的启动与退出

6.1 DMDSC 的启动

DMDSC 是基于共享存储的数据库集群系统，包含多个数据库实例，因此，与单节点的达梦数据库不同，DMDSC 集群需要在节点间进行同步、协调，才能正常地启动、关闭。启动 DMDSC 集群之前，必须先启动集群同步服务 DMCSS，如果使用了 DMASM 文件系统，则 DMASMSVR 服务也必须先启动。

如果 DMCSS 配置了 DMASMSVR/DMSERVER 自动拉起命令，可以先仅启动 DMCSS，然后启动 DMCSSM，在 DMCSSM 控制台执行命令"ep startup grp_asm"启动 DMASMSVR 集群，执行"ep startup grp_dsc"启动 DMSERVER 集群（其中 grp_asm / grp_dsc 为 DMASMSVR/DMSERVER 集群的组名）。

Linux 环境下，DMCSS/DMASMSVR/DMSERVER 可以配置成操作系统服务，每次开机自动启动，或者通过 Linux 命令"service XXX start/stop/restart"(XXX 为配置的服务名)完成服务的启动、关闭。服务脚本在达梦安装包里提供，可能还需要用户根据实际情况修改部分参数才能使用。关于服务脚本的详细介绍请参考《DM8_Linux 服务脚本使用手册》。

6.2 DMDSC 的退出

完整退出 DMDSC 集群的正确步骤为：依次退出 DMSERVER、DMASMSVR、DMCSS，未按照该顺序进行将导致节点异常退出。

6.2.1 使用 DMCSSM 监视器提供的命令退出集群

DMCSSM 提供了两种退出节点的命令：

```
ep stop group_name //退出指定组（所有节点）
```

```
ep halt group_name.ep_name //强制退出指定组的指定节点
```

使用 DMCSSM 监视器提供的命令退出集群时，可以先使用 show 命令查看 ASM/DB 组每个节点的自动拉起标记。

```
[CSS0] auto check = TRUE, global info:
[ASM0] auto restart = FALSE
[DSC02] auto restart = FALSE
[CSS1] auto check = TRUE, global info:
[ASM1] auto restart = FALSE
[DSC01] auto restart = FALSE
```

如果当前标记 auto restart 为 TRUE，则故障节点会尝试重新拉起，若此时直接进行退出会导致节点退出后再次被拉起，可以使用 set group_name auto restart off（其中 group_name 为指定的组名）关闭指定组的自动拉起功能。

使用 ep stop 命令退出 ASM 组时，需要保证 DB 组已经退出，否则会报错处理。该命令执行前，如果 CSS 对指定组的自动拉起功能是打开的，则会先通知 CSS 关闭对指定组的自动拉起功能，再通知指定组退出，避免命令执行成功后节点再次被自动拉起，可以省略 set group_name auto restart off 的步骤。执行 ep stop 命令，如果所有节点原本都是正常状态，那么所有节点同步正常退出；如果原先有节点处于故障退出状态，那么其余正常节点全部正常退出。

使用 ep halt 命令直接退出 ASM 组的节点时，DB 组的对应节点将一并异常退出。该命令不会关闭指定节点的自动拉起功能，需要先执行 set group_name auto restart off，再进行退出。

CSS 类型的组不允许执行 ep stop 和 ep halt 命令。

对于上述使用 ep stop 命令和 ep halt 命令进行退出的节点，用户可以分别通过以下方式查看 log 和节点的当前状态，判断所有节点是否同步正常退出，还是单节点故障退出。

可以在使用 ep stop 命令正常退出的节点对应的 log 中看到如下信息：

```
nsvr_shutdown->utl_stub_sys_close 100.

[for dem]SYSTEM SHUTDOWN SUCCESS.

DM Database Server shutdown successfully.

nsvr_notify_exit wakeup main thread to exit
```

在 DMCSSM 控制台使用 show [group_name]命令查看正常退出的节点状态，如下：

```
inst_status          vtd_status          is_ok          active

SHUTDOWN          SHUTDOWN          OK          FALSE
```

可以在使用 ep halt 命令强制退出的节点对应的 log 中可以看到如下信息：

```
[for dem]SYSTEM SHUTDOWN ABORT.

[for dem]SYSTEM SHUTDOWN ABORT.

VTD_CMD_SYS_HALT

VTD_CMD_SYS_HALT
```

在 DMCSSM 控制台使用 show [group_name]命令查看强制退出的节点状态，如下：

```
inst_status                vtd_status          is_ok          active

OPEN                       SYSHALT          ERROR          FALSE
```

6.2.2 使用控制台命令退出

DMASMSVR 和 DMSERVER 控制台执行 exit 命令，会通知所有其他节点一起退出；DMCSS 需要手动退出所有节点，没有同步功能。

DMDSC 环境 DIsql 登录实例执行 stop instance，实例会正常退出，不影响其他活动节点正常运行。

6.2.3 通过 DEM 资源监控进行启停

使用 DEM 搭建的 DSC 集群可以通过进程或服务方式来进行启动或停止。下面以搭建 DSC 集群时注册服务的情况为例：

1） 关闭节点的自动拉起

退出 DSC 集群前需要先关闭节点的自动拉起功能。可以在 DEM 的监控模块中选择数据库监控，选中需要修改的节点，点击编辑，在弹出窗口中修改数据库状态，关闭节点的自动拉起。

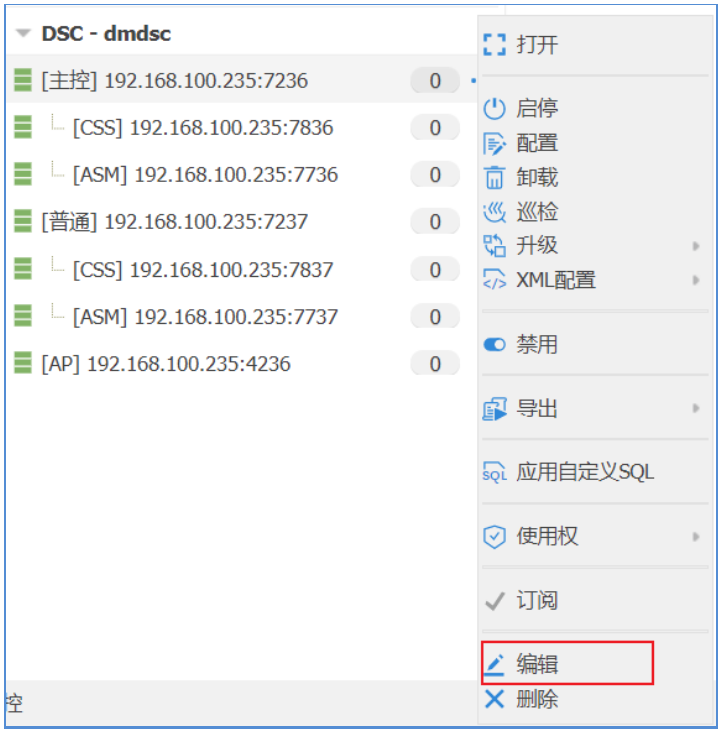


图 6.1 DEM 监控模块下数据库监控选中节点并编辑

在修改数据库弹窗中取消勾选自动拉起选项，界面截图如下：



图 6.2 DEM 修改数据库弹窗

或者在启动集群节点时取消勾选自动拉起。步骤为：在监控模块选择数据库监控，选中需要调整的节点，选择“启停”，进入数据库启动与停止界面。

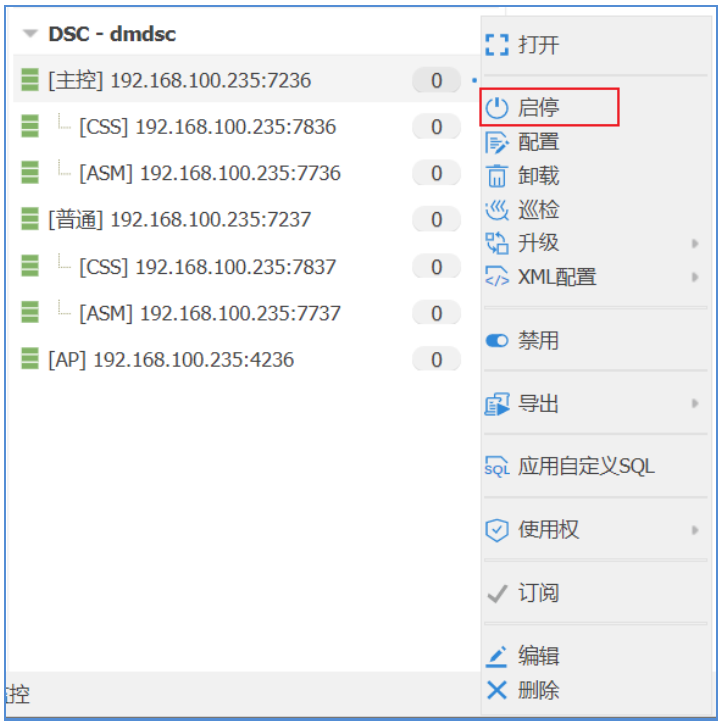


图 6.3 DEM 监控模块下数据库监控中选择启停

在已选择数据库列表中，取消勾选自动拉起选项，点击确认即可关闭节点的自动拉起。

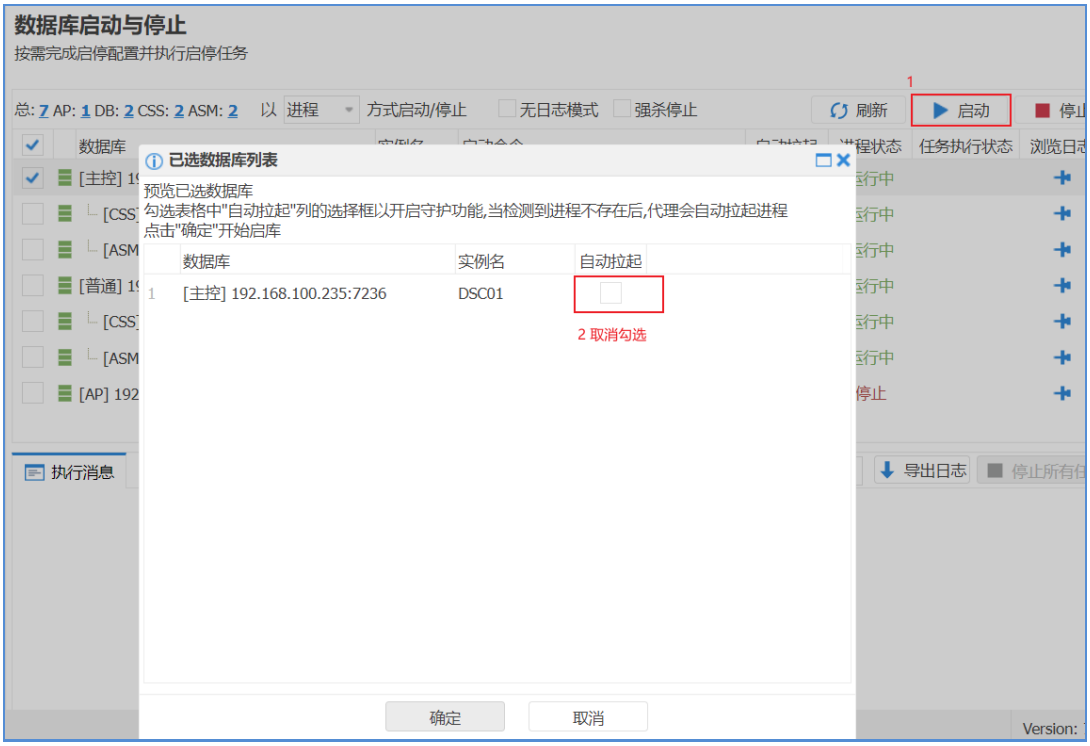


图 6.4 取消勾选“自动拉起”

2） 以进程方式停止

在监控模块选择数据库监控，选中需要调整的集群或集群中的节点，选择“启停”，进入数据库启动与停止界面。

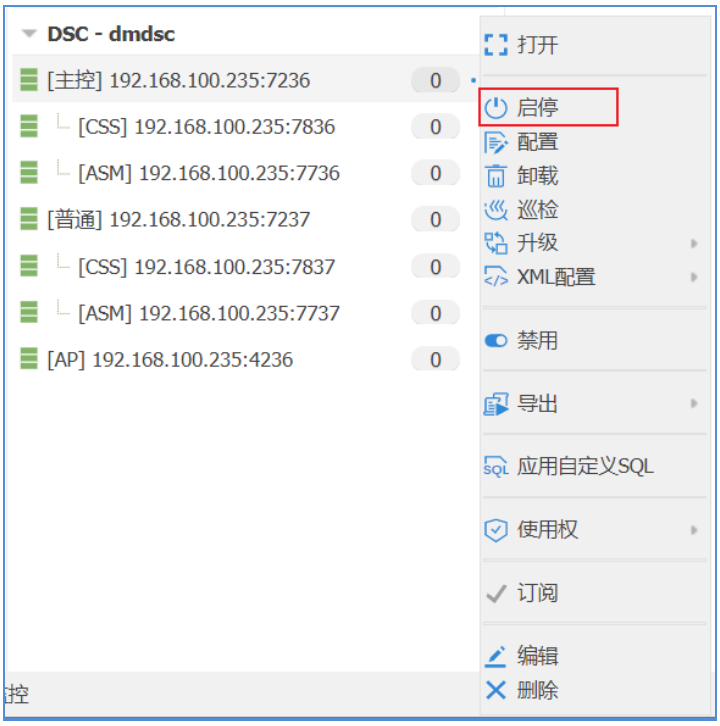


图 6.5 DEM 监控模块下数据库监控中选择启停

勾选需要退出的节点，选择以“进程”方式启动/停止后，点击停止按钮。

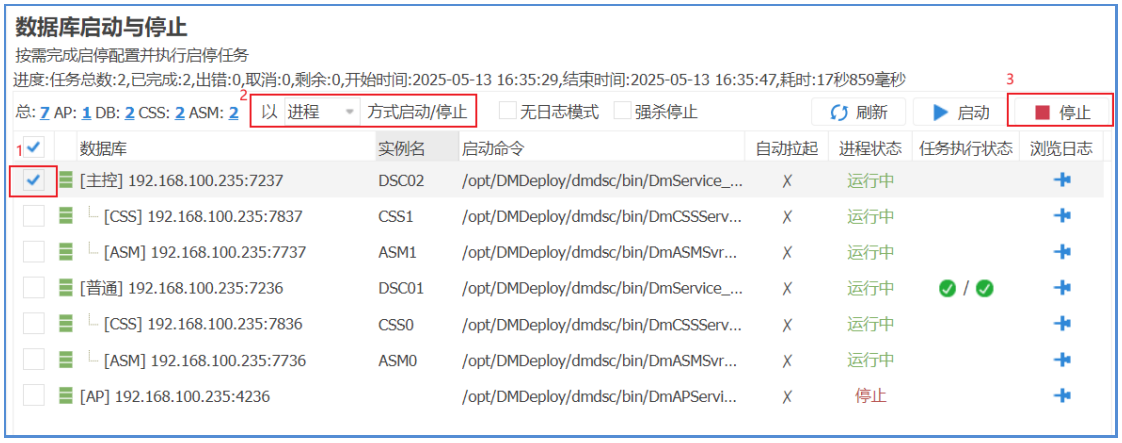


图 6.6 以进程方式退出节点

在已选择数据库列表弹框中预览已选数据库，然后点击“确定”开始退出节点。



图 6.7 以进程方式退出节点执行完成

3） 以服务方式停止

在监控模块选择数据库监控，选中需要调整的集群或集群中的节点，选择“启停”，进入数据库启动与停止界面，截图可参考 2）以进程方式停止。

勾选需要退出的节点，选择以“服务”方式启动/停止后，点击停止按钮。



图 6.8 以服务方式退出节点

在已选择数据库列表弹框中预览已选数据库，然后点击“确定”开始退出节点。



图 6.9 以服务方式退出节点执行完成

需要注意的是，强杀停止可能导致数据库文件损坏；以进程/服务方式启动的节点，只能以相应的方式退出；正常退出集群应依次退出[DB]、[ASM]、[CSS]各组节点，直接退出[ASM]节点将导致对应的[DB]节点异常退出，直接退出[CSS]节点将导致对应的[ASM]、[DB]节点异常退出。

6.2.4 通过 systemctl 命令退出

使用 systemctl 命令启动节点，必须通过 systemctl 命令退出节点。

必须正常退出一个节点后再依次退出其余节点。

如果当前节点未完全退出就使用命令退出下一个节点，下一个节点将收到系统发出的 signal 15 异常退出，在 DMCSSM 控制台使用 show [group_name]命令查看节点状态。

inst_name	inst_status	vtd_status	is_ok	active
DSC02	SHUTDOWN	OPEN	SYSHALT	FALSE
DSC01	SHUTDOWN	STOP	ERROR	FALSE

1）启动命令

```
systemctl start <服务名>
```

启动两节点 DSC。

```
systemctl start DmService_03_DSC01.service
```

```
systemctl start DmService_03_DSC02
```

两节点都启动成功后，在 DMCSSM 控制台使用 show [group_name]命令查看节点状态。

inst_name	inst_status	vtd_status	is_ok	active
DSC02	OPEN	WORKING	OK	TRUE
DSC01	OPEN	WORKING	OK	TRUE

2）节点顺序退出命令

```
systemctl stop <服务名>
```

先退出 DSC01。

```
systemctl stop DmService_03_DSC01.service
```

DSC01 退出后，在 DMCSSM 控制台使用 show [group_name]命令查看节点状态。

inst_name	inst_status	vtd_status	is_ok	active
DSC02	OPEN	WORKING	OK	TRUE
DSC01	SHUTDOWN	STOP	ERROR	FALSE

再退出 DSC02。

```
systemctl stop DmService_03_DSC02
```

DSC02 退出后，在 DMCSSM 控制台使用 show [group_name]命令查看节点状态。

inst_name	inst_status	vtd_status	is_ok	active
DSC02	SHUTDOWN	SHUTDOWN	OK	FALSE
DSC01	SHUTDOWN	STOP	ERROR	FALSE

6.2.5 通过 DMSERVER 脚本命令退出

使用 DMSERVER 脚本命令启动节点，必须通过 DMSERVER 脚本命令退出节点。

正常退出一个节点后再依次退出其余节点。

如果当前节点未完全退出就使用命令退出下一个节点，下一个节点将退出失败，仍然可以正常连接使用。

```
[root@test109 bin]# ./DmService_03_DSC02 stop
Stopping DmService_03_DES02:                [FAILED]
Kill the process(pid:67980) had timeout!
```

在 DMCSSM 控制台使用 show [group_name]命令查看节点状态，如下：

inst_name	inst_status	vtd_status	is_ok	active
DSC02	OPEN	WORKING	OK	TRUE
DSC01	SHUTDOWN	STOP	ERROR	FALSE

1） 启动命令

```
<DMSERVER 脚本名> start
```

先在 107 上执行启动命令。

```
[root@test107 bin]# ./ DmService_03_DSC01 start
Starting DmService_03_DSC01: connect dmasmtool（dmasmtoolm）sucessfully
上一次登录：2024 年 3 月 26 日 14:05:35 星期二 CST
                                                    [OK]
```

再在 109 上执行启动命令。

```
[root@test109 bin]# ./ DmService_03_DSC02 start
Starting DmService_03_DSC02: connect dmasmtool（dmasmtoolm）sucessfully
上一次登录：2024 年 3 月 26 日 14:05:53 星期二 CST
                                                    [OK]
```

在 DMCSSM 控制台使用 show [group_name]命令查看节点状态。

inst_name	inst_status	vtd_status	is_ok	active
DSC02	OPEN	WORKING	OK	TRUE

```
DSC01          OPEN          WORKING          OK          TRUE
```

2） 节点顺序退出命令 stop

stop 命令退出单节点。

```
<DMSERVER 脚本名> stop
```

先退出 DSC01。

```
[root@test107 bin]# ./DmService_03_DSC01 stop

Stopping DmService_03_DES01:                [OK]
```

DSC01 退出后，在 DMCSSM 控制台使用 show [group_name]命令查看节点状态。

```
inst_name      inst_status      vtd_status      is_ok      active

DSC02          OPEN             WORKING      OK            TRUE

DSC01          SHUTDOWN         STOP            ERROR      FALSE
```

再退出 DSC02。

```
[root@test109 bin]# ./DmService_03_DSC02 stop

Stopping DmService_03_DES02:                [OK]
```

DSC02 退出后，在 DMCSSM 控制台使用 show [group_name]命令查看节点状态。

```
inst_name      inst_status      vtd_status      is_ok      active

DSC02          SHUTDOWN         SHUTDOWN        OK         FALSE

DSC01          SHUTDOWN         STOP            ERROR      FALSE
```

3） 集群退出命令 stop_all

stop_all 命令退出集群所有节点，任一 DMSERVER 脚本都可以使用该命令，且只需要在其中一个节点上执行即可，不需要每个节点都执行。

```
<DMSERVER 脚本名> stop_all
```

执行如下命令：

```
[root@test109 bin]# ./DmService_03_DSC02 stop_all
```

```
Stopping DmService_03_DES02:                [OK]
```

在 DMCSSM 控制台使用 show [group_name]命令查看节点状态，如下：

inst_name	inst_status	vtd_status	is_ok	active
DSC02	SHUTDOWN	SHUTDOWN	OK	FALSE
DSC01	SHUTDOWN	SHUTDOWN	OK	FALSE

7 DMDSC 故障处理

7.1 故障处理

DMDSC 集群出现数据库实例或者节点硬件故障时，DMSERVER 的 VOTE 磁盘（非镜像环境下）或 DCRV 磁盘（镜像环境下）心跳信息不再更新，DMCSS 一旦监控到 DMSERVER 发生故障，会马上启动故障处理（参考 5 DMCSS 介绍），各节点 DMSERVER 收到故障处理命令后，启动故障处理流程。

在 DMDSC 故障处理机制下，一旦产生节点故障，登录到故障节点的所有连接将会断开，所有未提交事务将被强制回滚；活动节点上的用户请求可以继续执行，但是一旦产生节点间信息传递（比如：向故障节点发起 GBS/LBS 请求、或者发起 Remote Read 请求），当前操作就会被挂起；在 DMDSC 故障处理完成后，这些被挂起的操作可以继续执行。也就是说，DMDSC 产生节点故障时，活动节点上的所有连接会保留，正在执行的事务可能被阻塞一段时间，但最终可以正常完成，不会被强制回滚，也不会影响结果的正确性。

DMDSC 环境 DMSERVER 故障处理主要包括以下工作：

1. 暂停所有会话线程、工作线程、日志刷盘线程、检查点线程等，避免故障处理过程中产生并发错误；

2. 收集所有活动节点的 Buffer，丢弃无效数据页，获取最新数据页和需要重做 REDO 日志的数据页信息，并在排除故障节点后重新构造 LBS/GBS 信息；

3. 重做 REDO 日志；

4. 收集所有节点事务信息，重新构造锁对象，并重构相应的 LLS/GLS/LTV/GTV 系统；

5. 控制节点执行故障节点活动事务回滚和故障节点已提交事务修改的 PURGE 操作。

DMDSC 故障处理分为三个阶段；第一阶段由所有活动节点共同参与，进行全局的信息收集、重构；第二阶段由控制节点执行，重演 REDO 日志；第三阶段由控制节点执行，将故

障节点的活动事务回滚、并 PURGE 故障节点已提交事务的修改。在第一阶段执行期间，数据库实例不提供数据库服务，所有用户请求将被挂起。根据故障处理策略（由 INI 参数 DSC_CRASH_RECV_POLICY 控制）的不同，在第二阶段操作之前，会选择是否唤醒所有活动节点，正常提供数据库服务，也就是说故障处理第二阶段的操作与正常的数据库操作在系统内部同时进行。默认故障处理策略时，第二阶段重做所有节点 REDO 日志，且在此期间数据库服务挂起，直到第三阶段结束重新唤醒所有活动节点。选择新的故障处理策略，第二阶段只重做故障节点 REDO 日志，且在第二阶段开始前唤醒所有活动节点。第三阶段执行完成之前，DMDSC 故障处理仍然没有真正结束，在此期间，不能处理节点重加入。

DMDSC 支持多节点连续故障处理，即在故障处理过程中剩余活动节点又出现新的节点故障时，DMDSC 可以继续完成故障处理，不会造成其余活动节点退出，连接和事务的处理方法不变。需要注意的是，节点重加入流程中出现节点故障和集群非 OPEN 状态出现节点故障并不是多节点连续故障，仍然会导致节点 HALT。DMCSS 监控到发生节点连续故障后，立即发出故障回滚命令中止当前的故障处理流程。各节点 DMSERVER 收到故障回滚命令后，回滚当前正在执行的操作，中止日志重做和本次故障处理，重置和清理部分内容以便于执行下次故障处理。所有节点全部回滚完成后，DMCSS 发起新的对当前所有故障节点的故障处理命令。DMSERVER 收到新的故障处理命令后，重新开始故障处理流程，并根据情况跳过部分步骤。


说明：

1.通常意义上讲的故障处理，包含故障检测和故障处理两部分。

2.故障检测时间由 DCR_GRP_DSKCHK_CNT 决定。

3.影响故障处理时间的因素比较多，主要包括：故障节点 REDO 日志数量、修改涉及的数据页数量、需要 ROLLBACK、PURGE 的事务数，以及活动节点 Buffer 使用情况等。

7.2 故障日志

在 DMDSC 集群中出现实例故障时，DMCSS/DMASM/DMSERVER 实例都会打印大量日志对故障原因以及故障处理过程进行说明，这些日志目前已添加“[!!!DSC INFO!!!]”关键标识。由于日志量过大，本节内容将对故障处理过程中的特定日志以及故障排查流程进行相关说明，便于当 DMDSC 集群出现实例故障时，服务人员能够快速进行故障原因排查。

集群的绝大多数故障原因都会打印在 DMCSS/DMCSSM 监视器上，因此在进行故障检查时一般先检查 DMCSS/DMCSSM 日志，通过一些特定的关键日志找出故障节点数目、站点号以及故障节点类型。可以先通过[!!!DSC INFO!!!]关键字进行日志初步筛选，再根据关键日志信息，进行相关方向排查。

下面介绍一些常用的故障日志以及相应的故障排查方向。

7.2.1 实例故障

日志格式:

```
[!!!DSC INFO!!!][CSS]: dead instance detected, detect ep %s[%d] broken.
例：[!!!DSC INFO!!!][CSS]: dead instance detected, detect ep CSS0[0] broken.
```

日志说明:

DMCSS/DMCSSM 出现此条日志信息，说明对应实例已故障，一般在 DMCSSM 中也有相关的故障原因打印，根据故障日志的信息可进行相应方向故障排查。常见的故障原因有掉电、网络故障、存储异常等。若 DMCSSM 中没有直接的故障原因打印，可直接通过故障实例的日志查找实例故障原因，根据[!!!DSC INFO!!!]关键字或节点 halt 特定日志"[fordem]SYSTEM SHUTDOWN ABORT"进行日志筛查，快速定位故障原因。

7.2.2 实例组故障

日志格式:

[!!!DSC INFO!!!]All processers on EP[%d] have detected disk heartbeat issues.

例：[!!!DSC INFO!!!]All processers on EP[1] have detected disk heartbeat issues.

日志说明：

DMCSS/DMCSSM 出现此条日志信息，说明整个实例组，即属于这个实例组的 DMCSS/DMASM/DMSERVER 都已故障，一般在 DMCSSM 中也有相关的故障原因打印，根据故障日志的信息可进行相应方向故障排查。常见的故障原因有掉电、网络故障、存储异常等，可对整个实例组进行整体排查。该故障打印信息与上述的单个实例故障打印信息一般不同时出现。

7.2.3 共享存储异常

日志格式：

[%s][!!!DSC INFO!!!]Unable to access VOTING DISK! disk may be not mounted, suggest to check shared storage.

例：[CSS0][!!!DSC INFO!!!]Unable to access VOTING DISK! disk may be not mounted, suggest to check shared storage.

日志说明：

DMCSS/DMCSSM 出现此条日志信息，说明可能由于磁盘未挂载等原因导致共享存储异常，VTD 磁盘信息无法正常写入而导致实例故障，可以对共享储存进行故障原因排查。

7.2.4 网络异常

日志格式：

[!!!DSC INFO!!!]Network connect to IP address %s has been disconnected.

或 [!!!DSC INFO!!!]Network interface with IP address %s no longer running.

例：[!!!DSC INFO!!!]Network connect to IP address 0.0.0.0 has been disconnected.

或［!!!DSC INFO!!!]Network interface with IP address 0.0.0.0 no longer running.

日志说明:

实例（DMCSS/DMASM/DMSERVER）出现此条日志信息，说明集群中出现了网络故障。该日志常与 DMCSS/DMCSSM 监视器上的打印日志“设置命令[LINK_CHECK]，目标站点...”以及“The [%d]th subnet of network connectivity of [ep_name]:[%d] ERROR”日志共同出现。当实例故障后，若查看 DMCSS/DMCSSM 监视器出现此类日志，需要对故障节点的网络进行相关排查。

7.2.5 机房整体故障

日志格式:

[!!!DSC INFO!!!]CSSM detected all EP inactive, the entire cluster may be faulty or has already exited or network failure occurred.

日志说明:

DMCSSM 出现此条日志信息，说明有可能是机房整体故障，或者机房整体实例安全退出，可对整体集群状态进行排查。

8 DMDSC 节点重加入

故障节点恢复正常后，DMCSS 会启动节点重加入处理流程，将故障节点重新加入到 DMDSC 集群中。默认情况下，DMCSS 自动监控并处理节点重加入，不需要用户干预；我们也可以通过 DMCSSM 关闭自动监控功能，改成手动处理节点重加入（参考 15.1 DMCSSM 监视器）。

DMDSC 集群节点重加入操作会将 DMDSC 集群挂起一段时间，重加入过程中会中断正在执行的操作，暂停响应用户的数据库请求。但是，重加入操作不会终止这些活动事务，在重加入操作完成后，可以继续执行。

DMDSC 节点重加入的基本步骤包括：

1.根据各节点的 Buffer 使用情况，重构所有节点的 GBS、LBS 系统。

2.根据各节点的活动事务信息，重构全局的 GTV 系统、以及所有节点的本地 LTV 系统。

3.根据各节点的活动事务的封锁（LOCK）信息，重构全局的 GLS 系统、以及所有节点的本地 LLS 系统。

在节点重加入期间，不能处理新的节点故障，如果有新的节点故障会主动中止所有节点。


注意：

存在预提交状态事务的节点发生故障后，控制节点会将此故障节点事务信息收集到本地，重加入前必须在控制节点将这些事务回滚或提交。

DMDSC节点重加入时，集群内所有活动节点均不能处于MOUNT状态，否则节点重加入失败。

9 DMDSC 配置文件

与 DMDSC 相关的配置文件包括:

- DMDCR_CFG.INI
- DMDCR.INI
- DMINIT.INI
- MAL 系统配置文件（DMMAL.INI、DMASVRMAL.INI）
- DM.INI
- DMARCH.INI

下面分别介绍配置文件中各配置项的含义。

9.1 DMDCR_CFG.INI

DMDCR_CFG.INI 是格式化非镜像环境下 DCR DISK、VOTE DISK 和镜像环境下 DCRV 磁盘的配置文件。

配置信息包括三类：集群环境全局信息、集群组信息、以及组内节点信息。

集群环境全局信息为所有集群的公共信息，整个文件中只需要出现一次。例如:设置 DMCSS 集群、DMASM 集群和 DMDSC 集群三个集群的公共信息。

集群组信息用来设置一个集群中所有节点的公共信息，一个集群只需配置一个集群组信息。例如:设置 DMCSS 集群中两个节点 CSS0 和 CSS1 的公共信息。

组内节点信息，集群组中各节点的信息。例如：分别设置节点 CSS0 和 CSS1 的各自的节点信息。

使用 DMASMCMD 工具，可以根据 DMDCR_CFG.INI 配置文件，格式化 DCR DISK 和 VOTE DISK。

表 9.1 DMDCR_CFG.INI 配置项

集群环境全局信息	
DCR_VTD_PATH	VOTE DISK 路径
DCR_N_GRP	集群环境包括多少个组（即包括多少个集群），取值范围 1~16。例如：配置了 1 个 DMCSS 集群、1 个 DMASM 集群、1 个 DMDSC 集群，则 DCR_N_GRP=3。目前仅支持 3 个组
DCR_OGUID	消息标识，DMCSSM 登录 DMCSS 消息校验用。取值范围：正整数
集群组的信息	
DCR_GRP_TYPE	组类型：CSS、ASM 或 DB。分别表示 DMCSS 集群、DMASM 集群和 DMDSC 集群
DCR_GRP_NAME	组名，16 字节，配置文件内不可重复
DCR_GRP_N_EP	组内节点个数 N，最大 8
DCR_GRP_EP_ARR	组内包含的节点序列号，{0,1,2,...N-1} 用户不能指定，仅用于从 DCR 磁盘 export 出来可见
DCR_GRP_N_ERR_EP	组内故障节点个数 用户不能指定，仅用于从 DCR 磁盘 export 出来可见
DCR_GRP_ERR_EP_ARR	组内故障节点序列号 用户不能指定，仅用于从 DCR 磁盘 export 出来可见
DCR_GRP_DSKCHK_CNT	磁盘心跳机制，容错时间，单位秒 S，缺省为 60S，取值范围 5~600
节点信息，某些属性可能只针对某一类型节点，比如 SHM_KEY 只针对 ASM 节点有效	
DCR_EP_NAME	节点名，16 字节，配置文件内不可重复。 DB 的节点名必须和 DM.INI 里的 INSTANCE_NAME 保持一致，仅允许大小写不一致。

	ASM 的节点名必须和 DMASVRMAL.INI 里的 MAL_INST_NAME 一致，仅允许大小写不一致。 同一类型各节点的 EP_NAME 不能重复
DCR_EP_SEQNO	组内序号，CSS/ASM 不能配置，自动分配， DB 可以配置，0 ~ n_ep -1，组内不能重复，如不配置则自动分配
DCR_EP_HOST	节点 IP(实例所在机器的 IP 地址)。 CSS 中设置表示 DMCSSM 可以通过该 IP 连接 CSS，可以不配置。 dmasmsvr 允许通过 IP/PORT 远程访问，ASM 必须配置
DCR_EP_PORT	节点 TCP 监听端口(CSS/ASM/DB 有效，对应登录 CSS/ASM/DB 的端口号)，节点实例配置此参数，取值范围 1024~65534；发起连接端的端口在 1024~65535 之间随机分配。 特别对 DB 来说，DB 的 DCR_EP_PORT 与 DM.INI 中的 PORT_NUM 不一致时，DB 端口以 DM.INI 中的 PORT_NUM 为准
DCR_EP_SHM_KEY	共享内存标识，数值类型(ASM 有效，初始化共享内存的标识符)，应为大于 0 的 4 字节整数。同一机器上每个 ASM 节点对应的 DCR_EP_SHM_KEY 必须唯一，需要人为保证其唯一性
DCR_EP_SHM_SIZE	共享内存大小，单位 MB，取值范围 10~40000。共享内存大小与其能管理的磁盘大小的关系详见 10.5 DMASM 技术指标
DCR_EP_ASM_LOAD_PATH	ASM 磁盘扫描路径，Linux 下一般为/dev/raw，文件模拟情况，必须是全路径，不能是相对路径。 支持多路径配置，各路径以英文逗号','进行分隔，所有配置的路径都必须为有效路径，且暂不支持中文路径，例如： /dev/disk1,/dev/disk2,/dev/disk3。最多可以同时配置 8 个磁盘扫描

	路径，但配置的所有路径长度之和必须小于 256，若超过该长度，只能减小磁盘扫描路径个数

使用说明：

1. 在用 dmasmcmd 工具执行 init votedisk disk_path from dcr_cfg_path 时，指定的 disk_path 必须和 dcr_cfg_path 里面配置的 DCR_VTD_PATH 相同。

2. 如果配置 dmcssm，dmcssm 的 OGUID 必须和 DCR_OGUID 保持一致。

3. DCR_N_GRP 必须和实际配置的组数目保持一致。

4. CSS 和 ASM 组的 DCR_GRP_N_EP 要相等，DB 的 DCR_GRP_N_EP 要小于等于 CSS/ASM 的 DCR_GRP_N_EP。

5. ASM 节点的 DCR_EP_NAME 必须和 DMASM 系统使用的 DMASVRMAL.INI 配置文件里的 MAL_INST_NAME 保持一致，仅允许大小写不一致。

6. DB 节点的 DCR_EP_NAME 必须和数据库实例使用 DMMAL.INI 配置文件里的 MAL_INST_NAME、以及 DM.INI 配置文件里的 INSTANCE_NAME 保持一致，仅允许大小写不一致。

7. 所有 DB 节点的 DCR_EP_NAME 都不能重复，DB 组内的 DCR_EP_SEQNO 不能重复。

8. DMDCR_CFG.INI 配置文件中的所有路径均不支持中文路径。

举例说明：

```
//集群环境全局信息
DCR_N_GRP               = 3
DCR_VTD_PATH            = /dev/raw/raw2
DCR_OGUID           = 63635


//DMCSS 集群组的信息
```

```
[GRP]                    //[GRP]表示新建一个 Group
DCR_GRP_TYPE          = CSS
DCR_GRP_NAME          = CSS
DCR_GRP_N_EP          = 2
DCR_GRP_DSKCHK_CNT    = 60
//CSS0 节点的信息
[CSS]                    //[]里的是组名，与 DCR_GRP_NAME 对应
DCR_EP_NAME           = CSS0
DCR_EP_HOST           = 10.0.2.101
DCR_EP_PORT           = 9341
//CSS1 节点的信息
[CSS]                    //[]里的是组名，与 DCR_GRP_NAME 对应
DCR_EP_NAME           = CSS1
DCR_EP_HOST           = 10.0.2.102
DCR_EP_PORT           = 9343


//DMASM 集群组的信息
[GRP]                    //[GRP]表示新建一个 Group
DCR_GRP_TYPE          = ASM
DCR_GRP_NAME          = ASM
DCR_GRP_N_EP          = 2
DCR_GRP_DSKCHK_CNT    = 60
//ASM0 节点的信息
[ASM]                    //[]里的是组名，与 DCR_GRP_NAME 对应
```

```
DCR_EP_NAME            = ASM0

DCR_EP_SHM_KEY         = 93360

DCR_EP_SHM_SIZE        = 20

DCR_EP_HOST            = 10.0.2.101

DCR_EP_PORT            = 9349

DCR_EP_ASM_LOAD_PATH = /dev/raw

//ASM1 节点的信息

[ASM]                  //[]里的是组名，与 DCR_GRP_NAME 对应

DCR_EP_NAME            = ASM1

DCR_EP_SHM_KEY         = 93361

DCR_EP_SHM_SIZE        = 20

DCR_EP_HOST            = 10.0.2.201

DCR_EP_PORT            = 9351

DCR_EP_ASM_LOAD_PATH = /dev/raw



//DMDSC 集群组的信息

[GRP]                  //[GRP]表示新建一个 Group

DCR_GRP_TYPE           = DB

DCR_GRP_NAME           = DSC

DCR_GRP_N_EP           = 2

DCR_GRP_DSKCHK_CNT     = 60

//DSC01 节点的信息

[DSC]                  //[]里的是组名，与 DCR_GRP_NAME 对应

DCR_EP_NAME            = DSC01
```

```
DCR_EP_SEQNO       = 0
DCR_EP_PORT              = 5236
//DSC02 节点的信息
[DSC]                    //[]里的是组名，与 DCR_GRP_NAME 对应
DCR_EP_NAME        = DSC02
DCR_EP_SEQNO       = 1
DCR_EP_PORT              = 5237
```

9.2 DMDCR.INI

DMDCR.INI 是 DMCSS、DMASMSVR、DMASMTOOL 等工具的输入参数。记录了当前节点序列号以及 DCR 磁盘路径。

表 9.2 DMDCR.INI 配置项

DMDCR_PATH	记录 DCR 磁盘路径
DMDCR_SEQNO	记录当前节点序号（用来获取 ASM 登录信息）
DMDCR_MAL_PATH	保存 DMMAL.INI 配置文件的路径，仅对 DMASMSVR 有效
DMDCR_ASM_RESTART_INTERVAL	DMCSS 认定 DMASM 节点故障重启的时间间隔（取值范围 0~86400s），单位 S，DMCSS 只负责和 DMDCR_SEQNO 节点号相等的 DMASM 节点的故障重启，超过设置的时间后，如果 DMASM 节点的 active 标记仍然为 FALSE，则 DMCSS 会执行自动拉起。 如果配置为 0，则不会执行自动拉起操作，缺省为 60s
DMDCR_ASM_STARTUP_CMD	DMCSS 认定 DMASM 节点故障后，执行自动拉起的命令串，命令串长度不应超过 256，可以配置为服务方式或命令行方式启动，具体可参考表下方的 DMDCR.INI 使用说明

DMDCR_ASM_TRACE_LEVEL	指定日志级别。1：TRACE 级别；2：INFO 级别；3：WARN 级别；4：ERROR 级别；5：FATAL 级别。缺省为 2。日志级别越低（参数值越小），输出的日志越详细。当设置日志级别为较低级别时，兼容输出级别大的日志，如 WARN 级别也可输出 ERROR 级别的日志
DMDCR_DB_RESTART_INTERVAL	DMCSS 认定 DMDSC 节点故障重启的时间间隔(取值范围 0~86400s)，单位 S，DMCSS 只负责和 DMDCR_SEQNO 节点号相等的 DMDSC 节点的故障重启，超过设置的时间后，如果 DMDSC 节点的 active 标记仍然为 FALSE，则 DMCSS 会执行自动拉起。 如果配置为 0，则不会执行自动拉起操作，缺省为 60s
DMDCR_DB_STARTUP_CMD	DMCSS 认定 DMDSC 节点故障后，执行自动拉起的命令串，命令串长度不应超过 256，可以配置为服务方式或命令行方式启动，具体可参考表下方的 DMDCR.INI 使用说明
DMDCR_AUTO_OPEN_CHECK	指定时间内如果节点实例未启动，DMCSS 会自动将节点踢出集群环境，取值范围 30s~4294967294s，单位 S。不配置此参数时表示不启用此功能，即缺省为不启用
DMDCR_MESSAGE_CHECK	是否对 DMCSS 通信消息启用通信体校验（只有当消息的发送端和接收端都配置为 1 才启用通信体校验）。0：不启用；1：启用。缺省为 1
DMDCR_IPC_CONTROL	进程间互斥实现方式，0 表示用全局信号量控制；1 表示用全局互斥量控制；2 表示用全局互斥量控制，并且有分片功能。缺省为 2

DMDCR_LINK_CHECK_IP	第三方确认机器的 IP。可选参数，Linux 专用。确认机器为 DMDSC 集群各节点均可访问到的、DMDSC 集群环境之外的、一台独立机器，DMDCR_LINK_CHECK_IP 与内网通信网络位于同一网段。用于在 DMDSC 节点故障或意外断网时，各 DMSERVER 节点通过联通确认机器 IP 来检测网络是否畅通。DMCSS 若由于断网而导致与确认机器 IP 联通失败，则 DMDSC 集群会强制 HALT DMCSS 节点。配置了 DMDCR_LINK_CHECK_IP 之后，需要为 DMSERVER、DMASMSVR（非 DMASM 镜像）和 DMASMSVRM（DMASM 镜像）、DMCSS 赋予 ping 权限来完成联通功能。 使用 ROOT 权限执行以下语句分别进行赋权： DMSERVER 权限： sudo setcap cap_net_raw,cap_net_admin=eip /opt/dmdbms/bin/dmserver DMASMSVR 权限： sudo setcap cap_net_raw,cap_net_admin=eip /opt/dmdbms/bin/dmasmsvr DMASMSVRM 权限： sudo setcap cap_net_raw,cap_net_admin=eip /opt/dmdbms/bin/dmasmsvrm DMCSS 权限： sudo setcap cap_net_raw,cap_net_admin=eip /opt/dmdbms/bin/dmcss 其中，/opt/dmdbms/bin 为 dmserver、dmasmsvr、

	dmasmsvrm、dmcss 所在目录。 如果 DMDCR_LINK_CHECK_IP 配置成功，在 DMSERVER、DMASMSVR、DMASMSVRM 或 DMCSS 的事件日志中，会记录一条“comm_self_link_check success”日志，否则记录一条“comm_self_link_check create socket failed, ……”日志
DMDCR_NEED_PRE_LOAD	非镜像环境下，指定 DMASMSVR 启动时是否预加载文件 EXTENT；镜像环境下，指定 DMASMSVRM 启动时是否预加载文件 AU。0：否；1：是。预加载操作可以提升 DMSERVER 或 DMASMSVRM 运行时的性能，但是数据量非常大时，预加载操作比较耗时。缺省为 0
DMDCR_AU_PRE_LOAD_NUM	非镜像环境下，指定 DMASMSVR 预加载的文件 EXTENT 个数；镜像环境下，指定 DMASMSVRM 预加载的文件 AU 个数。取值范围 0~1024。若不指定该参数，则由 DMASMAPI 或 DMASMAPIM 根据读写文件大小来计算预加载的 EXTENT 或 AU 个数。该参数可以用于性能优化，一般情况下无需指定该参数
DMDCR_CSS_PORT	配置本节点 CSS 进程监听的端口号。默认为不配置，使用 DCR 磁盘配置的端口号；如果配置，则以当前配置为准，忽略 DCR 磁盘配置
DMDCR_ASM_PORT	配置本节点 ASM 进程监听的端口号。默认为不配置，使用 DCR 磁盘配置的端口号；如果配置，则以当前配置为准，忽略 DCR 磁盘配置

DMDCR_CHECK_CSS_ACTIVE	指定是否监测本节点 CSS 进程活动状态。0：否；1：是。缺省为 0。取值为 1 时，DMASMSVR 和 DMSERVER 定期监测 CSS 进程活动状态，若 CSS 故障，则 DMASMSVR 和 DMSERVER 主动 HALT
DMDCR_CRASH_CHECK_OPT_COUNT	远程节点 DMCSS/DMASMSVR/DMSERVER 被判为故障的标准。 0：心跳信息在 DCR_GRP_DSKCHK_CNT 时间内没有变化则判定为节点故障，缺省为 0； N：表示 DMCSS/DMASMSVR/DMSERVER 的磁盘心跳确认次数都超过 N 次，并且网络联通超时 N 秒没有联通，才判定为节点故障。取值范围 1~30
DMDCR_ASM_MAX_FILE_OPEN	DMASMSVR 进程内缓存 ASM 文件个数上限，达到上限会触发缓存淘汰。取值范围 0~4294967294，缺省为 4096

使用说明:

1. DMASMSVR 和 DMSERVER 使用不同的 MAL 系统，需要配置两套 MAL 系统，配置文件 DMMAL.INI 需要分别生成，保存到不同的目录下，并且 DMMAL.INI 中的配置项不能重复、冲突。

2. DMDSC 集群环境下，只有当所有 OK 节点实例都启动时，整个集群环境才能启动。可能存在某些场景，部分 OK 节点无法正常启动，导致整个集群环境无法正常启动。指定参数 DMDCR_AUTO_OPEN_CHECK 后，如果超过指定时间节点实例还未启动，DMCSS 自动将未启动节点踢出集群环境，变为 ERROR 节点，之后其他活动 OK 节点可以正常启动。

DMDSC 中至少一个 OK 节点启动后 DMCSS 才开始检查，所有 OK 节点都未启动情况下，DMCSS 不会主动踢出节点。

3. DMCSS 自动拉起故障 DMASMSVR 或 DMSERVER 实例。

故障认定间隔和启动命令串是配合使用的，DMASMSVR 和 DMSERVER 实例需要各自

配置，如果没有配置启动命令串，故障间隔即使配置为大于 0 的值 DMCSS 也不会执行自动拉起操作，DMDCR_ASM_RESTART_INTERVAL 和 DMDCR_DB_RESTART_INTERVAL 默认的 60S 只有在配置有启动命令串时才会起作用。

DMDCR_ASM_STARTUP_CMD 和 DMDCR_DB_STARTUP_CMD 的配置方法相同，只是执行码名称和 ini 配置文件路径有区别，可以配置为服务名或命令行启动方式。

- DMASMSVR 实例启动命令举例如下：

1） linux 命令行方式启动（不能出现带有空格的路径）：

```
DMDCR_ASM_STARTUP_CMD = /opt/dmdbms/bin/dmasmsvr
dcr_ini=/home/data/dmdcr.ini
```

2）linux 服务方式启动：

```
DMDCR_ASM_STARTUP_CMD = service dmasmserverd restart
```

3）Windows 命令行启动：

```
DMDCR_ASM_STARTUP_CMD = c:\dm\bin\dmasmsvr.exe  dcr_ini=d:\asmtest\dmdcr.ini
```

4）Windows 服务方式启动:

```
DMDCR_ASM_STARTUP_CMD = net start 注册服务名
```

- DMSERVER 实例启动命令如下：

1） linux 命令行方式启动（不能出现带有空格的路径）：

```
DMDCR_DB_STARTUP_CMD = /opt/dmdbms/bin/dmserver
path=/home/data/dsc0_config/dm.ini dcr_ini=/home/data/dmdcr.ini
```

2）linux 服务方式启动：

```
DMDCR_DB_STARTUP_CMD = service DmServiceDSC01 restart
```

3）Windows 命令行启动：

```
DMDCR_DB_STARTUP_CMD = c:\dm\bin\dmserver.exe
path=d:\asmtest\dsc0_config\dm.ini dcr_ini=d:\asmtest\dmdcr.ini
```

4）Windows 服务方式启动:

```
DMDCR_DB_STARTUP_CMD = net start 注册服务名
```

4. DMDCR_LINK_CHECK_IP 使用场景

当 2 节点 DSC 集群节点间出现 MAL 网络异常时，如果两个节点实例都正常运行，DMCSS 的处理逻辑是主动 HALT 节点号大的 DSC1 节点，保留节点号小的 DSC0 节点。如果真实情况是 DSC0 和应用的网络断了，而 DSC1 和应用的网络是好的，那么 DMCSS 这种保留节点号小的逻辑就不合适。因此，须配置 DMDCR_LINK_CHECK_IP 参数来解决上述问题。

DMDCR_LINK_CHECK_IP 成功配置之后，当集群中出现节点间 MAL 网络异常时，节点还需要尝试联通一下 DMDCR_LINK_CHECK_IP 配置的 IP（类似于 ping 功能），DMCSS 将联通结果共同作为处理网络异常的参考依据。例如：上述场景下，DSC0 和第三方确认 IP 无法联通，DSC1 和第三方确认 IP 可以联通，那么 DMCSS 综合判断之后主动 HALT 掉 DSC0 节点，保留了正常的 DSC1 节点。

如果没有配置 DMDCR_LINK_CHECK_IP，或者配置错误，或者没有赋予进程权限，集群所有节点尝试联通 DMDCR_LINK_CHECK_IP 时都会失败，那么 DMCSS 依然是保留节点号小的逻辑。

举例说明:

```
DMDCR_PATH=/dev/raw/raw1

DMDCR_SEQNO=0

DMDCR_MAL_PATH=/home/data/dmasvrmal.ini



//故障认定间隔，以及启动命令中的执行码路径和 INI 路径需要根据实际情况调整

DMDCR_ASM_RESTART_INTERVAL = 60

DMDCR_ASM_STARTUP_CMD = /opt/dmdbms/bin/dmasmsvr
```

```
dcr_ini=/home/data/dmdcr.ini



DMDCR_DB_RESTART_INTERVAL = 60

DMDCR_DB_STARTUP_CMD = /opt/dmdbms/bin/dmserver

path=/home/data/dsc0_config/dm.ini dcr_ini=/home/data/dmdcr.ini
```

9.3 DMINIT.INI

DMINIT.INI 是 DMINIT 工具初始化数据库环境的配置文件。与初始化库使用普通文件系统不同，使用 DMASM 文件系统，必须使用 DMINIT 工具的 control 参数指定 DMINIT.INI 文件。DMINIT 工具的命令行参数都可以放在 DMINIT.INI 中，比如 db_name、auto_overwrite 等，DMINIT.INI 参数分为全局参数和节点参数。DMINIT 工具具体用法可以参考《DM8_dminit 使用手册》，下面列出常用的一些参数。

表 9.3 DMINIT.INI 配置项

全局参数，对所有节点有效	
SYSTEM_PATH	初始化数据库存放的路径
DB_NAME	初始化数据库名称
MAIN	MAIN 表空间路径
MAIN_SIZE	MAIN 表空间大小
SYSTEM	SYSTEM 表空间路径
SYSTEM_SIZE	SYSTEM 表空间大小
ROLL	ROLL 表空间路径
ROLL_SIZE	ROLL 表空间大小
CTL_PATH	DM.CTL 控制文件路径

CTL_SIZE	DM.CTL 控制文件大小
LOG_SIZE	日志文件大小
HUGE_PATH	MAIN 表空间的 HUGE 数据文件路径
AUTO_OVERWRITE	文件存在时的处理方式
DCR_PATH	DCR 磁盘路径
DCR_SEQNO	DCR 序号
ASM_PORT	DMASM 进程监听的端口号
节点参数，对具体节点有效	
[XXX]	具体节点都是以[XXX]开始，节点实例名就是 XXX
CONFIG_PATH	配置文件存放路径
PORT_NUM	节点服务器监听通讯端口号，服务器配置此参数，取值范围 1024~65534，发起连接端的端口在 1024~65535 之间随机分配
MAL_HOST	节点 MAL 系统使用 IP。 对于 IPv6 地址，若当前环境存在多块网卡，需要用%号指定具体有效的网卡序号或网卡名称；若只有一块网卡或者已配置默认网卡，则可以不指定序号或名称。例如：MAL_HOST = fe80::610e:9715:5ec6:4ea8%ens01，其中 ens01 为当前环境使用的网卡名称
MAL_PORT	MAL 监听端口，用于数据守护、DSC、MPP 等环境中各节点实例之间 MAL 链路配置，监听端端口配置此参数，取值范围 1024~65534，发起连接端的端口在 1024~65535 之间随机分配

LOG_PATH	日志文件路径

使用说明:

1. SYSTEM_PATH 支持 ASM 文件系统，如果不指定 MAIN/SYSTEM/ROLL/CTL_PATH/HUGE_PATH，则数据文件、控制文件和 MAIN 表空间的 HUGE 数据文件目录都会默认生成在 SYSTEM_PATH/db_name 下面。

2. 在 DSC 集群环境里，除了 TEMP 表空间，所有其他表空间都只支持 ASM 文件路径，如果指定必须指定 SIZE。

3. 只有 DMINIT 工具使用 DMASM 文件系统，才会用到 DCR_PATH 和 DCR_SEQNO，并且不会写入其他配置文件。

4. CONFIG_PATH 暂时只支持本地磁盘，不支持 ASM 文件系统。DSC 环境配置，各节点的 CONFIG_PATH 要指定不同路径。

5. MAL_HOST 和 MAL_PORT 是为了自动创建 DMMAL.INI 文件使用，只有在初始化 DSC 环境时需要指定。

6. LOG_PATH 可以不指定，默认会在 SYSTEM_PATH 生成。如果指定，必须指定两个以上。

7. 必须指定实例名，也就是必须配置[XXX]。

举例说明:

```
system_path= +DMDATA/data

db_name= dsc

main= +DMDATA/data/dsc/main.dbf

main_size= 128

roll = +DMDATA/data/dsc/roll.dbf

roll_size= 128

system = +DMDATA/data/dsc/system.dbf
```

```
system_size= 128

ctl_path = +DMDATA/data/dsc/dm.ctl

ctl_size = 8

log_size = 256

dcr_path = /dev/raw/raw1

dcr_seqno = 0

auto_overwrite=1

SYSDBA_PWD=DMdba_123

SYSAUDITOR_PWD=DMauditor_123



[dsc01]

config_path = /home/data/config1

port_num = 5236

mal_host = 10.0.2.101

mal_port = 9340

log_path = +DMDATA/log/log101.log

log_path = +DMDATA/log/log102.log

[dsc02]

config_path = /home/data/config2

port_num = 5237

mal_host = 10.0.2.102

mal_port = 9341

log_path = +DMDATA/log/log201.log

log_path = +DMDATA/log/log202.log
```

9.4 MAL 系统配置文件

MAL 配置文件包括 DMMAL.INI 和 DMASVRMAL.INI。使用同一套 MAL 系统的所有实例，MAL 系统配置文件须严格保持一致。

DMDSC 使用 MAL 配置文件的前提条件：

一 DMASMSVR 和 DMDSC 集群中的 DMSERVER 实例需要分别配置一套独立的 MAL 系统，两者配置的 MAL 环境不能冲突。

二 DMASMSVR 组成的集群环境使用一套 MAL 系统进行通讯，需要在 DMDCR.INI 配置文件中配置 DMDCR_MAL_PATH 参数，指定 MAL 配置文件路径。例如：DMDCR_MAL_PATH =/home/data/dmasvrmal.ini。

三 使用 MAL 系统的 DMSERVER 实例，需要将 DM.INI 配置项 MAL_INI 设置为 1。同时 MAL 系统配置文件名称必须为 DMMAL.INI。

9.4.1 DMMAL.INI

表 9.4 DMMAL.INI 配置项

配置项	配置含义
MAL_CHECK_INTERVAL	MAL 链路检测时间间隔，取值范围 0~1800，单位秒（s），缺省为 30s，配置为 0 表示不进行 MAL 链路检测
MAL_CONN_FAIL_INTERVAL	判定实例之间 MAL 链路断开的时间，取值范围 2~1800，单位秒（s），缺省为 10s。 仅当 MAL_CHECK_INTERVAL 不为 0 时有效
MAL_CHECK_TIMEOUT	单个实例的 MAL 网络最大延迟时间，单位毫秒（ms），取值范围 0~65535，缺省为 0，表示不进行 MAL 网络延迟时间检测。需要和 MAL_CHECK_IP 配合使用。

	仅当 MAL_CHECK_INTERVAL 不为 0 时有效
MAL_CHECK_IP	第三方确认机器的 IP，用于检测各个实例的 MAL 网络延迟时间。需要和 MAL_CHECK_TIMEOUT 配合使用。确认机器为 MAL 链路所有实例均可访问到的独立外网机器，MAL 链路实例包括 DMSERVER、DMASMSVR（非 DMASM 镜像）和 DMASMSVRM（DMASM 镜像）。实例每隔 MAL_CHECK_INTERVAL 时间 ping 一次 MAL_CHECK_IP，若连接耗时超过 MAL_CHECK_TIMEOUT 指定时间，则该实例主动 HALT。 开启 MAL 网络延迟时间检测后，需要为 MAL 链路所有实例赋予 ping 权限，具体方法可以参考 DMDCR_LINK_CHECK_IP 参数介绍
MAL_SEND_THRESHOLD	最大待发送邮件数，用于控制待发送邮件总数，防止发送邮件堆积。取值范围 1024~10000，缺省为 2048
MAL_LEAK_CHECK	是否打开 MAL 内存泄露检查。0：关闭；1：打开。缺省为 0
MAL_LOGIN_TIMEOUT	MPP/DBLINK 等实例间登录时的超时检测间隔，取值范围 3~1800，单位 S，缺省为 15s
MAL_BUF_SIZE	单个 MAL 缓存大小限制，以兆为单位。当此 MAL 的缓存邮件超过此大小，则会将邮件存储到文件中。取值范围 0~500000，缺省为 100
MAL_SYS_BUF_SIZE	MAL 系统总内存大小限制，单位 MB。取值范围 0~500000，缺省为 0，表示 MAL 系统无总内存限制
MAL_VPOOL_SIZE	MAL 系统使用的内存初始化大小，单位 MB。取值范围 1~500000，缺省为 128，此值一般要设置的比 MAL_BUF_SIZE 大一些
MAL_COMPRESS_LEVEL	MAL 消息压缩等级，取值范围 0~10。缺省为 0，不进行压缩；1~9 表示采用 zip 算法，从 1 到 9 表示压缩速度依次递减，压缩率依次递增；

	10 表示采用 snappy 算法，压缩速度高于 zip 算法，压缩率相对低
MAL_TEMP_PATH	指定临时文件的目录。当邮件使用的内存超过 mal_buf_size 或者 mal_sys_buf_size 时，将新产生的邮件保存到临时文件中。如果缺省，则新产生的邮件保存到 temp.dbf 文件中
MAL_COMBIN_BUF_SIZE	指定 MAL 消息合并发送的缓冲区大小，单位 KB。MAL 发送消息时进行合并，减少网络发送次数。取值范围 0~131072。0 表示不进行消息合并，为缺省值。最小缓冲区大小 128kB，当设置值大于 0 小于 128 时，自动调整为 128
MAL_MESSAGE_CHECK	是否对 MAL 通信消息启用消息体校验（只有当消息的发送端和接收端都配置为 1 才启用消息体校验）。0：不启用；1：启用。缺省为 1
MAL_MSG_ENCRYPT	MAL 系统是否启用 SSL 通讯加密。0：不启用；1：启用。缺省为 0
MAL_USE_RDMA	Linux 环境下，MAL 是否使用 RDMA 通讯方式。 0：否，使用 TCP 方式；1：是，使用 RDMA 方式。 当 MAL_USE_RDMA=1 时，MAL_HOST 必须是 RDMA 网卡所绑定的 IP。使用 RDMA 之前需先安装好迈络思 ofed 驱动（RDMA 协议的基础库 libibverbs.so 和 librdmacm.so）
[MAL_NAME]	MAL 名称，同一个配置文件中 MAL 名称需保持唯一性
MAL_INST_NAME	数据库实例名，与 DM.INI 的 INSTANCE_NAME 配置项保持一致，MAL 系统中数据库实例名要保持唯一
MAL_HOST	MAL IP 地址，使用 MAL_HOST + MAL_PORT 创建 MAL 链路。 对于 IPv6 地址，若当前环境存在多块网卡，需要用%号指定具体有效的网卡序号或网卡名称；若只有一块网卡或者已配置默认网卡，则可以不指定序号或名称。例如：MAL_HOST =

	fe80::610e:9715:5ec6:4ea8%ens01，其中 ens01 为当前环境使用的网卡名称
MAL_PORT	MAL 监听端口，用于数据守护、DSC、MPP 等环境中各节点实例之间 MAL 链路配置，监听端端口配置此参数，取值范围 1024~65534，发起连接端的端口在 1024~65535 之间随机分配
MAL_INST_HOST	MAL_INST_NAME 实例对外服务 IP 地址
MAL_INST_PORT	MAL_INST_NAME 实例服务器监听通讯端口号，服务器配置此参数，取值范围 1024~65534，发起连接端的端口在 1024~65535 之间随机分配 此参数的配置应与 DM.INI 中的 PORT_NUM 保持一致
MAL_DW_PORT	MAL_INST_NAME 实例守护进程的监听端口，其他守护进程或监视器使用 MAL_HOST + MAL_DW_PORT 创建与该实例守护进程的 TCP 连接，监听端配置此参数，取值范围 1024~65534，发起连接端的端口在 1024~65535 之间随机分配。缺省为 0，表示不配置端口号，无法监听
MAL_INST_DW_PORT	节点实例监听守护进程的端口，实例服务器配置此参数，取值范围 1024~65534，发起连接端的端口在 1024~65535 之间随机分配。缺省为 0，表示不配置端口号，无法监听
MAL_RDMA_MAGIC	MAL 链路通信方式标识，取值范围 0~65534，缺省为 0，属于为每个实例配置的局部变量，为了支持 RDMA/TCP 混合通信。在 MAL_USE_RDMA 开启的情况下，配置相同 MAL_RDMA_MAGIC 的节点间使用 RDMA 通信，不同则使用 TCP 通信。若不配置 MAL_RDMA_MAGIC，缺省为 0，可以和 MAL_RDMA_MAGIC 为

	0 的节点进行 RDMA 通信。

9.4.2 DMASVRMAL.INI

表 9.5 DMASVRMAL.INI 配置项

配置项	配置含义
MAL_CHECK_INTERVAL	MAL 链路检测时间间隔，取值范围 0~1800，单位秒（s），缺省为 30s， 配置为 0 表示不进行 MAL 链路检测
MAL_CONN_FAIL_INTERVAL	判定实例之间 MAL 链路断开的时间，取值范围 2~1800，单位秒（s）， 缺省为 10s。 仅当 MAL_CHECK_INTERVAL 不为 0 时有效
MAL_CHECK_TIMEOUT	单个实例的 MAL 网络最大延迟时间，单位毫秒（ms），取值范围 0~65535，缺省为 0，表示不进行 MAL 网络延迟时间检测。需要和 MAL_CHECK_IP 配合使用。 仅当 MAL_CHECK_INTERVAL 不为 0 时有效
MAL_CHECK_IP	第三方确认机器的 IP，用于检测各个实例的 MAL 网络延迟时间。需 要和 MAL_CHECK_TIMEOUT 配合使用。确认机器为 MAL 链路所有 实例均可访问到的独立外网机器，MAL 链路实例包括 DMSERVER、 DMASMSVR（非 DMASM 镜像）和 DMASMSVRM（DMASM 镜像）。 实例每隔 MAL_CHECK_INTERVAL 时间 ping 一次 MAL_CHECK_IP，若连接耗时超过 MAL_CHECK_TIMEOUT 指定时 间，则该实例主动 HALT。 开启 MAL 网络延迟时间检测后，需要为 MAL 链路所有实例赋予 ping 权限，具体方法可以参考 DMDCR_LINK_CHECK_IP 参数介绍
MAL_LOGIN_TIMEOUT	MPP/DBLINK 等实例间登录时的超时检测间隔，取值范围 3~1800，

	单位 S，缺省为 15s
MAL_BUF_SIZE	单个 MAL 缓存大小限制，以兆为单位。当此 MAL 的缓存邮件超过此大小，则会将邮件存储到文件中。取值范围 0~500000，缺省为 100
MAL_SYS_BUF_SIZE	MAL 系统总内存大小限制，单位 MB。取值范围 0~500000，缺省为 0，表示 MAL 系统无总内存限制
MAL_VPOOL_SIZE	MAL 系统使用的内存初始化大小，单位 MB。取值范围 1~500000，缺省为 128，此值一般要设置的比 MAL_BUF_SIZE 大一些
MAL_COMPRESS_LEVEL	MAL 消息压缩等级，取值范围 0~10。缺省为 0，不进行压缩；1~9 表示采用 zip 算法，从 1 到 9 表示压缩速度依次递减，压缩率依次递增；10 表示采用 snappy 算法，压缩速度高于 zip 算法，压缩率相对低
MAL_TEMP_PATH	指定临时文件的目录。当邮件使用的内存超过 mal_buf_size 或者 mal_sys_buf_size 时，将新产生的邮件保存到临时文件中。如果缺省，则新产生的邮件保存到 temp.dbf 文件中
MAL_MESSAGE_CHECK	是否对 MAL 通信消息启用消息体校验（只有当消息的发送端和接收端都配置为 1 才启用消息体校验）。0：不启用；1：启用。缺省为 1
[MAL_NAME]	MAL 名称，同一个配置文件中 MAL 名称需保持唯一性
MAL_INST_NAME	数据库实例名，与 DM.INI 的 INSTANCE_NAME 配置项保持一致，MAL 系统中数据库实例名要保持唯一
MAL_HOST	MAL IP 地址，使用 MAL_HOST + MAL_PORT 创建 MAL 链路。 对于 IPv6 地址，若当前环境存在多块网卡，需要用%号指定具体有效的网卡序号或网卡名称；若只有一块网卡或者已配置默认网卡，则可以不指定序号或名称。例如：MAL_HOST = fe80::610e:9715:5ec6:4ea8%ens01，其中 ens01 为当前环境使用

	的网卡名称
MAL_PORT	MAL 监听端口，用于数据守护、DSC、MPP 等环境中各节点实例之间 MAL 链路配置，监听端端口配置此参数，取值范围 1024~65534，发起连接端的端口在 1024~65535 之间随机分配
MAL_INST_HOST	MAL_INST_NAME 实例对外服务 IP 地址
MAL_INST_PORT	MAL_INST_NAME 实例服务器监听通讯端口号，服务器配置此参数，取值范围 1024~65534，发起连接端的端口在 1024~65535 之间随机分配 此参数的配置应与 DMDCR_CFG.INI 中的 PORT_NUM 保持一致
MAL_LINK_MAGIC	用以区分不同网段。取值范围 0-65534。默认为 0。

9.5 DM.INI

DM.INI 是 DMSERVER 使用的配置文件，详细介绍可以参考《DM8 系统管理员手册》相关章节。

各 DMDSC 节点之间的部分 INI 参数必须保持一致，如果不一致，会导致后启动的节点启动失败，日志会记录失败原因。每个节点使用 SQL 语句或者系统函数修改本节点的 INI 时，DMDSC 会将新修改值同步到其它节点，始终保持 INI 参数值的一致性。

对于节点间必须保持一致的 INI 参数，用户使用 SQL 语句或者系统函数修改参数值时，建议指定修改 INI 文件中的参数值，若仅修改内存中的参数值，则当部分节点故障重启时，可能出现节点间参数不一致的情况。

须保持一致的 INI 参数是：BUFFER、BUFFER_POOLS、BUFFER_MODE、FAST_POOL_PAGES、FAST_ROLL_PAGES、KEEP、RECYCLE、RECYCLE_POOLS、ROLLSEG、ROLLSEG_POOLS、MULTI_PAGE_GET_NUM、ENABLE_INJECT_HINT、ORDER_BY_NULLS_FLAG、OPTIMIZER_MODE、DIRECT_IO、MAX_SESSIOS、

TRX_VIEW_MODE、FAST_LOGIN、TS_RESERVED_EXTENTS、TS_SAFE_FREE_EXTENTS、TS_MAX_ID、TS_FIL_MAX_ID、RLOG_APPEND_LOGIC、ISOLATION_LEVEL、LOCK_DICT_OPT、BACKSLASH_ESCAPE、STR_LIKE_IGNORE_MATCH_END_SPACE、MS_PARSE_PERMIT、COMPATIBLE_MODE、JSON_MODE、CASE_COMPATIBLE_MODE、COUNT_64BIT、CALC_AS_DECIMAL、CMP_AS_DECIMAL、CAST_VARCHAR_MODE、PL_SQLCODE_COMPATIBLE、LEGACY_SEQUENCE、DM6_TODATE_FMT、DROP_CASCADE_VIEW、ENABLE_FREQROOTS、ENABLE_OFFLINE_TS、DSC_N_CTLS、DSC_TRX_VIEW_SYNC、DSC_REMOTE_READ_MODE、CHECK_SVR_VERSION、COMMIT_WRITE、PARAM_DOUBLE_TO_DEC、RLOG_SYNC_MODE、DSC_NTP_MAX_DIFFTIME、DSC_NTP_POLICY、DSC_REQUEST_TIMEOUT、PAGE_FLUSH_CHECK、DSC_SLOT_WAIT_TIMEOUT、BINDDATA_COLTYPE、USE_JSON_DATATYPE、ENABLE_RLS。

另外，下面几个参数需要按要求进行配置：

- INSTANCE_NAME：DMSERVER 如果需要使用 DMASM 文件系统，INSTANCE_NAME 必须和 DMDCR_CFG.INI 里面配置的 DCR_EP_NAME 相同；
- MAL_INI：必须为 1，可缺省。在 DMDSC 系统中，为了确保通信正常，MAL_INI 缺省或为 0 的情况下，系统会将 MAL_INI 强制设置为 1。
- DSC_USE_SBT：是否使用辅助的平衡二叉树。取值 0 或 1。1：是，0：否。缺省为 1。DM 的检查点机制要求 BUFFER 更新链表保持有序性，所有被修改过的数据页，要根据其第一次修改的 LSN 值（FIRST_MODIFIED_LSN）从小到大有序排列。而 DMDSC 的缓存交换机制要求数据页在节点间传递，当一个更新页 P1 由普通节点 EP0 传递到节点 EP1 时，为了不破坏节点 EP1 更新链表 FIRST_MODIFIED_LSN 的有序性，需要扫描更新链，将 P1 加入到更新链中的合适位置。极端情况下，可能需要扫描整个更新链，才能找到 P1 的插入位置，在 BUFFER 比较大，更新链表比较长的情况下，这种扫描、定位代价十分昂贵，特别是

在高并发情况下，会引发严重的并发冲突，影响并发性能。开启 DSC_USE_SBT 参数后，系统内部维护一个平衡二叉树，在将数据页加入更新链表时，根据这个平衡二叉树进行 log_2^N 次比较，就可以找到插入位置；

- DSC_N_POOLS：LBS/GBS 池数目。取值范围 2~1024，缺省为 19。与 BUFFER_POOLS 类似，DSC_N_POOLS 将系统中的 LBS/GBS 根据页号进行分组，以降低 LBS/GBS 处理的并发冲突；
- CONFIG_PATH：指定 DMSERVER 所读取的配置文件（DMMAL.INI、DMARCH.INI、DMTIMER.INI 等）的路径。不允许指定 ASM 目录。缺省使用 SYSTEM_PATH 路径。如果 SYSTEM_PATH 保存在 ASM 上，则直接报错；
- TRACE_PATH：存放系统 trace 文件的路径。不允许指定 ASM 目录。默认的 TRACE_PATH 是 SYSTEM_PATH；如果 SYSTEM_PATH 保存在 ASM 上，则../config_path/trace 作为 TRACE_PATH。

9.6 DMARCH.INI

DMARCH.INI 是开启本地归档或远程归档时使用的配置文件。不开启归档，可不用配置 DMARCH.INI。更多 DMARCH.INI 用法请参考《DM8 系统管理员手册》。

9.7 DMCSSM.INI

DMCSSM.INI 是 DMCSSM 监视器的配置文件。具体请参考 15.1.2 配置文件 DMCSSM.INI。

10DMASM 介绍

10.1 DMASM 概述

DM 自动存储管理器（DM Auto Storage Manager，简称 DMASM）是一个专用的分布式文件系统。

DMDSC 如果直接使用块设备作为共享存储来存放数据库文件，会因为块设备本身的诸多功能限制，造成 DMDSC 集群在使用、维护上并不是那么灵活方便。为了克服块设备的这些使用限制，DM 专门设计了一款分布式文件系统 DMASM，来管理块设备的磁盘和文件。DMASM 的出现为 DMDSC 灵活管理和使用块设备提供了完美的解决方案。

使用 DMASM 自动存储管理方案，可以帮助用户更加便捷地管理 DMDSC 集群的数据库文件。DMASM 的主要部件包括：提供存储服务的块设备、DMASMSVR 服务器、DMASMAPI 接口、初始化工具 DMASMCMD 和管理工具 DMASMTOOL 等。

下图为一个部署了 DMASM 的 DMDSC 集群结构图。

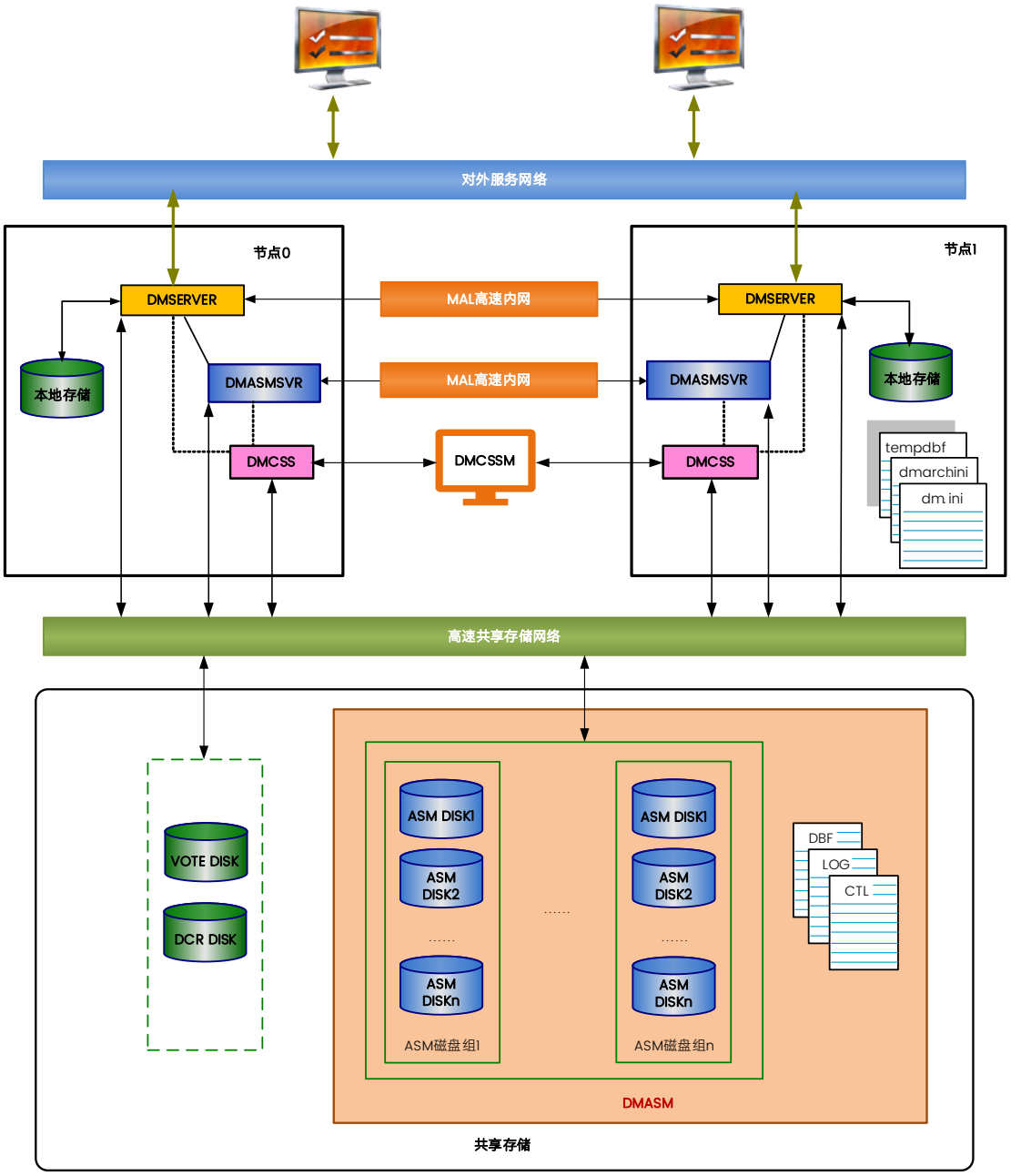


图 10.1 一个部署了 DMASM 的 DMDSC 集群结构图

10.2 使用 DMASM 的好处

下面通过对比的方式，展示在 DMDSC 中使用 DMASM 的好处。

如果直接将块设备作为 DMDSC 的共享存储，存在多项限制，增加了 DBA 的管理难度。具体限制在以下几点：

1. 不支持动态扩展文件大小；在创建数据文件时，就必须指定文件大小，并且文件无

法动态扩展。

2. 数据文件必须占用整个裸或块设备盘，造成空间浪费。

3. 不支持类 linux 的文件操作命令，使用不方便。

4. 操作系统支持最大块设备数目较小，无法创建足够的数据库文件。

为了克服上述限制，推荐用户使用 DMASM 来管理块设备的磁盘和文件。DMASM 提供了基本的数据文件访问接口，可以有效降低 DMDSC 共享存储的维护难度。DMASM 提供的主要功能包括:

1. 分布式管理

支持多台机器并发访问 ASM 磁盘和文件，提供全局并发控制。

2. 磁盘组管理

支持创建和删除磁盘组,将块设备格式化为DMASM格式,并由DMASMSVR统一管理;一个磁盘组可以包含一个或者多个 ASM 磁盘；磁盘组支持在线增加 ASM 磁盘，实现动态存储扩展。

3. 文件管理

支持创建、删除、截断文件等功能；支持创建目录；支持动态扩展文件；文件可以存放在一个磁盘组的多个磁盘中，文件大小不再受限于单个磁盘大小。

4. 完善、高效的访问接口

通过 DMASMAPI 可以获得各种文件管理功能。

5. 通用功能的管理工具

DMASMTOOL 提供一套类 Linux 的文件操作命令用于管理 ASM 文件，降低用户学习、使用 DMASM 文件系统的难度。

10.3 DMASM 术语和基本概念

10.3.1 DMASM 术语

本节介绍 DMASM 所用到的术语。术语的详细概念请参考 10.3.2 DMASM 基本概念。

表 10.1 中英文对照术语表

中文术语	英文术语
DM 自动存储管理器	DM Auto Storage Manager，简称 DMASM
ASM 磁盘	ASM DISK
磁盘组	DISK GROUP
ASM 文件	ASM FILE
簇	extent
数据分配单元	Allocation Units，简称 AU
DM 集群注册表	DM Clusterware Registry，简称 DCR
DCR 磁盘	DCR DISK
VOTE 磁盘	VOTE DISK，简称 VTD
DMASM 文件系统初始化工具	DMASMCMD（专门用于 DMASM 非镜像环境）
DMASM 服务器	DMASMSVR（专门用于 DMASM 非镜像环境）
DMASM 应用程序访问接口	DMASMAPI（专门用于 DMASM 非镜像环境）
DMASM 管理工具	DMASMTOOL（专门用于 DMASM 非镜像环境）

10.3.2 DMASM 基本概念

下面详细介绍各概念：

DM 自动存储管理器（DM Auto Storage Manager，简称 DMASM）

DMASM 是一个分布式文件系统，用来管理块设备的磁盘和文件。

ASM 磁盘

ASM 磁盘是指经过 DMASMCMD 工具格式化，可以被 DMASMSVR 识别的物理磁盘。ASM 磁盘是组成磁盘组的基本单位，一个块设备只能格式化为一个 ASM 磁盘，不支持分割使用。

磁盘组

磁盘组由一个或多个 ASM 磁盘组成，是存储 ASM 文件的载体；一块 ASM 磁盘只能属于一个磁盘组。DMASM 支持动态添加 ASM 磁盘。DMDSC 集群中，一般建议将日志文件和数据文件保存到不同的磁盘组中。

ASM 文件（ASMFILE）

在 ASM 磁盘组上创建的文件，称之为 ASM 文件。一个 ASM 文件只能保存在一个磁盘组中，但一个 ASM 文件的数据可以物理存放在同一磁盘组的多个 ASM 磁盘中。DMDSC 集群中，需要多个节点共享访问的数据库文件、日志文件、控制文件等，一般会创建为 ASM 文件。

DMASM 文件路径都以“+GROUP_NAME”开头，使用“/”作为路径分隔符，任何以“+”开头的文件，DM 都认为是 DMASM 文件，“GROUP_NAME”是磁盘组名称。比如，“+DATA/ctl/dm.ctl”表示 dm.ctl 文件，保存在 DMASM 文件系统的“DATA”磁盘组的 ctl 目录下。"+"号只能出现在全路径的第一位，出现在任意其他地方的路径都是非法的。

DMASM 只提供文件级别并发控制，访问 ASM 文件时系统会进行封锁操作。比如，正在访问的数据文件不允许被删除。但是 DMASM 并不提供数据文件的读写并发控制。DMASM 允许多个用户同时向同一个文件的相同偏移写入数据，一旦发生这种并发写，系统无法预知最终写入磁盘的数据是什么。因此，DMASM 必须由使用 DMASM 的应用程序来控制数据文件的读写并发，即避免多个程序同时写同一个数据文件的相同数据块。

不提供读写并发控制主要原因有两个:

原因一 达梦数据库管理软件已经提供了数据读写的并发控制机制，确保不会同时读、写相同的数据页，因此 DMASM 不需要再实现一套重复的并发控制策略。

原因二 DMASM 镜像不提供读写并发控制可提升 ASM 文件的读写效率。如果提供读、写操作的全局并发控制，虽然可避免并发写入导致数据不一致，但这种策略会影响读、写性能。

簇（Extent）

簇是 ASM 文件的最小分配单位，一个簇由物理上连续的一组 AU 构成。簇的大小为 4，也就是说一个 ASM 文件至少占用 4 个 AU，也就是 4M 的物理存储空间。

数据分配单元（Allocation Units，简称 AU）

DMASM 存储管理的最小单位。AU 的大小为 1M，为系统固定大小，无需用户指定。DMASM 以 AU 为单位将磁盘划分为若干逻辑单元，ASM 文件也是由一系列 AU 组成。根据 AU 的不同用途，系统内部定义了一系列 AU 类型，包括：desc AU、inode AU、REDO AU、和 data AU。

DCR 磁盘（DCR DISK）

DM 集群注册表（DM Clusterware Registry，简称 DCR）磁盘专门用于存储 DCR 文件。DCR 文件记录了存储、维护集群配置的详细信息。整个集群环境共享 DCR 磁盘信息，包括集群（DMDSC、DMASM、DMCSS）资源、实例名、监听端口、集群中故障节点信息等。DCR 必须存储在集群中所有节点都可以访问到的共享存储中，并且只支持保存在 DMASM 文件系统管辖范围之外的共享存储上。在一个集群环境中只能配置一个 DCR 磁盘。

VOTE 磁盘（VOTE DISK）

VOTE 磁盘专门用于存储 VTD 文件。VTD 文件记录了集群成员信息。DM 集群通过 VOTE DISK 进行心跳检测，确定集群中节点的状态，判断节点是否出现故障。当集群中出现网络故障时，使用 VOTE DISK 来确定哪些 DMDSC 节点应该被踢出集群。VOTE 磁盘还用来传递命令，在集群的不同状态（启动、节点故障、节点重加入等）DMCSS 通过 VOTE DISK 传递

控制命令，通知节点执行相应命令。VOTE DISK 必须存储在集群中所有节点都可以访问到的共享存储中，并且只支持保存在 DMASM 文件系统管辖范围之外的共享存储上。在一个集群环境中只能配置一个 VOTE 磁盘。

集群中各实例启动时，通过访问 DCR DISK 获取集群配置信息。被监控实例从 VOTE DISK 读取监控命令，并向 VOTE DISK 写入命令响应以及自身心跳信息；DMCSS 也向 VOTE DISK 写入自己的心跳信息，并从 VOTE DISK 访问各被监控节点的运行情况，并将监控命令写入 VOTE DISK，供被监控实例访问执行。

DMASM 文件系统初始化工具（DMASMCMD）

专门用于 DMASM 非镜像环境。将在 10.6 DMASM 主要部件中详细介绍。

DMASM 服务器（DMASMSVR）

专门用于 DMASM 非镜像环境。将在 10.6 DMASM 主要部件中详细介绍。

DMASM 应用程序访问接口（DMASMAPI）

专门用于 DMASM 非镜像环境。将在 10.6 DMASM 主要部件中详细介绍。

DMASM 管理工具（DMASMTOOL）

专门用于 DMASM 非镜像环境。将在 10.6 DMASM 主要部件中详细介绍。

10.3.3 DMASM 和达梦数据库文件系统对照表

DMASM 的实现主要参考了达梦数据库文件系统，因此，一些概念和实现原理与达梦数据库基本类似，熟悉达梦数据库的用户，就会更加容易理解 DMASM。

表 10.2 DMASM/达梦文件系统概念对照表

DMASM	达梦文件系统
磁盘组（DISK GROUP）	表空间（TABLESPACE）
ASM 磁盘（DISK）	数据文件（DATAFILE）
ASM 文件（FILE）	段（SEGMENT）

簇（EXTENT）	簇（EXTENT）
AU（ALLOCATION UNITS）	页（PAGE）
描述 AU（DESC AU）	描述页（DESC PAGE）
INODE AU（INODE AU）	INODE 页（INODE PAGE）

10.4 DMASM 关键技术

为了帮助用户更好的理解、使用 DMASM，本节从 ASM 磁盘与文件管理、DMASM REDO 日志、簇映射表等方面介绍 DMASM 原理。

10.4.1 ASM 磁盘与文件管理

DMASMCMD 将物理磁盘格式化后，变成可识别、可管理的 ASM 磁盘，再通过 ASM 磁盘组将一个或者多个 ASM 磁盘整合成一个整体提供文件服务。

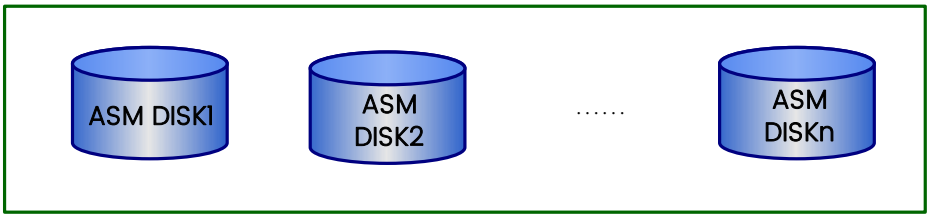


图 10.2 ASM 磁盘组结构

ASM 磁盘格式化以后，会逻辑划分为若干簇（Extent），簇是管理 ASM 磁盘的基本单位，ASM 文件的最小分配单位也是簇。

这些逻辑划分的簇根据其用途可以分为 DESC 描述簇、INODE 簇和 DATA 数据簇。

描述簇由多个描述项组成。每一个描述项存储一个 INODE 簇或一个数据簇的元数据。元数据包括簇所属文件 ID，簇的前一位簇，簇的后一位簇等。

INODE 簇由多个 INODE 项组成。每一个 INODE 项存放一个 ASM 文件的元数据。元数据包括文件完整路径、大小、创建时间等信息。

数据簇用于存储用户数据。

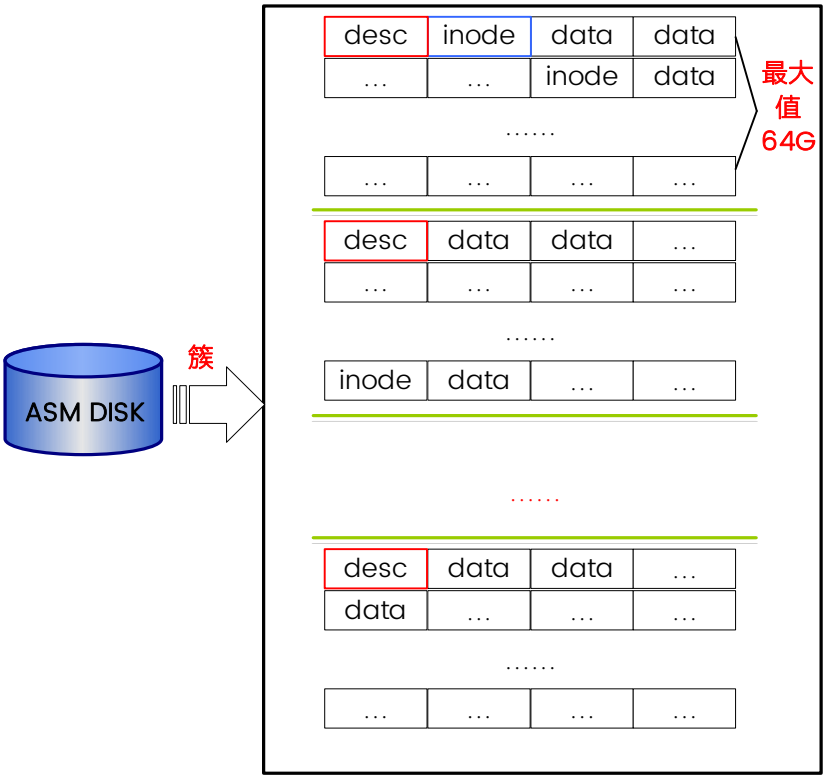


图 10.3 ASM 磁盘逻辑结构

创建、删除 ASM 文件操作，在 DMASM 系统内部会转换成修改、维护 INODE AU 的具体动作。而扫描全局的 INODE AU 链表就可以获取到磁盘组上所有的 ASM 文件信息。

10.4.2 DMASM REDO 日志

DMASM 采用重做日志机制保证在各种异常（比如系统掉电重启）情况下数据不被损坏。创建、删除 ASM 文件等 DDL 操作过程中，所有针对 DMASM 描述 AU、INODE AU 的修改，都会生成 REDO 日志，并且在描述 AU、INODE AU 的修改写入磁盘之前，必须确保 REDO 日志已经写入磁盘。DMASM 中，只针对描述 AU 和 INODE AU 的修改产生 REDO 日志，用户修改数据 AU 的动作并不会产生 REDO 日志。

DMASM 所有 DDL 操作（创建文件、删除文件、增加磁盘等）都是串行执行的，并且在操作完成之前，会确保所有修改的描述项、INODE 项写入磁盘；一旦 DDL 操作完成，所有 REDO 日志就可以被覆盖了。

DDL 操作过程中出现异常时，如果 REDO 日志尚未写入磁盘，则当前操作对系统没有任何影响；如果 REDO 日志已经写入磁盘，那么重新启动后，系统会重演 REDO 日志，修改描述 AU 和 INODE AU，将此 DDL 继续完成。

10.4.3 簇映射表

创建 ASM 文件后，用户操作 ASM 文件的一般流程是：调用 ASM 文件的 OPEN、READ、WRITE 接口，打开 ASM 文件并获取一个句柄，再使用这个句柄从文件的指定偏移读取数据、或者写入数据。用户在使用 DMASM 的过程中，只需要获取一个 ASM 文件句柄，并不需要知道数据最终保存在物理磁盘的什么位置。

DMASM 使用簇映射表机制维护 ASM 文件与物理磁盘地址的映射关系，访问 ASM 文件时，根据文件号、文件偏移等信息，通过簇映射表可以快速获取到物理磁盘地址。

由于 DMASM 并不缓存任何用户数据，与直接读、写块设备相比，ASM 文件的读、写操作仅仅增加了簇映射的代价，而这个代价与 IO 代价相比几乎可以忽略，因此，使用 DMASM 并不会引起读、写性能的降低。

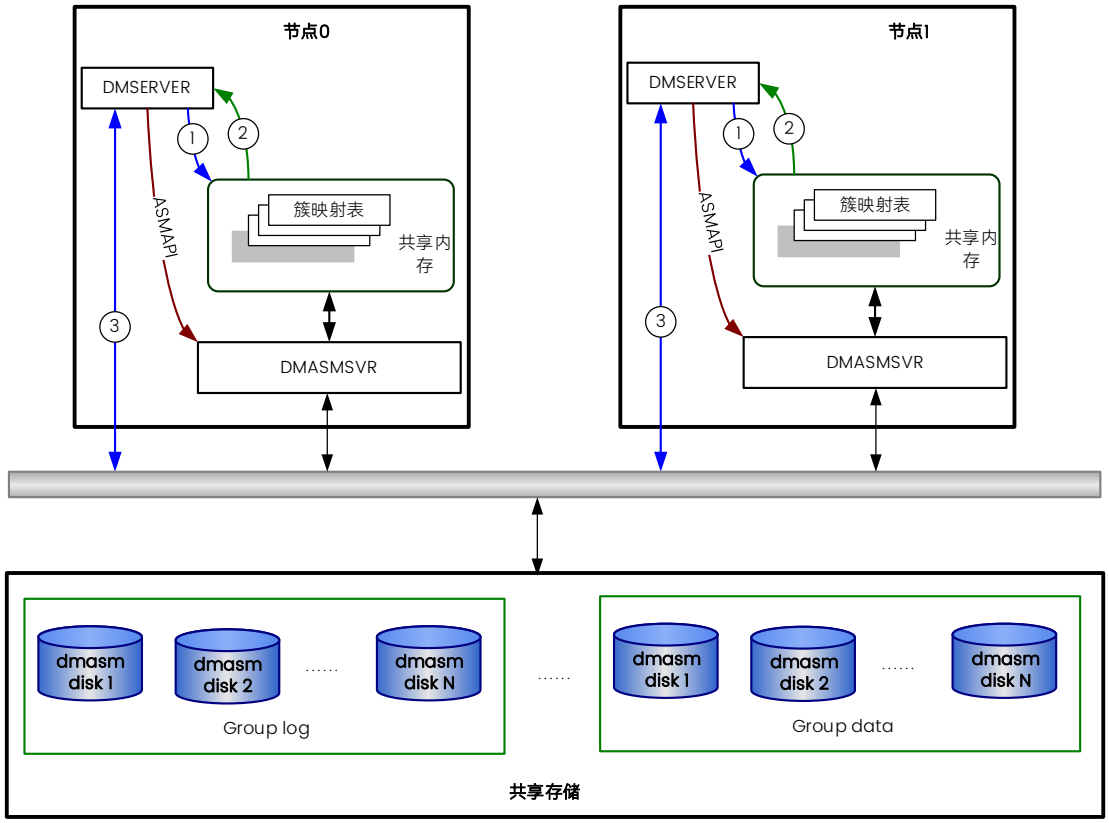


图 10.4 DMASM 数据访问

10.5 DMASM 技术指标

本节主要介绍一下 DMASM 中的一些重要技术指标。

表 10.3 DMASM 技术指标

项	大小	说明
AU 大小	1024 * 1024	一个 AU 占用 1MB 存储空间
簇大小	4	一个簇包含 4 个物理上连续的 AU
描述项大小	32	一个簇描述项占用 32 个字节的存储空间
描述 AU 管理的最大簇数目	16 * 1024	一个描述 AU 最多管理 16384 个簇
描述 AU 管理的最大 AU 数	64 * 1024	一个描述 AU 最多可以管理 65536 个 AU
描述 AU 管理的最大磁盘空间	64GB	一个描述 AU 最多可以管理 64GB 磁盘空间
Inode 项大小	512	一个 ASM 文件描述项大小，每个 ASM 文件/目录都对

		应着一个文件描述项目
Inode AU 可管理的最大文件数	2048	一个 Inode AU 最多可以管理 2048 个文件
ASM 文件最小尺寸	4MB	每个 ASM 文件最少包含一个簇
ASM 文件最大尺寸	4PB	一个 ASM 文件最多可以包含 4294967295 个 AU，每个 AU 是 1MB，理论上单个 ASM 文件的最大尺寸是 4PB
一个用户连接可同时打开的 ASM 文件数	65536	一个用户连接，最多打开 65536 个 ASM 文件
ASM 文件数上限	8388607	一个磁盘组，最多可以创建 8388607 个 ASM 文件
磁盘组个数上限	124	最多可创建 124 个磁盘组
每 10M 共享内存大小能管理的磁盘大小	约 440GB	每个簇描述项大概占用 64byte 内存空间，每个簇描述项对应 4MB 磁盘空间，共享内存还有部分内存用来保持必要的控制信息。440GB 左右的磁盘，需要 10MB 大小的共享内存能保证使用过程中簇描述项不被淘汰

10.6 DMASM 主要部件

10.6.1 DMASMCMD

DMASMCMD 是 DMASM 文件系统初始化工具，用来将块设备格式化为 ASM 磁盘，并初始化 DCR DISK、VOTE DISK。格式化 ASM 磁盘就是在块设备的头部写入 ASM 磁盘特征描述符号，包括 DMASM 标识串、ASM 磁盘名以及 ASM 磁盘大小等信息。其中 VOTE DISK 和 DCR DISK 也会被格式化为 ASM 磁盘。

DMASMCMD 工具的主要功能包括：

1. 通过 HELP 指令查看帮助信息。
2. 创建空文件模拟块设备。

3. 创建用于模拟块设备的磁盘文件（用于单机模拟 DMDSC 环境）。
4. 初始化 DCR DISK，同时指定密码。
5. 初始化 VOTE DISK。
6. 导出、导入 DCR DISK 配置信息。
7. 联机修改 DCR 磁盘，增加节点。
8. 校验 DCR 磁盘。
9. 清理 DCR DISK 中指定组的故障节点信息。
10. 列出指定路径下面磁盘属性。

10.6.1.1 使用脚本文件

10.6.1.1.1 查看 DMASMCMD HELP

通过 DMASMCMD HELP 可查看 DMASMCMD 工具的用法。

```
dmasmcmd V8

version: 03134284389-20250411-268655-10000

书写格式: ./dmasmcmd KEYWORD=<value> {KEYWORD=<value>}

        其中{}表示大括号内的关键词可为 0~N 个，N 为正整数，{}本身无需出现在语句中

使用示例: ./dmasmcmd SCRIPT_FILE=asmcmd.txt

--------------------------------------------------------------------------------

关键词（KEYWORD）        说明                                          缺省值        是否必选项

--------------------------------------------------------------------------------

SCRIPT_FILE    asmcmd 脚本文件路径                                    无            否

RET_FLAG    执行 asmcmd 脚本文件时，出错是否立即返回。0：忽略，1：返回    0        否

HELP            打印帮助信息                                          无            否
```

```
--------------------------------------------------------------------------
```

参数说明:

SCRIPT_FILE: 用于指定 ASMCMD 脚本文件绝对路径。脚本文件（例如 asmcmd.txt）集中存放了 DMASMCMD 用到的命令。

- 用户没有通过 SCRIPT_FILE 指定脚本文件，则 DMASMCMD 进入交互模式运行，逐条解析、运行命令。
- 用户通过 SCRIPT_FILE 指定脚本文件（比如 asmcmd.txt），则以行为单位读取文件内容，并依次执行，执行完成以后，自动退出 DMASMCMD 工具。脚本文件必须以"#asm script file"开头，否则认为是无效脚本文件；脚本中其它行以"#"开头表示注释；脚本文件大小不超过 1MB。

RET_FLAG: 用于设置执行 ASMCMD 脚本文件时，出错是否立即返回。取值 0 或 1，0 不返回，1 返回。缺省值为 0。

10.6.1.1.2 脚本文件用法

通过 SCRIPT_FILE 参数来指定脚本文件。

语法如下:

```
dmasmcmd SCRIPT_FILE=filepath  [RET_FLAG=value]
```

参数说明:

SCRIPT_FILE: 用于指定 asmcmd 脚本文件路径。脚本文件（例如 asmcmd.txt）中存放一条或多条 DMASMCMD 命令。用户没有通过 SCRIPT_FILE 指定脚本文件，则 DMASMCMD 进入交互模式运行，逐条解析、运行命令。用户通过 SCRIPT_FILE 指定脚本文件，则以行为单位读取文件内容，并依次执行，执行完成后，自动退出 DMASMCMD 工具。脚本文件必须以"#asm script file"开头，否则认为是无效脚本文件；脚本中其他行以"#"开头表示注释。

RET_FLAG：用于设置执行 ASMCMD 脚本文件时，出错是否立即返回。取值 0 或 1。0 不返回，即执行出错时忽略当前出错的行，继续执行下一条；1 返回，即执行出错时立即停止，并返回一个错误码。缺省值为 0。

举例说明：

第一步，创建脚本文件 asmcmd.txt。例如脚本文件内容如下：

```
#asm script file
create dcrdisk '/home/test/dameng/asmdisks/disk1.asm' 'dcr1'
create votedisk '/home/test/dameng/asmdisks/disk2.asm' 'vote'
create asmdisk '/home/test/dameng/asmdisks/disk3.asm' 'data1'
create asmdisk '/home/test/dameng/asmdisks/disk4.asm' 'data2'
```

第二步，通过使用 SCRIPT_FILE 参数指定脚本，执行出错立即停止并返回一个错误码。

```
./dmasmcmd SCRIPT_FILE=/home/test/dameng/asmcmd.txt RET_FLAG=1
```

执行结果如下：

```
dmasmcmd V8
ASM>create dcrdisk '/home/test/dameng/asmdisks/disk1.asm' 'dcr1'
[TRACE]The ASM initialize dcrdisk /home/test/dameng/asmdisks/disk1.asm to name DMASMdcr1
Used time: 5.384(ms).
ASM> create votedisk '/home/test/dameng/asmdisks/disk2.asm' 'vote'
[TRACE]The ASM initialize dcrdisk /home/test/dameng/asmdisks/disk2.asm to name DMASMvote
Used time: 19.871(ms).
ASM>create asmdisk '/home/test/dameng/asmdisks/disk3.asm'  'data1'
[TRACE]The ASM initialize asmdisk /home/test/dameng/asmdisks/disk3.asm to name
```

```
DMASMdata1

Used time: 6.754(ms).

ASM>create asmdisk '/home/test/dameng/asmdisks/disk4.asm'  'data2'

[TRACE]The ASM initialize asmdisk /home/test/dameng/asmdisks/disk4.asm to name
DMASMdata2

Used time: 10.430(ms).
```

10.6.1.2 DMASMCMD 命令用法

10.6.1.2.1 查看工具的 HELP 指令

在 DMASMCMD 工具中输入 HELP，可查看 DMASMCMD 工具如何使用 DMASMCMD 命令。

```
ASM>help

Format: create emptyfile file_path size(M) num

Usage:   create emptyfile '/dev_data/asmdisks/disk0.asm' size 100



Format: create asmdisk disk_path disk_name [size(M)] [op_type=<value>]

Usage:   create asmdisk '/dev_data/asmdisks/disk0.asm' 'DATA0'

Usage:   create asmdisk '/dev_data/asmdisks/disk0.asm' 'DATA0' 100

Usage:   create asmdisk '/dev_data/asmdisks/disk0.asm' 'DATA0' 100 op_type=0



Format: create dcrdisk disk_path disk_name [size(M)] [size(M)] [op_type=<value>]

Usage:   create dcrdisk '/dev_data/asmdisks/disk0.asm' 'DATA0'

Usage:   create dcrdisk '/dev_data/asmdisks/disk0.asm' 'DATA0' 100
```

```
Usage:    create dcrdisk '/dev_data/asmdisks/disk0.asm' 'DATA0' 100 op_type=0


Format: create votedisk disk_path disk_name [size(M)] [size(M)] [op_type=<value>]

Usage:    create votedisk '/dev_data/asmdisks/disk0.asm' 'DATA0'

Usage:    create votedisk '/dev_data/asmdisks/disk0.asm' 'DATA0' 100

Usage:    create votedisk '/dev_data/asmdisks/disk0.asm' 'DATA0' 100 op_type=0


Format: init dcrdisk disk_path from ini_path identified by password

Usage:    init dcrdisk '/dev_data/asmdisks/disk0.asm' from '/data/dmdcr_cfg.ini' identified by
'DCRpsd_123'


Format: init votedisk disk_path from ini_path

Usage:    init votedisk '/dev_data/asmdisks/disk0.asm' from '/data/dmdcr_cfg.ini'


Format: export dcrdisk disk_path to ini_path

Usage:    export dcrdisk '/dev_data/asmdisks/disk0.asm' to '/data/dmdcr_cfg.ini'


Format: import dcrdisk ini_path to disk_path

Usage:    import dcrdisk '/data/dmdcr_cfg.ini' to '/dev_data/asmdisks/disk0.asm'


Format: extend dcrdisk disk_path from ini_path

Usage:    extend dcrdisk '/dev_data/asmdisks/disk0.asm' from '/data/dmdcr_cfg.ini'


Format: check dcrdisk disk_path
```

```
Usage:    check dcrdisk '/dev_data/asmdisks/disk0.asm'


Format: clear dcrdisk err_ep_arr disk_path group_name

Usage:    clear dcrdisk err_ep_arr '/dev_data/asmdisks/disk0.asm' 'GRP_RAC'


Format: listdisks path{,path}

Usage:    listdisks '/dev_data/asmdisks/'

Usage:    listdisks '/dev_data/asmdisks/,/dev_data/asmdisks2/,/dev_data/asmdisks3/'
```

10.6.1.2.2 创建空文件模拟块设备

创建名为 xxx.asm 的模拟块设备，注意必须以.asm 结尾。模拟测试用，真实环境不建议使用。块设备作为普通数据磁盘时，size 最小值为 64MB；块设备作为 DCR 或 VOTE 磁盘时，size 最小值为 32MB，无上限限制。

语法格式：

```
CREATE EMPTYFILE file_path SIZE num
```

举例说明：

创建名为 DISK0.asm 的模拟块设备。

```
CREATE EMPTYFILE '/OPT/DATA/ASMDISKS/DISK0.asm' SIZE 100
```

10.6.1.2.3 创建磁盘文件

- **创建 ASM 磁盘**

用来将块设备格式化为 ASM DISK，会在块设备头部写入 ASM DISK 标识信息。size 取值最小 64。

语法格式：

```
CREATE ASMDISK disk_path disk_name [size] [op_type=<value>]
```

举例说明：

```
CREATE ASMDISK '/HOME/ASMDISKS/DISK0.asm' 'DATA0'

CREATE ASMDISK '/HOME/ASMDISKS/DISK0.asm' 'DATA0' 100

CREATE ASMDISK '/HOME/ASMDISKS/DISK0.asm' 'DATA0' 100 op_type=0
```

- **创建 DCR 磁盘**

用来将块设备格式化为 DCR 磁盘，会在块设备头部写入 DCR 标识信息。size 取值最小 32。

语法格式：

```
CREATE DCRDISK disk_path disk_name [size] [op_type=<value>]
```

举例说明：

```
CREATE DCRDISK '/HOME/ASMDISKS/DISK0.asm' 'DATA0'

CREATE DCRDISK '/HOME/ASMDISKS/DISK0.asm' 'DATA0' 100

CREATE DCRDISK '/HOME/ASMDISKS/DISK0.asm' 'DATA0' 100 op_type=0
```

- **创建 VOTE 磁盘**

用来将块设备格式化为 VOTE DISK，会在块设备头部写入 VOTE DISK 标识信息。size 取值最小 32。

语法格式：

```
CREATE VOTEDISK disk_path disk_name [size] [op_type=<value>]
```

举例说明：

```
CREATE VOTEDISK '/HOME/ASMDISKS/DISK0.asm' 'DATA0'

CREATE VOTEDISK '/HOME/ASMDISKS/DISK0.asm' 'DATA0' 100

CREATE VOTEDISK '/HOME/ASMDISKS/DISK0.asm' 'DATA0' 100 op_type=0
```

以上三个命令中 size 参数和 op_type 参数可以省略，程序会计算 disk_path 的大小；但是某些操作系统计算 disk_path 大小会失败，这时候还是需要用户指定 size 信息。op_type 参数表示操作类型，取值为 0 或 1，为 0 表示创建新版本（4100）格式磁盘，为 1 表示创建旧版本（4099）版本格式磁盘，缺省值为 0。

10.6.1.2.4 初始化 DCR DISK & VOTE DISK

- **初始化 DCR DISK**

根据配置文件 DMDCR_CFG.INI 的内容，初始化 DCR 磁盘。设置登录 ASM 文件系统的密码，密码要用单引号括起来。

语法格式：

```
INIT DCRDISK disk_path FROM ini_path IDENTIFIED BY password
```

举例说明：

```
INIT DCRDISK '/DEV/RAW/RAW2' FROM '/HOME/ASM/DMDCR_CFG.INI' IDENTIFIED BY 'DCRpsd_123'
```

- **初始化 VOTE DISK**

根据配置文件 DMDCR_CFG.INI 的内容，初始化 VOTE DISK。

语法格式：

```
INIT VOTEDISK disk_path FROM ini_path
```

举例说明：

```
INIT VOTEDISK '/DEV/RAW/RAW3' FROM '/HOME/ASM/DMDCR_CFG.INI'
```



注意：

1.DMASM 要求同一目录下只能有一套 ASM 文件系统，为了避免操作失误违反这一要求，不允许在已有 DCR（VOTE）磁盘的目录下新初始化其它的 DCR（VOTE）磁盘。

2.重新初始化 DCR（VOTE）磁盘时，需要将原 DCR（VOTE）磁盘重新初始

化为新 DCR（VOTE）磁盘，或者先将原 DCR（VOTE）磁盘格式化再选择其它磁盘初始化为 DCR（VOTE）磁盘，否则会由于违反 1.的要求报错。

10.6.1.2.5 导出 DCR 的配置文件

解析 DCR 磁盘内容，导出到 DMDCR_CFG.INI 文件。

语法格式：

```
EXPORT DCRDISK disk_path TO ini_path
```

举例说明：

```
EXPORT DCRDISK '/DEV/RAW/RAW2' TO '/HOME/ASM/DMDCR_CFG.INI'
```

10.6.1.2.6 导入 DCR 的配置文件

将修改后的 DMDCR_CFG.INI 文件导入到 DCR 磁盘。

必须以组为单位进行修改，其中 DCR_VTD_PATH 、DCR_N_GRP 、DCR_GRP_N_EP 、DCR_GRP_NAME 、DCR_GRP_TYPE 、DCR_GRP_EP_ARR 、DCR_EP_NAME、EP 所属的组类型、CHECKSUM 值、密码不可修改。特别的，在动态增加节点时，可以并需要对 DCR_GRP_N_EP 和 DCR_GRP_EP_ARR 进行修改。

语法格式：

```
IMPORT DCRDISK ini_path TO disk_path
```

举例说明：

```
IMPORT DCRDISK '/DATA/DMDCR_CFG.INI' TO '/DATA/ASMDISKS/DISK0.asm'
```

10.6.1.2.7 联机修改 DCR 磁盘，增加节点

联机修改 DCR 磁盘，增加节点，会将新增节点信息写回 dcr 磁盘。

语法格式：

```
EXTEND DCRDISK disk_path FROM ini_path
```

举例说明：

```
EXTEND DCRDISK 'D:\ASMDISKS\DISK0.asm' FROM 'D:\DMDCR_CFG.INI'
```

10.6.1.2.8 校验 DCR 磁盘

校验 DCR 磁盘信息是否正常。打印“ASMCMD check DCRDISK success”，表示 DCR 磁盘正常；打印“ASMCMD check DCRDISK failed……”，需要根据打印的信息进一步判断。如果 DCR 磁盘故障，需要重新初始化。

语法格式：

```
CHECK DCRDISK disk_path
```

举例说明：

```
CHECK DCRDISK '/DEV/RAW/RAW2'
```

10.6.1.2.9 清理指定组的故障节点信息

清理 DCR DISK 中指定组的故障节点信息，可借助 export 命令查看对应组的 DCR_GRP_N_ERR_EP 和 DCR_GRP_ERR_EP_ARR 信息，清理成功后，指定组的 DCR_GRP_N_ERR_EP 值为 0，DCR_GRP_ERR_EP_ARR 内容为空。

语法格式：

```
CLEAR DCRDISK err_ep_arr disk_path group_name
```

举例说明：

```
CLEAR DCRDISK ERR_EP_ARR '/DEV/RAW/RAW2' 'GRP_DSC'
```

10.6.1.2.10 显示指定路径下面磁盘属性

显示 path 路径下面所有磁盘的信息，分为三种类型：NORMAL DISK：普通磁盘；UNUSED ASMDISK：初始化未使用的 ASMDISK；USED ASMDISK：已经使用的 ASMDISK。

语法格式：

```
LISTDISKS path{,path}
```

举例说明：

```
LISTDISKS '/DEV/RAW/'

LISTDISKS '/DEV/RAW0/,/DEV/RAW1/,/DEV/RAW2/'
```

限制：

支持指定多个路径，显示多路径下的磁盘信息，各路径以英文逗号','进行分隔，所有配置的路径都必须为有效路径，最多可以同时配置 8 个路径，且不支持中文路径。

如果查询模拟 ASM 磁盘的磁盘信息，则对 path 路径没有限制；如果查询真实环境中共享存储路径下的磁盘信息，则 path 路径的根目录名称必须以“dev”开头，例如：listdisks '/dev/DMDATA/' 或 listdisks '/dev_data/DMDATA/'，否则将不会显示该路径下的磁盘信息。

10.6.2 DMASMSVR

DMASMSVR 是提供 DMASM 服务的主要载体，每个提供 DMASM 服务的节点都必须启动一个 DMASMSVR 服务器，这些 DMASMSVR 一起组成共享文件集群系统，提供共享文件的全局并发控制。DMASMSVR 启动时扫描/dev/raw/路径下的所有块设备，加载 ASM 磁盘，构建 ASM 磁盘组和 DMASM 文件系统。DMASMSVR 实例之间使用 MAL 系统进行信息和数据的传递。

DMASMSVR 集群的启动、关闭、故障处理等流程由 DMCSS 控制，DMASMSVR 定时

向 VOTE DISK 写入时间戳、状态、命令、以及命令执行结果等信息，DMCSS 控制节点定时从 VOTE DISK 读取信息，检查 DMASMSVR 实例的状态变化，启动相应的处理流程。

DMASMSVR 集群中，只有一个控制节点，控制节点以外的其他节点叫做普通节点，DMASMSVR 控制节点由 DMCSS 选取；所有 DDL 操作（比如创建文件，创建磁盘组等）都是在控制节点执行，用户登录普通节点发起的 DDL 请求，会通过 MAL 系统发送到控制节点执行并返回；而 ASM 文件的读、写等操作，则由登录节点直接完成，不需要传递到控制节点执行。

通过 DMASMSVR HELP 可查看 DMASMSVR 的启动用法。

```
dmasmsvr V8

version: 03134284389-20250411-268655-10000

书写格式: ./dmasmsvr KEYWORD=<value> {KEYWORD=<value>}

        其中{}表示大括号内的关键词可为 0~N 个，N 为正整数，{}本身无需出现在语句中

使用示例: ./dmasmsvr DCR_INI=/opt/data/DAMENG/dmdcr.ini

------------------------------------------------------------------------

关键词（KEYWORD）          说明                    缺省值          是否必选项

------------------------------------------------------------------------

DCR_INI                  dmdcr.ini 路径          无              是

-NOCONSOLE               以服务方式启动          无              否

LISTEN_IPV6              是否监听 IPV6           Y               否

HELP                     打印帮助信息            无              否

------------------------------------------------------------------------
```

参数说明：

DCR_INI：DMDCR.INI 的路径。DMDCR.INI 配置文件记录了 DCR 磁盘路径、实例序列号等信息；如果不指定 DCR_INI 参数，DMASMSVR 默认在当前路径下查找 DMDCR.INI 文

件。

LISTEN_IPV6：是否监听 IPV6。


说明：**使用 DMASMTOOL 或 DMASMAPI 函数 dmasm_connect() 登录 DMASMSVR 时，用户名只能为“ASMSYS”。若为本地登录，不校验密码；若为远程登录，应使用初始化 DCR 磁盘时设置的登录 ASM 文件系统的密码。**

10.6.3 DMASMAPI

DMASMAPI 是 DMASM 文件系统的应用程序访问接口，通过调用 DMASMAPI 接口，用户可以访问、操作 ASM 文件。与达梦数据库接口 DPI 类似，访问 ASM 文件之前，必须先分配一个 conn 对象，并登录到 DMASMSVR 服务器，再使用这个 conn 对象进行创建磁盘组、创建文件、删除文件、读取数据和写入数据等 DMASM 相关操作。

DMASMAPI 接口的详细使用说明请参考 19 附录。

10.6.4 DMASMTOOL

DMASMTOOL 是 DMASM 文件系统管理工具，提供了一套类 Linux 文件操作命令，用于管理 ASM 文件，是管理、维护 DMASM 的好帮手。DMASMTOOL 工具使用 DMASMAPI 连接到 DMASMSVR，并调用相应的 DMASMAPI 函数，实现创建、拷贝、删除等各种文件操作命令；DMASMTOOL 还支持 ASM 文件和操作系统文件的相互拷贝。

DMASMTOOL 可以登录本地 DMASMSVR，也可以登录位于其他节点的 DMASMSVR，并执行各种文件操作命令。一般建议登录本地 DMASMSVR 服务器，避免文件操作过程中的网络开销，提升执行效率。

通过 DMASMTOOL HELP 可查看 DMASMTOOL 的启动方法。

```
dmasmtool V8

version: 03134284389-20250411-268655-10000
```

书写格式: ./dmasmtool KEYWORD=<value> {KEYWORD=<value>}

其中{}表示大括号内的关键词可为 0~N 个，N 为正整数，{}本身无需出现在语句中

使用示例: ./dmasmtool DCR_INI=/opt/data/DAMENG/dmdcr.ini

连接本地 asmsvr:

关键词（KEYWORD） 说明 缺省值

是否必选项

DCR_INI dmdcr.ini 路径 dmdcr.ini 是

HOST asm 服务器地址，IPV6 地址要用[]包围 无 否

PORT_NUM asm 服务器端口号 无 否

USERID 登录 asm 服务器用户名密码（格式：ASMSYS/PASSWORD） 无 否

SCRIPT_FILE asmtool 脚本文件路径 无 否

HELP 打印帮助信息 无 否

连接远程 asmsvr:

关键词（KEYWORD） 说明 缺省值 是否必选项

DCR_INI dmdcr.ini 路径 dmdcr.ini 否

HOST asm 服务器地址，IPV6 地址要用[]包围 无 是

PORT_NUM asm 服务器端口号 无 是

USERID 登录 asm 服务器用户名密码 (格式: ASMSYS/PASSWORD) 无 是

SCRIPT_FILE asmtool 脚本文件路径 无 否

HELP	打印帮助信息	无	否

1) DMASMTOOL 的最大命令长度是 1024。

2) 用户没有指定脚本文件，则 DMASMTOOL 进入交互模式运行，逐条解析、运行命令；用户指定脚本文件(比如 asmtool.txt)，则以行为单位读取文件内容，并依次执行，执行完成以后，自动退出 DMASMTOOL 工具。脚本文件必须以"#asm script file"开头，否则认为是无效脚本文件；脚本中其他行以"#"开始表示注释。


注意：

3) DMASMTOOL 命令直接输入的 host/port 信息配置的是连接 ASMSVR 的信息，可以在 DMDCR_CFG.INI 里面找到，分别为要连接的 ASM 节点的 DCR_EP_HOST 和 DCR_EP_PORT。

DMASMTOOL 支持的命令说明：

1. 创建 ASM 磁盘组，添加 ASM 磁盘，删除 ASM 磁盘组，删除 ASM 磁盘

创建磁盘组，或为磁盘组添加磁盘时，以下情况可能导致失败：

- DMASMSVR 服务器没有访问对应磁盘的权限；
- 磁盘路径不在 DMDCR_CFG.INI 配置文件中配置的 DCR_EP_ASM_LOAD_PATH 路径下；
- 磁盘大小不够，最少需要 32MB。

在磁盘组中添加 ASM 磁盘时，可能出现组中的 ASM 磁盘版本号不一致的情况，此时 ASM 磁盘自动向下兼容，即高版本 ASM 磁盘自动转换为低版本 ASM 磁盘。关于 ASM 磁盘版本的介绍请参考 17.13 ASM 磁盘版本。

- **创建 ASM 磁盘组**

语法格式：

```
CREATE DISKGROUP name ASMDISK file_path
```

参数说明：

name：磁盘组名，最长不能超过 32 字节。

file_path：磁盘组路径，必须是全路径，不能是相对路径。

举例说明：

```
CREATE DISKGROUP 'DMDATA' ASMDISK '/DEV/RAW/RAW3'
```

- **添加 ASM 磁盘**

语法格式：

```
ALTER DISKGROUP name ADD ASMDISK file_path
```

参数说明：

name：磁盘组名，最长不能超过 32 字节。

file_path：ASMDISK 路径。必须是全路径，不能是相对路径。

举例说明：

```
ALTER DISKGROUP 'DMDATA' ADD ASMDISK '/DEV/RAW/RAW4'
```

- **删除 ASM 磁盘组**

语法格式：

```
DROP DISKGROUP name
```

参数说明：

name：磁盘组名。

举例说明：

```
DROP DISKGROUP 'DMDATA'
```

- **删除 ASM 磁盘**

删除 ASM 磁盘。仅支持删除磁盘组中的最后一个磁盘，如果磁盘组中仅有一个磁盘，则不支持删除；如果磁盘空间曾被分配使用过，则不支持删除。

语法格式：

```
ALTER DISKGROUP name DROP ASMDISK file_path
```

参数说明：

name：磁盘组名。

file_path：ASMDISK 路径。必须是全路径，不能是相对路径。

举例说明：

```
ALTER DISKGROUP 'DMDATA' DROP ASMDISK '/DEV/RAW/RAW4'
```

2. 创建文件，扩展文件，截断文件，删除文件

- **创建文件**

语法格式：

```
CREATE ASMFILE file_path SIZE num
```

参数说明：

file_path：ASM 文件路径。必须是全路径，不能是相对路径。

num：ASM 文件大小，单位 M。取值范围：0~2^{24} 个 AU，需转换为单位 M。

举例说明：

```
CREATE ASMFILE '+DMDATA/SAMPLE.DTA' SIZE 20
```

- **扩展文件**

语法格式：

```
ALTER ASMFILE file_path EXTEND TO size
```

参数说明：

file_path：ASM 文件路径。必须是全路径，不能是相对路径。

size：扩展到指定大小，单位 M。取值范围：0~2^{24} 个 AU，需转换为单位 M。

举例说明：

```
ALTER ASMFILE '+DMDATA/SAMPLE.DTA' EXTEND TO 20
```

- **截断文件**

语法格式：

```
ALTER ASMFILE file_path TRUNCATE TO size
```

参数说明：

file_path：ASM 文件路径。必须是全路径，不能是相对路径。

size：截断到指定大小，单位 M。取值范围：0~2^{24} 个 AU，需转换为单位 M。

举例说明：

```
ALTER ASMFILE '+DMDATA/SAMPLE.DTA' TRUNCATE TO 20
```

- **删除文件**

删除 ASM 文件。已经 open、正在访问的 DMASM 文件不允许删除。

语法格式：

```
DELETE ASMFILE file_path
```

参数说明：

file_path：ASM 文件路径。必须是全路径，不能是相对路径。

举例说明：

```
DELETE ASMFILE '+DMDATA/SAMPLE.DTA'
```

- **重定向输出文件**

重定向输出文件。如果多次重定向文件，第一次成功打开重定向文件之后，如果未关闭，则不再打开其他重定向文件。

语法格式：

```
SPOOL file_path [CREATE|REPLACE|APPEND]
```

参数说明：

file_path：输出文件路径。

CREATE：如果重定向文件不存在，则创建；如果存在，创建失败。

REPLACE：如果重定向文件不存在，则创建；如果存在，则替换掉。缺省为 REPLACE。

APPEND：如果重定向文件不存在，则创建；如果存在，则追加到文件末尾。

举例说明：

```
SPOOL /HOME/DATASPOOL.TXT
```

- **关闭重定向文件**

语法格式：

```
SPOOL OFF
```

举例说明：

```
SPOOL OFF
```

3. 兼容 LINUX 一些命令，功能受限，但是很实用

- **到达某目录**

语法格式：

```
CD [path]
```

参数说明：

path：目标路径。

举例说明：

```
CD +DMDATA/TEST
```

- **拷贝**

语法格式：

```
CP [-RF] <src_file_path> <dst_file_path >

或

CP [-RF] <src_dir_path> < dst_dir_path>
```

参数说明：

-R：递归拷贝，将指定目录下的子文件和子目录一并拷贝。

-F：执行拷贝，不给出提示。

src_file_path：源文件路径。必须是全路径，不能是相对路径。

dst_file_path：目标文件路径。必须是全路径，不能是相对路径。

src_dir_path：源文件所在磁盘路径。

dst_file_path：目标文件所在磁盘路径。

举例说明：

```
CP '+DMDATA/AA/SAMPLE.DTA' '+DMDATA/A/B.DTA'

CP -R '+DMDATA/AA' '+DMDATA/BB'

CP –F '+DMDATA/AA/SAMPLE.DTA' '+DMDATA/A/B.DTA'
```

- **删除**

语法格式：

```
RM [-rf] <file_path|dir_path>
```

参数说明：

-r：递归删除，将指定目录下的子文件和子目录一并删除。

-f：执行删除，不给出提示。

file_path：文件路径。

dir_path：磁盘路径。

举例说明：

```
RM '+DMDATA/A/SAMPLE.DTA'

RM -r '+DMDATA/A/'

RM –f '+DMDATA/B/'
```

- **创建目录**

语法格式：

```
MKDIR [-P] dir_path
```

参数说明：

-P：自动创建不存在的中间目录。

dir_path：磁盘路径。

举例说明：

```
MKDIR '+DMDATA/A'

MKDIR -P '+DMDATA/NODIR/BB'
```

- **查找**

语法格式：

```
FIND [path] <file_name>
```

参数说明：

path：磁盘路径。

file_name：文件名。

举例说明：

```
FIND +DMDATA/A 'SAMPLE.DTA'
```

- **显示**

语法格式：

```
LS [-lr] <file_name|dir_name>
```

参数说明：

-l：显示文件详细信息。

-r：递归显示。

file_name：文件名。

dir_name：磁盘名。

举例说明：

```
LS '+DMDATA/A'

LS -l '+DMDATA/A'

LS –r '+DMDATA/A'
```

- **显示存储信息**

语法格式：

```
DF
```

举例说明：

```
DF
```

- **当前目录**

语法格式：

```
PWD
```

举例说明：

```
PWD
```

- **估算文件空间已使用情况**

语法格式：

```
DU [dir_path{,dir_path}]
```

参数说明：

dir_path：要估算的文件目录，未指定则默认为当前目录。

举例说明：

```
du

du DMDATA

du DMDATA,DMLOG

du DMDATA DMLOG
```

估算结果中，出现的单位有缺省（字节）、K、M 和 G 等情况。

```
ASM>du

5.3G        +DATA/data/rac00/arch

5.3G        +DATA/data/rac00

586.6M     +DATA/data/rac01/arch

586.6M     +DATA/data/rac01

0           +DATA/data/rac/bak
```

```
631.5K     +DATA/data/rac/ctl_bak
0          +DATA/data/rac/HMAIN
1012.6M    +DATA/data/rac
6.8G       +DATA/data
6.8G       +DATA
```

- **移动、重命名文件。不支持跨磁盘组移动文件**

语法格式：

```
MV src_file_path dst_file_path
```

参数说明：

src_file_path：源文件路径。必须是全路径，不能是相对路径。

dst_file_path：目标文件路径。必须是全路径，不能是相对路径。

举例说明：

```
MV '+DMDATA/a/sample.dta' '+DMDATA/a/sample2.dta'
```

4. **DMASM 特有的一些命令**

- **列出所有的磁盘组**

语法格式：

```
LSDG
```

举例说明：

```
LSDG
```

- **列出所有的 ASM 磁盘**

语法格式：

```
LSDSK
```

举例说明：

```
LSDSK
```

- 列出系统属性，包括 AU_SIZE、簇信息、系统编码语言、共享内存属性等

语法格式：

```
LSATTR
```

举例说明：

```
LSATTR
```

- 列出所有的信息，包括文件等

语法格式：

```
LSALL
```

举例说明：

```
LSALL
```

- 修改密码

语法格式：

```
PASSWORD
```

举例说明：

```
PASSWORD
```

- 登录，在断开连接后，重新登录

语法格式：

```
LOGIN
```

举例说明：

```
LOGIN
```

- **测试 ASM 环境下磁盘的读写速度，包括随机读/写速度和顺序读/写速度**

语法格式：

```
IOTEST [DISKGROUP=<diskgroup_name>] [ASMFILE=<file_name>] [SIZE=<file_size>]
```

参数说明：

<diskgroup_name>：磁盘组名。若指定则在指定磁盘组下生成临时文件，若不指定则在当前路径下生成临时文件，如果当前路径为“+”目录，则默认在 0 号 group 根目录下生成临时文件。

<file_name>：指定生成的临时文件名称，若不指定则系统自动生成临时文件名。

<file_size>：指定临时文件大小，取值范围 0~2147483647，单位 MB。若不指定则默认临时文件大小为 1024MB。

使用说明：

DM 系统文件 IO 接口封装了 Windows 的 ReadFile/WriteFile 函数和 Linux 的 pread/pwrite 函数，DM 系统 IO 线程从系统 IO 任务链表中获取 IO 任务后调用 DM 系统文件 IO 接口读取指定文件相关内容。

Iotest 命令属于 ASM 文件系统，ASM 文件系统的存储最小单位为扇区，ASM 文件读写接口读写的字节数必须是扇区(512 字节)的倍数，如果 ASM 文件系统和 ASM 应用部署在同一台机器上，则使用本地读写接口，如果在不同的机器上，则使用远程读写接口。ASM

本地读写接口设置好 IO 任务参数后将 IO 任务加入 DM 系统 IO 任务链表，等待 DM 系统 IO 线程执行完该 IO 任务后回调唤醒 ASM 本地读写接口线程，完成文件读写操作。ASM 远程读写接口使用 MAL 链路网络通信，通知 ASM 文件系统向 DM 系统 IO 任务链表添加对应的文件 IO 任务，待文件 IO 完成后发送完成信件给 IO 调用节点，唤醒 IO 调用节点的对应线程，完成文件读写操作。

测试随机读写和顺序读写的方式：

随机读写的文件读写偏移随机为文件大小之内的当前读写单位大小的倍数，即如果文件大小为 1G，当前读写单位大小为 8M，则会进行 1G/8M=128 次文件读写，读写偏移为 1G 内的 8M 倍数，如 8M、16M 等。顺序读写的文件读写偏移则从 0 开始以当前读写单位顺序增长到文件大小，即如果文件大小为 1G，当前读写单位大小为 8M，则会进行 1G/8M=128 次文件读写，读写偏移为 0，8M，16M 直到 1016M。

Iotest 命令以单线程执行，并且不会走服务器缓存。

现有 ASM 系统只能测试磁盘组的文件读写速度，如果一个磁盘组由多个磁盘组成，则只能测试正在使用的未满磁盘的文件读写速度。预期磁盘组的 AU_SIZE 越小，文件读取速度越快。

执行 iotest 命令时，会在指定路径或当前路径下创建一个后缀名为.iotest 的临时文件，测试完成后自动删除该临时文件。出现以下情况时将报错：

- 若磁盘空间不足，则报错；
- 若临时文件生成路径下已经存在同名文件，则报错，需要手动删除同名文件后才能继续执行 iotest 命令。

举例说明：

```
IOTEST DISKGROUP=DMDATA SIZE=128
```

1. DMASMTOOL 工具的上下键，查找历史记录，以及 TAB 键的自动补齐功能是基于 readline 实现的，由于 readline 输入不支持中文，因此目前 DMASMTOOL 工具不支持中文输入。
2. 若文件名或路径中包含中文字符，当客户端与服务器的编码不一致时，可能出现乱码，因此建议不要使用包含中文字符的文件名或路径。


注意：

3. 在 DMASMTOOL 工具执行命令时，如果路径名或者文件名首字母为数字，需要给字符串加单引号'。
4. 在 DMASMTOOL 工具使用 cd 命令，首字母为'+', '\', '/'符号都会从目录最上层开始查找目录。
5. ASM 文件全路径长度不能超过 256 字节。

- 文件校验

检查 ASM 文件各个 AU 分布情况。

语法格式：

```
CHECK [file_path]
```

参数说明：

file_path：ASM 文件路径。

举例说明：

```
CHECK

或

CHECK +DMDATA/SAMPLE.DTA
```

- 升级 ASM 文件系统所有 ASM 文件的簇描述项 init_flag 值，从未格式化状态升级为格式化状态

语法格式：

```
upgrade
```

举例说明：

```
upgrade
```

10.7 DMASM 使用说明

DMASM 的配置和使用大致分为三步。

第一步 环境准备

- 硬件

  准备多台部署 DMDSC 集群的机器。机器数和集群节点数一致。

- 存储设备

  准备共享磁盘。用户根据实际数据量来准备共享磁盘的数量。至少一块。

- 操作系统

  为所有机器配置相同的操作系统。支持 Windows 和 Linux 系统。用户根据实际需求选择操作系统。

- 网络配置

  每台机器至少配置两块网卡。一块用于配置内网网段，一块用于配置外网网段。

- 用户权限准备

  如果用户选择 Linux 操作系统，那么需要在每台机器上使用 root 用户创建用于搭建环境的用户和组。

- 目录规划

  为每台机器各创建三个目录。

  目录一 使用 DMDBA 用户创建用于镜像环境搭建的目录，不同机器上此目录必须相同。

  目录二 DM 执行码和工具存放于目录，不同机器上此目录必须相同。

目录三 配置文件存放于目录，不同的机器上此目录可以不同。

- 磁盘准备

首先使用多路径软件为共享存储的磁盘创建目录名，即将磁盘链接到一个目录中。然后将磁盘的用户权限修改为和目录的用户权限一致。

第二步 搭建 DMASM

搭建按照下面顺序依次进行:

1. 使用 DMASMCMD 工具创建所有磁盘。具体为 DCR 磁盘、VOTE 磁盘和 ASM 磁盘，其中 ASM 磁盘用于存放数据(DATA)和日志(LOG)。
2. 配置并启动 DMCSS。
3. 配置并启动 DMASMSVR。
4. 使用 DMASMTOOL 工具创建 ASMDISK 磁盘组。具体为 DATA 磁盘组和 LOG 磁盘组两种，分别管理 DATA 磁盘和 LOG 磁盘。
5. 配置并启动 DMSERVER。
6. 配置并启动 DMCSSM。

至此，一个使用了 DMASM 镜像的 DMDSC 部署完成。

第三步 正常使用数据库

用户登录到任意一个 DM 实例，即可获得完整的达梦数据库服务。同时，用户可通过两种方式来管理 ASM 文件: 一是编写用户代码, 通过调用 DMASMAPI 接口来管理 ASM 文件; 二是使用 DMASMCMD 和 DMASMTOOL 工具来管理 ASM 文件。

11 DMASM 镜像介绍

11.1 DMASM 镜像概述

共享存储上的数据非常宝贵，为了保障这些数据的安全性和高可用性，达梦提供了 DMASM 镜像功能。镜像是 DMASM 的一个重要功能。

DMASM 镜像提供了多副本和条带化功能。多副本技术保证同一数据的多个副本会分别写入到不同的磁盘中。多个副本中只有一个作为主副本对外提供服务，其余副本均作为镜像副本。当主副本发生故障后，系统会从镜像副本中重新自动挑选一个继续提供服务。条带化技术可保证写入的数据均匀分布到磁盘组内的不同磁盘中，实现负载均衡。

DMDSC 采用配置镜像功能的 DMASM 管理的块设备作为共享存储，当出现磁盘损坏或数据丢失时，既可以利用其他镜像副本继续提供数据库服务，又可以使用其他镜像副本进行数据恢复。

下图为一个部署了 DMASM 镜像的 DMDSC 集群结构图。

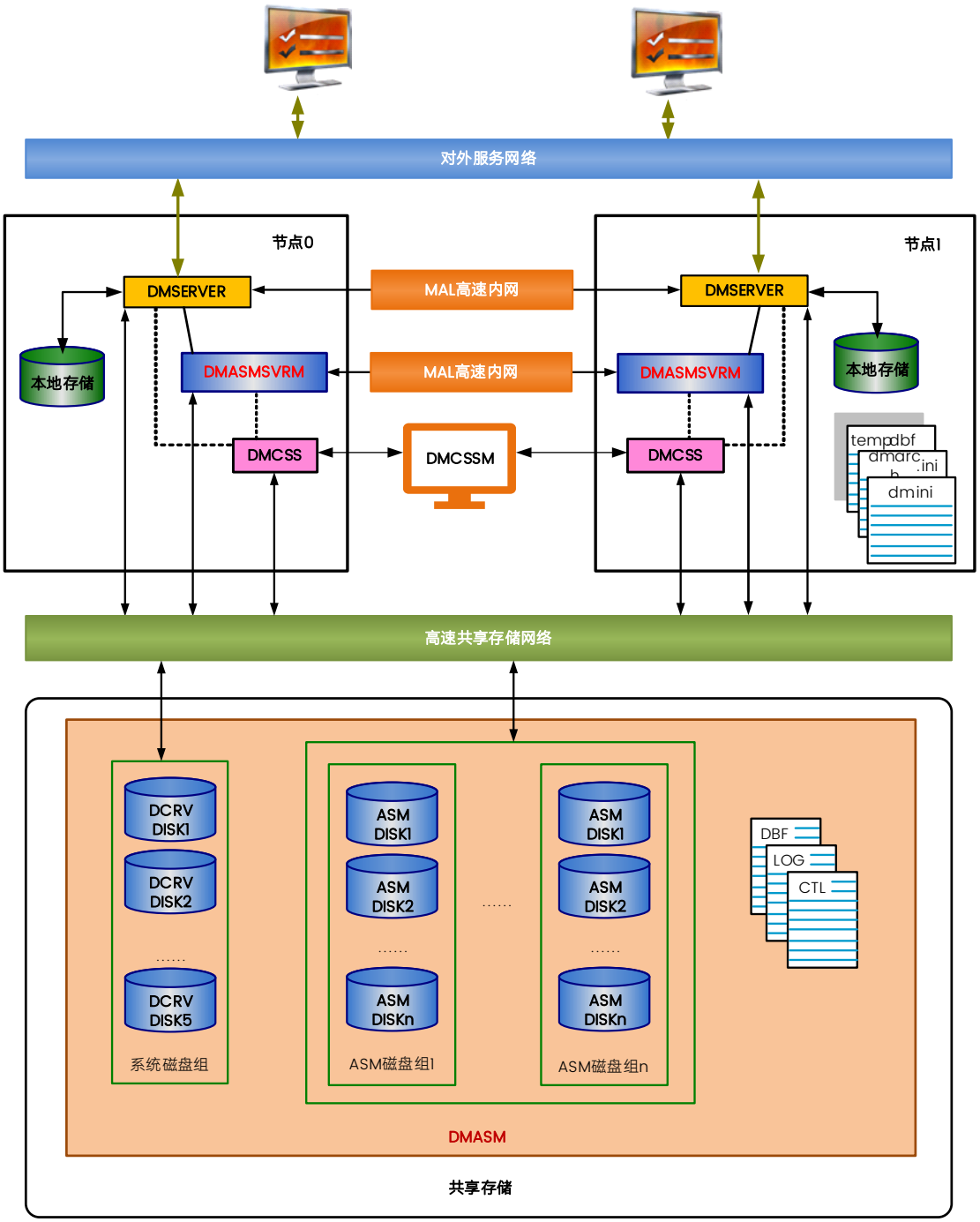


图 11.1 一个部署了 DMASM 镜像的 DMDSC 集群

一个 DMDSC 系统中可能同时拥有多个磁盘组。一个磁盘组中完整的 DMASM 镜像结构包括副本、磁盘和故障组。副本数由用户指定。故障组专门用来存放 ASM 文件的不同镜像副本。一个磁盘组中包含多个故障组，故障组是磁盘组的子集。下图以一个磁盘组中的二副本 DMASM 镜像为例，展示 DMASM 镜像的结构。

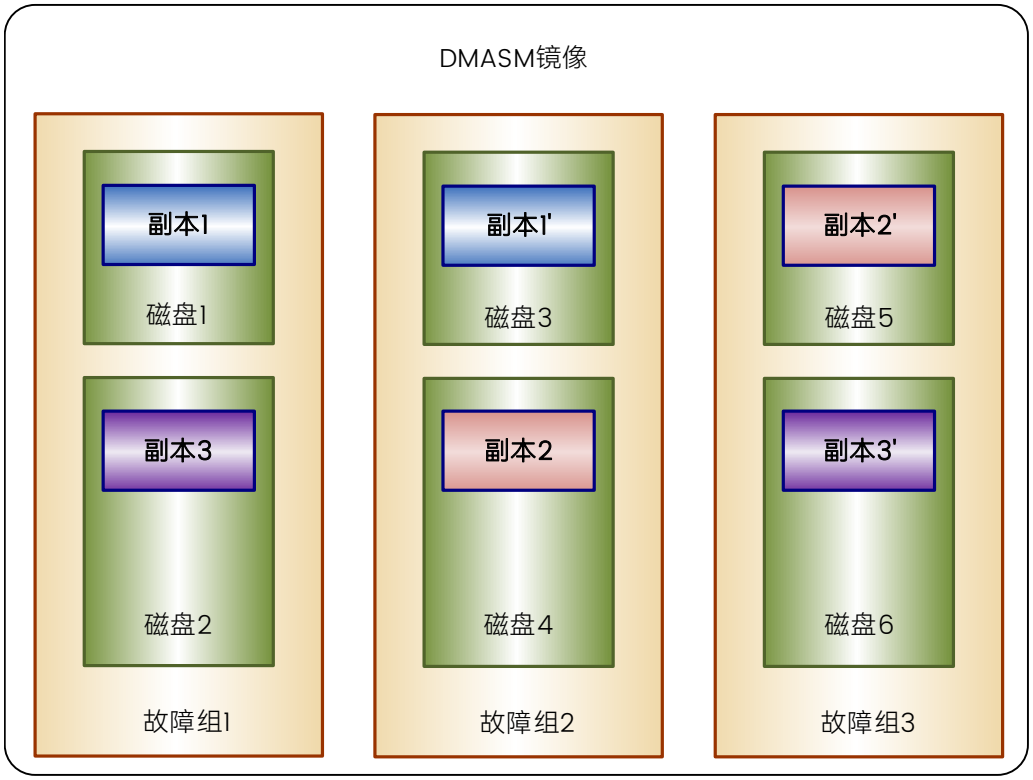


图 11.2　一个磁盘组中的两副本 DMASM 镜像结构

11.2 DMASM 镜像术语和基本概念

11.2.1　DMASM 镜像术语

本章介绍 DMASM 镜像所用到的术语。术语的详细概念介绍请参考 11.2.2 DMASM 镜像基本概念。

表 11.1 DMASM 镜像术语中英文对照表

中文术语	英文术语
DM 自动存储管理器	DM Auto Storage Manager，简称 DMASM
磁盘	DISK
磁盘组	DISK GROUP
系统磁盘	DCRV DISK
系统磁盘组	DCRV DISK GROUPS

ASM 磁盘	ASM DISK
ASM 磁盘组	ASM DISK GROUP
数据分配单元	ALLOCATION UNITS，简称 AU
DM 集群注册表	DM Clusterware Registry，简称 DCR
ASM 文件	ASMFILE
文件镜像	MIRRORING
故障组	FAILURE GROUPS
DMASM 文件系统初始化工具	DMASMCMDM（专门用于 DMASM 镜像环境）
DMASM 服务器	DMASMSVRM（专门用于 DMASM 镜像环境）
DMASM 应用程序访问接口	DMASMAPIM（专门用于 DMASM 镜像环境）
DMASM 管理工具	DMASMTOOLM（专门用于 DMASM 镜像环境）

11.2.2 DMASM 镜像基本概念

DM 自动存储管理器

DMASM 是一个分布式文件系统，用来管理块设备的磁盘和文件。

磁盘（DISK）

磁盘用于存储 ASM 文件的数据。DMASM 镜像环境中用到的磁盘均为经过 DMASMCMDM 工具创建的磁盘，并非普通的裸盘。

DMASM 镜像系统的磁盘分为 DCRV 磁盘和 ASM 磁盘两种。DCRV 磁盘由 CREATE DCRVDISK……语法创建。ASM 磁盘由 CREATE ASMDISK……语法创建。

磁盘组（DISK GROUP）

DMASM 镜像使用磁盘组管理 ASM 文件；每个磁盘组可以包含多块磁盘。ASM 文件数据均匀存储在各个磁盘上，能充分利用多块磁盘 IO。

DMASM 镜像系统的磁盘组分为 DCRV 系统磁盘组和 ASM 磁盘组两种。

DCRV 磁盘组通过 CREATE SYSTEM DISKGROUP ASMDISK……语法创建。DCRV 磁盘组名称为 SYS，系统自动生成，不需要用户指定。

ASM 磁盘组通过 CREATE DISKGROUP diskgroup_name……语法创建。

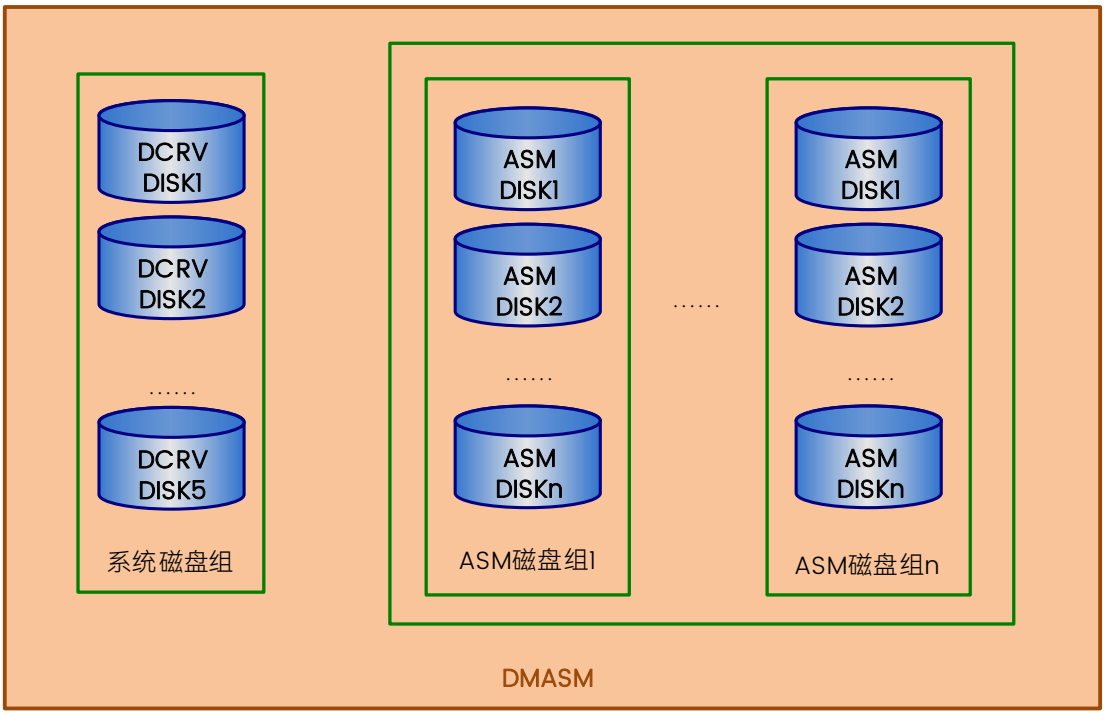


图 11.3 DMASM 镜像系统中磁盘组的结构

ASM 磁盘（ASM DISK）

ASM 磁盘是组成 ASM 磁盘组的基本组成单元。ASM 磁盘可以是一块完整的磁盘，也可以是磁盘的某一块分区。所有的 ASM 磁盘都需要在指定的搜索路径下面。

创建 ASM DISK 时可以为 ASM DISK 指定 ASM 磁盘名称。在操作系统层，同一块 ASM Disk 在不同机器需要被绑定为相同路径。

ASM 磁盘组（ASM DISK GROUP）

ASM 磁盘组可以联机添加或者删除某块磁盘。添加磁盘后，用户可以通过 DMASMTOOLM 执行指定命令发起 AU 重平衡，DMASM 系统会将 ASM 文件有效数据在各磁盘中重新分布；删除磁盘（或者磁盘故障）后，DMASM 系统会定期检查磁盘组，通过 AU 重平衡恢复丢失的数据。整个 AU 重分布的过程不影响 DMDSC 集群环境正常使用。当

软硬件资源充裕的情况下，可通过调大重平衡并行度来提高执行重平衡的速度。

一个 ASM 磁盘组内包含多块 ASM 磁盘，是 ASM 文件系统的基本组成元素。每个 ASM 磁盘组都是自描述的，包含运行 ASM 文件系统所需的所有信息；DMASMSVRM 实例自动扫描指定路径下所有可访问的磁盘，解析磁盘头格式，加载符合 ASM 磁盘组特征的磁盘，进一步解析得到磁盘组内日志信息、文件信息、文件数据映射等信息。

ASM 文件系统支持多个 ASM 磁盘组。一个 ASM 磁盘组内支持多个故障组；ASM 磁盘个数有限的场景，应该尽量保证磁盘组内有多个故障组（至少 3 个），每个故障组内有多个磁盘（至少 2 个），建议磁盘组一般不超过两个。

系统磁盘（DCRV DISK）

系统磁盘用于存放镜像系统的系统信息。每块 DCRV 磁盘均有一份 DCR 文件和一份 VTD 文件。无故障情况下，每个磁盘上的 DCR 文件和 VTD 文件均相同。数据写入 DCR 文件和 VTD 文件时，会向所有 DCRV 磁盘同时写入，只有超过半数的 DCRV 磁盘写入成功才能最终写入成功，否则写入失败。读取 DCR 文件或 VTD 文件时，只有从超过半数 DCRV 磁盘读取到一致内容才能最终读取到数据，否则读取失败。

系统磁盘组（DCRV GROUP）

DCRV 系统磁盘组是专供 DMDSC 集群使用的，与其余 ASM 磁盘组不同。DCRV 磁盘组专用于存储 DCR 文件和 VTD 文件。用户既不能删除 DCRV 磁盘组，也不能在系统磁盘组内创建文件、修改系统磁盘组属性、修改 DCR 文件和 VTD 文件。仅可以读取 DCR 文件和 VTD 文件。DCRV 磁盘组仅以 ASM 磁盘组的形式加载到 ASM 文件系统中，实现机制（读写一致与故障容错不同）与 ASM 磁盘组并不相同。

DCRV 系统磁盘组可以由 1~5 块 DCRV 磁盘组成。系统磁盘组提供联机增删磁盘的功能，每次仅允许增删一块磁盘。该命令虽然同样由 ASMTOOLM 发起，但实际由主 DMCSS 控制，DMCSS、DMASMSVRM 与 DMSERVER 协同完成。

DCRV 系统磁盘组在 ASM 文件系统内是一个单副本磁盘组，每块磁盘分别属于单独的

故障组，每份 DCR 文件或 VTD 文件都是一个单独的 ASM 文件。

数据分配单元（ALLOCATION UNITS）

ASM DISK GROUP 的每块磁盘按同一大小分为一个个 AU，创建 DISK GROUP 时可以指定 AU 大小（1、2、4、8、16、32、64M）。创建完 DISK GROUP，AU SIZE 不能更改。不同 DISK GROUP 允许使用不同 AU SIZE。

AU 分为描述 AU、INODE AU 和数据 AU 三种。

描述 AU 由多个描述 AU 项组成。每一个描述 AU 项存储一个 INODE AU 或一个数据 AU 的元数据。元数据包括 AU 所属文件 ID，AU 的前一位 AU，AU 的后一位 AU、其他副本地址等。

INODE AU 由多个 INODE 项组成。每一个 INODE 项存放一个 ASM 文件的元数据。元数据包括文件完整路径、大小、副本数、条带化大小、创建时间等信息。

数据 AU 用于存储用户数据。

DCR 磁盘（DCR DISK）

DM 集群注册表（DM Clusterware Registry，简称 DCR）磁盘专门用于存储 DCR 文件。DCR 文件记录了存储、维护集群配置的详细信息。整个集群环境共享 DCR 磁盘信息，包括集群（DMDSC、DMASM、DMCSS）资源、实例名、监听端口、集群中故障节点信息等。DCR 必须存储在集群中所有节点都可以访问到的共享存储中，并且只支持保存在 DMASM 文件系统管辖范围之外的共享存储上。在一个集群环境中只能配置一个 DCR 磁盘。

ASM 文件

在 DMASM 镜像环境中创建 ASM 文件时，可以指定文件副本数（单副本、两副本或三副本），在出现某块磁盘故障时，也可以保证数据完整。

在 DISK GROUP 里面创建 ASM 文件，系统会在 INODE AU 为该文件分配一个 INODE 项存放 ASM 文件元数据信息，在数据区分配多个数据 AU 存放 ASM 文件数据。一个 ASM 文件的所有 AU 都在同一 DISK GROUP 内，不会跨 DISK GROUP。

文件镜像（MIRRORING）

文件镜像是一种文件存储形式。将一个磁盘上的数据在其他磁盘上存储一份完全相同的副本，这种存储方式即为镜像。镜像是采用数据冗余的方式来保证数据的安全性。

故障组（FAILURE GROUPS）

故障组专门用来存放 ASM 文件的不同镜像副本。将在 11.3.6 故障组中详细介绍。

DMASM 文件系统初始化工具（DMASMCMDM）

专门用于 DMASM 镜像环境。将在 11.5 DMASM 镜像主要部件中详细介绍。

DMASM 服务器（DMASMSVRM）

专门用于 DMASM 镜像环境。将在 11.5 DMASM 镜像主要部件中详细介绍。

DMASM 应用程序访问接口（DMASMAPIM）

专门用于 DMASM 镜像环境。将在 11.5 DMASM 镜像主要部件中详细介绍。

DMASM 管理工具（DMASMTOOLM）

专门用于 DMASM 镜像环境。将在 11.5 DMASM 镜像主要部件中详细介绍。

11.3 DMASM 镜像关键技术

为了帮助用户更好的理解、使用 DMASM 镜像，本节从 ASM 磁盘与文件管理、DMASM REDO 日志、AU 描述项等方面介绍 DMASM 镜像原理。

11.3.1 磁盘与文件管理

DMASMCMDM 将物理磁盘格式化后，变成可识别可管理的磁盘。再通过磁盘组将一个或者多个格式化后的磁盘整合成一个整体提供文件服务。

磁盘被格式化后会被逻辑划分为一个个 AU。AU 是管理 DMASM 文件系统中磁盘的基本单位，ASM 文件的最小分配单位也是 AU。这些逻辑划分的 AU 根据其用途可以分为描述 AU、INODE AU 和数据 AU。

在 DMASM 文件系统中，一个磁盘中可有多个描述 AU，一个描述 AU 可管理多个 INODE AU 和数据 AU。在不同的场景中，每个描述 AU 管理的 INODE AU 和数据 AU 数量和顺序均不相同。下图展示了 DMASM 镜像环境下一个磁盘的逻辑结构，其中 desc、inode 和 data 分别为描述 AU、INODE AU 和数据 AU，MAX 为一个描述 AU 管理的最大磁盘空间。

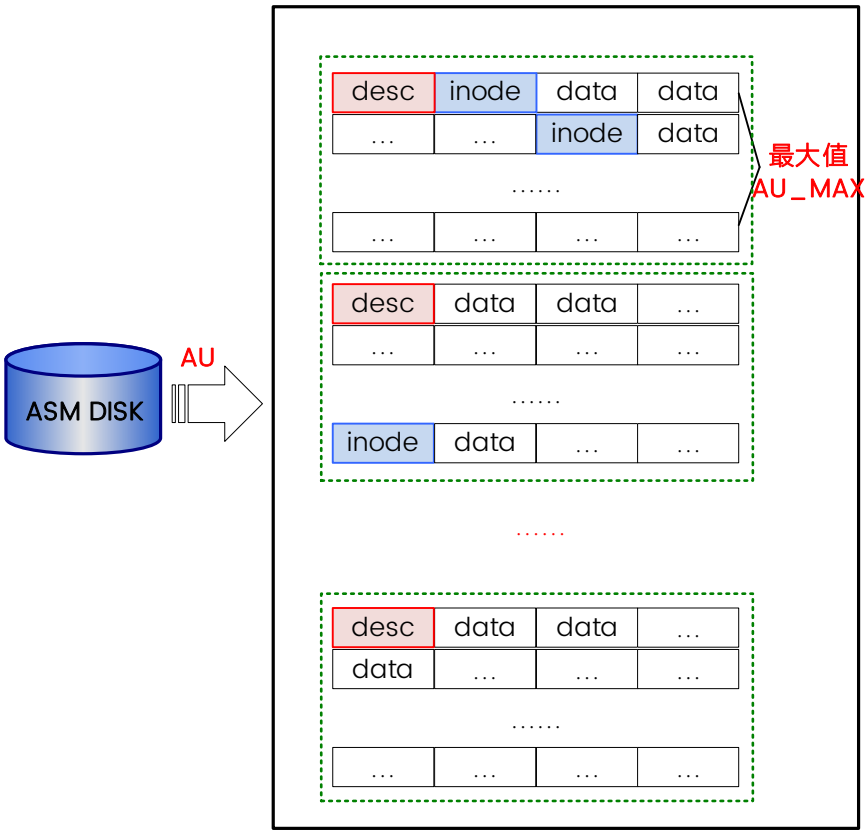


图 11.4 一个 DMASM 镜像磁盘的逻辑结构

创建、删除 ASM 文件的操作，在 DMASM 系统内部被转换成修改、维护 INODE AU 的具体动作。扫描全局的 INODE AU 链表就可以获取到磁盘组上所有的 ASM 文件信息。

11.3.2 DMASM REDO 日志

DMASM 采用重做日志机制，保证在各种异常（比如系统掉电）情况下数据不被损坏。创建、删除、扩展、截断（可以称为 DDL 操作）ASM 文件过程中，所有针对 DMASM 描述 AU、INODE AU 的修改，都会生成 REDO 日志，并且在描述 AU、INODE AU 的修改写入磁

盘之前，必须确保 REDO 日志已经写入磁盘。DMASM 中，只有针对描述 AU 和 INODE AU 的修改会产生 REDO 日志，用户修改数据 AU 的动作并不会产生 REDO 日志。

DMASM 所有 DDL 操作都是串行执行的，并且在操作完成之前，会确保所有修改的描述 AU、INODE AU 写入磁盘；DDL 操作过程中出现异常时，如果 REDO 日志尚未写入磁盘，则当前操作对系统没有任何影响；如果 REDO 日志已经写入磁盘，那么重新启动后，系统会重演 REDO 日志，修改描述 AU 和 INODE AU，将此次 DDL 继续完成。

11.3.3 AU 映射表

创建 ASM 文件后，用户操作 ASM 文件的一般流程是：调用 ASM 文件的 OPEN、READ、WRITE 接口；打开 ASM 文件并获取一个句柄，再使用这个句柄从文件的指定偏移读取或者写入数据。用户在使用 DMASM 的过程中，只需要获取一个 ASM 文件句柄，并不需要知道数据最终保存在物理磁盘的什么位置。

DMASM 使用 AU 映射表机制维护 ASM 文件与物理磁盘的映射管理，访问 ASM 文件时，根据文件句柄、文件偏移信息，通过 AU 映射表可以快速获取到数据对应的物理磁盘地址。

由于 DMASM 并不缓存任何用户数据，与直接读、写块设备相比，ASM 文件的读、写操作仅仅增加了获取 AU 映射表的代价，而这个代价与 IO 相比几乎可以忽略，因此，使用 DMASM 并不会引起读、写性能的下降。

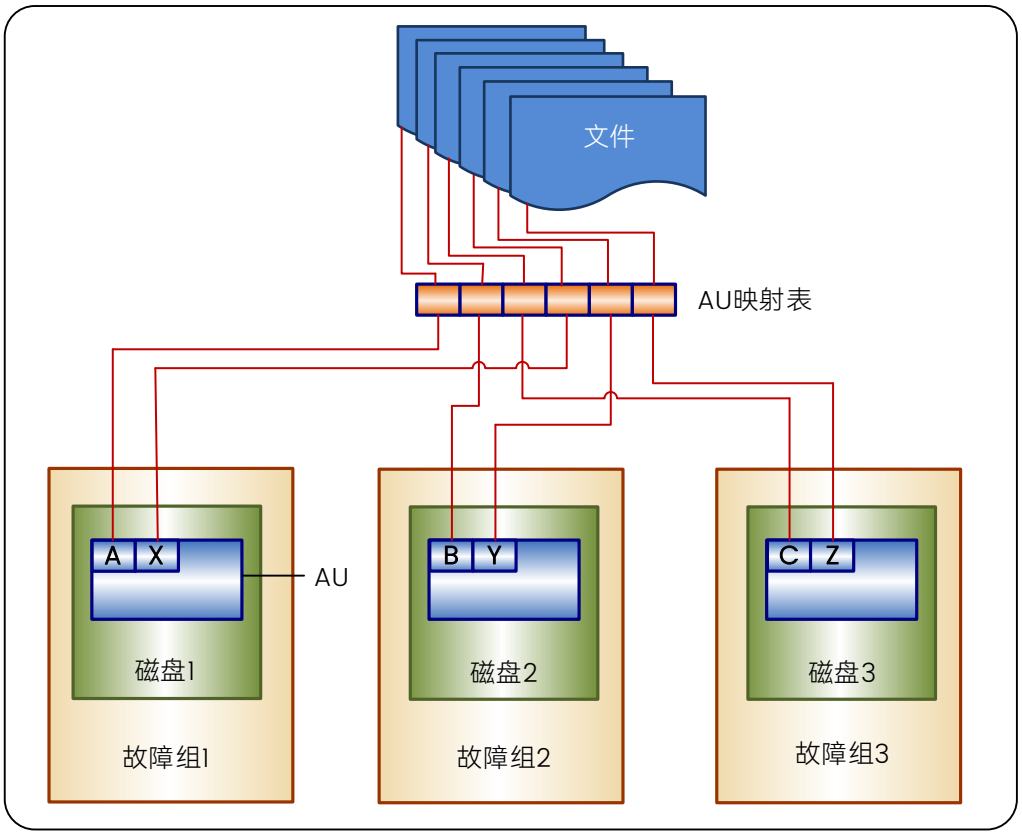


图 11.5 AU 映射表示意图

11.3.4 条带化

ASM 文件条带化技术是将一块连续的数据按照条带化粒度分割成多个数据块，并把它们分别存储到不同的磁盘中。

条带化分为两种类型：粗粒度条带化和细粒度条带化。条带化粒度是执行条带化的重要参数，取值 0、32、64、128 和 256，单位 KB。

- 数据文件的条带化粒度通过 DMINIT.INI 文件的 DATA_STRIPING 参数进行配置。
- 控制文件的条带化粒度固定为粗粒度，无需用户指定。
- 联机日志的条带化粒度通过 DMINIT.INI 文件的 LOG_STRIPING 参数进行配置。
- 归档日志的条带化粒度通过 DMARCH.INI 文件的 ARCH_ASM_STRIPING 参数进行配置。

- ASM 文件的条带化粒度在创建 ASM 文件时指定。
- IOTEST 临时文件的条带化粒度在执行 IOTEST 命令测试 ASM 环境下磁盘读写速度时指定。

条带化粒度取值 0 表示粗粒度。粗粒度条带是将文件按 AU 大小分割为一个个数据块。下图展示了粗粒度条带化分割示意图，其中以 AU 大小 1MB，文件大小 3MB 为例，文件被分为 3 个块，3 个块均匀地分布在磁盘组的三个磁盘上。

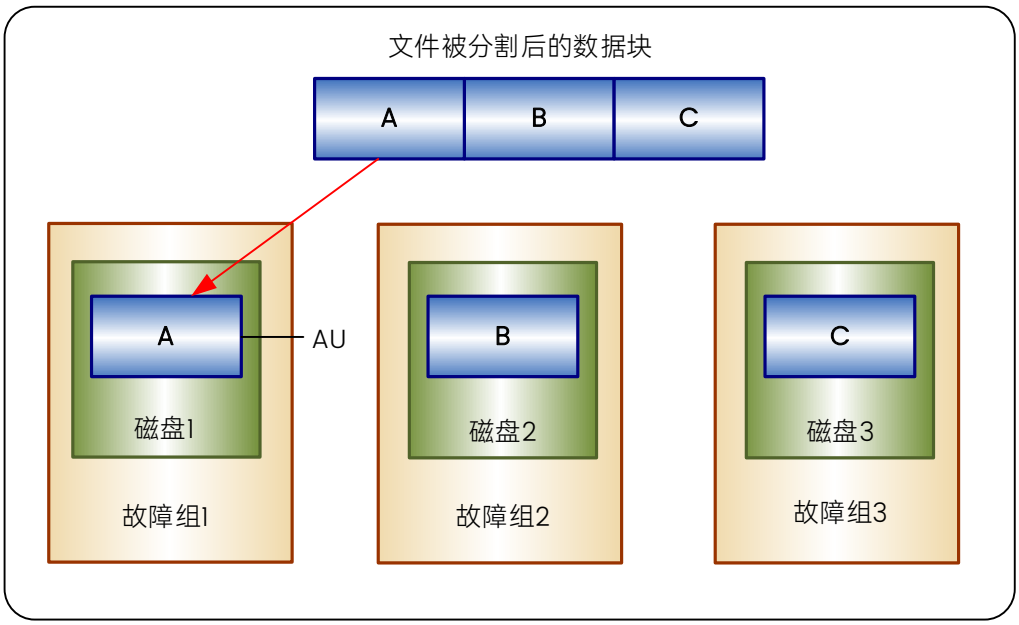


图 11.6 粗粒度条带化示意图

条带化粒度取值 32~256 表示细粒度。细粒度条带是将数据按照条带化粒度大小分割为一个个数据块。下图展示了细粒度条带化的分割示意图，其中以 AU 大小 1MB，文件大小 192KB，条带化粒度取值 32K 为例，文件被分为 6 个块，6 个块均匀地分布在磁盘组的三个磁盘上。

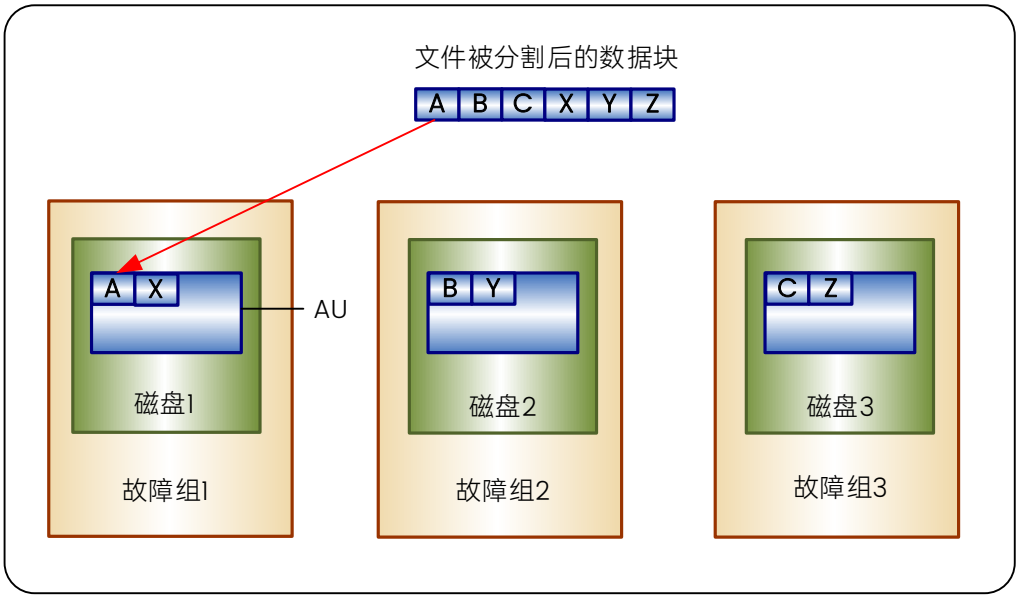


图 11.7 细粒度条带化示意图

条带化的性能与使用的共享存储紧密相关，条带化粒度需要根据实际情况配置。一般情况下，推荐数据文件使用 32K 的粒度，联机日志、归档日志等使用 256K 的粒度。粗粒度条带化基本等效于没有条带化，连续分配文件空间的情况。细粒度条带化有合并读写功能，对于大量内容的读写不是逐条带进行的，而是将同一 AU 的内容合并后一次完成的。因此，对于大量顺序读写的文件，也可以使用细粒度条带化。细粒度条带化提升性能的原理是将 IO 分散到多块磁盘，同时利用多块磁盘的性能。如果磁盘组内磁盘较少，不存在分散 IO 利用更多磁盘的条件，推荐使用粗粒度条带化。细粒度条带化提升的是 IO 性能，磁盘本身性能较高（SSD 等）且使用中数据库 IO 量较小的情况，条带化对性能没有影响。

使用 ASM 文件条带化将数据均匀分布到 Disk Group 内的磁盘中以后，多个用户同时访问数据的不同部分时，分别向多个不同的 AU 发出请求，由于这些 AU 存储在不同的磁盘上，因此不会造成磁盘冲突，可以获得最大程度的 IO 并行能力，从而提高数据库操作性能。

在 DMASM 镜像中，不建议用户将一块共享存储逻辑划分成多块 ASM 磁盘，进而使用条带化和文件镜像功能。因为一块共享存储逻辑划分成的多块磁盘并不能利用条带化和文件镜像的优势，无法并行地利用多块共享存储的 IO，反而可能破坏共享存储在顺序读写方面

的优化能力，降低 IO 速度；只有相互独立的物理磁盘才能发挥出优势。如果用户需要在逻辑划分成的多块磁盘上使用 DMASM 镜像，请将所有文件配置为粗粒度条带化。

11.3.5 文件镜像

为了保障数据安全性，可采用文件镜像的方式对文件进行存储，即将待存储的文件数据分割成大小相同的多个数据块，再将每个数据块按镜像类型指定的副本数分别存储在不同的磁盘上。数据镜像是一种高级的冗余技术，通过增加数据冗余来提高数据安全性。

文件镜像类型有三种：单副本、两副本和三副本。在创建文件的语法中分别用 EXTERNAL、NORMAL、HIGH 表示。单副本表示外部冗余（EXTERNAL），不提供任何镜像，完全依靠数据外部的磁盘高度可靠来保证数据的安全。双副本表示正常冗余（NORMAL），提供双向镜像，可通过副本较好的保证数据完整性。三副本表示高度冗余（HIGH），提供三向镜像，能够通过副本更好的保证数据的完整性。

多个镜像副本中只有一个作为主镜像副本对外提供服务，其余均作为从镜像副本。当主镜像副本发生故障后，系统会从镜像副本中重新自动挑选一个继续提供服务。

下图展示一个 ASM 文件经过细粒度的条带化技术分割成多个数据块 ABC 之后，按照两副本的镜像类型分别存储在不同的磁盘中。

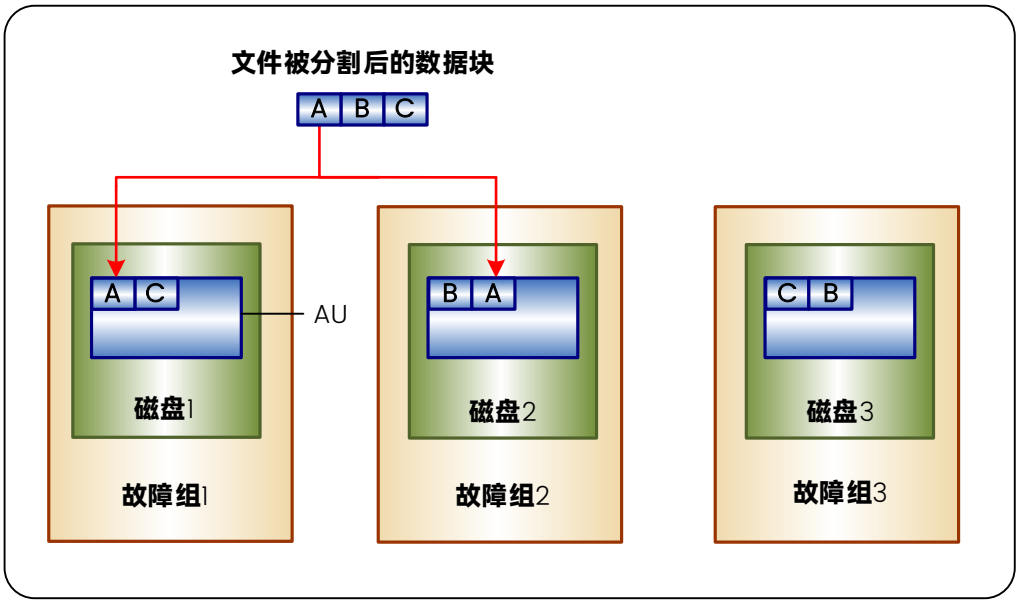


图 11.8 两副本的文件镜像示意图

不同类型的文件指定镜像类型的方式不同。文件镜像类型指定的副本数需要小于等于待存储的磁盘组的副本数，否则创建不成功。

- 数据库文件（控制文件、数据文件和日志文件）的镜像类型通过 DMINIT.INI 文件的 CTL_MIRROR、DATA_MIRROR 和 LOG_MIRROR 参数进行配置。其中日志文件包含联机日志和归档日志。
- 磁盘组元数据文件的镜像类型在创建磁盘组时指定。磁盘组元数据是指描述 AU、INODE AU 和 DMASM REDO 日志的相关信息。
- ASM 文件的镜像类型在创建 ASM 文件时指定。
- 拷贝之后 ASM 文件的镜像类型在执行 CP 命令时指定。
- IOTEST 临时文件的镜像类型在执行 iotest 命令测试 ASM 环境下磁盘读写速度时指定。

11.3.6 故障组

一个故障组是由多个磁盘组成的集合。一个磁盘组中可包含 1 个或多个故障组。此处的

磁盘均为经过 DMASMCMDM 工具创建的磁盘，并非普通的裸盘。一个故障组只能属于一个磁盘组。

故障组专门用来存放文件的不同镜像副本。故障组以 AU 为基本单位来存储不同的镜像副本。故障组中的一个 AU 可存储多个数据副本。为了数据的安全，系统会将同一个数据块的多个副本分别存储在不同的故障组中。

故障组的数量在创建 ASM Disk Group 时指定。创建两副本磁盘组时，至少需要两个故障组；创建三副本磁盘组时，至少需要三个故障组。如果只创建了一个故障组，则不能使用多副本功能。

下图展示了一个磁盘组中的故障组示意图。一个磁盘组中含有 6 块磁盘、三个故障组。两个 ASM 文件条带化粒度假定均为 32K。两副本 ASM 文件被条带化技术分割为 A、B、C、D、E 五个数据块；单副本 ASM 文件被条带化技术分割为 X、Y 二个数据块。

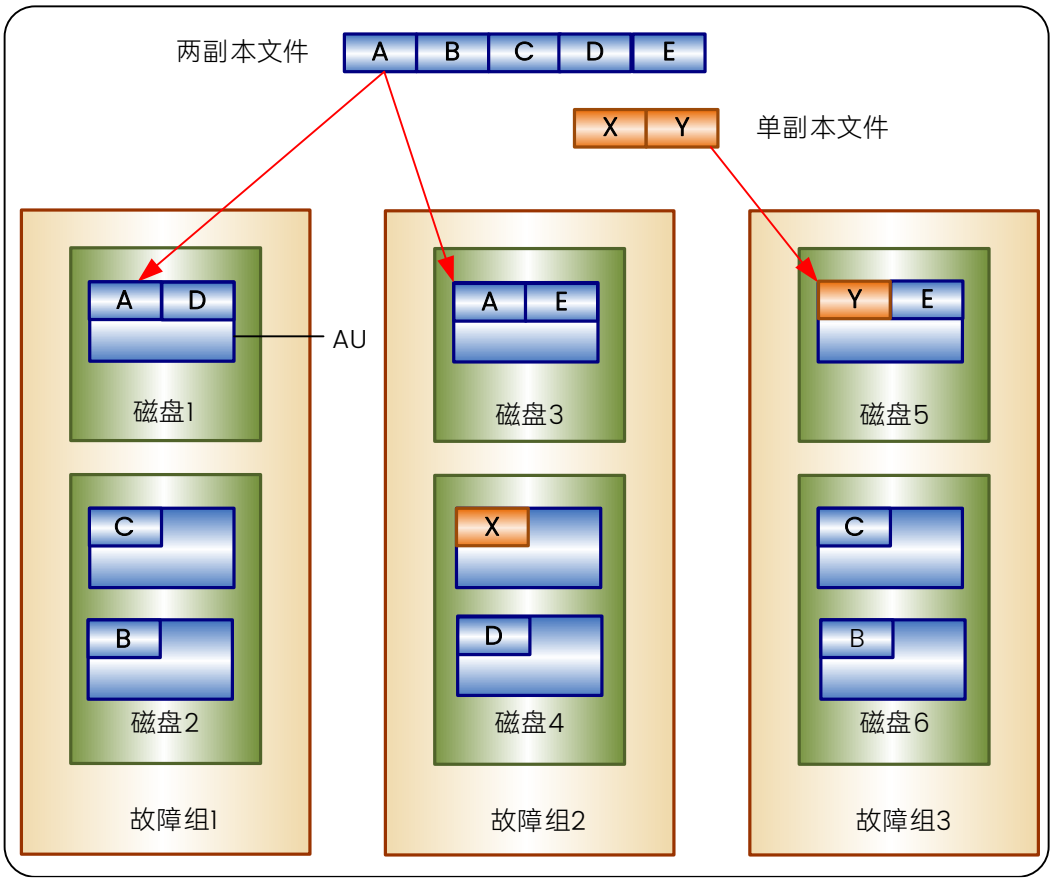


图 11.9 一个磁盘组中的故障组示意图

11.3.7 数据重分布

手动删除 ASM 磁盘或者物理磁盘故障可能导致 ASM 文件镜像数据丢失。在磁盘空间足够，且开启自动重平衡（DMDCR_AUTO_RBL_TIME）的条件下，系统会自动为 ASM 文件重新分配 AU 并补齐镜像数据。也可以手动触发故障磁盘重平衡，从而立即尝试恢复 ASM 文件丢失的镜像数据。当新添加磁盘时，用户可以手动执行命令发起磁盘空间重平衡，使磁盘使用率整体上是均衡的。

11.3.8 伙伴磁盘

伙伴磁盘用于限制磁盘 AU 的镜像副本在磁盘组中的分布，AU 的镜像副本仅分布在该 AU 所在磁盘的伙伴磁盘上。

伙伴磁盘具有如下性质（A、B、C 代指磁盘）：

1．A 是 B 的伙伴，则 B 一定也是 A 的伙伴。

2．A 和 B 是伙伴，B 和 C 是伙伴，A 和 C 不一定是伙伴。

3．伙伴磁盘与原磁盘位于不同故障组。

一般情况下，一个磁盘最多有 7 个伙伴磁盘，系统预留了第 8 个伙伴用于实现磁盘替换等功能。在创建磁盘组、添加磁盘时，系统会自动为新磁盘分配伙伴，添加磁盘也可以手动指定伙伴。日常维护时，允许手动增加磁盘伙伴关系。

11.3.9 磁盘状态

磁盘状态是磁盘的基本属性，表明了 DMASM 使用该磁盘的限制和该磁盘操作的限制。

磁盘状态有如下 5 种：

正常（NORMAL）：最基础的状态，磁盘加入磁盘组后未发生过故障或删除时就处于该状态。DMASM 会读写该磁盘，会分配、释放该磁盘的 AU。允许删除、替换该磁盘。

离线（OFFLINE）：磁盘发生故障后的状态，表示该磁盘发生故障不久可以通过重连快速

恢复正常。DMASM 不会读写该磁盘，仅会释放该磁盘的 AU。允许删除、替换、重连，不允许故障重平衡该磁盘。

重连中（RECONNECT）：重连操作过程中磁盘的状态。DMASM 会读写该磁盘，仅会释放该磁盘的 AU。不允许删除、替换、重连、故障重平衡该磁盘。

故障（FAIL）：磁盘处于离线状态超过指定时间后的状态，表示该磁盘彻底故障，对应的物理磁盘无法重连回磁盘组。DMASM 不会读写该磁盘，仅会释放该磁盘的 AU。允许替换、故障重平衡，不允许删除、重连该磁盘。

已删除（DELETED）：手动删除的磁盘的状态，类似故障状态。DMASM 不会读写该磁盘，仅会释放该磁盘的 AU。允许替换、故障重平衡，不允许删除、重连该磁盘。

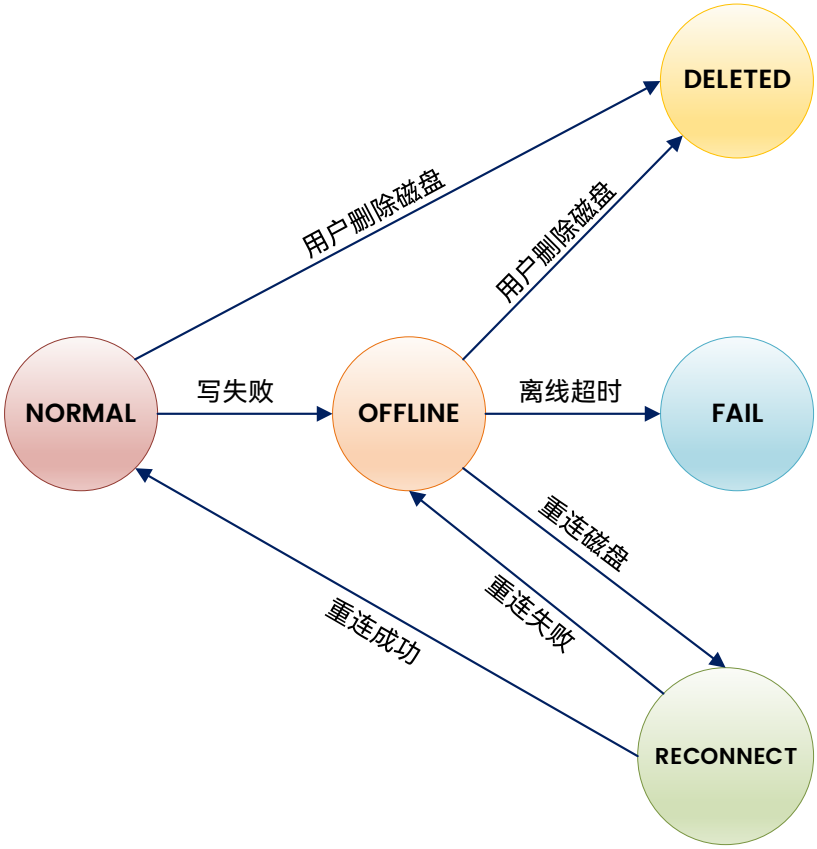


图 11.10 磁盘状态转换示意图

11.3.10 系统磁盘组在 DMASM 中的映射

系统磁盘组与普通的磁盘组不同，虽然也遵循 DMASM 的物理存储格式，但不由

DMASM 管理和使用。系统磁盘组以特殊的方式映射到 DMASM 文件系统中，通过 DMASM 提供有限的功能：

1. 读取、导出每副本 DCR、VTD 的内容。系统磁盘组仅支持读取，且不存储任何用户数据。
2. 查询 DCRV 磁盘的详细信息。DCRV 磁盘仅有 NORMAL、UNUSED、FAILED 共 3 种状态，对应正常、未使用、故障的磁盘。
3. 联机增删 DCRV 磁盘。该功能以 DMASM 为入口，由 CSS 协调 DMASMSVRM、DMSERVER 共同完成。

11.4 DMASM 镜像技术指标

本节主要介绍一下 DMASM 中的一些重要技术指标。

表 11.2 DMASM 技术指标

项	大小	说明
AU 大小（AU_SIZE）	支持 1、2、4、8、16、32、64	一个 AU 占用的存储空间。单位 M
AU 描述项大小	占用物理存储空间为 32 字节； 占用共享内存空间为 64 字节	一个 AU 描述项占用 32 字节的物理存储空间；加载到共享内存中占用 64 字节
一个描述 AU 管理的最大 AU 数	AU_SIZE*1024*1024 / 32	以 AU_SIZE=4 为例，一个描述 AU 最多可以管理 131072 个 AU
一个描述 AU 管理的最大磁盘空间（假设为 AU_MAX）	AU_SIZE*1024*1024 / 32 * AU_SIZE	以 AU_SIZE=4 为例，一个描述 AU 可以管理的最大磁盘空间为 524288MB，即 512GB
INODE 项大小	512 字节	一个 ASM 文件描述项大小，每个 ASM 文件对应着一个文件描述项

INODE AU 可管理的最大文件数	AU_SIZE*1024*1024 / 512	以 AU_SIZE=1 为例，一个 INODE AU 最多可以管理 2048 个文件
ASM 文件最小尺寸	AU_SIZE	每个 ASM 文件最少包含一个 AU，占用一个 AU_SIZE 大小的空间。目录只分配 INODE 项不分配 AU，不占用磁盘空间
ASM 文件最大尺寸	0X1000000 * AU_SIZE	一个 ASM 文件最多可以包含 0X1000000 个 AU，以 AU_SIZE=4 为例，理论上单个 ASM 文件的最大尺寸是 67108864MB，即 64TB
一个用户连接可以同时打开的 ASM 文件个数	65536	一个用户连接，最多打开 65536 个 ASM 文件
ASM 文件数上限	65532	一个磁盘组，最多可以创建 65532 个 ASM 文件
ASM 磁盘组个数上限	125	一个 DMASM 中可创建 1~125 个磁盘组
一个 ASM 磁盘组中的磁盘个数上限	8192	一个磁盘组中可包含 1~8192 个 ASM 磁盘
DCRV 磁盘组个数上限	1	一个 DMASM 镜像中只能有一个 DCRV 磁盘组
一个 DCRV 磁盘组中 DCRV 磁盘个数上限	5	一个 DCRV 磁盘组可包含 1、3 或 5 个 DCRV 磁盘

每 100M 共享内存大小能管理的磁盘大小上限	100*1024*1024 / 64 * AU_SIZE	每个 AU 描述项占用 64byte 共享内存空间，以 AU_SIZE=1 为例，100M 共享内存能管理的磁盘大小上限为 1638400MB，约 1.56TB。由于共享内存中还包含一些额外的控制信息，因此 100M 共享内存实际能管理的磁盘空间不足 1.56TB

11.5 DMASM 镜像主要部件

DMASM 镜像的主要部件包括：提供存储功能的共享存储、初始化工具 DMASMCMDM、服务器 DMASMSVRM、应用程序访问接口 DMASMAPIM、文件系统管理工具 DMASMTOOLM。

11.5.1 DMASMCMDM

DMASMCMDM 是 DMASM 镜像系统初始化工具，用来创建 ASM 磁盘和 DCRV 系统磁盘组等。

DMASMCMDM 工具用法分两种：一是在 DMASMCMDM 工具中使用脚本文件，脚本文件中存放一条或多条 DMASMCMDM 命令。二是在 DMASMCMDM 工具中使用 DMASMCMDM 命令。

11.5.1.1使用脚本文件

11.5.1.1.1 查看 DMASMCMDM HELP

通过 DMASMCMDM HELP 可查看 DMASMCMDM 工具如何使用脚本文件。

```
d:\dmdbms\bin>DMASMCMDM HELP
```

```
dmasmcmdm V8

version: 03134284368-20250430-272000-20149 Pack1

书写格式: dmasmcmdm KEYWORD=<value> {KEYWORD=<value>}

        其中{}表示大括号内的关键词可为 0~N 个，N 为正整数，{}本身无需出现在语句中

使用示例: dmasmcmdm SCRIPT_FILE=asmcmd.txt

--------------------------------------------------------------------------------

关键词（KEYWORD）        说明                                          缺省值      是否必选项

--------------------------------------------------------------------------------

SCRIPT_FILE       asmcmd 脚本文件路径                                NOTHING       OPTIONAL

RET_FLAG     执行 asmcmd 脚本文件时，出错是否立即返回。0：忽略，1：返回    0     OPTIONAL

HELP              打印帮助信息                                       NOTHING       OPTIONAL

--------------------------------------------------------------------------------
```

11.5.1.1.2 脚本文件用法

通过 SCRIPT_FILE 参数来指定脚本文件。

语法如下:

```
dmasmcmdm SCRIPT_FILE=filepath  [RET_FLAG=value]
```

参数说明:

SCRIPT_FILE：用于指定 asmcmd 脚本文件路径。脚本文件（例如 asmcmd.txt）中存放一条或多条 DMASMCMDM 命令。用户没有通过 SCRIPT_FILE 指定脚本文件，则 DMASMCMDM 进入交互模式运行，逐条解析、运行命令。用户通过 SCRIPT_FILE 指定脚本文件，则以行为单位读取文件内容，并依次执行，执行完成后，自动退出 DMASMCMDM 工具。脚本文件必须以"#asm script file"开头，否则认为是无效脚本文件；脚本中其他行以"#"开头表示注释。

RET_FLAG：用于设置执行 ASMCMD 脚本文件时，出错是否立即返回。取值 0 或 1。0 不返回，即执行出错时忽略当前出错的行，继续执行下一条；1 返回，即执行出错时立即停止，并返回一个错误码。缺省值为 0。

举例说明：

第一步，创建脚本文件 asmcmd.txt。例如脚本文件内容如下：

```
create dcrvdisk '/dev_DSC_HDD/DCRV1' 'dcrv1'

create dcrvdisk '/dev_DSC_HDD/DCRV2' 'dcrv2'

create asmdisk '/dev_DSC_HDD/MIR_HDD_1'   'data1'

create asmdisk '/dev_DSC_HDD/MIR_HDD_2'   'data2'
```

第二步，通过使用 SCRIPT_FILE 参数指定脚本，执行出错立即停止并返回一个错误码。

```
./dmasmcmdm SCRIPT_FILE=/opt/dmdbms/script/asmcmd.txt RET_FLAG=1
```

11.5.1.2 使用 DMASMCMDM 命令

11.5.1.2.1 查看 ASM HELP

在 DMASMCMDM 工具中输入 HELP，可查看 DMASMCMDM 工具如何使用 DMASMCMDM 命令。

```
d:\dmdbms\bin>DMASMCMDM

dmasmcmdm V8

ASM>HELP

Format: create emptyfile file_path size(MB) num

Usage:   create emptyfile 'd:\asmdisks\disk0.asm' size 100



Format: create asmdisk disk_path disk_name [size(MB)]
```

Usage: create asmdisk 'd:\asmdisks\disk0.asm' 'DATA0'

Usage: create asmdisk 'd:\asmdisks\disk0.asm' 'DATA0' 100

Format: create dcrvdisk disk_path disk_name [size(MB)]

Usage: create dcrvdisk 'd:\asmdisks\dmdcrv0.asm' 'DCRV0'

Usage: create dcrvdisk 'd:\asmdisks\dmdcrv0.asm' 'DCRV0' 100

Format: create system diskgroup asmdisk disk_path{,disk_path} attribute config=ini_path, passwd=password

Usage: create system diskgroup asmdisk 'd:\asmdisks\dmdcrv0.asm' attribute config='d:\dmdcr_cfg.ini', passwd='DCRpsd_123'

Usage: create system diskgroup asmdisk 'd:\asmdisks\dmdcrv0.asm','d:\asmdisks\dmdcrv1.asm','d:\asmdisks\dmdcrv2.asm' attribute config='d:\dmdcr_cfg.ini', passwd='DCRpsd_123'

Format: export dcrvdisk disk_path to ini_path

Usage: export dcrvdisk 'd:\asmdisks\dmdcrv0.asm' to 'd:\dmdcr_cfg.ini'

Format: import dcrvdisk ini_path to disk_path{,disk_path}

Usage: import dcrvdisk 'd:\dmdcr_cfg.ini' to 'd:\asmdisks\dmdcrv0.asm'

Usage: import dcrvdisk 'd:\dmdcr_cfg.ini' to 'd:\asmdisks\dmdcrv0.asm', 'd:\asmdisks\dmdcrv1.asm', 'd:\asmdisks\dmdcrv2.asm'

Format: extend dcrvdisk disk_path{,disk_path} from ini_path

Usage: extend dcrvdisk 'd:\asmdisks\dmdcrv0.asm' from 'd:\dmdcr_cfg.ini'

Usage: extend dcrvdisk 'd:\asmdisks\dmdcrv0.asm', 'd:\asmdisks\dmdcrv1.asm', 'd:\asmdisks\dmdcrv2.asm' from 'd:\dmdcr_cfg.ini'

Format: check dcrvdisk disk_path

Usage: check dcrvdisk 'd:\asmdisks\dmdcrv0.asm'

Format: recover dcrvdisk load_path{,load_path}

Usage: recover dcrvdisk 'd:\asmdisks\'

Usage: recover dcrvdisk 'd:\asmdisks\,d:\asmdisks2\,d:\asmdisks3\'

Format: clear dcrvdisk err_ep_arr disk_path_dir{,disk_path_dir} group_name

Usage: clear dcrvdisk err_ep_arr 'd:\asmdisks\' 'GRP_DSC'

Usage: clear dcrvdisk err_ep_arr 'd:\asmdisks\,d:\asmdisks2\,d:\asmdisks3\' 'GRP_DSC'

Format: listdisks path{,path}

Usage: listdisks 'd:\asmdisks\'

Usage: listdisks 'd:\asmdisks\,d:\asmdisks2\,d:\asmdisks3\'

Format: check_grp disk_path load_path{,load_path}

Usage: check_grp 'd:\asmdisks\dmdata0.asm' 'd:\asmdisks\'

Usage: check_grp 'd:\asmdisks\dmdata0.asm' 'd:\asmdisks\,d:\asmdisks2\,d:\asmdisks3\'

11.5.1.2.2 命令用法

当 SCRIPT_FILE 和 RET_FLAG 参数全部缺省时，即进入命令行界面。

语法如下：

```
dmasmcmdm
```

可在 DMASMCMDM 命令行界面中直接输入 DMASMCMDM 命令并执行。下面详细介绍 DMASMCMDM 工具支持的所有 DMASMCMDM 命令。

11.5.1.2.2.1 创建空磁盘文件

专门用于单机模拟 DMDSC 环境。

创建名为 xxx.asm 的空磁盘文件，用于模拟块设备，注意必须以.asm 结尾。模拟测试用，真实环境不建议使用。

语法如下：

```
CREATE EMPTYFILE file_path SIZE num
```

参数说明：

file_path：空磁盘文件路径。

num：指定空文件大小。取值为大于等于 32 的整数。单位 MB。

举例说明：

创建名为 disk0.asm 的空磁盘文件。

```
CREATE EMPTYFILE 'd:\asmdisks\disk0.asm' SIZE 100
```

11.5.1.2.2.2 创建 ASM 磁盘

创建 ASM 磁盘，即将块设备格式化为 ASM DISK，并在块设备头部写入 ASM DISK 标识信息（DMASM 标识串、ASM 磁盘名以及 ASM 磁盘大小等信息）。

语法如下：

```
CREATE ASMDISK disk_path disk_name [size]
```

参数说明：

size：指定 ASMDISK 大小。取值为大于等于 32 的整数。单位 MB。size 缺省时系统自动获取磁盘大小。

举例说明：

```
CREATE ASMDISK 'd:\asmdisks\disk0.asm' 'DATA0'

或

CREATE ASMDISK 'd:\asmdisks\disk0.asm' 'DATA0' 100
```

11.5.1.2.2.3 创建 DCRV 磁盘

创建 DCRV 磁盘，即将块设备格式化为 DCRV 磁盘，并在块设备头部写入 DCRV 标识信息。

语法如下：

```
CREATE DCRVDISK disk_path disk_name [size(MB)]
```

参数说明：

size：指定 DCRVDISK 大小。取值为大于等于 32 的整数。单位 MB。size 缺省时系统自动获取磁盘大小。

举例说明：

```
CREATE DCRVDISK 'd:\asmdisks\dmdcrv0.asm' 'DCRV0'

或

CREATE DCRVDISK 'd:\asmdisks\dmdcrv0.asm' 'DCRV0' 100
```

11.5.1.2.2.4 创建 DCRV 系统磁盘组

根据配置文件 DMDCR_CFG.INI 内容，创建 DCRV 系统磁盘组。同时设置登录 ASM 文件系统的密码，密码必须用单引号括起来。DCRV 系统磁盘组名称为 SYSTEM，系统自动生成，无需用户指定。

语法如下：

```
CREATE SYSTEM DISKGROUP ASMDISK disk_path{,disk_path} ATTRIBUTE CONFIG=ini_path,
PASSWD=password
```

参数说明：

CONFIG：DMDCR_CFG.INI 路径。

举例说明：

```
CREATE SYSTEM DISKGROUP ASMDISK '/dev/DMDATA/dmdcrv0' ATTRIBUTE
config='/home/dmdbms/config/dmdcr_cfg.ini', passwd='DCRpsd_123'
或
CREATE SYSTEM DISKGROUP ASMDISK
'/dev/DMDATA/dmdcrv0','/dev/DMDATA/dmdcrv1','/dev/DMDATA/dmdcrv2' ATTRIBUTE
config='/home/dmdbms/config/dmdcr_cfg.ini', passwd='DCRpsd_123'
```

11.5.1.2.2.5 导出 DCRV 配置文件

用户如果要查看当前 DCRV 磁盘的配置信息，可以使用 EXPORT 命令将 DCRV 磁盘内容导出到文本文件。导出过程不影响系统正常使用。

语法如下：

```
EXPORT DCRVDISK disk_path TO ini_path
```

举例说明：

```
EXPORT DCRVDISK '/dev/DMDATA/dmdcrv0' TO '/home/dmdbms/config/dmdcr_cfg1.ini'
```

11.5.1.2.2.6 导入 DCRV 配置文件

用户如果需要修改当前 DCRV 磁盘的配置信息，可以使用 IMPORT 命令修改 DCRV 磁盘内容。只有所有数据库实例都退出的情况才能使用 IMPORT 命令修改 DCRV 磁盘内容。

语法如下：

```
IMPORT DCRVDISK ini_path TO disk_path{,disk_path}
```

参数说明：

ini_path：DMDCR_CFG.INI 路径。

举例说明：

```
IMPORT DCRVDISK '/home/dmdbms/config/dmdcr_cfg.ini' TO '/dev/DMDATA/dmdcrv0'
或
IMPORT DCRVDISK '/home/dmdbms/configdmdcr_cfg.ini' TO '/dev/DMDATA/dmdcrv0',
'/dev/DMDATA/dmdcrv1', '/dev/DMDATA/dmdcrv2'
```


注意：

DCR_VTD_PATH、DCR_N_GRP、DCR_GRP_N_EP、DCR_GRP_NAME、DCR_GRP_TYPE、DCR_GRP_EP_ARR、DCR_EP_NAME、EP 所属的组类型、checksum 值，密码不可修改。必须以组为单位进行修改。

特别的，在动态增加节点时，可以并需要对 DCR_GRP_N_EP 和 DCR_GRP_EP_ARR 进行修改

11.5.1.2.2.7 联机修改 DCRV 磁盘

联机修改 DCRV 磁盘，增加节点，会将新增节点信息写回 DCRV 磁盘。支持同时修改多个 DCRV 磁盘。

语法格式：

```
EXTEND DCRVDISK disk_path{,disk_path} FROM ini_path
```

参数说明：

ini_path：DMDCR_CFG.INI 路径。

举例说明：

```
EXTEND DCRVDISK '/dev/DMDATA/dmdcrv0' FROM '/home/dmdbms/config/dmdcr_cfg.ini'

或

EXTEND DCRVDISK '/dev/DMDATA/dmdcrv0', '/dev/DMDATA/dmdcrv1', '/dev/DMDATA/dmdcrv2'

FROM '/home/dmdbms/config/dmdcr_cfg.ini'
```

11.5.1.2.2.8 校验 DCRV 磁盘

校验 DCRV 磁盘。

语法如下：

```
CHECK DCRVDISK disk_path
```

参数说明：

disk_path：磁盘的路径。

举例说明：

```
CHECK DCRVDISK 'd:\asmdisks\dmdcrv0.asm'
```

11.5.1.2.2.9 离线修复 VTD 文件

如果因为强杀等原因造成 VTD 文件副本不一致，导致 DMDSC 无法启动，那么可使用此命令将最新的 VTD 副本同步到其余副本，使 VTD 恢复到一致的状态。前提要求 DMCSS、DMASM、DMSERVER 全部退出，且超过半数 VTD 副本可以正常读写。

如果某一块 VTD 的所有副本全部被破坏，使用此命令则会将该块内容清零。清零之后 DMDSC 可能仍然无法启动，那么需要使用 DMASMCMDM 重新初始化 VTD 文件。

语法如下：

```
RECOVER DCRVDISK load_path{,load_path}
```

参数说明：

load_path：DCRV 磁盘所在路径。

支持指定多路径，各路径以英文逗号','进行分隔，所有配置的路径都必须为有效路径，最多可以同时配置 8 个路径，且暂不支持中文路径。

举例说明：

```
RECOVER DCRVDISK '/dev_data/asmdisks/'

RECOVER DCRVDISK '/dev_data/asmdisks0/,/dev_data/asmdisks1/,/dev_data/asmdisks2/'
```

11.5.1.2.2.10 清理 DCRV 中指定组的故障节点信息

清理 DCRV 中指定组的故障节点信息。

语法如下：

```
CLEAR DCRVDISK ERR_EP_ARR disk_path_dir{,disk_path_dir} group_name
```

参数说明：

disk_path_dir：磁盘的搜索路径。支持指定多路径，各路径以英文逗号','进行分隔，所有配置的路径都必须为有效路径，最多可以同时配置 8 个路径，且暂不支持中文路径。

group_name：磁盘组名。

举例说明：

```
CLEAR DCRVDISK ERR_EP_ARR 'd:\asmdisks\' 'GRP_DSC'

CLEAR DCRVDISK ERR_EP_ARR 'd:\asmdisks0\,d:\asmdisks1\,d:\asmdisks2\' 'GRP_DSC'
```

11.5.1.2.2.11 显示指定路径下的磁盘属性

显示 path 路径下面所有磁盘的信息，分为三种类型：normal disk、unused asmdisk 和 used asmdisk。normal disk 为不属于 ASM 文件系统的常规磁盘。unused asmdisk 为未使用的 ASMDISK。used asmdisk 为已使用的 ASMDISK。

支持指定多路径，各路径以英文逗号','进行分隔，所有配置的路径都必须为有效路径，

最多可以同时配置 8 个路径，且暂不支持中文路径。

语法如下：

```
LISTDISKS path{,path}
```

举例说明：

```
LISTDISKS '/dev/DMDATA'

LISTDISKS '/dev/DMDATA,/dev/DMLOG,/dev/RAW'
```



注意：如果查询模拟 ASM 磁盘的磁盘信息，对 path 路径没有限制；如果查询真实环境中共享存储路径下的磁盘信息，则 path 路径的根目录名称必须以“/dev/“开头，例如：listdisks ‘/dev/DMDATA/’或者 listdisks ‘/dev_data/DMDATA/’，否则将不会显示该路径下的磁盘信息。

11.5.1.2.2.12 校验用户创建的磁盘组数据

校验用户创建的磁盘组数据，包括元数据和用户数据。

语法如下：

```
CHECK_GRP disk_path load_path{,load_path}
```

参数说明：

disk_path：磁盘的路径。

load_path：磁盘的加载路径，为文件夹路径。支持指定多路径，各路径以英文逗号','进行分隔，所有配置的路径都必须为有效路径，最多可以同时配置 8 个路径，且暂不支持中文路径。

举例说明：

```
CHECK_GRP 'd:\asmdisks\dmdata0.asm' 'd:\asmdisks\'

CHECK_GRP 'd:\asmdisks\dmdata0.asm' 'd:\asmdisks0\,d:\asmdisks1\,d:\asmdisks2\'
```

11.5.2 DMASMSVRM

DMASMSVRM 是 DMASM 服务器，负责管理 DMASM 文件系统。每个提供 DMASM 服务的节点都必须启动一个 DMASMSVRM 服务器，这些 DMASMSVRM 服务器一起组成 DMASMSVRM 集群。DMASMSVRM 集群提供 DMASM 文件系统的全局并发控制。DMASMSVRM 集群需要和 DMCSS 集群配合使用，由 DMCSS 管理 DMASMSVRM 的启动、故障处理和节点重加入。

DMASMSVRM 启动时扫描指定路径（比如：/dev/DMDATA/）下的所有块设备，加载 ASM 磁盘，构建 ASM 磁盘组和 DMASM 文件系统。DMASMSVRM 服务器之间使用 MAL 系统进行信息和数据的传递。

DMASMSVRM 集群的启动、关闭、故障处理等流程由 DMCSS 控制，DMASMSVRM 定时向 DCRV 磁盘写入时间戳、实例状态等信息，DMCSS 控制节点定时从 DCRV 读取信息，检查 DMASMSVRM 实例的状态变化，启动响应的处理流程。

DMASMSVRM 集群中，只有一个控制节点，控制节点以外的其他节点叫做普通节点，DMASMSVRM 控制节点由 DMCSS 选取；所有 DDL 操作（比如创建、删除文件，创建、删除磁盘等）都是在控制节点执行，用户登录普通节点发起的 DDL 请求，会通过 MAL 系统转发到控制节点执行并返回；而 ASM 文件的读、写操作，则由登录节点直接完成，不需要经过 DMASMSVRM 实例。

通过 DMASMSVRM HELP 可查看 DMASMSVRM 的启动用法。

```
dmasmsvrm V8
version: 03134284389-20250411-268655-10000
书写格式: ./dmasmsvrm KEYWORD=<value> {KEYWORD=<value>}
        其中{}表示大括号内的关键词可为 0~N 个，N 为正整数，{}本身无需出现在语句中
使用示例: ./dmasmsvrm DCR_INI=/opt/data/DAMENG/dmdcr.ini
```

```
------------------------------------------------------------------------
关键词（KEYWORD）          说明                    缺省值          是否必选项
------------------------------------------------------------------------
DCR_INI                    dmdcr.ini 路径          无              是
-NOCONSOLE                 以服务方式启动          无              否
LISTEN_IPV6                是否监听 IPV6           Y               否
HELP                       打印帮助信息            无              否
------------------------------------------------------------------------
```

参数说明：

DCR_INI：DMDCR.INI 配置文件路径。DMDCR.INI 记录了 DCRV 磁盘路径、实例序列号等信息；如果不指定 DCR_INI 参数，dmasmsvrm 默认在当前路径下查找 DMDCR.INI 文件。

LISTEN_IPV6：是否监听 IPV6。


注意：

使用 DMASMTOOLM 或者 DMASMAPIM 函数 dmasmm_conect()登录 DMASMSVRM 时，用户名只能为"ASMSYS"。若为本地登录，不校验密码；若为远程登录，应使用初始化 DCR 磁盘时设置的登录 ASM 文件系统的密码。

11.5.3 DMASMAPIM

DMASMAPIM 是 DMASM 文件系统的应用程序访问接口，通过调用 DMASMAPIM，用户可以访问和操作 ASM 文件。

与达梦数据库 DPI 接口类似，访问 ASM 文件之前，必须先分配一个 conn 连接对象，并登录到 DMASMSVRM 服务器，再使用这个 conn 对象进行创建磁盘组、创建文件、删除文件、读取数据和写入数据等 DMASM 相关操作。

DMASMAPIM 接口的详细介绍请参考 19 附录。

11.5.4 DMASMTOOLM

DMASMTOOLM 是 DMASM 文件系统管理工具，提供了一套类 Linux 文件操作命令，用于管理 ASM 文件，是管理和维护 DMASM 的好帮手。DMASMTOOLM 工具使用 DMASMAPIM 连接到 DMASMSVRM，并调用相应的 DMASMAPIM 函数，实现创建、拷贝、删除等各种文件操作命令；DMASMTOOLM 还支持 ASM 文件和操作系统文件的相互拷贝。

DMASMTOOLM 可以登录本地 DMASMSVRM，也可以登录位于其他节点的 DMASMSVRM，并执行各种文件操作命令。一般建议登录本地 DMASMSVRM 服务器，避免文件操作过程中的网络开销，提升执行效率。

DMASMTOOLM 工具不支持中文输入。若文件名或路径中包含中文字符，当客户端与服务器的编码不一致时，可能出现乱码，因此建议不要使用包含中文字符的文件名或路径。



说明：在 DMASMTOOLM 工具执行命令时，如果路径名称或者文件名首字母为数字，需要给字符串加单引号’。

在 DMASMTOOLM 工具中使用 cd 命令，首字母为’+’，’\’，‘/’符号，都会从目录最上层开始查找目录。

通过 DMASMTOOLM HELP 可查看 DMASMTOOLM 的启动用法。

```
DMASMTOOLM V8

version: 03134284368-20250430-272000-20149 Pack1

书写格式: dmasmtoolm KEYWORD=<value> {KEYWORD=<value>}

        其中{}表示大括号内的关键词可为 0~N 个，N 为正整数，{}本身无需出现在语句中

使用示例: dmasmtoolm DCR_INI=d:\data\DAMENG\dmdcr.ini

--------------------------------------------------------------------------------
```

关键词（KEYWORD）	说明	缺省值	是否必选项
DCR_INI	dmdcr.ini 路径	dmdcr.ini	是
HOST	asm 服务器地址，IPV6 地址要用[]包围	无	否
PORT_NUM	asm 服务器端口号	无	否
USERID	登录 asm 服务器用户名密码（格式：ASMSYS/PASSWORD）	无	否
SCRIPT_FILE	asmtool 脚本文件路径	无	否
HELP	打印帮助信息	无	否


注意：

1.用户没有指定脚本文件，则 DMASMTOOLM 进入交换模式运行，逐条解析、运行命令；用户指定脚本文件（比如 asmtool.txt），则以行为单位读取文件内容，并依次执行，执行完成以后，自动退出 DMASMTOOLM 工具。脚本文件必须以"#asm script file"开头，否则认为是无效脚本文件；脚本中其它行以"#"开始标识注释。

2.dmasmtoolm 命令直接输入的 host/port 信息配置的是连接 dmasmsvrm 的信息，可以在 DMDCR_CFG.INI 里面找到，分别为要连接的 DMASM 节点的 DCR_EP_HOST 和 DCR_EP_PORT。

3.当 asm 服务器地址使用 IPv6 地址时，为了方便区分端口，需要用[]封闭 IP 地址。

DMASMTOOLM 支持的命令主要分三类：一是磁盘组相关命令；二是 ASM 文件相关命令；三是兼容 Linux 相关命令。

11.5.4.1 磁盘组命令

- 创建 ASM 磁盘组

语法如下：

```
CREATE DISKGROUP diskgroup_name [<EXTERNAL | NORMAL | HIGH> REDUNDANCY] <磁盘组子句>
{<磁盘组子句>}
<磁盘组子句>::= FAILGROUP  failgroup_name ASMDISK file_path{, file_path} [ATTRIBUTE
AU_SIZE=num, REDO_SIZE=num]
```

参数说明：

diskgroup_name：为磁盘组名。最长不能超过 32 个字符。

failgroup_name：为故障组名。最长不能超过 32 个字符。

EXTERNAL、NORMAL、HIGH：用于指定镜像类型，即文件的副本数。EXTERNAL 表示单副本；NORMAL 表示双副本；HIGH 表示三副本。缺省为单副本。

FILE_PATH：为磁盘路径，路径必须是全路径，不能是相对路径。全路径长度不能超过 256 字节。

ATTRIBUTE：为属性子句，用于指定的 AU_SIZE 和 DMASM REDO 日志的 REDO_SIZE 大小，以 MB 为单位。

举例说明：

创建一个名为 DMDATA3 的磁盘组，镜像类型为 3 副本（HIGH REDUNDANCY）。3 副本至少需要 3 个故障组，本例以 3 个故障组和 6 块磁盘为例，将 6 块磁盘均匀放入 3 个故障组 data1、data2、data3 中。

```
CREATE DISKGROUP 'DMDATA3' HIGH REDUNDANCY FAILGROUP 'data1' ASMDISK
'/dev/DMDATA/disk0','/dev/DMDATA/disk1' FAILGROUP 'data2' ASMDISK '/dev/DMDATA/disk2',
'/dev/DMDATA/disk3' FAILGROUP 'data3' ASMDISK '/dev/DMDATA/disk4','/dev/DMDATA/disk5'
```

```
ATTRIBUTE AU_SIZE=4, REDO_SIZE=128
```

创建一个名为 DMDATA1 的磁盘组，镜像类型为 1 副本（EXTERNAL REDUNDANCY）。1 个副本可以使用 1 个或多个故障组，磁盘数可以是 1 个或多个，数据会自动均匀地分布在各磁盘中。本例以 1 个故障组和 2 块磁盘为例，将 2 块磁盘放入故障组 data1 中。

```
CREATE DISKGROUP 'DMDATA1' EXTERNAL REDUNDANCY FAILGROUP 'data1' ASMDISK
'/dev/DMDATA/disk0','/dev/DMDATA/disk1' ATTRIBUTE AU_SIZE=4, REDO_SIZE=128
```

- 添加 ASM 磁盘

语法如下：

```
ALTER DISKGROUP name ADD <磁盘子句> {<磁盘子句>} [FREE_RATE num]
<磁盘子句>::= [FAILGROUP failgroup_name] ASMDISK file_path {, file_path} [PARTNER
(disk_name1{,disk_name2})]
```

参数说明：

file_path：ASMDISK 路径。必须是全路径，不能是相对路径。全路径长度不能超过 256 字节。

failgroup_name：指定故障组名。如未指定故障组，则系统会自动创建一个故障组，名称为 file_path 中的磁盘名称，例如/dev/DMDATA/disk1 中的 disk1。

disk_name1、disk_name2：磁盘名。为新磁盘指定伙伴磁盘，可以是老磁盘，也可以是新磁盘。如果指定的伙伴磁盘为新磁盘，则另一个新磁盘也需要指定当前磁盘为伙伴磁盘。通过指定伙伴磁盘方式添加磁盘将不自动生成伙伴关系。

num：空闲比例值。为小于等于该指定空闲比例值的老磁盘增加一个新磁盘作为伙伴磁盘，num 取值范围为 0~100。未配置则默认为 100，表示不检查磁盘空闲空间大小；为 0 表示只为磁盘空间已满的老磁盘新增一个新磁盘作为伙伴磁盘；为 10 表示为磁盘空闲比例小于等于 10%的老磁盘新增一个新磁盘作为伙伴磁盘。

举例说明：

例 1 为 DMDATA 磁盘组添加两块磁盘。

```
//磁盘放入故障组 disk1 中。
ALTER DISKGROUP 'DMDATA' ADD FAILGROUP 'data1' ASMDISK '/dev/DMDATA/disk1'
//两个磁盘分别放入故障组 data1 和 data2 中
ALTER DISKGROUP 'DMDATA' add FAILGROUP 'data1' ASMDISK
'/dev/DMDATA/disk1','/dev/DMDATA/disk2' FAILGROUP 'data2' ASMDISK
'/dev/DMDATA/disk3','/dev/DMDATA/disk4' FREE_RATE 10
```

例 2 如原有三个故障组，每个故障组各一个磁盘，分别为 DMASMDATA0、DMASMDATA1、 DMASMDATA2，为每个故障组各增加一块磁盘。

```
ALTER DISKGROUP 'DMDATA' ADD FAILGROUP 'data1' ASMDISK '/dev/DMDATA/disk3'
PARTNER('DMASMDATA4', 'DMASMDATA5', 'DMASMDATA1') FAILGROUP 'data2' ASMDISK
'/dev/DMDATA/disk4' PARTNER('DMASMDATA3', 'DMASMDATA5', 'DMASMDATA2') FAILGROUP
'data3' ASMDISK '/dev/DMDATA/disk5' PARTNER('DMASMDATA3', 'DMASMDATA4', 'DMASMDATA0')
```



注意：

创建磁盘组，或为磁盘组添加磁盘时，以下情况可能导致失败：

1.DMASMSVRM 进程没有访问对应磁盘的权限。

2. 磁盘路径不在 DMDCR_CFG.INI 配置文件中配置的 DCR_DISK_LOAD_PATH 路径下。

3.磁盘大小不够，最少需要(32 * AU_SIZE)MB 或（REDO_SIZE/AU_SIZE + 32）MB，两者中的较大值为磁盘大小下限。其中 AU_SIZE 和 REDO_SIZE 的单位均为 MB。

4.磁盘名称重复。

- 指定磁盘之间的伙伴关系

语法如下：

```
ALTER DISKGROUP name SET PARTNER DISK disk_name1 WITH disk_name2{, disk_name3 WITH
```

```
disk_name4}
```

参数说明:

name:磁盘组名。

disk_name1、disk_name2、disk_name3、disk_name4：磁盘名称。

举例说明:

指定磁盘 DMASMDATA1 和 DMASMDATA2、DMASMDATA3 和 DMASMDATA4 的伙伴关系。

```
ALTER DISKGROUP 'DMDATA' SET PARTNER DISK 'DMASMDATA1' WITH 'DMASMDATA2', 'DMASMDATA3'
WITH 'DMASMDATA4'
```

- 文件减少副本恢复

可以对单个文件或磁盘组中所有文件尝试进行减少副本异步恢复，尽可能多地恢复副本。

语法如下:

```
ALTER DISKGROUP name REPAIR [ASMFILE file_path] [POWER value]
```

参数说明:

name：磁盘组名称。

file_path：ASMDISK 路径。必须是全路径，不能是相对路径。全路径长度不能超过 256 字节。

value：指定恢复时并行度，取值范围 1~16，未指定则默认为 8。

举例说明:

指定恢复时的并行度为 2。

```
ALTER DISKGROUP 'DMDATA' REPAIR ASMFILE '+DMDATA/sample.dta' POWER 2
ALTER DISKGROUP 'DMDATA' REPAIR POWER 2
```

- 快速删除 ASM 磁盘

本命令可以快速删除正常、离线的磁盘，通过标记磁盘状态为已删除实现，并未重平衡

磁盘内的数据。本命令简便快捷，命令完成后 ASM 磁盘不会再被使用，可以随意处理。但磁盘内的数据相当于处于故障状态，需要通过故障重平衡恢复。磁盘内的所有数据均恢复后，磁盘自动从磁盘组中彻底删除。

本命令无法删除含有仅存的正常副本 AU 的磁盘，例如单副本 AU、2 副本已经有 1 副本故障的 AU 和 3 副本已经有 2 副本故障的 AU。如确定不再需要这些 AU，可以先删除 AU 所在文件，再执行本命令。

本命令支持使用磁盘名指定磁盘，对于正常磁盘和部分离线磁盘也支持使用磁盘路径。推荐使用磁盘名指定磁盘。

语法如下：

```
ALTER DISKGROUP name [FAST] DROP ASMDISK <disk>{, <disk>}

<disk>::= <disk_name> | <disk_path>
```

参数说明：

name：磁盘组名称。

disk_name：ASM 磁盘名，包含“DMASM”前缀的完整名称。

disk_path：ASM 磁盘路径，必须是绝对路径，不能是相对路径。

举例说明：

快速删除 DMDATA 磁盘组中的磁盘 DMASMDISK1 和 DMASMDISK2。

```
ALTER DISKGROUP 'DMDATA' DROP ASMDISK 'DMASMDISK1', '/dev/DMDATA/disk2'
```

- 完全删除 ASM 磁盘

本命令可以对所有非重连中的磁盘执行，执行时先将磁盘内所有 AU 重平衡到其它磁盘，重平衡成功后再直接彻底删除磁盘。重平衡失败，本命令将报错。重平衡过程中该磁盘故障或被移除，可能导致重平衡失败，甚至导致数据丢失。

本命令执行成功后，ASM 磁盘彻底删除，才可以随意处理对应物理磁盘。

本命令支持使用磁盘名指定磁盘，对于正常磁盘和部分其它磁盘也支持使用磁盘路径。

推荐使用磁盘名指定磁盘。

语法如下:

```
ALTER DISKGROUP name FULLY DROP ASMDISK <disk>{, <disk>} [POWER value]

<disk>::= <disk_name> | <disk_path>
```

参数说明:

name：磁盘组名称。

disk_name：ASM 磁盘名，包含“DMASM”前缀的完整名称。

disk_path：ASM 磁盘路径，必须是绝对路径，不能是相对路径。

value：重平衡并行度，取值范围 1~16。未指定则使用磁盘组默认并行度。

举例说明:

完全删除 DMDATA 磁盘组中的磁盘 DMASMDISK1 和 DMASMDISK2，使用 8 并行度重平衡磁盘。

```
ALTER DISKGROUP 'DMDATA' FULLY DROP ASMDISK 'DMASMDISK1', '/dev/DMDATA/disk2' POWER 8
```

- 添加 DCRV 磁盘

语法如下:

```
ALTER SYSTEM DISKGROUP ADD ASMDISK <disk_path>
```

参数说明:

disk_path：DCRV 磁盘路径，必须是绝对路径，不能是相对路径，路径长度不能超过 256 字节。

举例说明:

在 DCRV 系统磁盘组中添加一块 DCRV 磁盘，磁盘路径为/dev/DMDATA/dmdcrv1。

```
ALTER SYSTEM DISKGROUP ADD ASMDISK '/dev/DMDATA/dmdcrv1'
```

- 删除 DCRV 磁盘

语法如下:

```
ALTER SYSTEM DISKGROUP DROP ASMDISK <disk_name>|<disk_path>
```

参数说明:

disk_name：DCRV 磁盘名，包含“DMASM”前缀的完整名称。

disk_path：DCRV 磁盘路径，必须是绝对路径，不能是相对路径。

举例说明:

在 DCRV 系统磁盘组中删除一块 DCRV 磁盘，磁盘名为 DMASMDCRV1。

```
ALTER SYSTEM DISKGROUP DROP ASMDISK 'DMASMDCRV1'
```

在 DCRV 系统磁盘组中删除一块 DCRV 磁盘，磁盘路径为/dev/DMDATA/dmdcrv1。

```
ALTER SYSTEM DISKGROUP DROP ASMDISK '/dev/DMDATA/dmdcrv1'
```

- 删除 ASM 磁盘组

语法如下:

```
DROP DISKGROUP name
```

参数说明:

name：磁盘组名称。

举例说明:

删除名为 DMDATA 的磁盘组。

```
DROP DISKGROUP 'DMDATA'
```

- 修改重平衡并行度

当磁盘资源充裕（比如大量空闲）且希望每一次重平衡操作能快速完成，可适当调大重平衡并行度。当磁盘资源紧张，为了既节约磁盘资源，又不影响数据库或其他软件的正常使用，可适当调小重平衡并行度。

语法如下:

```
ALTER DISKGROUP name SET REBALANCE POWER num
```

参数说明:

name：磁盘组名称。

num：重平衡并行度。取值范围 1~16。

举例说明:

将磁盘组 DMDATA 的重平衡并行度修改为 8。

```
ALTER DISKGROUP 'DMDATA' set REBALANCE POWER 8
```

- 修改磁盘组自动重平衡时间间隔

用于修改 DMDCR.INI 参数 DMDCR_AUTO_RBL_TIME。

语法如下:

```
ALTER DISKGROUP SET ATTRIBUTE AUTO_RBL_TIME = value
```

参数说明:

value：修改 DMDCR_AUTO_RBL_TIME 的值。取值范围 0~86400，单位 S。

举例说明:

```
ALTER DISKGROUP SET ATTRIBUTE AUTO_RBL_TIME = 300
```

- 启用/禁止重平衡

语法如下:

```
ALTER DISKGROUP name REBALANCE <ENABLE | DISABLE>
```

参数说明:

name：磁盘组名称。

举例说明:

启用或禁止磁盘组 DMDATA 的重平衡功能。

```
ALTER DISKGROUP 'DMDATA' REBALANCE ENABLE
```

- 磁盘组重平衡

该命令用于对磁盘组中状态正常的磁盘进行空间重平衡，使磁盘组中各磁盘已分配 AU

的比率趋于相等。

语法如下:

```
ALTER DISKGROUP name REBALANCE [POWER value]
```

参数说明:

name：磁盘组名称。

value：重平衡并行度。取值范围 1~16。

举例说明:

对磁盘组 DMDATA 执行重平衡。

```
ALTER DISKGROUP 'DMDATA' REBALANCE POWER 8
```

- 停止当前正在执行的重平衡、磁盘重连和磁盘替换

包括故障重平衡、新加磁盘重平衡、磁盘重连、磁盘替换。

语法如下:

```
ALTER SYSTEM REBALANCE STOP
```

举例说明:

```
ALTER SYSTEM REBALANCE STOP
```

- 新加磁盘重平衡

语法如下:

```
ALTER DISKGROUP name REBALANCE DISK disk_name [POWER value]
```

参数说明:

name：磁盘组名称。

value：重平衡并行度。取值范围 1~16。

disk_name：磁盘名称。

举例说明:

新加磁盘后，对磁盘组 DMDATA 执行重平衡。

```
ALTER DISKGROUP 'DMDATA' REBALANCE DISK 'DMASMdisk1' POWER 8
```

- 磁盘替换

语法如下：

```
ALTER DISKGROUP name REPLACE disk <替换子句> [POWER value]

<替换子句>: : = disk_name WITH disk_path{,disk_name WITH disk_path}
```

参数说明：

name：磁盘组名称。

disk_name：磁盘名称。

disk_path：文件路径必须是新创建的磁盘，不能是已被使用的磁盘。

value：重平衡并行度。取值范围 1~16。

举例说明：

```
ALTER DISKGROUP 'DMDATA' REPLACE DISK 'DMASMdisk1' WITH '/dev/DMDATA/disk2'

ALTER DISKGROUP 'DMDATA' REPLACE DISK 'DMASMdisk1' WITH '/dev/DMDATA/disk2','DMASMdisk3'
with '/dev/DMDATA/disk4'
```

- 故障磁盘重平衡的手动触发命令

该命令用于对磁盘组中状态故障的磁盘进行空间重平衡。

语法如下：

```
ALTER DISKGROUP name FAIL DISK REBALANCE START [POWER value]
```

参数说明：

name：磁盘组名称。

value：重平衡并行度。取值范围 1~16。

举例说明：

手动触发磁盘组 DMDATA 中的故障磁盘重平衡。

```
ALTER DISKGROUP 'DMDATA' FAIL DISK REBALANCE START POWER 8
```

- 重启被暂停的磁盘替换

语法如下:

```
ALTER DISKGROUP name DISK REPLACE CONTINUE [POWER value]
```

参数说明:

name：磁盘组名称。

value：重平衡并行度。取值范围 1~16。

举例说明:

重启磁盘组 DMDATA 中暂停的磁盘替换。

```
ALTER DISKGROUP 'DMDATA' DISK REPLACE CONTINUE
```

- 磁盘重连

用于将离线磁盘重新连回磁盘组，需要保证磁盘上的数据在离线后没有发生过变化，即整块磁盘和离线时内容一致。

语法如下:

```
ALTER DISKGROUP name ONLINE DISK <重连子句> [POWER value]

<重连子句>: : = disk_name WITH disk_path{, disk_name WITH disk_path}
```

参数说明:

name：磁盘组名称。

disk_name：磁盘名称。

disk_path：文件路径必须是原磁盘所在路径。通常，磁盘路径不会发生变化，此路径即为创建磁盘组时指定的路径、lsdsk 显示的路径。

value：重平衡并行度。取值范围 1~16。

举例说明:

```
ALTER DISKGROUP 'DMDATA' ONLINE DISK 'DMASMdisk1' WITH '/dev/DMDATA/disk1'
```

```
ALTER DISKGROUP 'DMDATA' ONLINE DISK 'DMASMdisk1' WITH '/dev/DMDATA/disk1', 'DMASMdisk2'
with '/dev/DMDATA/disk2'
```

- 列出所有的磁盘组

语法如下:

```
LSDG
```

举例说明:

```
LSDG
```

- 列出所有的 ASM 磁盘

语法如下:

```
lsdsk
```

举例说明:

```
lsdsk
```

- 列出指定 ASM 磁盘的属性

语法如下:

```
lsdskattr dg_id, disk_id
```

参数说明:

dg_id：磁盘组 ID。

disk_id：磁盘 ID。

举例说明:

列出组 ID=0 和磁盘 ID=0 的磁盘的属性。

```
lsdskattr 0, 0
```

- 测试 ASM 环境下磁盘读写速度

包括随机读/写速度和顺序读/写速度。速度会打印在屏幕上。

执行 iotest 命令时，会在当前路径下创建一个后缀名为.iotest 的临时文件，大小为 1G。支持指定条带化粒度和镜像类型，测试完成后会自动删除该临时文件。如果当前路径为"+"目录，则默认在 0 号 Disk Group 根目录下创建临时文件。更多关于 iotest 命令实现机制的介绍请参考 10.6.4DMASMTOOL。

出现以下情况将报错：

1） 若磁盘空间不足 1G，则报错，创建文件失败。

2） 若临时文件生成路径下已经存在.iotest 文件，则报错，需要手动删除后才能执行 iotest 命令。

语法如下：

```
IOTEST [DISKGROUP name][<EXTERNAL | NORMAL | HIGH> REDUNDANCY] [STRIPING value]
```

参数说明：

name：待测试的磁盘组名。缺省为测试所有磁盘组。

EXTERNAL、NORMAL、HIGH：用于指定镜像类型，即文件的副本数。分别表示单副本、两副本、三副本。缺省为 EXTERNAL。

value：指定 IOTEST 测试的条带化粒度。取值 0、32、64、128 和 256，单位 KB。用户可根据系统软硬件资源的实际情况选取条带化粒度，资源充裕时，可适当调高条带化粒度。缺省为 0，表示粗粒度。

举例说明：

测试所有磁盘组的读写速度。

```
IOTEST
```

测试 DMDATA 磁盘组的读写速度。采用默认值单副本和条带化粒度 0。

```
IOTEST DISKGROUP 'DMDATA'
```

测试 DMDATA 磁盘组的读写速度。采用两副本和条带化粒度 32。

```
IOTEST DISKGROUP 'DMDATA' NORMAL REDUNDANCY STRIPING 32
```

11.5.4.2 ASM 文件命令

- 创建文件

语法如下：

```
CREATE ASMFILE file_path SIZE num[<EXTERNAL | NORMAL | HIGH> REDUNDANCY] [STRIPING value]
```

参数说明：

file_path：ASM 文件路径。

num：用于指定文件大小。取值范围 0~16777216*AU_SIZE，单位 MB。

EXTERNAL、NORMAL、HIGH：用于指定镜像类型，即文件的副本数。分别表示单副本、两副本、三副本。创建文件时指定的副本数需要小于等于待存储的磁盘组的副本数。缺省为单副本。

value：用于指定条带化粒度。取值 0、32、64、128 和 256。缺省为 0。

举例说明：

```
CREATE ASMFILE '+DMDATA/sample.dta' SIZE 20

或

CREATE ASMFILE '+DMDATA/sample.dta' SIZE 20 HIGH REDUNDANCY STRIPING 128
```

- 扩展文件

将文件扩大到指定的大小。扩展文件时指定的 num 大小一定要比原文件更大，否则不予执行，直接退出。

语法如下：

```
ALTER ASMFILE file_path EXTEND TO num
```

参数说明：

file_path：ASM 文件路径。

num：目标大小。取值范围 0~16777216*AU_SIZE。单位 MB。

举例说明：

```
ALTER ASMFILE '+DMDATA/sample.dta' EXTEND TO 20
```

- 截断文件

将文件截断缩小到指定大小。截断时指定的 num 大小一定要比原文件更小，否则不予执行，直接退出。

语法如下：

```
ALTER ASMFILE file_path TRUNCATE TO num
```

参数说明：

file_path：ASM 文件路径。

num：目标大小。取值范围 0~16777216*AU_SIZE。单位 MB。

举例说明：

```
ALTER ASMFILE '+DMDATA/sample.dta' TRUNCATE TO 20
```

- 删除文件

已经打开、正在使用的 ASM 文件不能被删除。

语法如下：

```
DELETE ASMFILE file_path
```

参数说明：

file_path：ASM 文件路径。

举例说明：

```
DELETE ASMFILE '+DMDATA/sample.dta'
```

- 文件校验

查看 ASM 文件各个 AU 分布情况或校验文件是否存在缺失副本。

语法如下：

```
CHECK [file_path] [fast]
```

参数说明：

file_path：ASM 文件路径。

fast：不指定 fast 关键字则校验各镜像副本数据是否一致，否则不校验（非本地登录 asm 也不会校验）。

举例说明：

```
CHECK

或

CHECK +DMDATA/sample.dta
```

校验文件是否存在缺失副本（n_copy 表示文件应有副本数，r_copy 表示文件最大缺失副本数）。

```
CHECK sample1.dta fast

file: +DATA/sample1.dta fil_id:0x80040021 n_copy:3 r_copy:1

check file consistency skipped, require local connection and without fast keyword.

(au_seq group_id [disk_id, disk_auno])

(0          0          [0, 203] [2, 203] [65535, -1])

(1          0          [1, 203] [0, 204] [65535, -1])
```

- 重定向输出文件

重定向功能可以将工具控制台执行命令写入指定文本文件中。多次 SPOOL 重定向文件，第一次成功打开重定向文件之后，如果未关闭，则不再打开其他重定向文件。文件记录了控制台输入命令和输出结果。

语法如下：

```
SPOOL file_path [CREATE|REPLACE|APPEND]
```

参数说明：

file_path：输出文件路径。

CREATE：如果重定向文件不存在，则创建；如果已经存在，创建失败。

REPLACE：如果重定向文件不存在，则创建；如果已经存在，则替换掉。默认为 REPLACE。

APPEND：如果重定向文件不存在，则创建；如果已经存在，则追加到文件末尾。

举例说明：

```
SPOOL d:\asmdisks\spool.txt
```

- 关闭重定向文件

语法如下：

```
SPOOL OFF
```

举例说明：

```
SPOOL OFF
```

- 列出系统属性

系统属性包括：磁盘组 ID、AU_SIZE、簇信息、系统编码语言、共享内存属性、自动重平衡时间等。

语法如下：

```
LSATTR
```

举例说明：

```
LSATTR
```

- 列出所有的信息。

所有信息包括：磁盘组、磁盘、所有文件等。

语法如下：

```
LSALL
```

举例说明：

```
LSALL
```

- 修改密码

修改 DMASMSVRM 的密码。当 DMASMTOOLM 和 DMASMSVRM 不在同一台机器上时，DMASMTOOLM 登录 DMASMSVRM 时使用此密码。DMASMTOOLM 和 DMASMSVRM 在同一台机器时免密。

语法如下:

```
password
```

举例说明:

```
password
```

- 登录，在断开连接后，重新登录

语法如下:

```
LOGIN
```

举例说明:

```
LOGIN
```

11.5.4.3 Linux 兼容命令

DMASMTOOLM 兼容了部分 Linux 命令用法。这些命令对辅助管理和使用 DMASM 非常有用。

- 到达指定目录

语法如下:

```
CD [path]
```

参数说明:

path：目标目录。缺省为当前目录。

举例说明:

```
cd +DMDATA/test
```

- 文件拷贝

语法如下：

```
CP [<-RF>|<-R>|<-F>] src_file_path dst_file_path [<EXTERNAL | NORMAL | HIGH> REDUNDANCY]
```

参数说明：

-F：执行拷贝，不给出提示。

-R：递归拷贝，将指定目录下的子文件和子目录一并拷贝。

src_file_path：源文件路径。必须是全路径，不能是相对路径。

dst_file_path：目标文件路径。必须是全路径，不能是相对路径。

EXTERNAL、NORMAL 和 HIGH：指定镜像类型，即文件副本数。只对拷贝本地文件到 DMASM 有效。缺省为单副本。

举例说明：

```
cp '+DMDATA/a/sample.dta' '+DMDATA/a/b.dta'

cp -r '+DMDATA/a/newfile.dat' '+DMDATA/b'

cp -f '+DMDATA/a/*' '+DMDATA/c'

cp '/home/test/a.txt' '+DMDATA' HIGH REDUNDANCY
```

- 删除文件或目录

语法如下：

```
RM [-f] file_path

RM -r[f] directories
```

参数说明：

-f：执行删除，不给出提示。

-r：递归删除，将指定目录下的子文件和子目录一并删除。

file_path：待删除的文件。

directories：待删除的目录。

举例说明：

```
rm '+DMDATA/a/sample.dta'

rm -r '+DMDATA/a/'
```

- 移动、重命名文件。不支持跨磁盘组移动文件

语法如下：

```
MV src_file_path dst_file_path
```

参数说明：

src_file_path：源文件路径。必须是全路径，不能是相对路径。

dst_file_path：目标文件路径。必须是全路径，不能是相对路径。

举例说明：

```
mv '+DMDATA/a/sample.dta' '+DMDATA/a/sample2.dta'
```

- 创建目录

语法如下：

```
MKDIR [-P] dir_path
```

参数说明：

-p：表示自动创建不存在的中间目录。如未指定-P 且不存在中间目录，则报错。

dir_path：目标目录。

举例说明：

```
mkdir '+DMDATA/a'

mkdir -p '+DMDATA/a'
```

- 查找文件

语法如下：

```
FIND [PATH] file_name
```

参数说明：

file_name：目标文件名。

举例说明：

```
find +DMDATA/a 'sample.dta'
```

- 显示文件详细信息

语法如下：

```
LS [<-l>|<-r>|<-lr>] [dir_path]
```

参数说明：

-l：显示详细信息。

-r：表示递归的意思，不是 linux 中的逆序显示。

dir_path：目标文件路径。

举例说明：

```
ls

ls -l

ls -r b*
```

- 显示存储信息

语法如下：

```
DF
```

举例说明：

```
df
```

- 当前目录

语法如下：

```
PWD
```

举例说明：

```
pwd
```

- 估算文件空间已使用情况

语法如下：

```
DU [dir_path{,dir_path}]
```

参数说明：

dir_path：要估算的文件目录，未指定则默认为当前目录

举例说明：

```
du

du DMDATA

du DMDATA,DMLOG

du DMDATA DMLOG
```

估算结果中，出现的单位有缺省（字节）、K 和 M 等情况。

```
ASM>du

0       +DATA/dsc_mirror2/bak

18.0K         +DATA/dsc_mirror2/ctl_bak

0       +DATA/dsc_mirror2/HMAIN

660.1M      +DATA/dsc_mirror2

792.1M      +DATA
```

11.6 DMASM 镜像配置文件

镜像环境下需用到 DMDCR_CFG.INI、DMDCR.INI、DMINIT.INI、DMARCH.INI、MAL 系统配置文件和 DM.INI 配置文件。其中，DMDCR_CFG.INI、DMDCR.INI、DMINIT.INI 和 DMARCH.INI 配置文件中增加了镜像相关参数。MAL 系统配置文件和 DM.INI 参数未发生改

变，与 9 DMDSC 配置文件中用法完全相同。

增加了镜像参数的配置文件包括：

- DMDCR_CFG.INI
- DMDCR.INI
- DMINIT.INI
- DMARCH.INI

下面分别介绍配置文件中各配置项的含义。

11.6.1 DMDCR_CFG.INI

DMDCR_CFG.INI 是 DMASMCMDM 格式化 DCRV 磁盘的配置文件。配置信息包括三类：集群环境全局信息、集群组信息、以及组内节点信息。

在 DMASM 镜像环境中，DMDCR_CFG.INI 取消了 DCR_EP_ASM_LOAD_PATH 和 DCR_VTD_PATH 参数，新增了 DCR_DISK_LOAD_PATH 参数。除此之外，其他的 DMDCR_CFG.INI 参数保持不变，请参考 9 DMDSC 配置文件。

表 11.3 DMDCR_CFG.INI 配置项

集群环境全局信息	
DCR_DISK_LOAD_PATH	ASM 磁盘扫描路径，Linux 下一般为/dev/DMDATA，或者/dev_DMDATA/。 支持多路径配置，各路径以英文逗号','进行分隔，所有配置的路径都必须为有效路径，且暂不支持中文路径，例如：/dev/disk1,/dev/disk2,/dev/disk3。最多可以同时配置 8 个磁盘扫描路径，但配置的所有路径长度之和必须小于 256，若超过该长度，只能减小磁盘扫描路径个数
DCR_N_GRP	集群环境包括多少个组，取值范围 1~16

DCR_OGUID	消息标识，DMCSSM 登录 DMCSS 消息校验用。取值范围：正整数
集群组信息	
DCR_GRP_TYPE	组类型。取值：CSS、ASM 和 DB。CSS 代表 DMCSS 集群、ASM 代表 DMASM 集群、DB 代表 DMDSC 集群
DCR_GRP_NAME	组名，16 字节，配置文件内不可重复
DCR_GRP_N_EP	组内节点个数 N，最大 8
DCR_GRP_EP_ARR	组内包含的节点序列号，{0,1,2,...N-1} 用户不能指定，仅用于从 DCR 磁盘 export 出来可见
DCR_GRP_N_ERR_EP	组内故障节点个数 用户不能指定，仅用于从 DCR 磁盘 export 出来可见
DCR_GRP_ERR_EP_ARR	组内故障节点序列号 用户不能指定，仅用于从 DCR 磁盘 export 出来可见
DCR_GRP_DSKCHK_CNT	磁盘心跳机制，容错时间，单位秒，缺省 60S，取值范围 5~600
节点信息，某些属性可能只针对某一类型节点，比如 SHM_KEY 只针对 ASM 节点有效	
DCR_EP_NAME	节点名，16 字节，配置文件内不可重复， DB 的节点名必须和 DM.INI 里的 INSTANCE_NAME 保持一致， ASM 的节点名必须和 DMASVRMAL.INI 里的 MAL_INST_NAME 一致， 同一类型各节点的 EP_NAME 不能重复
DCR_EP_SEQNO	组内序号，CSS/ASM 组不能配置，自动分配， DB 组可以配置，0 ~ n_ep -1，组内不能重复，如不配置则自动分配
DCR_EP_PORT	节点 TCP 监听端口(CSS/ASM/DB 有效，对应登录 CSS/ASM/DB 的端口号)，节点实例配置此参数，取值范围 1024~65534；发起连接端的端口在

	1024~65535 之间随机分配。 特别对 DB 来说，DB 的 DCR_EP_PORT 与 DM.INI 中的 PORT_NUM 不一致时，DB 端口以 DM.INI 中的 PORT_NUM 为准
DCR_EP_SHM_KEY	共享内存标识，数值类型(ASM 有效，初始化共享内存的标识符)，应为大于 0 的 4 字节整数。同一机器上每个 ASM 节点对应的 DCR_EP_SHM_KEY 必须唯一，需要人为保证其唯一性
DCR_EP_SHM_SIZE	共享内存大小（ASM 有效，初始化共享内存大小），单位 M，取值范围 10~40000。共享内存大小与其能管理的磁盘大小的关系详见 10.5 DMASM 技术指标

举例说明

一个完整的 DMDCR_CFG.INI 文件。

```
DCR_N_GRP                = 3
DCR_DISK_LOAD_PATH       = /dev/DMDATA/
DCR_OGUID                = 63635

[GRP]                    #[GRP]表示新建一个 Group
    DCR_GRP_TYPE         = CSS
    DCR_GRP_NAME         = GRP_CSS
    DCR_GRP_N_EP         = 2
    DCR_GRP_DSKCHK_CNT   = 60

[GRP]
    DCR_GRP_TYPE         = ASM
```

```
    DCR_GRP_NAME              = GRP_ASM
    DCR_GRP_N_EP              = 2
    DCR_GRP_DSKCHK_CNT        = 60

[GRP]
    DCR_GRP_TYPE              = DB
    DCR_GRP_NAME              = GRP_DSC
    DCR_GRP_N_EP              = 2
    DCR_GRP_DSKCHK_CNT        = 60

[GRP_CSS]                     #[ ]里的是组名，与 DCR_GRP_NAME 对应
    DCR_EP_NAME               = CSS0
    DCR_EP_HOST               = 192.168.1.102
    DCR_EP_PORT               = 9941

[GRP_CSS]
    DCR_EP_NAME               = CSS1
    DCR_EP_HOST               = 192.168.1.102
    DCR_EP_PORT               = 9942

[GRP_ASM]
    DCR_EP_NAME               = ASM0
    DCR_EP_SHM_KEY            = 78910
    DCR_EP_SHM_SIZE           = 10
```

```
    DCR_EP_HOST             = 192.168.1.102
    DCR_EP_PORT             = 9949

[GRP_ASM]
    DCR_EP_NAME             = ASM1
    DCR_EP_SHM_KEY          = 78911
    DCR_EP_SHM_SIZE         = 10
    DCR_EP_HOST             = 192.168.1.102
    DCR_EP_PORT             = 9950

[GRP_DSC]
    DCR_EP_NAME             = DSC0
    DCR_EP_PORT             = 5236

[GRP_DSC]
    DCR_EP_NAME             = DSC1
    DCR_EP_PORT             = 5237
```

11.6.2 DMDCR.INI

DMDCR.INI 是 DMCSSM、DMASMSVRM、DMASMTOOLM 等工具的输入参数。记录了当前节点序列号以及 DCR 磁盘路径。

在 DMASM 镜像环境中，对 DMDCR_PATH 参数进行了进一步定义，同时新增了 DMDCR_AUTO_RBL_TIME 参数。除此之外，其他的 DMDCR.INI 参数保持不变，请参考 9 DMDSC 配置文件。

表 11.4 镜像环境中新增的 DMDCR.INI 配置项

字段	字段意义
DMDCR_PATH	记录 DCR 磁盘路径。和 DMDCR_CFG.INI 里 DCR_DISK_LOAD_PATH 相同。支持多路径配置，各路径以英文逗号','进行分隔，所有配置的路径都必须为有效路径，且暂不支持中文路径，例如：/dev/disk1,/dev/disk2,/dev/disk3。最多可以同时配置 8 个磁盘扫描路径，但配置的所有路径长度之和必须小于 256，若超过该长度，只能减小磁盘扫描路径个数
DMDCR_AUTO_RBL_TIME	镜像自动重平衡时间间隔，单位秒，取值范围 0~86400s，缺省配置为 0，表示不进行自动重平衡。主从节点配置不一致以主 ASM 节点配置为准

11.6.3 DMINIT.INI

DMINIT.INI 是 DMINIT 工具初始化数据库环境的配置文件。

在 DMASM 镜像环境中，DMINIT.INI 中新增了 CTL_MIRROR、DATA_MIRROR、LOG_MIRROR、DATA_STRIPING 和 LOG_STRIPING 参数，修改了 DCR_PATH 参数。除此之外，其他的 DMINIT.INI 参数保持不变，请参考 9 DMDSC 配置文件。

表 11.5 镜像环境中存在差异的 DMINIT.INI 配置项

字段	字段意义
新增参数	
CTL_MIRROR	dm.ctl、dmtemp.ctl 和 dm_service.prikey 文件副本数，取值 1、2 或 3；缺省为 1
DATA_MIRROR	SYSTEM/MAIN/ROLL 表空间数据文件副本数，取值 1、2 或 3；缺省为 1
LOG_MIRROR	联机日志文件副本数，取值 1、2 或 3；缺省为 1

DATA_STRIPING	数据文件条带化粒度，取值 0、32、64、128、256。缺省为 32。单位 KB。 条带化的效果受软硬件资源的影响，不同环境中最佳的 DATA_STRIPING 参数值不同，需 DBA 根据实际情况进行调整。一般情况下建议 DATA_STRIPING 和页大小保持一致
LOG_STRIPING	联机日志条带化粒度，取值 0、32、64、128、256。缺省为 64。单位 KB。 条带化的效果受软硬件资源的影响，不同环境中最佳的 LOG_STRIPING 参数值不同，需 DBA 根据实际情况进行调整。一般情况下建议直接使用缺省值 64
修改参数	
DCR_PATH	DCR 磁盘路径。在镜像环境下，支持多路径配置，各路径以英文逗号','进行分隔，所有配置的路径都必须为有效路径，且暂不支持中文路径，最多可以同时配置 8 个磁盘扫描路径，但配置的所有路径长度之和必须小于 256，若超过该长度，只能减小磁盘扫描路径个数

11.6.4 DMARCH.INI

DMARCH.INI 是开启本地归档和远程归档使用的配置文件。

在 DMASM 镜像环境中，DMARCH.INI 中新增了 ARCH_ASM_MIRROR 和 ARCH_ASM_STRIPING 参数。除此之外，其他的 DMARCH.INI 参数保持不变，请参考 9 DMDSC 配置文件。

表 11.6 DMARCH.INI 的配置项

项目	字段	字段意义
全局配置项	ARCH_ASM_MIRROR	归档文件配置在 ASM 上镜像类型，取值 1/2/3，缺省取值和 LOG_MIRROR 参数值一样
	ARCH_ASM_STRIPING	归档文件配置在 ASM 上条带化类型，取值 0 表示粗粒度，32/64/128/256 表示细粒度条带化大小，缺省取值和 LOG_STRIPING

		参数值一样

11.7 DMASM 镜像使用说明

DMASM 镜像的配置和使用大致分为三步。

第一步 环境准备

- 硬件

  准备多台部署 DMDSC 集群的机器。机器数和集群节点数一致。

- 存储设备

  准备共享磁盘。用户根据实际数据量来准备共享磁盘的数量。至少一块。

- 操作系统

  为所有机器配置相同的操作系统。支持 Windows 和 Linux 系统。用户根据实际需求选择操作系统。

- 网络配置

  每台机器至少配置两块网卡。一块用于配置内网网段，一块用于配置外网网段。

- 用户权限准备

  如果用户选择 Linux 操作系统，那么需要在每台机器上使用 root 用户创建用于搭建环境的用户和组。

- 目录规划

  为每台机器各创建三个目录。

  目录一 使用 dmdba 用户创建用于镜像环境搭建的目录，不同机器上此目录必须相同。

  目录二 DM 执行码和工具存放于目录，不同机器上此目录必须相同。

  目录三 配置文件存放于目录，不同的机器上此目录可以不同。

- 磁盘准备

  首先使用多路径软件为共享存储的磁盘创建目录名，即将磁盘链接到一个目录中。然后

将磁盘的用户权限修改为和目录的用户权限一致。

第二步 搭建 DMASM 镜像

搭建按照下面顺序依次进行:

1. 使用 DMASMCMDM 工具创建所有磁盘。具体为 DCRV 磁盘和 ASM 磁盘，其中 ASM 磁盘用于存放数据和日志。

2. 使用 DMASMCMDM 工具创建 DCRV 系统磁盘组。用于统一管理 DCRV 磁盘。

3. 配置并启动 DMCSS。

4. 配置并启动 DMASMSVRM。

5. 使用 DMASMTOOLM 工具创建 ASMDISK 磁盘组。具体为 DATA 磁盘组和 LOG 磁盘组两种，分别管理 DATA 磁盘和 LOG 磁盘。在创建磁盘组的时候指定副本数。

6. 配置并启动 DMSERVER。

7. 配置并启动 DMCSSM。

至此，一个使用了 DMASM 镜像的 DMDSC 部署完成。

第三步 正常使用数据库

用户登录到任意一个 DM 实例，即可获得完整的达梦数据库服务。同时，用户可通过两种方式来管理 ASM 文件：一是编写用户代码，通过调用 DMASMAPIM 接口来管理 ASM 文件；二是使用 DMASMCMDM 和 DMASMTOOLM 工具来管理 ASM 文件。

12DMDSC 搭建

本章将举例说明 DMDSC 的搭建过程，并介绍搭建 DMDSC 的一些注意事项。


说明：**真实的生产环境中，建议至少配置两块共享磁盘，分别用来存放联机日志文件和数据文件。**

在本章的磁盘准备中，介绍了 2 种共享存储磁盘固定方式：一种是使用 UDEV 工具，详见 12.1.1 环境准备或 12.2.1 环境准备的磁盘准备中；一种是使用 multipath 软件，详见 12.2.1 环境准备的磁盘准备中。因为两种方式效果一样，但 UDEV 方式搭建步骤更简单，所以推荐用户使用 UDEV 方式。

固定磁盘时建议将 DMDSC 系统中的共享存储磁盘固定到一个自行创建的独立目录下，因为 DMDSC 启动时会扫描磁盘目录下所有磁盘，使用独立目录可以避免扫描到目录中的其他无关磁盘文件而导致 DMDSC 启动失败。

12.1 基于 DMASM 的 DMDSC

本节用到的 DMASM 功能和 10 DMASM 介绍中一样。

一个典型的基于 DMASM 的 DMDSC 一般具备两个 DMDSC 节点。使用块设备的真实物理磁盘做共享存储，并使用 DMASM 来管理块设备。

磁盘数依据磁盘组数目不同而存在不同的最低磁盘数要求。本示例中 1 块 DCR 磁盘、1 块 VOTE 磁盘、1 个 DMDATA 磁盘组（含 1 块磁盘）和 1 个 DMLOG 磁盘组（含 1 块磁盘），共 4 块共享磁盘即可。

本示例以两节点为例搭建 DMDSC 环境。规划将 IP 为 107 的机器作为控制节点。

表 12.1 集群规划

节点机器	实例名称	IP	PORT_NUM

107	CSS0	192.168.100.107	9836
	ASM0		5836
	DSC01		6636
110	CSS1	192.168.100.110	9837
	ASM1		5837
	DSC02		6637

在 4 块共享磁盘中：2 块较小的磁盘（1G）用于创建 DCR、VOTE 磁盘；2 块较大的磁盘（2T）用于创建 ASM 磁盘组（数据磁盘组 DMDATA 和联机日志磁盘组 DMLOG）。

12.1.1 环境准备

从硬件、存储设备、操作系统、网络配置、用户准备、目录规划和磁盘准备这 7 个方面进行准备，下面分别介绍。

12.1.1.1 硬件

IP 地址为 192.168.100.107 和 192.168.100.110 的两台相同配置的机器（380G 内存，2 块网卡）。

12.1.1.2 存储设备

4 块由存储服务器提供的共享存储磁盘，存储服务器通过 SAN 交换机与两台机器相连提供存储服务。

12.1.1.3 操作系统

eulerosv2r7.x86_64，使用 OS 默认的参数配置即可，同一套 DMDSC 环境要求其所在机器的操作系统相同。

12.1.1.4 网络配置

eth0 网卡 192.168.102.x 为内网网段，eth1 网卡 192.168.100.x 为外网网段。

12.1.1.5 用户准备

使用 root 用户创建用于搭建环境的用户和组，以下两机器都要做。

```
#新建组 dmdba

groupadd dmdba

#在组 dmdba（第一个）中创建一个用户 dmdba（第二个）

useradd -g dmdba dmdba

#为用户 dmdba 设置密码

passwd dmdba

#用户输入密码，本例以******代替

******
```

12.1.1.6 目录规划

两机器都要做。

使用 dmdba 用户创建用于 DSC 环境搭建的目录：/home/dmdba/dmdsc。

DM 执行码和工具存放于目录：/home/dmdba/dmdsc/bin。

配置文件存放于目录：107 机器/home/dmdba/dmdsc/data/DSC01 和 110 机器/home/dmdba/dmdsc/data/DSC02。

12.1.1.7 磁盘准备

两机器都要做。本节使用 UDEV 工具来固定磁盘。如果用户想使用 multipath 软件来

固定磁盘，请参考 12.2.1 环境准备的磁盘准备。

1. 通过 scsi_id 获取磁盘信息。

```
[root@test107 ~]# /usr/lib/udev/scsi_id -g -u /dev/sdgp
234074fa79680f431
[root@test110 ~]# /usr/lib/udev/scsi_id -g -u /dev/sdcf
234074fa79680f431
```

以上可以看出 107 的 /dev/sdgp 和 110 的 /dev/sdcf 对应的共享存储服务器上的同一块磁盘。同样取得其他 4 块磁盘的 scsi id 信息。

本步骤的目的是为了在下一步中将相同 scsi_id 的磁盘创建相同的盘符。例如，下一步骤中 RESULT=="234074fa79680f431"即为将 scsi_id 为 234074fa79680f431 的磁盘命名为 DCR 磁盘，并将 DCR 磁盘软链接到/dev_DSC2 目录下并命名为 DCR。

2. 创建磁盘链接。

编写/etc/udev/rules.d/66-dmdevices.rules 配置信息，创建磁盘链接。书写时一个 KERNEL 为一行，不能换行。

```
## DCR 磁盘配置,且在软链接之前创建文件夹 /dev_DSC2
KERNEL=="sd*",SUBSYSTEM=="block",PROGRAM=="/usr/lib/udev/scsi_id --whitelisted
--replace-whitespace
--device=/dev/$name",RESULT=="234074fa79680f431",SYMLINK+="DCR", OWNER="dmdba",
GROUP="dmdba", MODE="0660", RUN+="/bin/sh -c 'chown dmdba:dmdba /dev/$name;mkdir
-p /dev_DSC2; ln -s /dev/DCR /dev_DSC2/DCR'"
## VOTE 磁盘配置
KERNEL=="sd*",SUBSYSTEM=="block",PROGRAM=="/usr/lib/udev/scsi_id --whitelisted
--replace-whitespace
--device=/dev/$name",RESULT=="234074fa746a006ac",SYMLINK+="VOTE", OWNER="dmdba",
```

```
GROUP="dmdba", MODE="0660", RUN+="/bin/sh -c 'chown dmdba:dmdba /dev/$name; ln -s
/dev/VOTE /dev_DSC2/VOTE'"
## DMDATA 磁盘配置
KERNEL=="sd*",SUBSYSTEM=="block",PROGRAM=="/usr/lib/udev/scsi_id --whitelisted
--replace-whitespace
--device=/dev/$name",RESULT=="234074fa7f643c18b",SYMLINK+="DMDATA", OWNER="dmdba",
GROUP="dmdba", MODE="0660", RUN+="/bin/sh -c 'chown dmdba:dmdba /dev/$name; ln -s
/dev/DMDATA /dev_DSC2/DMDATA'"
## DMLOG 磁盘配置，且在搭建完成之后，将权限直接赋予 dmdba 组的 dmdba 用户
KERNEL=="sd*",SUBSYSTEM=="block",PROGRAM=="/usr/lib/udev/scsi_id --whitelisted
--replace-whitespace
--device=/dev/$name",RESULT=="234074fa795eb6057",SYMLINK+="DMLOG", OWNER="dmdba",
GROUP="dmdba", MODE="0660", RUN+="/bin/sh -c 'chown dmdba:dmdba /dev/$name; ln -s
/dev/DMLOG /dev_DSC2/DMLOG ; chown -R dmdba:dmdba /dev_DSC2
'"
```

从配置信息可以看出，通过 scsi id 找到磁盘后，在 /dev/下命名了磁盘信息，然后再做软链接到 /dev_DSC2 下。软链接的目录可以是/dev 下的独立目录，也可以是其他自行创建的独立目录，例如本例中的/dev_DSC2，目录路径必须以“/dev”开始，否则 DM 不会认为这是使用物理磁盘的真实环境。

之所以要软链接到独立目录，是因为 DMDSC 启动时会扫描目录下所有磁盘，如果目录中存在与当前 DMDSC 系统 GROUP_ID/DISK_ID 有冲突的磁盘（例如存在两块 GROUP_ID=0 且 DISK_ID=0 的磁盘），则 DMDSC 将启动失败。因此需要软链接到独立目录，保证该目录下仅包含当前 DMDSC 系统使用的共享存储磁盘，避免 DMDSC 启动失败。

以上配置在 107 和 110 机器完全相同，2 台机器均完成之后，进行下一步。

3. 重启 systemd-udev-trigger 服务。

```
[root@test107 ~]# systemctl restart systemd-udev-trigger

[root@test110 ~]# systemctl restart systemd-udev-trigger
```

搭建之前磁盘权限信息如下:

```
[dmdba@test107 ~]# ls -lth /dev_DSC2/

总用量 0

lrwxrwxrwx 1 dmdba dmdba 32 10 月 31 14:06 DMDATA -> /dev/DMDATA

lrwxrwxrwx 1 dmdba dmdba 32 10 月 31 14:06 DMLOG -> /dev/DMLOG

lrwxrwxrwx 1 dmdba dmdba 27 10 月 31 14:05 VOTE -> /dev/VOTE

lrwxrwxrwx 1 dmdba dmdba 27 10 月 31 14:05 DCR -> /dev/DCR



[dmdba@test110 ~]# ls -lth /dev_DSC2/

总用量 0

lrwxrwxrwx 1 dmdba dmdba 32 10 月 31 14:06 DMDATA -> /dev/DMDATA

lrwxrwxrwx 1 dmdba dmdba 32 10 月 31 14:06 DMLOG -> /dev/DMLOG

lrwxrwxrwx 1 dmdba dmdba 27 10 月 31 14:05 VOTE -> /dev/VOTE

lrwxrwxrwx 1 dmdba dmdba 27 10 月 31 14:05 DCR -> /dev/DCR
```

12.1.2 搭建两节点 DMDSC

搭建的配置文件分别存放于 107 机器的/home/dmdba/dmdsc/data/DSC01 和 110 机器的/home/dmdba/dmdsc/data/DSC02 下。

1. 准备配置文件 DMDCR_CFG.INI 文件，保存到 107 机器的 /home/dmdba/dmdsc/data/DSC01 下。

```
DCR_N_GRP= 3
```

```
DCR_VTD_PATH=/dev_DSC2/VOTE

DCR_OGUID= 1071107589

[GRP]

  DCR_GRP_TYPE = CSS

  DCR_GRP_NAME = GRP_CSS

  DCR_GRP_N_EP = 2

  DCR_GRP_DSKCHK_CNT = 60



[GRP_CSS]

  DCR_EP_NAME = CSS0

  DCR_EP_HOST = 192.168.102.107

  DCR_EP_PORT = 9836



[GRP_CSS]

  DCR_EP_NAME = CSS1

  DCR_EP_HOST = 192.168.102.110

  DCR_EP_PORT = 9837



[GRP]

DCR_GRP_TYPE= ASM

DCR_GRP_NAME= GRP_ASM

DCR_GRP_N_EP= 2

DCR_GRP_DSKCHK_CNT= 60
```

```
[GRP_ASM]

DCR_EP_NAME= ASM0

DCR_EP_SHM_KEY= 64735

DCR_EP_SHM_SIZE= 512

DCR_EP_HOST= 192.168.102.107

DCR_EP_PORT= 5836

DCR_EP_ASM_LOAD_PATH= /dev_DSC2



[GRP_ASM]

DCR_EP_NAME= ASM1

DCR_EP_SHM_KEY= 54736

DCR_EP_SHM_SIZE= 512

DCR_EP_HOST= 192.168.102.110

DCR_EP_PORT= 5837

DCR_EP_ASM_LOAD_PATH= /dev_DSC2



[GRP]

DCR_GRP_TYPE= DB

DCR_GRP_NAME= GRP_DSC

DCR_GRP_N_EP= 2

DCR_GRP_DSKCHK_CNT= 60



[GRP_DSC]

DCR_EP_NAME= DSC01
```

```
DCR_EP_SEQNO= 0

DCR_EP_PORT= 6636



[GRP_DSC]

DCR_EP_NAME= DSC02

DCR_EP_SEQNO= 1

DCR_EP_PORT= 6637
```

2. 在 107 机器使用 DMASMCMD 工具初始化所有磁盘。

```
create dcrdisk '/dev_DSC2/DCR' 'DCR'

create votedisk '/dev_DSC2/VOTE' 'VOTE'

create asmdisk '/dev_DSC2/DMDATA' 'DMDATA'

create asmdisk '/dev_DSC2/DMLOG'  'DMLOG'

init  dcrdisk  '/dev_DSC2/DCR'  from  '/home/dmdba/dmdsc/data/DSC01/dmdcr_cfg.ini'

identified by 'DCRpsd_123'

init votedisk  '/dev_DSC2/VOTE' from '/home/dmdba/dmdsc/data/DSC01/dmdcr_cfg.ini'
```

3. 准备 DMASM 的 MAL 配置文件 DMASVRMAL.INI，保存到 107 的 /home/dmdba/dmdsc/data/DSC01 下。更多关于 DMASVRMAL.INI 的介绍请参考 9.4 MAL 系统配置文件。

```
[MAL_INST1]

MAL_INST_NAME= ASM0

MAL_HOST= 192.168.102.107

MAL_PORT= 4836

[MAL_INST2]

MAL_INST_NAME= ASM1
```

```
MAL_HOST= 192.168.102.110
MAL_PORT= 4837
```

如果 MAL_HOST 为 IPv6 地址，并且当前环境存在多块网卡，则需要用%号指定具体有效的网卡序号或网卡名称。假设当前机器上 IPv6 网卡名为 ens01，则 DMASVRMAL.INI 文件可以写为如下形式：

```
[MAL_INST1]
MAL_INST_NAME= ASM0
MAL_HOST= fe80::610e:9715:5ec6:4ea8%ens01
MAL_PORT= 4836
[MAL_INST2]
MAL_INST_NAME= ASM1
MAL_HOST= fe80::8261:5fff:fe1b:4620%ens01
MAL_PORT= 4837
```

4. 为 DSC02 配置 DMASVRMAL.INI，和 DSC01 的 DMASVRMAL.INI 内容完全一样。保存到 110 的/home/dmdba/dmdsc/data/DSC02 下。如果 MAL_HOST 为 IPv6 地址，则将 DMASVRMAL.INI 复制到 DSC02 后，需要修改 MAL_HOST 中的网卡序号或网卡名称为当前环境下的网卡序号或网卡名称。假设当前机器上 IPv6 网卡名为 ens02，则 DMASVRMAL.INI 文件修改为如下形式：

```
[MAL_INST1]
MAL_INST_NAME= ASM0
MAL_HOST= fe80::610e:9715:5ec6:4ea8%ens02
MAL_PORT= 4836
[MAL_INST2]
MAL_INST_NAME= ASM1
```

```
MAL_HOST= fe80::8261:5fff:fe1b:4620%ens02

MAL_PORT= 4837
```

5. 准备 DMDCR.INI 文件，保存到 107 的/home/dmdba/dmdsc/data/DSC01 下。

```
DMDCR_PATH                      = /dev_DSC2/DCR

DMDCR_MAL_PATH                  = /home/dmdba/dmdsc/data/DSC01/dmasvrmal.ini

DMDCR_SEQNO                     = 0

DMDCR_ASM_RESTART_INTERVAL      = 0

DMDCR_ASM_STARTUP_CMD           = /home/dmdba/dmdsc/bin/dmasmsvr

dcr_ini=/home/dmdba/dmdsc/data/DSC01/dmdcr.ini

DMDCR_DB_RESTART_INTERVAL       = 0

DMDCR_DB_STARTUP_CMD            = /home/dmdba/dmdsc/bin/dmserver

path=/home/dmdba/dmdsc/data/DSC01/DSC01_conf/dm.ini

dcr_ini=/home/dmdba/dmdsc/data/DSC01/dmdcr.ini

DMDCR_LINK_CHECK_IP=192.168.1.88
```

设置了 DMDCR_LINK_CHECK_IP，须为 107 节点的 DMSERVER、DMASMSVR、DMCSS 赋予 ping 权限。

```
sudo setcap cap_net_raw,cap_net_admin=eip /home/dmdba/dmdsc/bin/dmserver

sudo setcap cap_net_raw,cap_net_admin=eip /home/dmdba/dmdsc/bin/dmasmsvr

sudo setcap cap_net_raw,cap_net_admin=eip /home/dmdba/dmdsc/bin/dmcss
```

6. 为 DSC02 配置 DMDCR.INI，保存到 110 的 /home/dmdba/dmdsc/data/DSC02 下。

```
DMDCR_PATH                      = /dev_DSC2/DCR

DMDCR_MAL_PATH                  = /home/dmdba/dmdsc/data/DSC02/dmasvrmal.ini
```

```
DMDCR_SEQNO                    = 1

DMDCR_ASM_RESTART_INTERVAL     = 0

DMDCR_ASM_STARTUP_CMD          = /home/dmdba/dmdsc/bin/dmasmsvr

dcr_ini=/home/dmdba/dmdsc/data/DSC02/dmdcr.ini

DMDCR_DB_RESTART_INTERVAL      = 0

DMDCR_DB_STARTUP_CMD           = /home/dmdba/dmdsc/bin/dmserver

path=/home/dmdba/dmdsc/data/DSC02/DSC02_conf/dm.ini

dcr_ini=/home/dmdba/dmdsc/data/DSC02/dmdcr.ini

DMDCR_LINK_CHECK_IP=192.168.1.88
```

设置了 DMDCR_LINK_CHECK_IP, 须为 110 节点的 DMSERVER、DMASMSVR、DMCSS 赋予 ping 权限。

```
sudo setcap cap_net_raw,cap_net_admin=eip /home/dmdba/dmdsc/bin/dmserver

sudo setcap cap_net_raw,cap_net_admin=eip /home/dmdba/dmdsc/bin/dmasmsvr

sudo setcap cap_net_raw,cap_net_admin=eip /home/dmdba/dmdsc/bin/dmcss
```

7. 启动 DMCSS、DMASM 服务程序。

主节点 107 启动 DMCSS:

```
./dmcss dcr_ini=/home/dmdba/dmdsc/data/DSC01/dmdcr.ini
```

另一节点 110 启动 DMCSS:

```
./dmcss dcr_ini=/home/dmdba/dmdsc/data/DSC02/dmdcr.ini
```

主节点 107 启动 DMASMSVR:

```
./dmasmsvr dcr_ini=/home/dmdba/dmdsc/data/DSC01/dmdcr.ini
```

另一节点 110 启动 DMASMSVR:

```
./dmasmsvr dcr_ini=/home/dmdba/dmdsc/data/DSC02/dmdcr.ini
```

8. 使用 DMASMTOOL 工具创建 ASM 磁盘组，在 107 上登录创建:

```
./dmasmtool dcr_ini=/home/dmdba/dmdsc/data/DSC01/dmdcr.ini
```

创建一个 DATA 磁盘组和一个 LOG 磁盘组。

```
#创建 DATA 磁盘组

CREATE DISKGROUP DMDATA    asmdisk '/dev_DSC2/DMDATA'

#创建 LOG 磁盘组

CREATE DISKGROUP DMLOG     asmdisk '/dev_DSC2/DMLOG'
```

9. 在 107 机器上准备 DMINIT.INI 配置文件，保存到 /home/dmdba/dmdsc/data/DSC01 目录下。

```
DB_NAME= dsc2

SYSTEM_PATH= +DMDATA/data

SYSTEM= +DMDATA/data/dsc2/system.dbf

SYSTEM_SIZE= 128

ROLL= +DMDATA/data/dsc2/roll.dbf

ROLL_SIZE= 128

MAIN= +DMDATA/data/dsc2/main.dbf

MAIN_SIZE= 128

CTL_PATH= +DMDATA/data/dsc2/dm.ctl

LOG_SIZE= 2048

DCR_PATH= /dev_DSC2/DCR

DCR_SEQNO= 0

AUTO_OVERWRITE= 2

PAGE_SIZE = 16

EXTENT_SIZE = 16

SYSDBA_PWD=DMdba_123
```

```
SYSAUDITOR_PWD=DMauditor_123



[DSC01]

CONFIG_PATH= /home/dmdba/dmdsc/data/DSC01/DSC01_conf

PORT_NUM = 6636

MAL_HOST= 192.168.102.107

MAL_PORT= 6536

LOG_PATH= +DMLOG/log/DSC01_log1.log

LOG_PATH= +DMLOG/log/DSC01_log2.log

[DSC02]

CONFIG_PATH= /home/dmdba/dmdsc/data/DSC02/DSC02_conf

PORT_NUM = 6637

MAL_HOST= 192.168.102.110

MAL_PORT= 6537

LOG_PATH= +DMLOG/log/DSC02_log1.log

LOG_PATH= +DMLOG/log/DSC02_log2.log
```

10. 使用 DMINIT 初始化一个节点的数据库环境。

选择一个节点，启动 DMINIT 初始化数据库，这里以 107 为例。DMINIT 执行完成后，会在 config_path 目录（/home/dmdba/dmdsc/data/DSC01/DSC01_conf 和 /home/dmdba/dmdsc/data/DSC02/DSC02_conf）下生成配置文件 DM.INI 和 DMMAL.INI。

```
./dminit control=/home/dmdba/dmdsc/data/DSC01/dminit.ini
```

11. 使用拷贝的方式配置另外一个节点的数据库环境。

将 107 上初始化库时产生的 DSC02 节点的配置文件（整个

/home/dmdba/dmdsc/data/DSC02 文件夹）复制到 110 机器的/home/dmdba/dmdsc/data/DSC02/目录下。之后就可以启动数据库服务器了。

```
scp -r /home/dmdba/dmdsc/data/DSC02/*
dmdba@192.168.100.110:/home/dmdba/dmdsc/data/DSC02/
```

需要注意的是，如果 DMMAL.INI 文件中的 MAL_HOST 为 IPv6 地址，则将配置文件复制到 110 机器上后，需要修改 MAL_HOST 中的网卡序号或网卡名称为当前环境下的网卡序号或网卡名称。

12. 启动数据库服务器

分别启动两个节点的服务器。

```
107 节点服务器启动:
cd /home/dmdba/dmdsc/bin
./dmserver dcr_ini=/home/dmdba/dmdsc/data/DSC01/dmdcr.ini
/home/dmdba/dmdsc/data/DSC01/DSC01_conf/dm.ini
110 节点服务器启动:
cd /home/dmdba/dmdsc/bin
./dmserver dcr_ini=/home/dmdba/dmdsc/data/DSC02/dmdcr.ini
/home/dmdba/dmdsc/data/DSC02/DSC02_conf/dm.ini
```

13. 配置并启动 DMCSSM 监视器。

现在我们搭建监视器，配置 DMCSSM.INI 文件。

DMCSSM 在任何机器上均可以启动，只要该台机器和 DMDSC 的真实机器网络是相通的，就可以监控 DMDSC 集群信息。

这里我们选择在 107 机器上搭建监视器。/home/dmdba/dmdsc/data 目录中 DMCSSM.INI 详细内容如下:

```
#和 DMDCR_CFG.INI 中的 DCR_OGUID 保持一致
```

```
CSSM_OGUID =     1071107589



#配置所有 CSS 的连接信息,

#与 DMDCR_CFG.INI 中 CSS 配置项的 DCR_EP_HOST 和 DCR_EP_PORT 保持一致

CSSM_CSS_IP = 192.168.102.107:9836

CSSM_CSS_IP = 192.168.102.110:9837



CSSM_LOG_PATH =     /home/dmdba/dmdsc/data/cssm_log #监视器日志文件存放路径

CSSM_LOG_FILE_SIZE          =     32          #每个日志文件最大 32M

CSSM_LOG_SPACE_LIMIT    =     0           #不限定日志文件总占用空间
```

创建 DMCSSM 的日志存放路径。

```
mkdir /home/dmdba/dmdsc/data/cssm_log
```

启动 DMCSSM 集群监视器。

```
./dmcssm ini_path=/home/dmdba/dmdsc/data/dmcssm.ini
```

DMCSSM 启动之后，可使用 show 命令在 DMCSSM 监视器中查看集群状态信息。

```
show



monitor current time:2024-10-21 17:33:15, n_group:3

==================== group[name = GRP_CSS, seq = 0, type = CSS, Control Node = 0]

=========================================



[CSS1] auto check = TRUE, global info:

[ASM1] auto restart = FALSE

[DSC1] auto restart = FALSE
```

```
[CSS2] auto check = TRUE, global info:

[ASM2] auto restart = FALSE

[DSC2] auto restart = FALSE

[CSS3] auto check = TRUE, global info:

[ASM3] auto restart = FALSE

[DSC3] auto restart = FALSE

[CSS4] auto check = TRUE, global info:

[ASM4] auto restart = FALSE

[DSC4] auto restart = FALSE


ep: css_time            inst_name      seqno          port       mode         inst_status

vtd_status         is_ok        active        guid               pid            ts

2024-10-21 17:33:14       CSS1         0           58004           Control Node          OPEN

WORKING            OK          TRUE        585773734          51318          585781129

2024-10-21 17:33:14       CSS2         1           58005           Normal Node          OPEN

WORKING            OK          TRUE        585774897          51501          585782289

2024-10-21 17:33:14       CSS3         2           58006           Normal Node          OPEN

WORKING            OK          TRUE        585776143          51558          585783532

2024-10-21 17:33:14       CSS4         3           58007           Normal Node          OPEN

WORKING            OK          TRUE        585777402          51631          585784789


==================== group[name = GRP_ASM, seq = 1, type = ASM, Control Node = 0]

=========================================
```

```
n_ok_ep = 4

ok_ep_arr(index, seqno):

(0, 0)

(1, 1)

(2, 2)

(3, 3)



sta = OPEN, sub_sta = STARTUP

break ep = NULL

recover ep = NULL



crash process over flag is TRUE

ep:css_time            inst_name       seqno       port          mode             inst_status

vtd_status       is_ok         active         guid              pid              ts

2024-10-21 17:33:14      ASM1             0           58008       Control Node       OPEN

WORKING          OK           TRUE        585784553       51855       585791922

2024-10-21 17:33:14      ASM2             1           58009         Normal Node       OPEN

WORKING          OK           TRUE        585785798       52034       585793165

2024-10-21 17:33:14      ASM3             2           58010         Normal Node       OPEN

WORKING          OK           TRUE         585787055       52087       585794418

2024-10-21 17:33:14      ASM4             3           58011          Normal Node         OPEN

WORKING          OK           TRUE         585788311          52157       585795672



==================== group[name = GRP_DSC, seq = 2, type = DB, Control Node = 0]
```

```
========================================


n_ok_ep = 4

ok_ep_arr(index, seqno):

(0, 0)

(1, 1)

(2, 2)

(3, 3)


sta = OPEN, sub_sta = STARTUP

break ep = NULL

recover ep = NULL


crash process over flag is TRUE

ep:  css_time              inst_name      seqno      port          mode           inst_status

vtd_status      is_ok      active         guid           pid            ts

2024-10-21 17:33:14        DSC1           0          58000       Control Node         OPEN

WORKING         OK         TRUE        120873204         54581       120880520

2024-10-21 17:33:14        DSC2           1          58001       Normal Node          OPEN

WORKING         OK         TRUE        120873800         54736       120881112

2024-10-21 17:33:14        DSC3           2          58002       Normal Node          OPEN

WORKING         OK         TRUE        120874447         54940       120881754

2024-10-21 17:33:14        DSC4           3          58003       Normal Node          OPEN

WORKING         OK         TRUE        120875004         55152       120882305
```

至此，基于 DMASM 的 DMDSC 已经搭建完成。

12.2 基于 DMASM 镜像的 DMDSC

本节用到的 DMASM 功能和 11 DMASM 镜像介绍中一样。

一个典型的基于 DMASM 镜像的 DMDSC 一般具备两个 DMDSC 节点。使用块设备的真实物理磁盘做共享存储，并使用 DMASM 镜像来管理块设备。磁盘数依据磁盘组数目和副本数不同而存在不同的最低磁盘数要求。例如：若全部为单副本则三个磁盘组（通常会有 SYSTEM、DATA、LOG 三个磁盘组）各只需要一块，共 3 块共享磁盘即可。

本实例以两节点三副本为例搭建镜像环境。规划将 IP 为 109 的机器作为控制节点。

表 12.2 集群规划

节点机器	实例名称	IP	PORT_NUM
109	CSS0	192.168.100.109	7936
	ASM0		7536
	DSC01		7236
107	CSS1	192.168.100.107	7937
	ASM1		7537
	DSC02		7237

本示例中使用 23 块共享磁盘，其中 5 块较小的磁盘（1G）用于创建 DCRV 系统磁盘组（SYSTEM），18 块较大的磁盘（2T）用于创建 ASM 磁盘组（数据磁盘组 DATA 和联机日志磁盘组 LOG）。

表 12.3 集群磁盘规划

磁盘组	副本数	故障组内磁盘数	单块磁盘大小	总磁盘数
SYSTEM	三副本	系统磁盘组比较特殊，每个磁盘位于一个故障组中	1G	5

DATA		3	2048G	9
LOG		3	2048G	9

12.2.1 环境准备

从硬件、存储设备、操作系统、网络配置、用户准备、目录规划和磁盘准备这 7 个方面进行准备，下面分别介绍。

12.2.1.1 硬件

IP 地址为 192.168.100.109 和 192.168.100.107 的两台相同配置的机器（380G 内存，2 块网卡）。

12.2.1.2 存储设备

11 块由存储服务器提供的共享存储磁盘，存储服务器通过 SAN 交换机与两台机器相连提供存储服务。

12.2.1.3 操作系统

eulerosv2r7.x86_64，使用 OS 默认的参数配置即可，同一套 DMDSC 环境要求其所在机器的操作系统相同。

12.2.1.4 网络配置

eth0 网卡 192.168.102.x 为内网网段，eth1 网卡 192.168.100.x 为外网网段。

12.2.1.5 用户准备

使用 root 用户创建用于搭建环境的用户和组，以下两机器都要做。

```
#新建组 dmdba

groupadd dmdba

#在组 dmdba（第一个）中创建一个用户 dmdba（第二个）

useradd -g dmdba dmdba

#为用户 dmdba 设置密码

passwd dmdba

#用户输入密码，本例以******代替

******
```

12.2.1.6 目录规划

两机器都要做。

使用 dmdba 用户创建用于镜像环境搭建的目录：/home/dmdba/dmdsc。

DM 执行码和工具存放于目录：/home/dmdba/dmdsc/bin。

配置文件存放于目录：109 机器/home/dmdba/dmdsc/data/DSC01 和 107 机器/home/dmdba/dmdsc/data/DSC02。

12.2.1.7 磁盘准备

两机器都要做。

本节提供两种磁盘固定方式：一是使用 UDEV 工具；二是使用 multipath 软件。用户采用其中一种即可。推荐用户使用第一种方式。

12.2.1.7.1 UDEV 方式

1. 通过 scsi_id 获取磁盘信息。

```
[root@test107 ~]# /usr/lib/udev/scsi_id -g -u /dev/sdgp
```

```
234074fa79680f431
[root@test110 ~]# /usr/lib/udev/scsi_id -g -u /dev/sdcf
234074fa79680f431
```

以上可以看出 107 的 /dev/sdgp 和 109 的 /dev/sdcf 对应的共享存储服务器上的同一块磁盘。同样取得其他 22 块磁盘的 scsi id 信息。

本步骤的目的是为了在下一步中将相同 scsi_id 的磁盘创建相同的盘符。例如，下一步骤中 RESULT=="234074fa79680f431"即为将 scsi_id 为 234074fa79680f431 的所有磁盘均命名为 DCRV1 磁盘，并将 DCRV1 磁盘软链接到 dev_DSC2/DCRV1 目录下。

2． 创建磁盘链接。

编写/etc/udev/rules.d/66-dmdevices.rules 配置信息，创建磁盘链接。书写时一个 KERNEL 为一行，不能换行。

```
## DCRV1 磁盘配置，且在软链接之前创建文件夹 /dev_DSC_HDD
KERNEL=="sd*",SUBSYSTEM=="block",PROGRAM=="/usr/lib/udev/scsi_id --whitelisted
--replace-whitespace
--device=/dev/$name",RESULT=="234074fa79680f431",SYMLINK+="DCRV1", OWNER="dmdba",
GROUP="dmdba", MODE="0660", RUN+="/bin/sh -c 'chown dmdba:dmdba /dev/$name;mkdir
-p /dev_DSC_HDD; ln -s /dev/DCRV1 /dev_DSC_HDD/DCRV1'"
## DCRV2 磁盘配置
KERNEL=="sd*",SUBSYSTEM=="block",PROGRAM=="/usr/lib/udev/scsi_id --whitelisted
--replace-whitespace
--device=/dev/$name",RESULT=="234074fa79680f432",SYMLINK+="DCRV2", OWNER="dmdba",
GROUP="dmdba", MODE="0660", RUN+="/bin/sh -c 'chown dmdba:dmdba /dev/$name; ln -s
/dev/DCRV2 /dev_DSC_HDD/DCRV2'"
## DCRV3 磁盘配置
```

```
KERNEL=="sd*",SUBSYSTEM=="block",PROGRAM=="/usr/lib/udev/scsi_id --whitelisted
--replace-whitespace
--device=/dev/$name",RESULT=="234074fa79680f433",SYMLINK+="DCRV3", OWNER="dmdba",
GROUP="dmdba", MODE="0660", RUN+="/bin/sh -c 'chown dmdba:dmdba /dev/$name; ln -s
/dev/DCRV3 /dev_DSC_HDD/DCRV3'"
## DCRV4 磁盘配置
KERNEL=="sd*",SUBSYSTEM=="block",PROGRAM=="/usr/lib/udev/scsi_id --whitelisted
--replace-whitespace
--device=/dev/$name",RESULT=="234074fa79680f434",SYMLINK+="DCRV4", OWNER="dmdba",
GROUP="dmdba", MODE="0660", RUN+="/bin/sh -c 'chown dmdba:dmdba /dev/$name; ln -s
/dev/DCRV4 /dev_DSC_HDD/DCRV4'"
## DCRV5 磁盘配置
KERNEL=="sd*",SUBSYSTEM=="block",PROGRAM=="/usr/lib/udev/scsi_id --whitelisted
--replace-whitespace
--device=/dev/$name",RESULT=="234074fa79680f435",SYMLINK+="DCRV5", OWNER="dmdba",
GROUP="dmdba", MODE="0660", RUN+="/bin/sh -c 'chown dmdba:dmdba /dev/$name; ln -s
/dev/DCRV5 /dev_DSC_HDD/DCRV5'"
## DMDATA 磁盘（MIR_HDD_1、MIR_HDD_2……MIR_HDD_9）配置
KERNEL=="sd*",SUBSYSTEM=="block",PROGRAM=="/usr/lib/udev/scsi_id --whitelisted
--replace-whitespace
--device=/dev/$name",RESULT=="234074fa7f643c18b",SYMLINK+="MIR_HDD_1",
OWNER="dmdba", GROUP="dmdba", MODE="0660", RUN+="/bin/sh -c 'chown dmdba:dmdba
/dev/$name; ln -s /dev/MIR_HDD_1 /dev_DSC_HDD/MIR_HDD_1'"
KERNEL=="sd*",SUBSYSTEM=="block",PROGRAM=="/usr/lib/udev/scsi_id --whitelisted
```

```
--replace-whitespace
--device=/dev/$name",RESULT=="234074fa7f643c18c",SYMLINK+="MIR_HDD_2",
OWNER="dmdba", GROUP="dmdba", MODE="0660", RUN+="/bin/sh -c 'chown dmdba:dmdba
/dev/$name; ln -s /dev/MIR_HDD_2 /dev_DSC_HDD/MIR_HDD_2'"
KERNEL=="sd*",SUBSYSTEM=="block",PROGRAM=="/usr/lib/udev/scsi_id --whitelisted
--replace-whitespace
--device=/dev/$name",RESULT=="234074fa7f643c18d",SYMLINK+="MIR_HDD_3",
OWNER="dmdba", GROUP="dmdba", MODE="0660", RUN+="/bin/sh -c 'chown dmdba:dmdba
/dev/$name; ln -s /dev/MIR_HDD_3 /dev_DSC_HDD/MIR_HDD_3'"
……省略部分为使用同样的方法配置 MIR_HDD_4……MIR_HDD_9
## DMLOG 磁盘配置（MIR_HDD_10、MIR_HDD_11……MIR_HDD_18），且在搭建完成之后，将权限直
接赋予 dmdba 组的 dmdba 用户
KERNEL=="sd*",SUBSYSTEM=="block",PROGRAM=="/usr/lib/udev/scsi_id --whitelisted
--replace-whitespace
--device=/dev/$name",RESULT=="234074fa7f643c18e",SYMLINK+="MIR_HDD_10",
OWNER="dmdba", GROUP="dmdba", MODE="0660", RUN+="/bin/sh -c 'chown dmdba:dmdba
/dev/$name; ln -s /dev/MIR_HDD_10/dev_DSC_HDD/ MIR_HDD_10; chown -R
dmdba:dmdba /dev_DSC_HDD'"
KERNEL=="sd*",SUBSYSTEM=="block",PROGRAM=="/usr/lib/udev/scsi_id --whitelisted
--replace-whitespace
--device=/dev/$name",RESULT=="234074fa7f643c18f",SYMLINK+="MIR_HDD_11",
OWNER="dmdba", GROUP="dmdba", MODE="0660", RUN+="/bin/sh -c 'chown dmdba:dmdba
/dev/$name; ln -s /dev/MIR_HDD_11/dev_DSC_HDD/MIR_HDD_11'" chown -R dmdba:dmdba
/dev_DSC_HDD'"
```

```
KERNEL=="sd*",SUBSYSTEM=="block",PROGRAM=="/usr/lib/udev/scsi_id --whitelisted
--replace-whitespace
--device=/dev/$name",RESULT=="234074fa7f643c18g",SYMLINK+="MIR_HDD_12",
OWNER="dmdba", GROUP="dmdba", MODE="0660", RUN+="/bin/sh -c 'chown dmdba:dmdba
/dev/$name; ln -s /dev/MIR_HDD_12/dev_DSC_HDD/MIR_HDD_12; chown -R
dmdba:dmdba /dev_DSC_HDD'"
……省略部分为使用同样的方法配置 MIR_HDD_13……MIR_HDD_18
```

从配置信息可以看出，通过 scsi id 找到磁盘后，在 /dev/下命名了磁盘信息，然后再做软链接到/dev_DSC_HDD 下。软链接的目录可以是/dev 下的独立目录，也可以是其他自行创建的独立目录，例如本例中的/dev_DSC_HDD，目录路径必须以“/dev”开始，否则 DM 不会认为这是使用物理磁盘的真实环境。

之所以要软链接到独立目录，是因为 DMDSC 启动时会扫描目录下所有磁盘，如果目录中存在与当前 DMDSC 系统 GROUP_ID/DISK_ID 有冲突的磁盘（例如存在两块 GROUP_ID=0 且 DISK_ID=0 的磁盘），则 DMDSC 将启动失败。因此需要软链接到独立目录，保证该目录下仅包含当前 DMDSC 系统使用的共享存储磁盘，避免 DMDSC 启动失败。

以上配置在 107 和 109 机器完全相同，2 台机器均完成之后，进行下一步。

3． 重启 systemd-udev-trigger 服务。

```
[root@test107 ~]# systemctl restart systemd-udev-trigger
[root@test109 ~]# systemctl restart systemd-udev-trigger
```

搭建之前磁盘权限信息如下：

```
[dmdba@test107 ~]# ls -lth /dev_DSC_HDD/
总用量 0
lrwxrwxrwx 1 dmdba dmdba 32 11 月  7 14:06 DCRV1 -> /dev/DCRV1
lrwxrwxrwx 1 dmdba dmdba 32 11 月  7 14:06 DCRV2 -> /dev/DCRV2
```

```
lrwxrwxrwx 1 dmdba dmdba 32 11 月  7 14:06 DCRV3 -> /dev/DCRV3
lrwxrwxrwx 1 dmdba dmdba 32 11 月  7 14:06 DCRV4 -> /dev/DCRV4
lrwxrwxrwx 1 dmdba dmdba 32 11 月  7 14:06 DCRV5 -> /dev/DCRV5
lrwxrwxrwx 1 dmdba dmdba 27 11 月  7 14:05 MIR_HDD_1 -> /dev/MIR_HDD_1
lrwxrwxrwx 1 dmdba dmdba 27 11 月  7 14:05 MIR_HDD_2 -> /dev/MIR_HDD_2
lrwxrwxrwx 1 dmdba dmdba 27 11 月  7 14:05 MIR_HDD_3 -> /dev/MIR_HDD_3
lrwxrwxrwx 1 dmdba dmdba 27 11 月  7 14:05 MIR_HDD_4 -> /dev/MIR_HDD_4
lrwxrwxrwx 1 dmdba dmdba 27 11 月  7 14:05 MIR_HDD_5 -> /dev/MIR_HDD_5
lrwxrwxrwx 1 dmdba dmdba 27 11 月  7 14:05 MIR_HDD_6 -> /dev/MIR_HDD_6

[dmdba@test109 ~]# ls -lth /dev_DSC_HDD/
总用量 0
lrwxrwxrwx 1 dmdba dmdba 32 11 月  7 14:06 DCRV1 -> /dev/DCRV1
lrwxrwxrwx 1 dmdba dmdba 32 11 月  7 14:06 DCRV2 -> /dev/DCRV2
lrwxrwxrwx 1 dmdba dmdba 32 11 月  7 14:06 DCRV3 -> /dev/DCRV3
lrwxrwxrwx 1 dmdba dmdba 32 11 月  7 14:06 DCRV4 -> /dev/DCRV4
lrwxrwxrwx 1 dmdba dmdba 32 11 月  7 14:06 DCRV5 -> /dev/DCRV5
lrwxrwxrwx 1 dmdba dmdba 27 11 月  7 14:05 MIR_HDD_1 -> /dev/MIR_HDD_1
lrwxrwxrwx 1 dmdba dmdba 27 11 月  7 14:05 MIR_HDD_2 -> /dev/MIR_HDD_2
lrwxrwxrwx 1 dmdba dmdba 27 11 月  7 14:05 MIR_HDD_3 -> /dev/MIR_HDD_3
lrwxrwxrwx 1 dmdba dmdba 27 11 月  7 14:05 MIR_HDD_4 -> /dev/MIR_HDD_4
lrwxrwxrwx 1 dmdba dmdba 27 11 月  7 14:05 MIR_HDD_5 -> /dev/MIR_HDD_5
lrwxrwxrwx 1 dmdba dmdba 27 11 月  7 14:05 MIR_HDD_6 -> /dev/MIR_HDD_6
```

12.2.1.7.2 multipath 方式

1. 多路径软件准备：使用 multipath 做多路径。

mutipath 通常为 Linux 自带，如果没有安装配置，那么可以参照如下流程进行安装并配置。

```
yum install *multipath* -y

mpathconf --enable  # 生成/etc/multipath.conf

systemctl start multipathd

systemctl enable multipathd
```

mutipath 配置完成。

完成后新加到系统上的磁盘都会在/dev/mapper 中显示，可以通过命令快速查看磁盘信息：

```
fdisk -l | grep mapper
```

2. 创建磁盘链接。

通过以上配置，共享存储服务器提供的磁盘被 mutipath 发现后会在/dev/mapper 下看到 mutipath 为其创建的链接。

```
[dmdba@test109 mapper]# ls

control  euleros-root  mpathad  mpathaf  mpathah  mpathaj  mpathap  mpathar
mpathat  mpathav  mpathax  mpathba  mpathbc  mpathbe  mpathbg  mpathbi
mpathbk  mpathbm  mpathbo  mpathbq  mpathbs  mpathh  mpathj  mpathl
euleros-home  euleros-swap  mpathae  mpathag  mpathai  mpathak  mpathaq
mpathas  mpathau  mpathaw  mpathay  mpathbb  mpathbd  mpathbf  mpathbh
mpathbj  mpathbl  mpathbn  mpathbp  mpathbr  mpathbt  mpathi  mpathk  mpathm
```

由于所有新加入的磁盘都会被放到该目录下，而 DMDSC 环境在启动时会读取配置目录

中所有设备的头信息，因此为了避免多套环境间互相影响建议将其中所需要用到的磁盘单独再链接到一个新的目录，该目录可以是/dev 下的独立目录，也可以是其他自行创建的独立目录，目录路径必须以“/dev”开始，否则 DM 不会认为这是使用物理磁盘的真实环境。这里将磁盘链接到/dev_DSC_HDD 目录下并重命名。

这里使用 root 用户创建目录/dev_DSC_HDD，并赋权给 dmdba 用户，两机器都要做。dev_DSC_HDD 目录名称可以自取，以 dev 开头即可。

```
[root@test109 /]# mkdir /dev_DSC_HDD
[root@test109 /]# chown -R dmdba:dmdba dev_DSC_HDD
[root@test107 /]# mkdir /dev_DSC_HDD
[root@test107 /]# chown -R dmdba:dmdba dev_DSC_HDD
```

链接之前需要注意：因为同样的磁盘在两机器/dev/mapper 下名称可能不同，不同的磁盘在两机器/dev/mapper 下名称也可能相同。这会导致不同节点的 DMDSC 对同一名称的磁盘读取出不同的内容，简单通过两个磁盘名称一样无法判断是否为同一磁盘。因此需要通过命令/usr/lib/udev/scsi_id -g -u mpathx 读取磁盘通用唯一识别码（UUID），从而判断两机器上的链接分别对应哪块磁盘。例如下面这个例子可以判断 109 机器上的/dev/mapper/mpathba 与 107 机器上的/dev/mapper/mpathbb 为同一磁盘，因此在链接时应该将这两个磁盘更改为同一名称。

```
[dmdba@test109 mapper]# /usr/lib/udev/scsi_id -g -u mpathba
3600b34203c36824d1178d8057d7900d0
[dmdba@test107 mapper]# /usr/lib/udev/scsi_id -g -u mpathbb
3600b34203c36824d1178d8057d7900d0
#两机器上创建链接并更名为相同的名称
[dmdba@test109 dev_DSC_HDD]$ ln -s /dev/mapper/mpathba MIR_HDD_1
[dmdba@test107 dev_DSC_HDD]$ ln -s /dev/mapper/mpathba MIR_HDD_1
```

之后使用 root 用户依次将所有磁盘链接到/dev_DSC_HDD 下并更名为便于识别的名称。如下所示，DCRV 为 1G 的磁盘用于创建 DCRV 系统磁盘组，MIR_HDD_*为 2T 的磁盘用于创建 DATA 和 LOG 磁盘组。

109 机器上查看创建完成的链接如下：

```
[dmdba@test109 dev_DSC_HDD]$ ll
总用量 0
lrwxrwxrwx 1 dmdba dmdba 18 9 月  20 10:48 DCRV1 -> /dev/mapper/mpathl
lrwxrwxrwx 1 dmdba dmdba 18 9 月  20 10:48 DCRV2 -> /dev/mapper/mpathm
lrwxrwxrwx 1 dmdba dmdba 18 9 月  20 10:48 DCRV3 -> /dev/mapper/mpathk
lrwxrwxrwx 1 dmdba dmdba 18 9 月  20 10:48 DCRV4 -> /dev/mapper/mpathi
lrwxrwxrwx 1 dmdba dmdba 18 9 月  20 10:48 DCRV5 -> /dev/mapper/mpathj
lrwxrwxrwx 1 dmdba dmdba 19 9 月  20 10:58 MIR_HDD_1 -> /dev/mapper/mpathba
lrwxrwxrwx 1 dmdba dmdba 19 9 月  20 10:48 MIR_HDD_2 -> /dev/mapper/mpathbb
lrwxrwxrwx 1 dmdba dmdba 19 9 月  20 10:48 MIR_HDD_3 -> /dev/mapper/mpathbc
lrwxrwxrwx 1 dmdba dmdba 19 9 月  20 10:48 MIR_HDD_4 -> /dev/mapper/mpathbd
lrwxrwxrwx 1 dmdba dmdba 19 9 月  20 10:48 MIR_HDD_5 -> /dev/mapper/mpathbe
lrwxrwxrwx 1 dmdba dmdba 19 9 月  20 10:48 MIR_HDD_6 -> /dev/mapper/mpathbf
```

107 机器上查看创建完成的链接如下：

```
[dmdba@test107 dev_DSC_HDD]$ ll
总用量 0
lrwxrwxrwx 1 dmdba dmdba 18 8 月  30 08:47 DCRV1 -> /dev/mapper/mpathi
lrwxrwxrwx 1 dmdba dmdba 18 8 月  30 08:47 DCRV2 -> /dev/mapper/mpathg
lrwxrwxrwx 1 dmdba dmdba 18 8 月  30 08:48 DCRV3 -> /dev/mapper/mpathj
lrwxrwxrwx 1 dmdba dmdba 18 8 月  30 08:48 DCRV4 -> /dev/mapper/mpathm
```

```
lrwxrwxrwx 1 dmdba dmdba 18 8 月   30 08:48 DCRV5 -> /dev/mapper/mpathh
lrwxrwxrwx 1 dmdba dmdba 19 8 月   29 17:45 MIR_HDD_1 -> /dev/mapper/mpathbb
lrwxrwxrwx 1 dmdba dmdba 19 8 月   29 17:45 MIR_HDD_2 -> /dev/mapper/mpathbc
lrwxrwxrwx 1 dmdba dmdba 19 8 月   29 17:45 MIR_HDD_3 -> /dev/mapper/mpathbd
lrwxrwxrwx 1 dmdba dmdba 19 8 月   29 17:45 MIR_HDD_4 -> /dev/mapper/mpathbe
lrwxrwxrwx 1 dmdba dmdba 19 8 月   29 17:45 MIR_HDD_5 -> /dev/mapper/mpathbf
lrwxrwxrwx 1 dmdba dmdba 19 8 月   29 17:45 MIR_HDD_6 -> /dev/mapper/mpathbg
```

3. 磁盘权限更改。

以上创建的链接虽然显示权限为 dmdba，但是链接对应的实际磁盘（通常为/dev/dm-*）权限还未更改。

首先，需要检查一下权限是否变动。如果系统重启，会导致权限变成 root，需要手动执行以下权限更新。使用 root 用户将权限赋给 dmdba:

```
[root@test109 dev_DSC_HDD]# chown dmdba:dmdba *
[root@test107 dev_DSC_HDD]# chown dmdba:dmdba *
```

然后，使用 ll 命令查看/dev/dm-*可以看到链接实际对应的磁盘权限应该已经变为 dmdba 用户了。以 109 上为例:

```
[dmdba@test109 dev_DSC_HDD]$ ll /dev/dm-*
brw-rw---- 1 dmdba dmdba 253, 16 9 月  19 14:53 /dev/dm-16
brw-rw---- 1 dmdba dmdba 253, 18 9 月  19 14:53 /dev/dm-18
brw-rw---- 1 dmdba dmdba 253, 21 9 月  20 09:00 /dev/dm-21
brw-rw---- 1 dmdba dmdba 253, 22 9 月  20 09:00 /dev/dm-22
brw-rw---- 1 dmdba dmdba 253, 23 9 月  20 09:00 /dev/dm-23
brw-rw---- 1 dmdba dmdba 253, 24 9 月  20 09:00 /dev/dm-24
brw-rw---- 1 dmdba dmdba 253, 25 9 月  20 09:00 /dev/dm-25
```

```
brw-rw---- 1 dmdba      dmdba      253, 26 9 月   20 09:00 /dev/dm-26
brw-rw---- 1 dmdba      dmdba      253,   3 9 月   19 14:53 /dev/dm-3
brw-rw---- 1 dmdba      dmdba      253,   4 9 月   19 14:53 /dev/dm-4
brw-rw---- 1 dmdba      dmdba      253,   5 9 月   19 14:53 /dev/dm-5
```

12.2.2 搭建两节点 DMDSC

搭建的配置文件分别存放于 109 机器的/home/dmdba/dmdsc/data/DSC01 和 107 机器的/home/dmdba/dmdsc/data/DSC02 下。

1. 准备配置文件 DMDCR_CFG.INI 文件，保存到 109 机器的 /home/dmdba/dmdsc/data/DSC01 下。与普通 DMDSC 环境相比，镜像环境中的 DCR_DISK_LOAD_PATH 不再指向具体的磁盘，而是 DCRV 磁盘所在的目录 /dev_DSC_HDD。

```
DCR_N_GRP= 3
DCR_DISK_LOAD_PATH = /dev_DSC_HDD
DCR_OGUID= 509317566
[GRP]
  DCR_GRP_TYPE = CSS
  DCR_GRP_NAME = GRP_CSS
  DCR_GRP_N_EP = 2
  DCR_GRP_DSKCHK_CNT = 60

[GRP_CSS]
  DCR_EP_NAME = CSS0
  DCR_EP_HOST = 192.168.102.109
```

```
  DCR_EP_PORT = 7936


[GRP_CSS]
  DCR_EP_NAME = CSS1
  DCR_EP_HOST = 192.168.102.107
  DCR_EP_PORT = 7937


[GRP]
DCR_GRP_TYPE= ASM
DCR_GRP_NAME= GRP_ASM
DCR_GRP_N_EP= 2
DCR_GRP_DSKCHK_CNT= 60


[GRP_ASM]
DCR_EP_NAME= ASM0
DCR_EP_SHM_KEY= 54730
DCR_EP_SHM_SIZE= 512
DCR_EP_HOST= 192.168.102.109
DCR_EP_PORT= 7536


[GRP_ASM]
DCR_EP_NAME= ASM1
DCR_EP_SHM_KEY= 54731
DCR_EP_SHM_SIZE= 512
```

```
DCR_EP_HOST= 192.168.102.107

DCR_EP_PORT= 7537



[GRP]

DCR_GRP_TYPE= DB

DCR_GRP_NAME= GRP_DSC

DCR_GRP_N_EP= 2

DCR_GRP_DSKCHK_CNT= 60



[GRP_DSC]

DCR_EP_NAME= DSC01

DCR_EP_SEQNO= 0

DCR_EP_PORT= 7236



[GRP_DSC]

DCR_EP_NAME= DSC02

DCR_EP_SEQNO= 1

DCR_EP_PORT= 7237
```

2. 在 109 机器使用 DMASMCMDM 工具初始化所有磁盘并创建 DCRV 系统磁盘组，DCRV 系统磁盘组中的磁盘数只能为 1，3，5 其中之一，这里以 5 块 DCRV 磁盘为例。

```
create dcrvdisk '/dev_DSC_HDD/DCRV1' 'dcrv1'

create dcrvdisk '/dev_DSC_HDD/DCRV2' 'dcrv2'

create dcrvdisk '/dev_DSC_HDD/DCRV3' 'dcrv3'

create dcrvdisk '/dev_DSC_HDD/DCRV4' 'dcrv4'
```

```
create dcrvdisk '/dev_DSC_HDD/DCRV5' 'dcrv5'

create asmdisk '/dev_DSC_HDD/MIR_HDD_1'   'data1'

create asmdisk '/dev_DSC_HDD/MIR_HDD_2'   'data2'

create asmdisk '/dev_DSC_HDD/MIR_HDD_3'   'data3'

create asmdisk '/dev_DSC_HDD/MIR_HDD_4'   'log1'

create asmdisk '/dev_DSC_HDD/MIR_HDD_5'   'log2'

create asmdisk '/dev_DSC_HDD/MIR_HDD_6'   'log3'

CREATE SYSTEM DISKGROUP   ASMDISK

'/dev_DSC_HDD/DCRV1','/dev_DSC_HDD/DCRV2','/dev_DSC_HDD/DCRV3','/dev_DSC_HDD/

DCRV4','/dev_DSC_HDD/DCRV5'   ATTRIBUTE

CONFIG='/home/dmdba/dmdsc/data/DSC01/dmdcr_cfg.ini', passwd='DCRpsd_123'
```

可以启动 DMASMCMDM 工具，依次输入以上命令，或者将命令写入 asmcmd.txt 文件，执行 DMASMCMDM SCRIPT_FILE=asmcmd.txt，只需在一台机器执行即可。

用户没有指定脚本文件，则 DMASMCMDM 进入交互模式运行，逐条解析、运行命令；用户指定脚本文件（比如 asmcmd.txt），则以行为单位读取文件内容，并依次执行，执行完成以后，自动退出 DMASMCMDM 工具。脚本文件必须以“#asm script file”开头，否则认为是无效脚本文件；脚本中其它行以“#”表示注释；脚本文件大小不超过 1M。

3. 准备 DMASM 的 MAL 配置文件 DMASVRMAL.INI，规则与普通的 DMDSC 环境一样，保存到 109 的/home/dmdba/dmdsc/data/DSC01 下。更多关于 DMASVRMAL.INI 的介绍请参考 9.4 MAL 系统配置文件。

```
[MAL_INST1]

MAL_INST_NAME= ASM0

MAL_HOST= 192.168.102.109

MAL_PORT= 7436
```

```
[MAL_INST2]

MAL_INST_NAME= ASM1

MAL_HOST= 192.168.102.107

MAL_PORT= 7437
```

4. 为 DSC02 配置 DMASVRMAL.INI，和 DSC01 的 DMASVRMAL.INI 内容完全一样。保存到 107 的/home/dmdba/dmdsc/data/DSC02 下。需要注意的是，如果 MAL_HOST 为 IPv6 地址，则将 DMASVRMAL.INI 复制到 DSC02 后，需要修改 MAL_HOST 中的网卡序号或网卡名称为当前环境下的网卡序号或网卡名称，详细介绍请参考 12.1.2 搭建两节点 DMDSC。

5. 准备 DMDCR.INI 文件，保存到 109 的/home/dmdba/dmdsc/data/DSC01 下，同样注意 DMDCR_PATH 不再指向具体的磁盘而是目录/dev_DSC_HDD。

```
DMDCR_PATH= /dev_DSC_HDD

DMDCR_SEQNO= 0

DMDCR_MAL_PATH= /home/dmdba/dmdsc/data/DSC01/dmasvrmal.ini

DMDCR_ASM_RESTART_INTERVAL= 0

DMDCR_ASM_STARTUP_CMD= /home/dmdba/dmdsc/bin/dmasmsvrm

dcr_ini=/home/dmdba/dmdsc/data/DSC01/dmdcr.ini

DMDCR_DB_RESTART_INTERVAL= 0

DMDCR_DB_STARTUP_CMD= /home/dmdba/dmdsc/bin/dmserver

path=/home/dmdba/dmdsc/data/DSC01/DSC01_conf/dm.ini

dcr_ini=/home/dmdba/dmdsc/data/DSC01/dmdcr.ini

DMDCR_IPC_CONTROL                = 2

DMDCR_LINK_CHECK_IP=192.168.1.88
```

设置了 DMDCR_LINK_CHECK_IP，须为 109 节点的 DMSERVER、DMASMSVRM、

DMCSS 赋予 ping 权限。

```
sudo setcap cap_net_raw,cap_net_admin=eip /home/dmdba/dmdsc/bin/dmserver

sudo setcap cap_net_raw,cap_net_admin=eip /home/dmdba/dmdsc/bin/dmasmsvrm

sudo setcap cap_net_raw,cap_net_admin=eip /home/dmdba/dmdsc/bin/dmcss
```

6. 为 DSC02 配置 DMDCR.INI，保存到 107 的 /home/dmdba/dmdsc/data/DSC02 下。

```
DMDCR_PATH= /dev_DSC_HDD

DMDCR_SEQNO= 1

DMDCR_MAL_PATH= /home/dmdba/dmdsc/data/DSC02/dmasvrmal.ini

DMDCR_ASM_RESTART_INTERVAL= 0

DMDCR_ASM_STARTUP_CMD= /home/dmdba/dmdsc/bin/dmasmsvrm

dcr_ini=/home/dmdba/dmdsc/data/DSC02/dmdcr.ini

DMDCR_DB_RESTART_INTERVAL= 0

DMDCR_DB_STARTUP_CMD= /home/dmdba/dmdsc/bin/dmserver

path=/home/dmdba/dmdsc/data/DSC02/DSC02_conf/dm.ini

dcr_ini=/home/dmdba/dmdsc/data/DSC02/dmdcr.ini

DMDCR_IPC_CONTROL                = 2

DMDCR_LINK_CHECK_IP=192.168.1.88
```

设置了 DMDCR_LINK_CHECK_IP，须为 107 节点的 DMSERVER、DMASMSVRM、DMCSS 赋予 ping 权限。

```
sudo setcap cap_net_raw,cap_net_admin=eip /home/dmdba/dmdsc/bin/dmserver

sudo setcap cap_net_raw,cap_net_admin=eip /home/dmdba/dmdsc/bin/dmasmsvrm

sudo setcap cap_net_raw,cap_net_admin=eip /home/dmdba/dmdsc/bin/dmcss
```

7. 启动 DMCSS、DMASM 服务程序，在镜像环境中对应的执行码为 DMCSS，

DMASMSVRM。

主节点 109 启动 DMCSS：

```
./dmcss dcr_ini=/home/dmdba/dmdsc/data/DSC01/dmdcr.ini

DMCSS V8

DMCSS IS READY

[2022-08-30 15:05:28:464] [CSS]：设置 EP CSS0[0]为控制节点
```

另一节点 107 启动 DMCSS：

```
./dmcss dcr_ini=/home/dmdba/dmdsc/data/DSC02/dmdcr.ini

DMCSS V8

DMCSS IS READY

[2022-08-30 15:06:23:136] [CSS]：设置 EP CSS0[0]为控制节点
```

主节点 109 启动 DMASMSVRM：

```
./dmasmsvrm dcr_ini=/home/dmdba/dmdsc/data/DSC01/dmdcr.ini



ASM SELF EPNO:0

DMASMSVRM V8

dmasmsvrm task worker thread startup

the ASM server is Ready.

check css cmd: START NOTIFY, cmd_seq: 24

check css cmd: EP START, cmd_seq: 25



ASM Control Node EPNO:0

check css cmd: EP OPEN, cmd_seq: 38

check css cmd: EP REAL OPEN, cmd_seq: 41
```

另一节点 107 启动 DMASMSVRM:

```
./dmasmsvrm dcr_ini=/home/dmdba/dmdsc/data/DSC02/dmdcr.ini



ASM SELF EPNO:1

DMASMSVRM V8

dmasmsvrm task worker thread startup

the ASM server is Ready.

check css cmd: EP START, cmd_seq: 27



ASM Control Node EPNO:0

check css cmd: EP OPEN, cmd_seq: 39

check css cmd: EP REAL OPEN, cmd_seq: 42
```

8. 使用 DMASMTOOLM 工具创建 ASM 磁盘组，选择一个节点启动 DMASMTOOLM 工具:

```
./dmasmtoolm dcr_ini=/home/dmdba/dmdsc/data/DSC01/dmdcr.ini
```

创建一个 DATA 磁盘组和一个 LOG 磁盘组，每个磁盘组包含 3 个副本。

```
#创建 DATA 磁盘组

ASM>CREATE DISKGROUP DMDATA HIGH REDUNDANCY FAILGROUP 'data1' asmdisk

'/dev_DSC_HDD/MIR_HDD_1','/dev_DSC_HDD/MIR_HDD_2','/dev_DSC_HDD/MIR_HDD_3' FAI

LGROUP 'data2' asmdisk

'/dev_DSC_HDD/MIR_HDD_4' ,'/dev_DSC_HDD/MIR_HDD_5','/dev_DSC_HDD/MIR_HDD_6'F

AILGROUP 'data3' asmdisk

'/dev_DSC_HDD/MIR_HDD_7','/dev_DSC_HDD/MIR_HDD_8','/dev_DSC_HDD/MIR_HDD_9' A

TTRIBUTE AU_SIZE=32, REDO_SIZE=128
```

```
#创建 LOG 磁盘组

ASM>CREATE DISKGROUP DMLOG HIGH REDUNDANCY FAILGROUP 'data1' asmdisk

'/dev_DSC_HDD/MIR_HDD_10','/dev_DSC_HDD/MIR_HDD_11','/dev_DSC_HDD/MIR_HDD_12' F

AILGROUP 'data2' asmdisk

'/dev_DSC_HDD/MIR_HDD_13','/dev_DSC_HDD/MIR_HDD_14' ,'/dev_DSC_HDD/MIR_HDD_15'

FAILGROUP 'data3' asmdisk

'/dev_DSC_HDD/MIR_HDD_16' ,'/dev_DSC_HDD/MIR_HDD_17','/dev_DSC_HDD/MIR_HDD_18'

ATTRIBUTE AU_SIZE=32, REDO_SIZE=128
```

9. 准备 DMINIT.INI 配置文件，保存到/home/dmdba/dmdsc/data/DSC01 目录下，这里创建 3 副本的数据和联机日志文件，且数据文件使用 32K 的条带化粒度，联机日志文件使用 64K 的条带化粒度。

```
DB_NAME= dameng

SYSTEM_PATH= +DMDATA/data

SYSTEM= +DMDATA/data/dameng/system.dbf

SYSTEM_SIZE= 128

ROLL= +DMDATA/data/dameng/roll.dbf

ROLL_SIZE= 128

MAIN= +DMDATA/data/dameng/main.dbf

MAIN_SIZE= 128

CTL_PATH= +DMDATA/data/dameng/dm.ctl

LOG_SIZE= 2048

DCR_PATH= /dev_DSC_HDD

DCR_SEQNO= 0

AUTO_OVERWRITE= 2
```

```
PAGE_SIZE = 32

EXTENT_SIZE = 16

SYSDBA_PWD=DMdba_123

SYSAUDITOR_PWD=DMauditor_123

DATA_MIRROR = 3

LOG_MIRROR = 3

DATA_STRIPING = 32

LOG_STRIPING = 64

[DSC01]

CONFIG_PATH= /home/dmdba/dmdsc/data/DSC01/DSC01_conf

PORT_NUM = 7236

MAL_HOST= 192.168.102.109

MAL_PORT= 7336

LOG_PATH= +DMLOG/log/DSC01_log1.log

LOG_PATH= +DMLOG/log/DSC01_log2.log

[DSC02]

CONFIG_PATH= /home/dmdba/dmdsc/data/DSC02/DSC02_conf

PORT_NUM = 7237

MAL_HOST= 192.168.102.107

MAL_PORT= 7337

LOG_PATH= +DMLOG/log/DSC02_log1.log

LOG_PATH= +DMLOG/log/DSC02_log2.log
```

10. 使用 DMINIT 初始化一个节点的数据库环境。

选择一个节点，启动 DMINIT 初始化数据库，这里以 109 为例。DMINIT 执行完成后，

会在 config_path 目录（/home/dmdba/dmdsc/data/DSC01/DSC01_conf 和 /home/dmdba/dmdsc/data/DSC02/DSC02_conf）下生成配置文件 DM.INI 和 DMMAL.INI。

```
./dminit control=/home/dmdba/dmdsc/data/DSC01/dminit.ini
```

11. 使用拷贝的方式配置另外一个节点的数据库环境。

将 109 上初始化库时产生的 DSC02 节点的配置文件（整个 /home/dmdba/dmdsc/data/DSC02 文件夹）复制到 107 机器的 /home/dmdba/dmdsc/data/DSC02/目录下。之后就可以启动数据库服务器了。

```
scp -r /home/dmdba/dmdsc/data/DSC02/* dmdba@192.168.100.107:/home/dmdba/dmdsc/data/DSC02/
```

需要注意的是，如果 DMMAL.INI 文件中的 MAL_HOST 为 IPv6 地址，则将配置文件复制到 107 机器上后，需要修改 MAL_HOST 中的网卡序号或网卡名称为当前环境下的网卡序号或网卡名称。

12. 启动数据库服务器。

分别启动两个节点的服务器。

```
109 节点服务器启动:
/home/dmdba/dmdsc/bin/dmserver
dcr_ini=/home/dmdba/dmdsc/data/DSC01/dmdcr.ini
/home/dmdba/dmdsc/data/DSC01/DSC01_conf/dm.ini
107 节点服务器启动:
/home/dmdba/dmdsc/bin/dmserver
dcr_ini=/home/dmdba/dmdsc/data/DSC02/dmdcr.ini
/home/dmdba/dmdsc/data/DSC02/DSC02_conf/dm.ini
```

13. 配置并启动 DMCSSM 监视器。

镜像的 DMCSSM 监视器搭建同普通的 DMDSC 集群相同，详情可以参考普通的 DMDSC DMCSSM 信息。

在/home/dmdba/dmdsc/data/路径下配置 DMCSSM.INI。

```
#和 DMDCR_CFG.INI 中的 DCR_OGUID 保持一致

CSSM_OGUID =     509317566



#配置所有 CSS 的连接信息，

#与 DMDCR_CFG.INI 中 CSS 配置项的 DCR_EP_HOST 和 DCR_EP_PORT 保持一致

CSSM_CSS_IP = 192.168.102.109:7936

CSSM_CSS_IP = 192.168.102.107:7937



CSSM_LOG_PATH =     /home/dmdba/dmdsc/data/cssm_log #监视器日志文件存放路径，没

有的话需要创建

CSSM_LOG_FILE_SIZE          =     32          #每个日志文件最大 32M

CSSM_LOG_SPACE_LIMIT     =     0           #不限定日志文件总占用空间
```

创建 DMCSSM 的日志存放路径。

```
mkdir /home/dmdba/dmdsc/data/cssm_log
```

启动 DMCSSM 集群监视器。

```
./dmcssm ini_path=/home/dmdba/dmdsc/data/dmcssm.ini
```

DMCSSM 启动之后，可使用 show 命令在 DMCSSM 监视器中查看集群状态信息。

```
show
```

至此，基于 DMASM 镜像的 DMDSC 环境搭建完成。

12.3 检验是否搭建成功

DMDSC 搭建完成后，可通过下面三种方式检验是否搭建成功。下面的示例以 12.2 基于 DMASM 镜像的 DMDSC 为例进行操作。

一 使用 DMASMTOOLM 登录到 ASM 文件系统，通过 ls 命令查看磁盘组信息是否正确。

```
[dmdba@test109 bin]$ ./dmasmtoolm

dcr_ini=/home/dmdba/dmdsc/data/DSC01/dmdcr.ini

ASM>ls

+

disk groups total [3]……

NO.1        name: DMDATA

NO.2        name: DMLOG

NO.3        name: SYS
```

二 通过 lsdsk 命令查看磁盘详细信息是否正确。

```
ASM>lsdsk

group DMDATA include 3 disks……

    NO.1 disk :

        id: 0

        name: DMASMd1

        failgroup: data1

        partner: 1 2

        path: /dev_DSC_HDD/MIR_HDD_1

        status: NORMAL

        size: 71423AU
```

```
            free_size: 71406AU

            free_rate: 100.0%

            create_time: 2022-09-20 13:56:34

            modify_time: 2022-09-20 13:56:52

            belong group: DMDATA

        NO.2 disk :

            ……

group DMLOG include 3 disks……

        NO.1 disk :

            id: 0

            name: DMASMd4

            failgroup: data1

            partner: 1 2

            path: /dev_DSC_HDD/MIR_HDD_4

            status: NORMAL

            size: 71423AU

            free_size: 71406AU

            free_rate: 100.0%

            create_time: 2022-09-20 13:56:34

            modify_time: 2022-09-20 13:57:03

            belong group: DMLOG

        NO.2 disk :

            ……

group SYS include 5 disks……
```

```
        NO.1 disk :
                id: 0
                name: DMASMdcrv1
                failgroup: SYS_FGRP0
                partner:
                path: /dev_DSC_HDD/DCRV1
                status: NORMAL
                size: 1024AU
                free_size: 1006AU
                free_rate: 100.0%
                create_time: 2022-09-20 13:52:33
                modify_time: 2022-09-20 13:52:33
                belong group: SYS
        NO.2 disk :
              ……
total 0 disks unused……
Used time: 5.106(ms).
```

三 使用 DIsql 工具，以 SYSDBA 身份登录服务器查看 V$ASMDISK 视图中的信息是否正确。

```
./disql SYSDBA/DMdba_123@192.168.100.109:7236
服务器[192.168.100.109:7236]:处于普通打开状态
登录使用时间 : 10.755(ms)
disql V8
SQL> SELECT GROUP_ID,DISK_ID,DISK_NAME,DISK_PATH,SIZE,FREE_MB,STATUS FROM V$ASMDISK;
```

```
行号          GROUP_ID      DISK_ID       DISK_NAME    DISK_PATH                    SIZE
FREE_MB       STATUS
---------- ----------- ----------- ---------- ---------------------- -----------
1             0             0             DMASMd1      /dev_DSC_HDD/MIR_HDD_1 2285536
2284320       NORMAL
2             0             1             DMASMd2      /dev_DSC_HDD/MIR_HDD_2 2285536
2284288       NORMAL
3             0             2             DMASMd3      /dev_DSC_HDD/MIR_HDD_3 2285536
2284288       NORMAL
……
```

13巧用服务名

客户端要连接到数据库，除了使用某一个具体的“服务器 IP 地址和端口号”，也可以使用数据库服务名。服务名是一个数据库系统中所有对外服务的实例的“IP 地址和端口号”的集合的名称。

连接 DMDSC 集群，建议客户端使用数据库服务名，以实现故障自动切换、只连集群的主控节点或者只连集群的第一个节点等功能。

13.1 配置服务名

连接服务名可以在 DM 提供的 JDBC、DPI 接口中使用。可以通过编辑 dm_svc.conf 文件配置连接服务名。

dm_svc.conf 是一个客户端配置文件，它包含了 DM 各接口和客户端工具所需要配置的一些参数。它必须和接口/客户端工具位于同一台机器上才能生效。

初始 dm_svc.conf 文件在 DM 安装时自动生成。不同平台的生成目录有所不同。

1. 32 位的 DM 安装在 Win32 操作平台下，此文件位于%SystemRoot%\system32 目录;

2. 64 位的 DM 安装在 Win64 操作平台下，此文件位于%SystemRoot%\system32 目录;

3. 32 位的 DM 安装在 Win64 操作平台下，此文件位于%SystemRoot%\SysWOW64 目录;

4. 在 Linux 平台下，此文件位于/etc 目录。

可以通过设置操作系统环境变量 DM_SVC_PATH 来修改 dm_svc.conf 文件路径。

dm_svc.conf 文件包含很多参数，详细请参考《DM8 系统管理员手册》。本节只介绍

DMDSC 相关的常用配置项：

- 服务名

用于连接的服务名，用户通过连接服务名访问数据库。

服务名格式：

```
服务名=(IP[:PORT],IP[:PORT],.....)
```

IP 为数据库所在的 IP 地址，如果是 IPv6 地址，为了方便区分端口，需要用[]封闭 IP 地址。PORT 为数据库使用的 TCP 连接端口，可选配置，不配置则使用默认的端口。

- SWITCH_TIMES

检测到数据库实例故障时，接口在服务器之间切换的次数；超过设置次数没有连接到有效数据库时，断开连接并报错。有效值范围 1~9223372036854775807，缺省值为 1。

- SWITCH_INTERVAL

表示在服务器之间切换的时间间隔，单位为毫秒，有效值范围 1~9223372036854775807，缺省值为 200。

- LOGIN_DSC_CTRL

服务名连接数据库时是否只选择 DMDSC 控制节点的库。0：否；1：是。缺省值为 0。

- EP_SELECTOR

连接数据库时采用何种模型建立连接。不同的接口取值不同。

在 JDBC 和 DPI 中，该配置项的取值为大于或等于 0 的整数。0：依次选取列表中的不同节点建立连接，使得所有连接均匀地分布在各个节点上；其余大于 0 的整数表示选择列表中第几个节点建立连接，只有当前节点无法建立连接时才会选择下一个节点进行连接，若配置的值超过服务名列表的节点数，则连接第 N 个节点。其中，N=EP_SELECTOR % 服务名列表节点数。

在.NET PROVIDER、NODE.JS、GO 中，该配置项的取值为 0 或 1。0：依次选取列表中的不同节点建立连接，使得所有连接均匀地分布在各个节点上；1：选择列表中最前面的节

点建立连接，只有当前节点无法建立连接时才会选择下一个节点进行连接。

缺省值为 0。

- AUTO_RECONNECT

连接发生异常或一些特殊场景下连接处理策略。0：关闭连接；1：当连接发生异常时自动切换到其他库，无论切换成功还是失败都会抛一个 SQLException，用于通知上层应用进行事务执行失败时的相关处理；2：配合 EP_SELECTOR>=1 使用，如果服务名列表前面的节点恢复了，将当前连接切换到前面的节点上；4：保持各节点会话动态均衡，通过后台线程检测节点及会话数变化，并切换连接使之保持均衡。

也可以将 AUTO_RECONNECT 置为上述几个值的组合值，表示同时进行多项配置，如置为 3 表示同时配置 1 和 2。缺省值为 0。

- CHECK_FREQ

JDBC 专用。服务名连接数据库时，循环检测连接是否需要重置的时间间隔。即每间隔设定时间检测连接对象是否发生改变，若连接对象发生改变，JDBC 连接会自动重置到新对象。单位 MS，取值范围 0~2147483647。缺省值为 300000。

13.2 集群的 dm_svc.conf 模板

在集群场景中，一个完整的 dm_svc.conf 包含两大模块：全局配置区和服务配置区。全局配置区在前，可配置 dm_svc.conf 中所有的配置项；服务配置区在后，以“[服务名]”开头，可配置 dm_svc.conf 中除了服务名和 WALLET_LOCATION 外的所有配置项。服务配置区中的配置优先级高于全局配置区。未设置的参数一律使用默认值。

例如：

```
# dm_svc.conf 文件
# 以#开头的行表示是注释
# 全局配置区
```

```
DMDSC1=(192.168.1.1:5236,192.168.1.3:5236)

DMDSC2=(192.168.1.5:5236,192.168.1.7:5236)

TIME_ZONE=(+480)    #表示+8:00 时区


#DMDSC1 服务配置区

#以下配置是每次定向连接 DMDSC1 服务名的第一个服务（192.168.1.1:5236），当服务器故障后，以间隔 1000 毫秒的节奏尝试连接第一个服务 60 次，若连接成功则进行使用，若连接失败则连接下一个服务（192.168.1.3:5236）。假设 2 号服务先连接成功，由于 AUTO_RECONNECT=(1)，因此当 1 号服务可以正常连接后当前连接也不会切换到 1 号服务。

[DMDSC1]

SWITCH_TIMES=(60)

SWITCH_INTERVAL=(1000)

EP_SELECTOR=(1)

AUTO_RECONNECT=(1)


#DMDSC2 服务配置区

#以下配置是每次定向连接 DMDSC2 服务名的第一个服务（192.168.1.5:5236），当服务器故障后，以间隔 1000 毫秒的节奏尝试连接第一个服务 60 次，若连接成功则进行使用，若连接失败则连接下一个服务（192.168.1.7:5236）。假设 2 号服务先连接成功，由于 AUTO_RECONNECT=(2)，因此当 1 号服务可以正常连接后当前连接会切换到 1 号服务。

[DMDSC2]

CLUSTER=(DSC)

SWITCH_TIMES=(60)

SWITCH_INTERVAL=(1000)
```

```
EP_SELECTOR=(1)

AUTO_RECONNECT=(2)
```

需要说明的是，如果对 dm_svc.conf 的配置项进行了修改，需要重启客户端工具，修改的配置才能生效。另外，如果 dm_svc.conf 配置文件中包含中文，则必须保证该配置文件的编码与客户端编码一致。

13.3 故障自动切换

当用户连接到 DM 数据共享集群时，实际上是连接到集群中的一个实例，用户的所有增删改查操作都是由该实例完成的。但是如果该实例出现故障，那么用户连接会被切换到其他正常实例。而这种切换对用户是透明的，用户的增删改查继续返回正确结果，感觉不到异常。这种功能就是故障自动切换。

实现故障自动切换的前提条件是在使用 DM 数据共享集群的时候，必须配置连接服务名。

本节以 12.1 基于 DMASM 的 DMDSC 成功搭建的数据共享集群系统为例，来体验一下故障自动切换功能。故障自动切换的步骤：

1. 配置一个名为 dmdsc_svc 的连接服务名，使用 dmdsc_svc 连接 DMDSC 集群中的数据库，接口会随机选择一个 IP 进行连接，如果连接不成功或者服务器状态不正确，则顺序获取下一个 IP 进行连接，直至连接成功或者遍历了所有 IP。

本例子使用 DIsql 作为客户端，因此 dm_svc.conf 位于 DIsql 所在的机器。

dm_svc.conf 配置如下：

```
#以下配置是每次定向连接 dmdsc_svc 服务名的第一个服务器，当服务器故障后，尝试 99 次，间隔 1000 毫秒的节奏一直连接第一个服务器；若连接不上，再循环下一个服务器，连接上之后进行使用。
dmdsc_svc=(10.0.2.101:5236,10.0.2.102:5237)
[dmdsc_svc]
SWITCH_TIMES=(99)
```

```
SWITCH_INTERVAL=(1000)
```

2. 以 SYSDBA 身份连接到 DMDSC。假设 SYSDBA 密码为 DMdba_123，连接语句如下：

```
DIsql SYSDBA/DMdba_123@dmdsc_svc
```

3.确认当前用户已经连接到的节点实例。

```
SQL> select name from v$instance;

行号          NAME

---------- ------- -------

1             DSC0
```

用户当前连接到节点 0 上 DSC0 实例。不要退出这个会话（假定为会话 1），第 4 步还是在这个会话中执行。

4.关闭 DSC0 实例，或者将节点 0 所在的这台主机关机。

5.在会话 1 中再次执行这条语句，服务器会返回提示正在切换当前连接。

```
SQL> select name from v$instance;

[-70065]:连接异常,切换当前连接成功.
```

6. 等待几秒后，切换成功。此时，在会话 1 中再次执行这条语句，可执行成功。可见，会话也已切换到 DSC1 实例，实现了故障自动切换。

```
SQL> select name from v$instance;

行号          NAME

---------- ------ --------

1             DSC1
```

13.4 只连集群的主控节点

本节以 12.1 基于 DMASM 的 DMDSC 成功搭建的数据共享集群系统为例进行介绍。

LOGIN_DSC_CTRL=1 表示使用服务名连接数据库时只选择 DMDSC 主控节点（CONTROL NODE）的库。dm_svc.conf 配置如下：

```
dmdsc_svc=(10.0.2.101:5236,10.0.2.102:5237)

[dmdsc_svc]

LOGIN_DSC_CTRL=(1)
```

此时退出 DSC1，系统中只剩 DSC2。DSC2 为主控节点。查询操作如下：

```
SQL> select EP_NAME,EP_SEQNO,EP_MODE,EP_STATUS from V$DSC_EP_INFO;

行号          EP_NAME EP_SEQNO      EP_MODE         EP_STATUS

---------- ------- ----------- ------------ ---------

1             DSC1    0               Normal Node   ERROR

2             DSC2    1               Control Node OK
```

以 SYSDBA 身份连接到 DMDSC。假设 SYSDBA 密码为 DMdba_123，连接语句如下：

```
DIsql SYSDBA/DMdba_123@dmdsc_svc
```

发现当前用户已经连接到的控制节点实例。

```
SQL> select name from v$instance;

行号          NAME

---------- ------- -------

1             DSC2
```

再次启动 DSC1 后，DSC1 作为普通节点加入集群，DSC2 依然是控制节点，因此 DIsql 连接的依然是 DSC2。

```
SQL> select name from v$instance;

行号          NAME

---------- ------- -------

1             DSC2
```

13.5 只连集群的第 N 个节点

此功能 DIsql 工具暂不支持。

本节以 12.1 基于 DMASM 的 DMDSC 成功搭建的数据共享集群系统为例进行介绍。

EP_SELECTOR=N（N>=1）和 AUTO_RECONNECT=2 表示使用服务名连接数据库时会选择“IP 地址和端口号”集合列表中第 N 个节点建立连接，只有当前节点无法建立连接时才会选择下一个节点进行连接，如果第 N 个节点恢复了，当前连接将自动切换到第 N 个节点上。

例 以 EP_SELECTOR=1 为例，dm_svc.conf 配置如下：

```
dmdsc_svc=(10.0.2.102:5237,10.0.2.101:5236)

[dmdsc_svc]

EP_SELECTOR=(1)

AUTO_RECONNECT=(2)

CHECK_FREQ=(60000)
```

使用 MANAGER 通过服务名连接到 DMDSC。10.0.2.102:5237 为 DSC1，10.0.2.102:5236 为 DSC0。

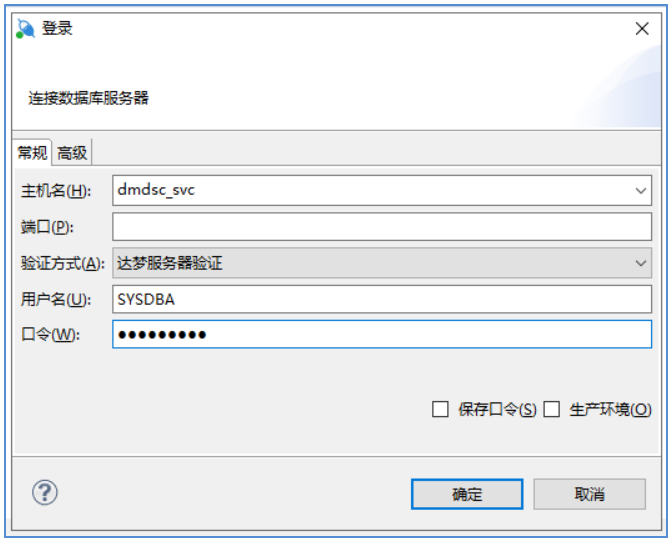


图 13.1 登录界面

使用此功能，MANAGER 工具需配置自动提交功能。窗口-选项-查询分析器：勾选自动提交。

在 MANAGER 界面中执行如下语句，会发现当前用户连接的是第一个节点。

```
select name from v$instance;

查询结果为:    DSC1
```

当关闭 DSC1 之后，MANAGER 会自动连接到 DSC0。

```
select name from v$instance;

查询结果为:        DSC0
```

再次启动 DSC1 之后，MANAGER 又自动连回 DSC1。

```
select name from v$instance;

查询结果为:    DSC1
```

因为数据库故障切换过程和 MANAGER 工具自动重置连接（CHECK_FREQ）均需要一定的时间，因此，只有等待两者都顺利完成，才能看到正确的结果。

14动态增删节点

DMDSC 集群（基于 DMASM 的或基于 DMASM 镜像的）支持动态增加节点，不支持动态删除节点。其中，数据守护环境中的 DMDSC 集群不支持动态增删节点。


注意：

增加节点过程中，不应该有修改数据库状态或模式的操作。

14.1 动态增加节点流程

基于 DMASM 的 DMDSC 和基于 DMASM 镜像的 DMDSC 均支持动态增加节点，二者流程基本一致。基于 DMASM 的 DMDSC 动态增加节点时，需要使用相应的 DMASMSVR、DMASMTOOL 和 DMASMCMD 工具；基于 DMASM 镜像的 DMDSC 动态增加节点时，需要使用相应的 DMASMSVRM、DMASMTOOLM 和 DMASMCMDM 工具。

动态增加节点要求当前 DMDSC 集群的所有节点都为 OK 状态，所有 DMSERVER 实例都处于 OPEN 状态，且可以正常访问。每次扩展可以在原有基础上增加一个节点。

本章节主要基于 12.1 基于 DMASM 的 DMDSC 成功架构的数据共享集群系统，介绍如何动态扩展一个节点。

已搭建好的 DMDSC 集群实例名为 DSC01、DSC02，在此基础上扩展一个节点 DSC03。

14.1.1 环境说明

新增节点环境为：

操作系统：eulerosv2r7.x86_64。

网络配置：eth0 网卡为 192.168.102.x 内网网段，新增节点为 192.168.102.106；eth1 为 192.168.100.x 外网网段，新增节点为 192.168.100.106。

DM 各种工具位于目录：/home/dmdba/dmdsc/bin。

配置文件位于目录：/home/dmdba/dmdsc/data/DSC03。

14.1 新增节点环境

节点机器	实例名称	IP	PORT_NUM
107	CSS0	192.168.100.107	9836
	ASM0		5836
	DSC01		6636
110	CSS1	192.168.100.110	9837
	ASM1		5837
	DSC02		6637
106	CSS2	192.168.100.106	9838
	ASM2		5838
	DSC03		6638

14.1.2 操作流程

1. 在 107 机器上使用 DMASMCMD 工具 export 出备份 DMDCR_CFG_BAK.INI

```
./dmasmcmd

Asm> export dcrdisk '/dev_DSC2/DCR' to '/home/dmdba/dmdsc/data/DSC01/dmdcr_cfg
_bak.ini'
```

若是基于 DMASM 镜像的 DMDSC，则会存在多个 DCRV 磁盘，此时使用 DMASMCMDM 工具 export 出任意一个磁盘备份即可。

2. 为新增节点准备日志文件

1）使用 DIsql 登录任意一个节点执行添加日志文件操作：

至少两个日志文件，路径必须是 ASM 文件格式，大小可以参考其他两个活动节点。

```
SQL>alter database add node logfile '+DMLOG/log/DSC03_log1.log' size 256,
```

```
'+DMLOG/log/DSC03_log2.log' size 256;
```

2)使用 DMCTLCVT 工具将 dm.ctl 转换为文本文件 dmctl.txt，查看 dmctl.txt，新增节点的日志文件信息已经添加进 dm.ctl。

```
./dmctlcvt TYPE=1 SRC=+DMDATA/data/dsc2/dm.ctl
DEST=/home/dmdba/dmdsc/data/DSC01/dmctl.txt
DCR_INI=/home/dmdba/dmdsc/data/DSC01/dmdcr.ini
```

3)使用 DMASMTOOL 工具登录 ASM 文件系统，也可以看到新增的节点日志文件

```
./dmasmtool DCR_INI=/home/dmdba/dmdsc/data/DSC01/dmdcr.ini
ASM>ls +DMLOG/log
```

3. 修改 DMMAL.INI 配置文件

修改 107 机器/home/dmdba/dmdsc/data/DSC01 目录以及 110 机器/home/dmdba/dmdsc/data/DSC02 目录下的 DMMAL.INI 配置文件，添加新增节点信息。

```
[MAL_INST3]
    MAL_INST_NAME   = DSC03
    MAL_HOST        = 192.168.102.106
    MAL_PORT        = 6538
```

4. 为新增节点准备 config_path

将 107 机器/home/dmdba/dmdsc/data/DSC01 路径下的 DSC01_conf 文件夹拷贝到 106 机器/home/dmdba/dmdsc/data/DSC03 路径下，修改文件夹名称为 DSC03_conf。

修改 DSC03_conf 文件夹下的配置文件:

1) 修改 DM.INI

```
CONFIG_PATH      = /home/dmdba/dmdsc/data/DSC03/DSC03_conf
```

```
INSTANCE_NAME    = DSC03

PORT_NUM = 6638
```

2） 如果打开了归档参数，修改 DMARCH.INI

5. 新建 DMDCR.INI 配置文件，保存到节点 106 的 /home/dmdba/dmdsc/data/DSC03 目录下面

注意设置 dmdcr_seqo 为 2，修改 DM.INI 路径。

```
DMDCR_PATH        = /dev_DSC2/DCR

DMDCR_MAL_PATH =/home/dmdba/dmdsc/data/DSC03/dmasvrmal.ini   #DMASMSVR 使用的 MAL 配置文件路径

DMDCR_SEQNO     = 2

#ASM 重启参数，命令行方式启动

DMDCR_ASM_RESTART_INTERVAL = 0

DMDCR_ASM_STARTUP_CMD             = /home/dmdba/dmdsc/bin/dmasmsvr dcr_ini=/home/dmdba/dmdsc/data/DSC03/dmdcr.ini

#DB 重启参数，命令行方式启动

DMDCR_DB_RESTART_INTERVAL       = 0

DMDCR_DB_STARTUP_CMD              = /home/dmdba/dmdsc/bin/dmserver path=/home/dmdba/dmdsc/data/DSC03/DSC03_conf/dm.ini dcr_ini=/home/dmdba/dmdsc/data/DSC03/dmdcr.ini

DMDCR_LINK_CHECK_IP=192.168.1.88
```

设置了 DMDCR_LINK_CHECK_IP，须为 106 节点的 DMSERVER、DMASMSVR、DMCSS 赋予 ping 权限。

```
sudo setcap cap_net_raw,cap_net_admin=eip /home/dmdba/dmdsc/bin/dmserver

sudo setcap cap_net_raw,cap_net_admin=eip /home/dmdba/dmdsc/bin/dmasmsvr
```

```
sudo setcap cap_net_raw,cap_net_admin=eip /home/dmdba/dmdsc/bin/dmcss
```

6. 修改当前环境的 DMASVRMAL.INI 配置文件

直接修改当前环境的 DMASVRMAL.INI 文件，添加新增节点信息，使用 DMASM 的所有节点都要配置，内容完全一样，并且将新增信息后的 DMASVRMAL.INI 文件拷贝到 106 机器的/home/dmdba/dmdsc/data/DSC03 目录下。

```
[MAL_INST2]

MAL_INST_NAME= ASM2

MAL_HOST= 192.168.102.106

MAL_PORT= 4838
```

7. 修改 DMDCR_CFG_BAK.INI，添加新增节点信息，CSS/ASMSVR/DB 都要配置所有组信息修改:

```
DCR_GRP_N_EP    =    3

DCR_GRP_EP_ARR      =    {0,1,2}
```

每个组增加一个节点信息，注意 DCR_EP_SHM_KEY、端口号不能冲突；各组信息要放在各自的后面，即[GRP_CSS]中 CSS2 放在 CSS1 后面，[GRP_ASM]中 ASM2 放在 ASM1 后面，DSC03 放在 DSC02 后面。

```
[GRP_CSS]

DCR_EP_NAME = CSS2

DCR_EP_HOST = 192.168.102.106

DCR_EP_PORT = 9838



[GRP_ASM]

DCR_EP_NAME     =    ASM2

DCR_EP_SHM_KEY  =    93362
```

```
DCR_EP_SHM_SIZE =    512

DCR_EP_HOST =        192.168.102.106

DCR_EP_PORT =        5838

DCR_EP_ASM_LOAD_PATH= /dev_DSC2



[GRP_DSC]

DCR_EP_NAME          =    DSC03

DCR_EP_SEQNO         =    2

DCR_EP_PORT          =    6638
```

8. 使用 DMASMCMD 工具将新增节点信息写回磁盘，新增节点作为 ERROR 节点

```
./dmasmcmd

Asm> extend dcrdisk '/dev_DSC2/DCR' from '/home/dmdba/dmdsc/data/DSC01/dmdcr_

cfg_bak.ini'
```

若是基于 DMASM 镜像的 DMDSC，则会存在多个 DCRV 磁盘，此时需要使用 DMASMCMDM 工具将新增节点信息写回所有 DCRV 磁盘。

9. 在 DMCSSM 控制台执行增加节点命令

```
extend node
```

程序会通知所有实例(DMCSS/DMASMSVR/DMSERVER)更新信息，在 CSS 控制台执行 SHOW 命令，能看到新增节点信息，DMASMSVR/DMSERVER 是 ERROR 节点，程序会通知 ASMSVR/DMSERVER 更新 MAL 信息。

10. 启动新的 DMCSS、DMASM 服务程序

在 106 机器启动 DMCSS、DMASMSVR 程序。

手动启动新的 DMCSS，dcr_ini 指向新的 DMDCR.INI 文件：

```
./dmcss DCR_INI=/home/dmdba/dmdsc/data/DSC03/dmdcr.ini
```

手动启动新的 DMASMSVR，dcr_ini 指向新的 DMDCR.INI 文件，DMASMSVR 启动故障重加入流程：

```
./dmasmsvr DCR_INI=/home/dmdba/dmdsc/data/DSC03/dmdcr.ini
```

如果 DMCSS 配置有自动拉起 DMASMSVR 的功能，可以等待 DMCSS 自动拉起 DMASMSVR 程序，不需要手动启动。

11. 启动新的数据库服务器

如果 DMCSS 配置有自动拉起 DMSERVER 的功能，可以等待 DMCSS 自动拉起实例，不需要手动启动。

如果需要手动启动，可参考下面的操作步骤：

106 机器：

```
./dmserver    /home/dmdba/dmdsc/data/DSC03/DSC03_conf/dm.ini
DCR_INI=/home/dmdba/dmdsc/data/DSC03/dmdcr.ini
```

12. 修改 DMCSSM.INI

修改 DMCSSM.INI，增加新扩节点 DMCSS 的 IP:PORT 配置项 CSSM_CSS_IP，并重启 DMCSSM。

```
#增加新增节点的 CSS 的连接信息
#和 DMDCR_CFG.INI 中 CSS 配置项的 DCR_EP_HOST 和 DCR_EP_PORT 保持一致
CSSM_CSS_IP = 192.168.102.106:9838
```


注意：

如果由于配置文件错误，动态增加节点失败，只能停掉所有实例，重新 init dcr 磁盘，不影响 DMSERVER 数据。

14.1.3 注意事项

1. 增加节点前由用户保证所有 DMCSS/DMASMSVR/DMSERVER 节点都是 OK 的，

且都是活动的；

2. 每次增加节点只能扩一个节点，扩展完成后可以再继续增加节点；

3. 增加节点的过程中不能出现修改实例状态或模式的操作；

4. 增加节点的过程中，如果发生 DMCSS/DMASMSVR/DMSERVER 实例故障，会导致扩展失败；

5. 扩展过程中操作失误（比如未修改 DMASVRMAL.INI，未增加日志文件），会导致扩展失败；

6. 执行完 Extend Node 命令，用户需要查看 log 文件，确认扩展操作是否成功；

7. 扩展失败可能会导致集群环境异常，需要退出所有 DMCSS/DMASMSVR/DMSERVER，重新 init dcr 磁盘。

14.2 动态删除节点流程

DMDSC 集群（基于 DMASM 的或基于 DMASM 镜像的）不支持动态删除节点。

15监控 DMDSC

DMDSC 集群的运行情况可以通过 DMCSSM 监视器（Dameng Cluster Synchronization Services Monitor，DMCSSM）进行查看，也可以查询 DMDSC 相关的动态视图获取更详细的信息。DMCSSM 监视器支持一些控制命令，可以用来启动、关闭 DMDSC 集群，还可以进行手动控制节点故障处理和节点重加入。

15.1 DMCSSM 监视器

配置了 DMCSS 的集群中，可配置 DMCSSM 对集群进行统一管理，可配置 0~10 个 DMCSSM。各 DMCSSM 作用一样，相互独立，互不干扰。

15.1.1 功能说明

1. 监控集群状态

DMCSS 每秒会发送集群中所有节点的状态信息、当前连接到 DMCSS 的监视器信息以及 DCR 的配置信息到活动的监视器上，监视器提供对应的 show 命令用于查看各类信息。

2. 打开/关闭指定组的自动拉起

DMCSSM 提供 SET AUTO RESTART ON/SET AUTO RESTART OFF 命令，通知 DMCSS 打开或关闭对指定组的自动拉起功能，此功能和 DMCSS 的监控打开或关闭没有关系。

3. 强制 OPEN 指定组

DMCSSM 提供 OPEN FORCE 命令，在启动 ASM 或 DB 组时，如果组中某个节点发生硬件故障等原因导致一直无法启动，可执行此命令通知 DMCSS 将 ASM 组或 DB 组强制 OPEN，不再等待故障节点启动成功。

4. 启动/退出集群

DMCSSM 提供 EP STARTUP/EP STOP 命令，可以通知 DMCSS 启动/退出指定的 ASM 或 DB 组。

5. 集群故障处理

DMCSSM 提供 EP BREAK/EP RECOVER 命令，在主 CSS 的监控功能被关闭的情况下，可以通过执行这些命令手动进行故障处理和故障恢复。另外在某些特殊场景下还可通过 EP HALT 命令强制退出指定节点，具体可参考 15.1.4 命令说明。

15.1.2 配置文件 DMCSSM.INI

DMCSSM 的配置文件名称为 DMCSSM.INI，所支持的配置项说明如下。

表 15.1 DMCSSM.INI 配置项

配置项	配置含义
CSSM_OGUID	用于和 DMCSS 通信校验使用，和 DMDCR_CFG.INI 中的 DCR_OGUID 值保持一致
CSSM_CSS_IP	集群中所有 DMCSS 所在机器的 IP 地址，以及 DMCSS 的监听端口，配置格式为“IP:PORT”的形式，其中 IP 和 PORT 分别对应 DMDCR_CFG.INI 中 DMCSS 节点的 DCR_EP_HOST 和 DCR_EP_PORT。 如果使用 IPv6 地址，为了方便区分端口，需要用[]封闭 IP 地址部分。 对于 IPv6，若当前环境存在多块网卡，需要用%号指定具体有效的网卡序号或网卡名称；若只有一块网卡或者已配置默认网卡，则可以不指定序号或名称。例如： CSSM_CSS_IP = [fe80::6aa7:3f02:59b3:bcb4%3]:52184
CSSM_LOG_PATH	日志文件路径，日志文件命名方式为 “dmcssm_年月日时分

	秒.log”，例如“dmcssm_20160614131123.log”。 如果 DMCSSM.INI 中配置有 CSSM_LOG_PATH 路径，则将 CSSM_LOG_PATH 作为日志文件路径，如果没有配置，则将 DMCSSM.INI 配置文件所在的路径作为日志文件路径
CSSM_LOG_FILE_SIZE	单个日志文件大小，取值范围 16~2048，单位为 MB，缺省为 64MB，达到最大值后，会自动生成并切换到新的日志文件中
CSSM_LOG_SPACE_LIMIT	日志总空间大小，取值 0 或者 256~4096，单位为 MB。缺省为 0，表示没有空间限制。如果达到设定的总空间限制，会自动删除创建时间最早的日志文件
CSSM_MESSAGE_CHECK	是否对 CSSM 通信消息启用通信体校验（只有当消息的发送端和接收端都配置为 1 才启用通信体校验）。0：不启用；1：启用。缺省为 1


说明：

在有日志写入操作时，如果日志路径下没有日志文件，会自动创建一个新的日志文件，如果已经有日志文件，则根据设定的单个日志文件大小（CSSM_LOG_FILE_SIZE）决定继续写入已有的日志文件或者创建新的日志文件写入。

创建新的日志文件时，根据设定的日志总空间大小（CSSM_LOG_SPACE_LIMIT）决定是否删除创建时间最早的日志文件。

15.1.3 配置步骤

同一个 DMDSC 集群中，允许最多同时启动 10 个监视器，建议监视器放在独立的第三方机器上，避免由于节点间网络不稳定等原因导致监视器误判节点故障。

下面举例说明监视器的配置步骤，以12 DMDSC 搭建中搭建的 2 节点 DSC 环境为基础，配置对应的监视器，监视器放在第三方机器上，为 windows 操作系统，DMCSSM.INI 配置

文件路径为 D:\cssm，可根据实际情况调整配置环境及路径。

1. 配置 DMCSSM.INI 文件

```
#和 DMDCR_CFG.INI 中的 DCR_OGUID 保持一致

CSSM_OGUID = 1071107589



#配置所有 CSS 的连接信息，

#和 DMDCR_CFG.INI 中 CSS 配置项的 DCR_EP_HOST 和 DCR_EP_PORT 保持一致

CSSM_CSS_IP = 192.168.102.107:9836

CSSM_CSS_IP = 192.168.102.110:9837



CSSM_LOG_PATH =D:\cssm\log        #监视器日志文件存放路径

CSSM_LOG_FILE_SIZE = 32           #每个日志文件最大 32M

CSSM_LOG_SPACE_LIMIT = 0          #不限定日志文件总占用空间
```

2. 启动 dmcssm 监视器

```
dmcssm.exe INI_PATH=D:\cssm\dmcssm.ini
```

15.1.4 命令说明

监视器提供一系列命令，支持集群的状态信息查看以及节点的故障处理，可输入 help 命令，查看命令使用说明。

表 15.2 监视器命令

命令名称	含义
help	显示帮助信息
show [group_name]	显示指定的组信息，如果没有指定 group_name，则显示所有组信息

show config	显示 DMDCR_CFG.INI 的配置信息
show monitor	显示当前连接到主 CSS 的所有监视器信息
set group_name auto restart on	打开指定组的自动拉起功能（只修改 dmcss 内存值）
set group_name auto restart off	关闭指定组的自动拉起功能（只修改 dmcss 内存值）
open force group_name	强制 open 指定的 ASM 或 DB 组
ep startup group_name	启动指定的 ASM 或 DB 组
ep stop group_name	退出指定的 ASM 或 DB 组
ep halt group_name.ep_name	强制退出指定组中的指定节点
extend node	联机增加节点
ep crash group_name.ep_name	手动指定节点故障
check crash over group_name	检查指定组故障处理是否真正结束
exit	退出监视器

命令使用说明：

1. help

显示帮助信息。

2. show [group_name]

显示指定的组信息，如果没有指定 group_name，则显示所有组信息。

返回的组信息优先从主 CSS 获取，如果主 CSS 故障或尚未选出，则任选一个从 CSS 返回组信息。

3. show config

显示 DMDCR_CFG.INI 的配置信息，对于 DB 类型的节点，会比 CSS/ASM 节点多一项 DCR_EP_SEQNO 的显示值，如果原本的 ini 文件中没有手动配置，则显示的是自动分配的序列值。

返回的配置信息优先从主 CSS 获取，如果主 CSS 故障或尚未选出，则任选一个从 CSS

返回信息。

4. show monitor

显示当前连接到主 CSS 的所有 CSSM 监视器信息，如果主 CSS 故障或尚未选出，则任选一个从 CSS 显示连接信息。

返回的信息中，第一行为当前执行命令的监视器的连接信息。

5. set group_name auto restart on

打开指定组的自动拉起功能。

可借助 show css 命令查看每个节点的自动拉起标记，每个 css 只能控制和自己的 DMDCR.INI 中配置的 DMDCR_SEQNO 相同节点的自动拉起。

6. set group_name auto restart off

关闭指定组的自动拉起功能。

可借助 show css 命令查看每个节点的自动拉起标记，每个 css 只能控制和自己的 DMDCR.INI 中配置的 DMDCR_SEQNO 相同节点的自动拉起。

DSC one node 模式下执行该命令无效果，仅会更改 DMCSS 中的自动拉起参数内存值，实际 DSC one node 模式下不会关闭自动拉起功能。

7. open force group_name

在启动 ASM 或 DB 组时，如果某个节点故障一直无法启动，可借助此命令将 ASM 或 DB 组强制 OPEN。

此命令需要发送到主 CSS 执行，并且主 CSS 的监控需要处于打开状态，如果主 CSS 故障或尚未选出，则命令执行失败。

8. ep startup group_name

通知 CSS 启动指定的 ASM 或 DB 组，如果 CSS 已经打开了指定组的自动拉起功能，则命令不允许执行，需要等待 CSS 自动检测故障并执行拉起操作。

每个 CSS 只负责拉起和自己的 DMDCR.INI 中配置的 DMDCR_SEQNO 相同的 ASM 或

DB 节点，因此需要所有 CSS 都处于活动状态，否则只通知当前活动的 CSS 自动拉起相对应的节点。


说明：

只有在 ASM 组正常启动到 OPEN 状态，并且所有活动的 ASM 节点都处于 OPEN 状态时，才允许启动 DB 组，否则执行 DB 组的启动命令会报错，CSS 自动拉起 DB 组时也需要满足此条件。

在命令执行前，如果 CSS 对指定组的自动拉起功能是关闭的，在节点拉起成功后，会打开对指定组的自动拉起功能。

9. ep stop group_name

退出指定的 ASM 或 DB 组，如果主 CSS 故障或尚未选出，则命令执行失败。


注意：

在退出 ASM 组时，需要保证 DB 组已经退出，否则会报错处理。

在命令执行前，如果 CSS 对指定组的自动拉起功能是打开的，则会先通知 CSS 关闭对指定组的自动拉起功能，再通知指定组退出，避免命令执行成功后节点再次被自动拉起。

DSC one node 模式下虽然会通知 CSS 关闭对指定组的自动拉起功能，但实际执行无效果，仅会更改 DMCSS 中的自动拉起参数内存值，实际 DSC one node 模式下不会关闭自动拉起功能。

10. ep halt group_name.ep_name

强制退出指定组的指定 EP。适用于下述场景：

场景一 某个 ASM 或 DB 集群节点故障，CSS 的心跳容错时间 DCR_GRP_DSKCHK_CNT 配置值很大，在容错时间内，CSS 不会调整故障节点的 active 标记，一直是 TRUE，CSS 认为故障 EP 仍然处于活动状态，不会自动执行故障处理，并且不允许手动执行故障处理。

另外，执行 EP STARTUP 或 EP STOP 命令时，会误认为故障 EP 仍然处于活动状态，导致执行结果与预期不符。此时可以通过执行 EP HALT 命令，通知 CSS 再次 HALT 故障 EP，

确认 EP 已经被 HALT 后，CSS 会及时调整 active 标记为 FALSE，在此之后，对自动/手动故障处理，EP STARTUP/EP STOP 命令都可以正常执行。

场景二 需要强制 HALT 某个正在运行的 ASM 或 DB 节点，也可以通过此命令完成。

11. extend node

DMDSC 集群联机增加节点时使用。

程序会通知所有实例(DMCSS/DMASMSVR/DMSERVER)更新 dcr 信息。

使用 show 命令能看到新增节点信息，新增 DMASMSVR/DMSERVER 为 ERROR 状态。

12. ep crash group_name.ep_name

手动指定节点故障，节点故障后，css 只有在 DCR_GRP_DSKCHK_CNT 配置的时间过后才会判定实例故障，开始故障处理流程。用户如果明确知道实例已经故障，可以收到执行此命令，CSS 可以立即开始故障处理流程。

13. check crash over group_name

DB 集群环境故障处理后，需要满足一定条件（控制节点重做 REDO 日志产生的数据页修改都已经刷盘完成），才允许故障节点重新加回集群环境。此命令用来显示 DB 集群环境故障处理是否真正结束。

14. exit

退出监视器。

15.2 动态视图

DMDSC 集群提供一系列动态视图来查看当前的系统运行信息。部分视图是全局的，任意节点登录查询的结果都是相同的，部分视图仅显示登录节点的信息。

15.2.1 DMDSC 通用视图

15.2.1.1 V$DSC_EP_INFO

显示实例信息，登录任意节点查询得到的结果一致。

序号	列	数据类型	说明
1	EP_NAME	VARCHAR(128)	实例名称
2	EP_SEQNO	INTEGER	DSC 节点序号
3	EP_GUID	BIGINT	EP 唯一标识码
4	EP_TIMESTAMP	BIGINT	EP 时间戳
5	EP_MODE	VARCHAR(32)	EP 模式，CONTROL NODE 或 NORMAL NODE
6	EP_STATUS	VARCHAR(32)	EP 状态，OK 或 ERROR

15.2.1.2 V$DSC_GBS_POOL

显示 GBS 控制结构的信息，仅显示登录节点的信息。

序号	列	数据类型	说明
1	N_CTL	INTEGER	GBS 控制块总数
2	N_FREE_CTL	INTEGER	空闲的 GBS 控制块数目
3	N_SUB_POOL	INTEGER	GBS_POOL 个数

15.2.1.3 V$DSC_GBS_POOLS_DETAIL

显示分片的 GBS_POOL 详细信息，仅显示登录节点的信息。

序号	列	数据类型	说明

1	POOL_ID	INTEGER	GBS_POOL 编号
2	N_USED_CTL	INTEGER	正在使用的 GBS 控制块数目
3	N_REQUEST	INTEGER	当已有的 GBS 控制块全部被使用时，系统会临时分配额外的 GBS 控制块，当临时分配的 GBS 控制块尚处于请求阶段时，会被统计至 N_REQUEST 字段
4	N_FREE_REQUEST	INTEGER	当临时分配的 GBS 控制块使用完成后会被放入空闲链表中，N_FREE_REQUEST 字段统计的是空闲链表中的临时分配的 GBS 控制块数目

15.2.1.4 V$DSC_GBS_CTL

显示 GBS 控制块信息。仅显示登录节点的信息。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	GBS_POOL 编号
2	TS_ID	INTEGER	表空间号
3	FILE_ID	INTEGER	文件号或-1。-1 表示全局表空间封锁的 LBS 页
4	PAGE_NO	INTEGER	页号或表空间大小。当 FILE_ID=-1 时，此处 PAGE_NO 表示表空间大小
5	ACCESS_MAP	INTEGER	曾经访问此数据页的 EP
6	FRESH_EP	INTEGER	最新数据所在 EP。-1 表示此数据页尚未被修改
7	FRESH_LSN	BIGINT	最新修改对应的 LSN 值
8	N_REPLACED	INTEGER	控制块被替换次数
9	N_REVOKED	INTEGER	权限被回收次数

10	N_OWNER	INTEGER	拥有权限的 EP 数
11	N_REQUEST	INTEGER	请求授权的 EP 数
12	N_REVOKING	INTEGER	正在回收权限的 EP 数
13	N_REVOKE_X_ONLY	INTEGER	页的优化次数

15.2.1.5 V$DSC_GBS_CTL_DETAIL

显示 GBS 控制块详细信息。仅显示登录节点的信息。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	GBS_POOL 编号
2	TS_ID	INTEGER	表空间号
3	FILE_ID	INTEGER	文件号或-1。-1 表示全局表空间封锁的 LBS 页
4	PAGE_NO	INTEGER	页号或表空间大小。当 FILE_ID=-1 时，此处 PAGE_NO 表示表空间大小
5	ACCESS_MAP	INTEGER	曾经访问此数据页的 EP
6	FRESH_EP	INTEGER	最新数据所在 EP。-1 表示此数据页尚未被修改
7	FRESH_LSN	BIGINT	最新修改对应的 LSN 值
8	N_REPLACED	INTEGER	控制块被替换次数
9	N_REVOKED	INTEGER	权限被回收次数
10	N_OWNER	INTEGER	拥有权限的 EP 数
11	N_REQUEST	INTEGER	请求授权的 EP 数
12	N_REVOKING	INTEGER	正在回收权限的 EP 数
13	TYPE	VARCHAR(32)	详细信息类型（OWNER/REQUEST/REVOKING）

14	MODE	INTEGER	封锁模式，0/1/2/4 分别表示 N_LATCH/X_LATCH/S_LATCH/F_LATCH。该字段值可能为上述值的组合值，如字段值为 3 表示同时拥有 X_LATCH 和 S_LATCH 两种封锁
15	EP_SEQNO	INTEGER	拥有、请求、或者回收封锁的 EP
16	REAL_FLUSH	CHAR	是否真正执行刷盘请求（Y/N）
17	N_REVOKE_X_ONLY	INTEGER	页的优化次数

15.2.1.6 V$DSC_GBS_CTL_LRU_FIRST

显示 GBS 控制块 LRU 链表首页信息。仅显示登录节点的信息。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	GBS_POOL 编号
2	TS_ID	INTEGER	表空间号
3	FILE_ID	INTEGER	文件号或-1。-1 表示全局表空间封锁的 LBS 页
4	PAGE_NO	INTEGER	页号或表空间大小。当 FILE_ID=-1 时，此处 PAGE_NO 表示表空间大小
5	ACCESS_MAP	INTEGER	曾经访问此数据页的 EP
6	FRESH_EP	INTEGER	最新数据所在 EP。-1 表示此数据页尚未被修改
7	FRESH_LSN	BIGINT	最新修改对应的 LSN 值
8	N_REPLACED	INTEGER	控制块被替换次数
9	N_REVOKED	INTEGER	权限被回收次数
10	N_OWNER	INTEGER	拥有权限的 EP 数

11	N_REQUEST	INTEGER	请求授权的 EP 数
12	N_REVOKING	INTEGER	正在回收权限的 EP 数
13	N_REVOKE_X_ONLY	INTEGER	页的优化次数

15.2.1.7 V$DSC_GBS_CTL_LRU_FIRST_DETAIL

显示 GBS 控制块 LRU 链表首页详细信息。仅显示登录节点的信息。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	GBS_POOL 编号
2	TS_ID	INTEGER	表空间号
3	FILE_ID	INTEGER	文件号或-1。-1 表示全局表空间封锁的 LBS 页
4	PAGE_NO	INTEGER	页号或表空间大小。当 FILE_ID=-1 时，此处 PAGE_NO 表示表空间大小
5	ACCESS_MAP	INTEGER	曾经访问此数据页的 EP
6	FRESH_EP	INTEGER	最新数据所在 EP。-1 表示此数据页尚未被修改
7	FRESH_LSN	BIGINT	最新修改对应的 LSN 值
8	N_REPLACED	INTEGER	控制块被替换次数
9	N_REVOKED	INTEGER	权限被回收次数
10	N_OWNER	INTEGER	拥有权限的 EP 数
11	N_REQUEST	INTEGER	请求授权的 EP 数
12	N_REVOKING	INTEGER	正在回收权限的 EP 数
13	TYPE	VARCHAR(32)	详细信息类型：OWNER/REQUEST/REVOKING
14	MODE	INTEGER	封锁模式，0/1/2/4 分别表示 N_LATCH/X_LATCH/S_LATCH/F_LATCH。

			该字段值可能为上述值的组合值，如字段值为 3 表示同时拥有 X_LATCH 和 S_LATCH 两种封锁
15	EP_SEQNO	INTEGER	拥有、请求、或者回收封锁的 EP
16	REAL_FLUSH	CHAR	是否真正执行刷盘请求（Y/N）
17	N_REVOKE_X_ONLY	INTEGER	页的优化次数

15.2.1.8 V$DSC_GBS_CTL_LRU_LAST

显示 GBS 控制块 LRU 链表尾页信息。仅显示登录节点的信息。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	GBS_POOL 编号
2	TS_ID	INTEGER	表空间号
3	FILE_ID	INTEGER	文件号或-1。-1 表示全局表空间封锁的 LBS 页
4	PAGE_NO	INTEGER	页号或表空间大小。当 FILE_ID=-1 时，此处 PAGE_NO 表示表空间大小
5	ACCESS_MAP	INTEGER	曾经访问此数据页的 EP
6	FRESH_EP	INTEGER	最新数据所在 EP。-1 表示此数据页尚未被修改
7	FRESH_LSN	BIGINT	最新修改对应的 LSN 值
8	N_REPLACED	INTEGER	控制块被替换次数
9	N_REVOKED	INTEGER	权限被回收次数
10	N_OWNER	INTEGER	拥有权限的 EP 数
11	N_REQUEST	INTEGER	请求授权的 EP 数
12	N_REVOKING	INTEGER	正在回收权限的 EP 数

13	N_REVOKE_X_ONLY	INTEGER	页的优化次数

15.2.1.9 V$DSC_GBS_CTL_LRU_LAST_DETAIL

显示 GBS 控制块 LRU 链表尾页详细信息。仅显示登录节点的信息。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	GBS_POOL 编号
2	TS_ID	INTEGER	表空间号
3	FILE_ID	INTEGER	文件号或-1。-1 表示全局表空间封锁的 LBS 页
4	PAGE_NO	INTEGER	页号或表空间大小。当 FILE_ID=-1 时，此处 PAGE_NO 表示表空间大小
5	ACCESS_MAP	INTEGER	曾经访问此数据页的 EP
6	FRESH_EP	INTEGER	最新数据所在 EP。-1 表示此数据页尚未被修改
7	FRESH_LSN	BIGINT	最新修改对应的 LSN 值
8	N_REPLACED	INTEGER	控制块被替换次数
9	N_REVOKED	INTEGER	权限被回收次数
10	N_OWNER	INTEGER	拥有权限的 EP 数
11	N_REQUEST	INTEGER	请求授权的 EP 数
12	N_REVOKING	INTEGER	正在回收权限的 EP 数
13	TYPE	VARCHAR(32)	详细信息类型：OWNER/REQUEST/REVOKING
14	MODE	INTEGER	封锁模式，0/1/2/4 分别表示 N_LATCH/X_LATCH/S_LATCH/F_LATCH。该字段值可能为上述值的组合值，如字段值为 3 表示同时拥有

			X_LATCH 和 S_LATCH 两种封锁
15	EP_SEQNO	INTEGER	拥有、请求、或者回收封锁的 EP
16	REAL_FLUSH	CHAR	是否真正执行刷盘请求（Y/N）
17	N_REVOKE_X_ONLY	INTEGER	页的优化次数

15.2.1.10 V$DSC_GBS_REQUEST_CTL

显示等待 GBS 控制块的请求信息。当已有的 GBS 控制块全部被使用时，系统会临时分配额外的 GBS 控制块，当临时分配的 GBS 控制块尚处于请求阶段时，会被记录至当前视图中。仅显示登录节点的信息。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	GBS_POOL 编号
2	TS_ID	INTEGER	表空间号
3	FILE_ID	INTEGER	文件号或-1。-1 表示全局表空间封锁的 LBS 页
4	PAGE_NO	INTEGER	页号或表空间大小。当 FILE_ID=-1 时，此处 PAGE_NO 表示表空间大小
5	MODE	INTEGER	封锁模式，0/1/2/4 分别表示 N_LATCH/X_LATCH/S_LATCH/F_LATCH。该字段值可能为上述值的组合值,如字段值为 3 表示同时拥有 X_LATCH 和 S_LATCH 两种封锁
6	EP_SEQNO	INTEGER	请求封锁的 EP
7	FREQ_CONFLICT	INTEGER	是否为高频冲突数据页

15.2.1.11 V$DSC_LBS_POOL

显示 LBS 控制结构的信息，仅显示登录节点的信息。

序号	列	数据类型	说明
1	N_CTL	INTEGER	LBS 控制块总数
2	N_FREE_CTL	INTEGER	空闲的 LBS 控制块数目
3	N_SUB_POOL	INTEGER	LBS_POOL 个数
4	N_FAST_REQUEST	BIGINT	LBS 请求本地直接命中的次数。该项已废弃，恒为 0
5	N_GBS_REQUEST	INTEGER	LBS 请求发送给 GBS 节点的次数。该项已废弃，恒为 0
6	N_PAGE_CREATE_REQUEST	INTEGER	创建数据页请求次数。该项已废弃，恒为 0
7	TOTAL_REQUEST	BIGINT	总的 LBS 请求次数。该项已废弃，恒为 0

15.2.1.12 V$DSC_LBS_POOLS_DETAIL

显示分片的 LBS_POOL 详细信息。仅显示登录节点的信息。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	LBS_POOL 编号
2	N_USED_CTL	INTEGER	正在使用的 LBS 控制块数目
3	N_REQUEST_SPACE	INTEGER	当已有的 LBS 控制块全部被使用时，系统会临时分配额外的 LBS 控制块，当临时分配的 LBS 控制块尚处于请求阶段时，会被统计至 N_REQUEST_SPACE 字段
4	N_FREE_REQUEST	INTEGER	当临时分配的 LBS 控制块使用完成后会被放入空闲链表中，N_FREE_REQUEST 字段统计的是空闲链表

			中的临时分配的 LBS 控制块数目
5	N_FAST_REQUEST	INTEGER	LBS 请求本地直接命中的次数。该项已废弃，恒为 0
6	N_GBS_REQUEST	INTEGER	LBS 请求发送给 GBS 节点的次数。该项已废弃，恒为 0
7	N_PAGE_CREATE_REQUEST	INTEGER	创建数据页请求次数。该项已废弃，恒为 0
8	TOTAL_REQUEST	INTEGER	总的 LBS 请求次数。该项已废弃，恒为 0

15.2.1.13 V$DSC_LBS_CTL_LRU_FIRST

显示 LBS 的 LRU_FIRST 控制块信息。仅显示登录节点的信息。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	LBS_POOL 编号
2	TS_ID	INTEGER	表空间号
3	FILE_ID	INTEGER	文件号或-1。-1 表示全局表空间封锁的 LBS 页
4	PAGE_NO	INTEGER	页号或表空间大小。当 FILE_ID=-1 时，此处 PAGE_NO 表示表空间大小
5	ACCESS_MAP	INTEGER	曾经访问此数据页的 EP
6	FRESH_EP	INTEGER	最新数据所在 EP。-1 表示此数据页尚未被修改
7	FRESH_LSN	BIGINT	最新修改对应的 LSN 值
8	N_REPLACED	INTEGER	控制块被替换次数
9	N_REVOKED	INTEGER	权限被回收次数
10	N_FIXED	INTEGER	引用计数
11	MODE	INTEGER	获得 GBS 授权的封锁模式

12	PHY_LSN	BIGINT	数据页上的最新 LSN 值
13	N_REQUEST	INTEGER	请求获得授权的工作线程数
14	N_REVOKE_X_ONLY	INTEGER	页的优化次数

15.2.1.14 V$DSC_LBS_CTL_LRU_LAST

显示 LBS 的 LRU_LAST 控制块信息。仅显示登录节点的信息。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	LBS_POOL 编号
2	TS_ID	INTEGER	表空间号
3	FILE_ID	INTEGER	文件号或-1。-1 表示全局表空间封锁的 LBS 页
4	PAGE_NO	INTEGER	页号或表空间大小。当 FILE_ID=-1 时，此处 PAGE_NO 表示表空间大小
5	ACCESS_MAP	INTEGER	曾经访问此数据页的 EP
6	FRESH_EP	INTEGER	最新数据所在 EP。-1 表示此数据页尚未被修改
7	FRESH_LSN	BIGINT	最新修改对应的 LSN 值
8	N_REPLACED	INTEGER	控制块被替换次数
9	N_REVOKED	INTEGER	权限被回收次数
10	N_FIXED	INTEGER	引用计数
11	MODE	INTEGER	获得 GBS 授权的封锁模式
12	PHY_LSN	BIGINT	数据页上的最新 LSN 值
13	N_REQUEST	INTEGER	请求获得授权的工作线程数

15.2.1.15 V$DSC_LBS_CTL_DETAIL

显示 LBS 控制块详细信息。仅显示登录节点的信息。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	LBS_POOL 编号
2	TS_ID	INTEGER	表空间号
3	FILE_ID	INTEGER	文件号或-1。-1 表示全局表空间封锁的 LBS 页
4	PAGE_NO	INTEGER	页号或表空间大小。当 FILE_ID=-1 时，此处 PAGE_NO 表示表空间大小
5	ACCESS_MAP	INTEGER	曾经访问此数据页的 EP
6	FRESH_EP	INTEGER	最新数据所在 EP。-1 表示此数据页尚未被修改
7	FRESH_LSN	BIGINT	最新修改对应的 LSN 值
8	N_REPLACED	INTEGER	控制块被替换次数
9	N_REVOKED	INTEGER	权限被回收次数
10	N_FIXED	INTEGER	引用计数
11	MODE	INTEGER	获得 GBS 授权的封锁模式
12	PHY_LSN	BIGINT	数据页上的最新 LSN 值
13	N_REQUEST	INTEGER	请求获得授权的工作线程数
14	REQUEST_MODE	INTEGER	请求的封锁模式
15	REVOKE_LSN	BIGINT	回收权限时，GBS 上的最新 LSN 值
16	N_REMOTE_READ	INTEGER	数据页 REMOTE READ 的次数，数值越大，说明该数据页在节点间冲突越高
17	TOTAL_REQUEST	INTEGER	数据页的总访问次数

15.2.1.16 V$DSC_LBS_CTL_LRU_FIRST_DETAIL

显示 LBS 的 LRU_FIRST 控制块详细信息。仅显示登录节点的信息。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	LBS_POOL 编号
2	TS_ID	INTEGER	表空间号
3	FILE_ID	INTEGER	文件号或-1。-1 表示全局表空间封锁的 LBS 页
4	PAGE_NO	INTEGER	页号或表空间大小。当 FILE_ID=-1 时，此处 PAGE_NO 表示表空间大小
5	ACCESS_MAP	INTEGER	曾经访问此数据页的 EP
6	FRESH_EP	INTEGER	最新数据所在 EP。-1 表示此数据页尚未被修改
7	FRESH_LSN	BIGINT	最新修改对应的 LSN 值
8	N_REPLACED	INTEGER	控制块被替换次数
9	N_REVOKED	INTEGER	权限被回收次数
10	N_FIXED	INTEGER	引用计数
11	MODE	INTEGER	获得 GBS 授权的封锁模式
12	PHY_LSN	BIGINT	数据页上的最新 LSN 值
13	N_REQUEST	INTEGER	请求获得授权的工作线程数
14	REQUEST_MODE	INTEGER	请求的封锁模式
15	REVOKE_LSN	BIGINT	回收权限时，GBS 上的最新 LSN 值

15.2.1.17 V$DSC_LBS_CTL_LRU_LAST_DETAIL

显示 LBS 的 LRU_LAST 控制块详细信息。仅显示登录节点的信息。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	LBS_POOL 编号
2	TS_ID	INTEGER	表空间号
3	FILE_ID	INTEGER	文件号或-1。-1 表示全局表空间封锁的 LBS 页
4	PAGE_NO	INTEGER	页号或表空间大小。当 FILE_ID=-1 时，此处 PAGE_NO 表示表空间大小
5	ACCESS_MAP	INTEGER	曾经访问此数据页的 EP
6	FRESH_EP	INTEGER	最新数据所在 EP。-1 表示此数据页尚未被修改
7	FRESH_LSN	BIGINT	最新修改对应的 LSN 值
8	N_REPLACED	INTEGER	控制块被替换次数
9	N_REVOKED	INTEGER	权限被回收次数
10	N_FIXED	INTEGER	引用计数
11	MODE	INTEGER	获得 GBS 授权的封锁模式
12	PHY_LSN	BIGINT	数据页上的最新 LSN 值
13	N_REQUEST	INTEGER	请求获得授权的工作线程数
14	REQUEST_MODE	INTEGER	请求的封锁模式
15	REVOKE_LSN	BIGINT	回收权限时，GBS 上的最新 LSN 值

15.2.1.18 V$DSC_GTV_SYS

显示 GTV 控制结构的信息，仅登录集群环境控制节点才能获取数据，登录其他节点返回数据无效。

序号	列	数据类型	说明
1	T_INFO_NUM	INTEGER	系统已提交、未 PURGE 事务所修改的表对象个数

2	NEXT_TRXID	BIGINT	下一个事务 ID
3	MAX_PURGABLE_TRIXID	BIGINT	最大可 PURGE 的事务 ID
4	UNDO_TRIXID	BIGINT	回滚段中，正在被访问的最小事务 ID
5	UNDO_CNT	INTEGER	UNDO_TRXID 被设置的次数

15.2.1.19 V$DSC_LBS_CTL

显示 LBS 控制块信息。仅显示登录节点的信息。

序号	列	数据类型	说明
1	POOL_ID	INTEGER	LBS_POOL 编号
2	TS_ID	INTEGER	表空间号
3	FILE_ID	INTEGER	文件号或-1。-1 表示全局表空间封锁的 LBS 页
4	PAGE_NO	INTEGER	页号或表空间大小。当 FILE_ID=-1 时，此处 PAGE_NO 表示表空间大小
5	ACCESS_MAP	INTEGER	曾经访问此数据页的 EP
6	FRESH_EP	INTEGER	最新数据所在 EP。-1 表示此数据页尚未被修改
7	FRESH_LSN	BIGINT	最新修改对应的 LSN 值
8	N_REPLACED	INTEGER	控制块被替换次数
9	N_REVOKED	INTEGER	权限被回收次数
10	N_FIXED	INTEGER	引用计数
11	MODE	INTEGER	获得 GBS 授权的封锁模式
12	PHY_LSN	BIGINT	数据页上的最新 LSN 值

13	N_REQUEST	INTEGER	请求获得授权的工作线程数
14	N_REMOTE_READ	INTEGER	数据页 REMOTE READ 的次数，数值越大，说明该数据页在节点间冲突越高
15	TOTAL_REQUEST	INTEGER	数据页总的访问次数
16	N_REVOKE_X_ONLY	INTEGER	页的优化次数
17	REVOKE_DELAY	INTEGER	BS_CTL 延迟权限回收的次数

15.2.1.20 V$DSC_LOCK

显示全局活动的事务锁信息，登录任意节点查询得到的结果一致。

序号	列	数据类型	说明
1	EP_SEQNO	INTEGER	拥有该锁的节点号
2	ADDR	BIGINT	全局封锁对象的地址
3	TRX_ID	BIGINT	所属事务 ID
4	LTYPE	VARCHAR(10)	锁类型：TID 锁、对象锁
5	LMODE	CHAR(2)	锁模式：S 锁、X 锁、IX 锁、IS 锁
6	BLOCKED	INTEGER	是否阻塞
7	TABLE_ID	INTEGER	对应表锁、字典对象 ID
8	ROW_IDX	BIGINT	被封锁事务 ID
9	IGN_FLAG	INTEGER	锁对象的 IGNORABLE 标记，INI 参数 LOCK_DICT_OPT 开启时有效。 取值 0：表示锁正在使用中，IGN_FLAG 最低位置为 0； 取值 1：表示事务 TRX1 已经提交，但是锁

			资源未释放，TRX1 重新封锁时可以直接使用，此时 IGN_FLAG 最低位置为 1; 取值 2：为 0+2 的组合值。当锁正被使用时，另一事务要封锁同一对象，将当前正被使用的锁的 IGN_FLAG 次低位置为 1; 取值 3: 为 1+2 的组合值。当 TRX1 已提交，但锁未释放时，另一事务 TRX2 要封锁同一对象，此时 TRX2 可以忽略此 IGN_FLAG=1 的锁，但是要将此锁的 IGN_FLAG 次低位置为 1

15.2.1.21 V$DSC_TRX

显示所有活动事务的信息。通过该视图可以查看所有系统中所有的事务以及相关信息，如锁信息等，登录任意节点查询得到的结果一致。

序号	列	数据类型	说明
1	EP_SEQNO	INTEGER	事务所在节点号
2	ID	BIGINT	当前活动事务的 ID 号
3	NEXTID	BIGINT	下一个事务 ID 号
4	MIN_ACTIVE_ID	BIGINT	所有活动事务 ID 号最小者
5	STATUS	VARCHAR(20)	当前事务的状态
6	ISOLATION	INTEGER	事务的隔离级别 0：读未提交 1：读提交

			2：可重复读 3：串行化
7	READ_ONLY	CHAR(1)	是否为只读事务（Y/N）
8	SESS_ID	BIGINT	当前事务所在的会话 ID
9	SESS_SEQ	INTEGER	会话序列号，用来唯一标识会话
10	INS_CNT	INTEGER	插入回滚记录个数
11	DEL_CNT	INTEGER	删除回滚记录个数
12	UPD_CNT	INTEGER	更新回滚记录个数
13	UPD_INS_CNT	INTEGER	更新插入回滚记录个数
14	UREC_SEQNO	INTEGER	当前 Undo 记录的递增序列号
15	WAITING	BIGINT	事务等待的锁
16	THRD_ID	INTEGER	当前事务对应的线程 ID

15.2.1.22 V$DSC_TRXWAIT

显示事务等待信息，登录任意节点查询得到的结果一致。当 DMDSC 系统的活动节点数超过 1 个时，使用 V$DSC_TRXWAIT 查询事务等待信息，当为单节点系统或 DMDSC 系统仅剩一个活动节点时，应查询 V$TRXWAIT。

序号	列	数据类型	说明
1	EP_SEQNO	INTEGER	事务所在节点号
2	TRX_ID	BIGINT	事务 ID
3	WAIT_FOR_ID	BIGINT	所等待的事务 ID
4	WAIT_SEQNO	TINYINT	等待的事务序列号
5	WAIT_TIME	INTEGER	当前已等待时间，单位毫秒

6	TRX_OBJ	BIGINT	事务对象
7	WAIT_TRX_OBJ	BIGINT	等待事务对象

15.2.1.23 V$DCR_INFO

查看 DCR 配置的全局信息，登录任意节点查询得到的结果一致。

序号	列	数据类型	说明
1	VERSION	INTEGER	DCR 版本号
2	N_GROUP	INTEGER	DCR 配置的组个数
3	VTD_PATH	VARCHAR(256)	VOTE 磁盘路径
4	UDP_FLAG	INTEGER	是否使用 UDP 心跳机制，已无效
5	UDP_OGUID	BIGINT	DCR 磁盘 GUID 标识，用于校验 DCR 磁盘
6	DCR_PATH	VARCHAR(256)	DCR 磁盘路径

15.2.1.24 V$DCR_GROUP

查看 DCR 配置的组信息，登录任意节点查询得到的结果一致。

序号	列	数据类型	说明
1	GROUP_TYPE	VARCHAR(32)	组类型，CSS/ASM/DB
2	GROUP_NAME	VARCHAR(128)	组名称
3	N_EP	INTEGER	组中配置的 EP 个数
4	DSKCHK_CNT	INTEGER	磁盘心跳超时时间。单位秒
5	NETCHK_TIME	INTEGER	预留参数，暂无意义，显示为 0

15.2.1.25 V$DCR_EP

查看 DCR 配置的节点信息，登录任意节点查询得到的结果一致。

序号	列	数据类型	说明
1	GROUP_NAME	VARCHAR(128)	节点所属的组名
2	EP_NAME	VARCHAR(128)	节点名称
3	EP_SEQNO	INTEGER	节点的组内序号: 对 CSS/ASM 组的节点，是自动分配的序号，对 DB 组的节点，如果没有配置，也是自动分配的序号，否则是实际的配置序号。
4	EP_HOST	VARCHAR(128)	节点的 IP 地址，对 CSS/ASM 组的节点有效，表示登录节点的 IP 地址
5	EP_PORT	INTEGER	节点的 TCP 监听端口，对应登录节点对应实例的端口号
6	UDP_PORT	INTEGER	节点的 UDP 监听端口，已无效
7	SHM_KEY	INTEGER	共享内存标识，初始化共享内存的标识符，对 ASM 组的节点有效
8	SHM_SIZE	INTEGER	共享内存大小，单位 MB，初始化共享内存大小，对 ASM 组的节点有效
9	ASM_LOAD_PATH	VARCHAR(256)	ASM 磁盘扫描路径，对 ASM 组的节点有效

15.2.1.26 V$DMDCR_INI

查看当前节点 dmdcr.ini 文件中的参数信息。

序号	列	数据类型	说明

1	NAME	VARCHAR(128)	参数名称
2	VALUE	VARCHAR(1024)	参数的值

15.2.1.27 V$DSC_REQUEST_STATISTIC

统计 DSC 环境内 TYPE 类型请求时间，仅显示登录节点的信息。

序号	列	数据类型	说明
1	TYPE	VARCHAR(64)	请求类型，可能取值: logic read, physical read, remote read, remote page send, remote read failed, remote read processing, remote read transmission, remote rlog flush, remote rlog flush wait, physical write, lbs request fast, lbs x latch owned, lbs s latch owned, lbs f latch owned, lbs x latch waited, lbs s latch waited, lbs f latch waited, lbs x latch waited by revoke, lbs s latch waited by revoke, lbs f latch waited by revoke, lbs sent x latch request to local gbs,

			lbs sent s latch request to local gbs, lbs sent f latch request to local gbs, lbs sent x latch request to remote gbs, lbs sent s latch request to remote gbs, lbs sent f latch request to remote gbs, lbs request rollback, gbs synchronous revoke conflicts, gbs synchronous revoke roll conflicts, gbs asynchronous revoke conflicts, gbs asynchronous revoke roll conflicts, lls lock table, lls lock tid, lls lock release, lls dict remove, ltv view get
2	TOTAL_REQUEST_COUNT	BIGINT	总请求次数
3	MAX_REQUEST_TIME	INTEGER	最大请求时间，单位为微秒
4	MIN_REQUEST_TIME	INTEGER	最小请求时间，单位为微秒

5	AVERAGE_REQUEST_TIME	INTEGER	平均请求时间，单位为微秒
6	TOTAL_REQUEST_TIME	BIGINT	总的请求时间，单位为微秒

15.2.1.28 V$DSC_CRASH_OVER_INFO

显示 DSC 环境各节点数据页最小 first_modified_lsn，以及故障节点 file_lsn。

如果活动节点 buffer 中不存在更新页则 min_first_modified_lsn 为 NULL；节点故障后，只有在所有 OK 节点 min_first_modified_lsn 都大于或等于故障节点 file_lsn 之后，才允许故障节点重加入；满足所有 OK 节点 min_first_modified_lsn 都大于 crash_lsn 之后，crash_lsn 会清零。

序号	列	数据类型	说明
1	EP_SEQNO	INTEGER	DSC 节点号
2	OK_FLAG	INTEGER	节点是否 OK。1 是，表示 OK；0 否，表示故障
3	MIN_FIRST_MODIFIED_LSN	BIGINT	最小 first_modified_lsn
4	CRASH_LSN	BIGINT	内存中记录的故障节点 file_lsn

15.2.1.29 V$DSC_GLS_SYS

显示 DSC 环境下的系统全局封锁相关信息。若发起一次封锁请求时，DSC 环境下全局封锁的缓存系统中与其冲突的封锁数目大于 1024，认为发生了一次死锁。

序号	列	数据类型	说明
1	TOTAL_SUCCESS	INTEGER	封锁成功的次数
2	TOTAL_WAIT	INTEGER	封锁冲突等待的次数
3	TOTAL_DEADLOCK	INTEGER	发生死锁的次数

4	HLCK_SEQ	BIGINT	DSC 集群全局封锁的缓存系统当前使用的序列号

15.2.1.30 V$DSCIDS

显示 DMDSC 中所有节点活动事务相关的信息。

序号	列	数据类型	说明
1	EPSEQ	INTEGER	DMDSC 节点序号
2	MIN_ACTIVE_ID	BIGINT	最小活动事务号
3	NEXT_ID	BIGINT	即将产生的事务号
4	N_ACTIVE	INTEGER	活动事务个数
5	MAX_CMT_LSN	BIGINT	事务提交生成的最大 LSN
6	MIN_CMT_TID	BIGINT	已提交、且日志已刷盘的事务号
7	MIN_LAST_CMT_LSN	BIGINT	最小活动事务上一次提交 LSN
8	NOTIFY_VERSION	BIGINT	活动事务广播信息版本号

15.2.1.31 V$DSC_TRX_VIEW

显示 DSC 环境下活动事务视图信息。

序号	列	数据类型	说明
1	TRXID	BIGINT	当前活动事务的 ID 号
2	NEXT_ID	BIGINT	下一个事务 ID 号，大于等于 next_id 的事务不可见
3	MIN_ACTIVE_ID	BIGINT	所有活动事务最小 ID, 小于 min_active_id 的事务不可见
4	EGTS_ENO	BIGINT	刷新视图添加 snap 任务时的 egts 执行号

5	TEMP_ENO	BIGINT	获取最小事务号时代替 trx 登记缓存事务提交计数的执行号
6	TEMP_CMTSEQ	BIGINT	获取最小事务号时代替 trx 登记的缓存事务提交计数
7	N_LONG	INTEGER	长事务数目
8	LONG_TIDS	CLOB	长事务事务 ID 数组
9	N_PE	INTEGER	数据页损坏事务信息数目
10	PE_TIDS	CLOB	数据页损坏事务信息数组
11	MVCC_PAGE_OPT	INTEGER	数据页可见性判断优化标记
12	LAST_CMT_LSN	BIGINT	上次事务提交产生的 LSN
13	TRX_START_LSN	BIGINT	第一次分配回滚页时的 LSN
14	MIN_LAST_CMT_LSN	BIGINT	可见视图里事务号最小事务的 last_cmt_lsn
15	REMOTE_NEXT_ID	VARCHAR(322)	DSC 集群其它节点的下一个事务 ID 号
16	REMOTE_MIN_ID	VARCHAR(322)	DSC 集群其它节点的所有活动事务最小 ID
17	MAX_CMT_LSN	VARCHAR(322)	DSC 集群其它节点的最新已提交事务 LSN
18	MIN_CMT_TID	VARCHAR(322)	DSC 集群其它节点的最小已提交事务 ID
19	N_INHASH_IDS	INTEGER	活动事务 ID 数组已使用数目
20	INHASH_IDS	CLOB	活动事务 ID 数组

15.2.2 DMASM 通用视图

15.2.2.1 V$ASMATTR

如果使用有 ASM 文件系统，可通过此视图查看 ASM 文件系统相关属性，登录任意节点查询得到的结果一致。

序号	列	数据类型	说明
1	AU_SIZE	INTEGER	单个 AU 大小，单位字节。在镜像环境中，该值为 NULL
2	EXTENT_SIZE	INTEGER	一个簇包含的 AU 个数
3	LOCAL_CODE	VARCHAR(64)	当前所连接的 ASMSERVER 的编码格式
4	LOCAL_LANG	VARCHAR(64)	当前所连接的 ASMSERVER 使用的语言： CN：中文 EN：英文
5	USE_SHM	VARCHAR(8)	是否使用共享内存，TRUE/FALSE
6	EXTENT_ARR_SIZE	INTEGER	共享内存能存放的 extent 个数，可以通过修改 DCR_EP_SHM_SIZE 来控制，EXTENT_ARR_SIZE 等于 FREE_EXTENTS_SIZE 与 LRU_EXTENTS_SIZE 的和
7	AUTO_RBL_TIME	INTEGER	镜像专用。INTEGER ASMSERVER 自动重平衡间隔，单位为 s。非镜像环境下无意义
8	FREE_EXTENTS_SIZE	INTEGER	共享内存中的空闲簇个数
9	LRU_EXTENTS_SIZE	INTEGER	共享内存中的已使用的簇个数

15.2.2.2 V$ASMGROUP

如果使用有 ASM 文件系统，可通过此视图查看 ASM 磁盘组信息，登录任意节点查询得到的结果一致。

序号	列	数据类型	说明

1	GROUP_ID	INTEGER	磁盘组 ID
2	GROUP_NAME	VARCHAR(128)	磁盘组名称
3	N_DISK	INTEGER	磁盘组中包含的磁盘个数
4	AU_SIZE	INTEGER	单个 AU 大小，单位为字节
5	EXTENT_SIZE(AU)	INTEGER	一个簇包含的 AU 个数
6	TOTAL_SIZE	INTEGER	磁盘组总大小，单位为 AU 个数
7	FREE_SIZE	INTEGER	磁盘组空闲大小，单位为 AU 个数
8	TOTAL_FILE_NUM	INTEGER	磁盘组中总的文件个数，包括文件和目录
9	TYPE	VARCHAR(16)	镜像专用。磁盘组副本数：EXTERNAL 单副本；NORMAL 两副本；HIGH 三副本；UNDEFINED：未知。非镜像环境下为 EXTERNAL
10	REDO_SIZE(MB)	INTEGER	镜像专用。磁盘组日志文件大小，单位 MB。非镜像环境下为 1
11	RBL_STAT	INTEGER	镜像专用。磁盘组重平衡状态。DISABLE：未启用；ENABLE 已启用。非镜像环境下为 DISABLE
12	RBL_PWR	VARCHAR(8)	镜像专用。磁盘组重平衡并行度。非镜像环境下为 0
13	AU_SIZE_MB	INTEGER	单个 AU 大小，单位为 MB
14	TOTAL_MB	INTEGER	磁盘组总大小，单位为 MB
15	FREE_MB	INTEGER	磁盘组空闲大小，单位为 MB

15.2.2.3 V$ASMDISK

如果使用有 ASM 文件系统，可通过此视图查看所有的 ASM 磁盘信息，登录任意节点查询得到的结果一致。

序号	列	数据类型	说明
1	GROUP_ID	INTEGER	所在的磁盘组 ID， 如果是未使用的磁盘，则值为-1
2	DISK_ID	INTEGER	磁盘 ID， 如果是未使用的磁盘，则值为-1
3	DISK_NAME	VARCHAR(128)	磁盘名称
4	DISK_PATH	VARCHAR(256)	磁盘路径
5	SIZE	INTEGER	磁盘大小，单位为 MB
6	FREE_AUNO	INTEGER	磁盘最大 AU 号
7	CREATE_TIME	VARCHAR(64)	磁盘创建时间
8	MODIFY_TIME	VARCHAR(64)	磁盘最近一次修改时间
9	FAILGROUP_NAME	VARCHAR(128)	镜像专用。故障磁盘组组名
10	FREE_MB	INTEGER	镜像专用。磁盘剩余空间。非镜像环境下为-1
11	STATUS	VARCHAR(16)	镜像专用。磁盘状态。非镜像环境下为 NORMAL。镜像状态取值包括：NORMAL：正常；DELETED：已删除；FAIL：故障；OFFLINE：离线；RECONNECT：重连中。若未使用磁盘，取值为 NULL
12	VERSION	INTEGER	非镜像专用。磁盘版本号。镜像环境下为

			0
13	IS_DESTROYED	INTEGER	磁盘物理信息是否损坏。0 表示未损坏，1 表示损坏。每隔 10min 检测一次磁盘

15.2.2.4 V$ASMFILE

如果使用有 ASM 文件系统，可通过此视图查看所有的 ASM 文件信息，登录任意节点查询得到的结果一致。

序号	列	数据类型	说明
1	FILE_ID	BIGINT	文件 ID
2	TYPE	VARCHAR(32)	类型，目录或文件
3	PATH	VARCHAR(256)	文件完整路径
4	SIZE_BYTES	BIGINT	文件实际大小，单位为字节， 目录类型的文件不占用空间，值为 0
5	SIZE_TOTAL	BIGINT	文件占用总空间大小，单位为字节， 目录类型的文件不占用空间，值为 0
6	CREATE_TIME	VARCHAR(64)	文件创建时间
7	MODIFY_TIME	VARCHAR(64)	文件修改时间
8	GROUP_ID	INTEGER	所在磁盘组 ID
9	DISK_ID	INTEGER	inode 项所在磁盘 ID
10	DISK_AUNO	INTEGER	inode 项所在磁盘 AU 编号
11	AU_OFFSET	INTEGER	inode 项在 AU 内的偏移
12	REDUNDANCY	VARCHAR(6)	镜像专用。文件副本数。非镜像环境下为 UNPROT

13	STRIPED	VARCHAR(6)	镜像专用。文件条带化粒度。字段值为 COARSE 时表示条带化粒度为 0，即粗粒度条带化。非镜像环境下为 NONE
14	REDUNDANCY_LOWERED	VARCHAR(2)	标记文件是否存在缺失副本，取值 Y/N

15.2.2.5 V$ASM_MAL_SYS

ASVRMAL 系统信息视图。仅 DMDSC 环境可使用。

序号	列	数据类型	说明
1	MAL_CHECK_INTERVAL	INTEGER	链路检测间隔
2	MAL_CONN_FAIL_INTERVAL	INTEGER	认定链路断开的时间间隔
3	MAL_CHECK_TIMEOUT	INTEGER	单个实例的 MAL 网络最大延迟时间，表示不进行 MAL 网络延迟时间检测。需要和 MAL_CHECK_IP 配合使用。仅当 MAL_CHECK_INTERVAL 不为 0 时有效。单位毫秒（ms），取值范围 0~65535，缺省为 0
4	MAL_CHECK_IP	VARCHAR(128)	第三方确认机器的 IP，用于检测各个实例的 MAL 网络延迟时间。需要和 MAL_CHECK_TIMEOUT 配合使用
5	MAL_LOGIN_TIMEOUT	INTEGER	MPP/DBLINK 等实例间登录时的超时检测间隔 3~1800，单位秒，缺省为 15
6	MAL_BUF_SIZE	INTEGER	单个 MAL 缓存大小限制，单位 MB。当 MAL 的缓存邮件超过此大小，会将邮件存储到文件

			中。取值范围 0~500000，缺省为 100，配置为 0 表示无限制
7	MAL_SYS_BUF_SIZE	INTEGER	MAL 系统总内存大小限制，单位 MB。取值范围 0~500000，缺省为 0，表示无限制
8	MAL_VPOOL_SIZE	INTEGER	MAL 配置的总的 POOL 大小，单位 MB。取值范围 1~500000，缺省为 128
9	MAL_MESSAGE_CHECK	INTEGER	是否对 MAL 通信消息启用消息体校验（只有当消息的发送端和接收端都配置为 1 才启用通信体校验）。0：不启用；1：启用。缺省为 1
10	MAL_COMPRESS_LEVEL	INTEGER	邮件压缩级别
11	MAL_USE_RDMA	INTEGER	Linux 环境下，MAL 是否使用 RDMA 通讯方式。0：否；1：是。缺省为 0

15.2.2.6 V$ASM_MAL_INI

ASVRMAL 参数信息。仅 DMDSC 环境可使用。

序号	列	数据类型	说明
1	MAL_NAME	VARCHAR (128)	MAL 名称
2	MAL_INST_NAME	VARCHAR (256)	实例名称
3	MAL_HOST	VARCHAR (256)	IP 地址
4	MAL_PORT	INTEGER	端口号
5	MAL_INST_HOST	VARCHAR (256)	对应实例 IP 地址
6	MAL_INST_PORT	INTEGER	对应实例的端口号

7	MAL_LINK_MAGIC	INTEGER	MAL 链路网段标识

15.2.3 DMASM 镜像专用视图

本节中 V$ASMPARTNER、V$ASM_OPERATION、V$ASM_REBALANCE_HISTORY 和 V$ASM_FAIL_AU 为镜像环境专用视图。

15.2.3.1 V$ASMPARTNER

如果使用有 DMASM 镜像，可通过此视图查看所有的 ASM 磁盘伙伴关系信息。

序号	列	数据类型	说明
1	GROUP_ID	INTEGER	磁盘组 ID
2	DISK_ID	INTEGER	磁盘 ID
3	PARTNER_ID	INTEGER	伙伴磁盘 ID
4	PARTNER_FAILGROUP	VARCHAR(128)	所属故障组名

15.2.3.2 V$ASM_OPERATION

如果使用有 DMASM 镜像，可通过此视图查看系统中重平衡或减少副本恢复进度信息。

序号	列	数据类型	说明
1	GROUP_ID	INTEGER	磁盘组 ID
2	DISK_ID	INTEGER	磁盘 ID
3	OPERATION	VARCHAR(16)	操作类型 ● REBALANCE(重平衡) ● REPAIR (缺失副本修复)
4	STATE	VARCHAR(5)	操作状态

			• RUN（正在运行） • WAIT（等待运行） • ERRS（发生错误停止）
5	POWER	INTEGER	重平衡并行度
6	SOFAR	INTEGER	已经迁移 AU 数
7	EST_WORK	INTEGER	预估需要迁移 AU 总数
8	EST_RATE	INTEGER	预估每分钟迁移 AU 数
9	EST_MINUTES	INTEGER	预估操作完成剩余时间（分钟）
10	BEGIN_TIME	TIMESTAMP	操作开始时间
11	END_TIME	TIMESTAMP	操作结束时间（发生错误停止运行时间）
12	PROGRESS	VARCHAR(8)	操作进度（百分比）
13	ERR_CODE	INTEGER	操作错误码

15.2.3.3 V$ASM_REBALANCE_HISTORY

DMASM 镜像环境中，查看 AU 重平衡历史信息。

序号	列	数据类型	说明
1	GROUP_ID	INTEGER	磁盘组 ID
2	SRC_DISK_ID	INTEGER	源磁盘 ID
3	SRC_AUNO	INTEGER	源 AU 号
4	DST_DISK_ID	INTEGER	目的磁盘 ID
5	DST_AUNO	INTEGER	目的 AU 号

15.2.3.4 V$ASM_FAIL_AU

DMASM 镜像环境中，查看故障 AU 信息。

序号	列	数据类型	说明
1	GROUP_ID	INTEGER	磁盘组 ID
2	DISK_ID	INTEGER	磁盘 ID
3	AU_NO	INTEGER	AU 编号
4	FILE_ID	BIGINT	所属文件 ID
5	AU_SEQ	INTEGER	文件内 AU 编号

15.3 DMDSC 日志文件

DMDSC 集群、DMCSS 集群和 DMASM 集群在运行过程中，各节点均会生成各自的日志文件，日志文件位于各节点的 log 目录里。

15.3.1 DMDSC 集群日志

对于 DMDSC 集群，每个 DMSERVER 节点会生成相应的事件日志文件。事件日志文件记录了 DM 数据库运行时的关键事件。例如：系统启动、关闭、内存申请失败、IO 错误等一些致命错误；数据库运行过程中的日志信息；备份还原过程中备份还原操作的阶段性信息等。

事件日志文件主要用于系统出现严重错误时进行查看并定位问题。事件日志文件随着 DMSERVER 的运行一直存在。

事件日志文件打印的是中间步骤的信息，所以出现部分缺失属于正常现象。事件日志信息格式为：时间+日志类型（INFO/WARNING/ERROR/FATAL）+进程（database）+进程 ID（P 开头）+线程（dm_sql_thd/main_thread 等）+日志内容。

DMSERVER 启动时，如遇到 dm.key、dm.ini 不存在等原因无法正常启动，因尚未获取到实例名，因此将生成的事件日志文件命名为“dm_unknown_年月”。例如：dm_unknown_202212。

DMSERVER 运行过程中产生的事件日志生成到当月的月度事件日志文件中，命名为“dm_实例名_年月”。例如：dm_DSC01_202212.log。

更多事件日志文件的介绍请参考《DM8 系统管理员手册》。

15.3.2 DMCSS 集群日志

对于 DMCSS 集群，每个 DMCSS 节点会产生各自的事件日志文件，记录 DMCSS 运行过程中的关键事件。例如：DMCSS 启动、关闭、操作数据库等。

DMCCS 事件日志文件命名为“dm_实例名_年月”。例如：dm_CSS1_202212.log。DMCSS 事件日志信息格式为：时间+日志类型（INFO/WARNING/ERROR/FATAL）+进程（dmcss）+进程 ID（P 开头）+线程 ID（T 开头）+日志内容。

15.3.3 DMASM 集群日志

DMASM 集群分为非镜像环境和镜像环境。两种环境下日志不同。下面分别进行介绍：

- **非 DMASM 镜像环境下的日志**

在非 DMASM 镜像环境中，对于 DMASM 集群，每个节点会产生以下几种日志文件：

一 DMASM 事件日志文件，记录 DMASMSVR 运行过程中的关键事件。例如：DMASMSVR 启动，关闭、MAL 通信检查、DMCSS 命令检查等。该日志文件命名为“dm_实例名_年月”。例如：dm_ASM1_202212.log。DMASM 事件日志信息格式为：时间+日志类型（INFO/WARNING/ERROR/FATAL）+进程（dmasmsvr）+进程 ID（P 开头）+线程 ID（T 开头）+日志内容。

二 DMASMCMD 工具日志文件，记录 DMASMCMD 执行的各种操作。该日志文件命名

为“dm_dmasmcmd_年月”。例如：dm_dmasmcmd_202212.log。该日志信息格式为：时间+日志类型（INFO/WARNING/ERROR/FATAL）+进程（dmasmcmd）+进程 ID（P 开头）+线程 ID（T 开头）+日志内容。

三 DMASMAPI 日志文件，记录通过 DMASMAPI 接口操作 DMASM 文件的内容。该日志文件命名为“dmasmapi_进程 ID（P 开头）_年月”。例如：dmasmapi_P0000540862_202210.log。DMASMAPI 日志信息格式为：时间+线程 ID（T 开头）+日志类型（TRACE/WARNING/ERROR/FATAL）+日志内容。

四 DMASMTOOL 工具日志文件，记录通过 DMASMTOOL 执行的各种操作。该日志文件命名为“dm_dmasmtool_年月”。例如：dm_dmasmtool_202212.log。该日志信息格式为：时间+线程 ID（T 开头）+日志类型（TRACE/WARNING/ERROR/FATAL）+日志内容。

五 DMASM 文件操作日志文件，记录系统对 DMASM 文件的打开、关闭、增加，删除等操作。该日志文件命名为“dmasm_节点序号_年月”，节点序号为 01、02……。例如：dmasm02_202212.log。DMASM 文件操作日志信息格式为：时间+DMASM+日志类型（TRACE/WARNING/ERROR/FATAL）+日志内容。

- **DMASM 镜像环境下的日志**

在 DMASM 镜像环境中，对于 DMASM 集群，每个节点会产生几种日志文件：

一 DMASM 事件日志文件，记录 DMASMSVRM 运行过程中的关键事件。该日志文件在镜像环境下和非镜像环境下，用法完全一致。具体请参考非镜像环境下的 DMASM 事件日志文件。

二 DMASMCMDM 工具日志文件，记录 DMASMCMDM 执行的各种操作。该日志文件命名为“dm_dmasmcmdm_年月”。例如：dm_dmasmcmdm_202212.log。该日志信息格式为：时间+日志类型（INFO/WARNING/ERROR/FATAL）+进程（dmasmcmdm）+进程 ID（P 开头）+线程 ID（T 开头）+日志内容。

三 DMASMAPIM 日志文件，记录通过 DMASMAPIM 接口操作 DMASM 文件的内容。该日志文件命名为“dmasmapi3_进程 ID（P 开头）_年月”。例如：dmasmapi3_P0000540862_202210.log。DMASMAPI 日志信息格式为：时间+线程 ID（T 开头）+日志类型（TRACE/WARNING/ERROR/FATAL）+日志内容。

四 DMASMTOOLM 工具日志文件，记录通过 DMASMTOOLM 执行的各种操作。该日志文件命名为“dm_dmasmtoolm_年月”。例如：dm_dmasmtoolm_202212.log。该日志信息格式为：时间+线程 ID（T 开头）+日志类型（TRACE/WARNING/ERROR/FATAL）+日志内容。

五 DMASM 文件操作日志文件，记录系统对 DMASM 文件的打开、关闭、增加，删除等操作。该日志文件命名为“dmasm_trace_进程 ID（P 开头）_年月日”。例如：dmasm_trace_P0000002432_20221202.log。DMASM 文件操作日志信息格式为：时间+DMASM+日志类型（TRACE/WARNING/ERROR/FATAL）+日志内容。

16备份还原

DMDSC 集群备份还原的功能、语法与单节点数据库基本保持一致，本章主要介绍 DMDSC 集群与单节点数据库备份、还原的使用方法差异，并说明在 DMDSC 集群中执行备份还原的一些注意事项。关于备份还原更详细的说明，请参考《DM8 备份与还原》手册。

16.1 DMDSC 和单节点差异

达梦数据库中，备份还原的对象包括：表、表空间和数据库。表备份还原的操作对象是数据页，而数据页是通过 BUFFER 获取的，与存储无关，因此 DMDSC 集群的表备份还原与单节点没有任何区别。表空间备份只需要访问属于这个表空间的数据文件，并不需要备份归档日志，因此 DMDSC 集群的表空间备份与单节点没有任何区别。表空间还原要求将表空间数据恢复到最新状态，需要重做归档日志，集群节点需要访问其他节点的本地归档日志。另外，数据库备份以及数据库恢复操作均需要访问其他节点的本地归档日志文件。因此 DMDSC 集群支持节点间相互配置远程归档，配置远程归档后集群中的任一节点均能成功访问其他所有节点的本地归档日志，解决了 DMDSC 集群备份还原过程中访问其他节点归档日志文件问题。DMDSC 集群备份恢复的使用方法与单节点基本保持一致。

16.2 远程归档

16.2.1 简介

远程归档（REMOTE ARCHIVE）专门用于 DMDSC 环境中，是将当前节点的远程归档目录配置为另一节点的本地归档目录，以此来共享它的本地归档日志文件。其中，另一节点的本地归档目录必须位于 ASM 共享存储或其他共享磁盘上，以使当前节点可以成功访问。

远程归档采用双向配置的方式，即两个节点将自己的远程归档相互配置在对方机器上。

集群中所有的节点，都拥有一套包括所有节点的，完整的归档日志文件。

注意：**远程归档必须双向配置。否则，单向配置后远程归档会处于无效状态。**

共享本地归档的远程归档就是双向配置的两个节点都将对方的本地归档作为自己的远程归档。例如，节点 0 的本地归档配置在 ASM 共享存储或其他共享磁盘上。节点 1 可以通过将自己的远程归档目录设置为节点 0 的本地归档目录，将节点 0 的本地归档日志文件作为自己的远程归档日志文件。下图展示了一个共享本地归档的远程归档的双向配置示意图。

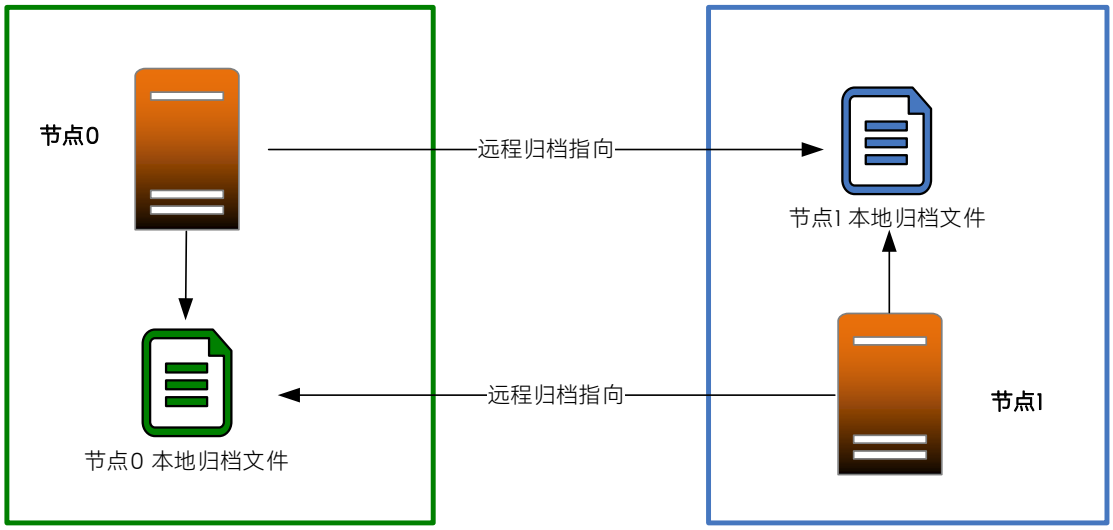


图 16.1 共享本地归档的远程归档

DMDSC 集群中，各个节点配置一个远程归档为其他节点的本地归档，通过这种共享本地归档的方式，可以在任意一个节点上，访问到 DMDSC 集群所有节点产生的、完整的归档日志文件。由于归档是双向配置的，若某一节点出现故障，故障处理过程中，其他正常节点会利用自身归档帮助故障节点进行远程归档修复，归档修复完成后，故障节点的本地归档路径下会生成名称中包含“REMOTE”的归档日志文件，该过程无需用户手动干涉，仅当 dmarch.ini 中 ARCH_FLUSH_BUF_SIZE 大于 0 启用归档刷盘缓存机制时，才会执行远程归档修复。

16.2.2 远程归档配置方法

与其他归档类型一样，远程归档也是配置在 DMARCH.INI 文件中，远程归档相关的主要几个配置项包括：

1. ARCH_TYPE 设置为 REMOTE，表示是远程归档；
2. ARCH_DEST 设置为远程数据库实例名；
3. ARCH_INCOMING_PATH 设置为远程数据库实例的本地归档日志文件存放路径；
4. ARCH_RECV_PATH 设置本地的归档备份路径。当 DMDSC 集群中出现故障节点时，控制节点会将联机日志检查点偏移后的所有有效日志备份到 ARCH_RECV_PATH 路径下，即便后续 ASM 文件系统损坏，也能保证 LOCAL 归档与 ARCH_RECV_PATH 路径下的归档合并起来是完整的。该参数没有默认值，可不配置。

16.2.3 DMDSC 归档配置方法

一般建议 DMDSC 集群中的节点，除了配置本地归档之外，再双向配置集群中所有其他节点的远程归档。查询 V$DM_ARCH_INI、V$ARCH_STATUS 等动态视图可以获取归档配置以及归档状态等相关信息。

下面以两节点 DMDSC 集群为例，说明如何配置远程归档，DSC0 和 DSC1 是 DMDSC 集群中的两个实例，双向配置共享本地归档的 REMOTE 归档：

DSC0 实例的 DMARCH.INI 配置：

```
[ARCHIVE_LOCAL1]
    ARCH_TYPE           = LOCAL
    ARCH_DEST           =+DMDATA/DSC0/arch
    ARCH_FILE_SIZE      = 128
    ARCH_SPACE_LIMIT    = 0
```

```
[ARCH_REMOTE1]
    ARCH_TYPE               = REMOTE
    ARCH_DEST               = DSC1
    ARCH_INCOMING_PATH      =+DMDATA/DSC1/arch
```

DSC1 实例的 DMARCH.INI 配置：

```
[ARCHIVE_LOCAL1]
    ARCH_TYPE               = LOCAL
    ARCH_DEST               =+DMDATA/DSC1/arch
    ARCH_FILE_SIZE          = 128
    ARCH_SPACE_LIMIT        = 0
[ARCH_REMOTE1]
    ARCH_TYPE               = REMOTE
    ARCH_DEST               = DSC0
    ARCH_INCOMING_PATH =+DMDATA/DSC0/arch
```



注意：**远程归档指定的本地归档目录必须和真实的本地归档路径完全一致，否则服务器启动过程中校验失败。**

16.3 DMDSC 备份集

备份集除了保存备份对象的数据（数据页和归档日志），还记录了备份库节点的描述信息。单节点库生成的备份集，可以认为是只包含一个节点的特殊备份集。与节点相关的描述信息主要包括：

1. DMDSC 库的节点数，单节点库为 1
2. 备份开始时 DMDSC 节点的状态，以及各节点 REDO 日志的起始 LSN 和 SEQ

3. 备份结束时 DMDSC 节点 REDO 日志的结束 LSN 和 SEQ

4. 备份集中记录了执行备份节点的 DM.INI 配置参数，还原时若指定了 REUSE DMINI，可使用备份集中的参数值覆盖目标库节点的 DM.INI 文件

备份操作可以在 DMDSC 集群的任意节点执行，生成的备份集可以存放在本地磁盘上，也可以存放到共享存储的 DMASM 目录中。但考虑到数据安全性，一般建议将备份集保存在本地磁盘上。可以通过以下方式，将备份集生成到本地磁盘：

1. 使用 DMINIT 初始化库时，将默认备份目录 bak_path 设置为本地磁盘

2. 修改 DMDSC 集群中所有节点的 DM.INI 配置文件，将 bak_path 设置为本地磁盘

3. 执行备份时，手动指定备份集路径为本地磁盘

16.4 非镜像环境下备份还原实例

下面以从归档恢复为例说明 2 节点（DSC0、DSC1）DMDSC 环境下的备份恢复：

1） 搭建 DMDSC 环境，每个节点都需要双向配置共享本地归档的远程归档。归档配置示例如下：

DSC0 实例的 DMARCH.INI 配置：

```
[ARCHIVE_LOCAL1]
    ARCH_TYPE           = LOCAL
    ARCH_DEST           =+DMDATA/DSC0/arch
    ARCH_FILE_SIZE      = 128
    ARCH_SPACE_LIMIT    = 0
[ARCH_REMOTE1]
    ARCH_TYPE           = REMOTE
    ARCH_DEST           = DSC1
    ARCH_INCOMING_PATH =+DMDATA/DSC1/arch
```

DSC1 实例的 DMARCH.INI 配置：

```
[ARCHIVE_LOCAL1]
    ARCH_TYPE           = LOCAL
    ARCH_DEST           = +DMDATA/DSC1/arch
    ARCH_FILE_SIZE      = 128
    ARCH_SPACE_LIMIT    = 0
[ARCH_REMOTE1]
    ARCH_TYPE           = REMOTE
    ARCH_DEST           = DSC0
    ARCH_INCOMING_PATH = +DMDATA/DSC0/arch
```

2） 启动 DIsql，联机备份数据库。备份其中任意一个节点即可备份整个 DMDSC 环境。

```
SQL>BACKUP DATABASE BACKUPSET '/home/dm_bak/db_full_bak_for_dsc';
```

3） 还原数据库。还原数据库之前可选择对备份文件进行校验。需要注意的是，待还原的目标库可以是单机库，也可以是 DMDSC 库，且节点个数允许不同。

```
RMAN> RESTORE DATABASE '/opt/dmdbms/data/DAMENG_FOR_RESTORE/dm.ini' FROM
BACKUPSET '/home/dm_bak/db_full_bak_for_dsc';
```

4） 恢复数据库。DMDSC 库恢复要求各节点归档完整性由用户保证，即各节点的本地归档都能够访问到。若本地归档配置在 ASM 共享存储上，RMAN 工具启动时，需要指定 DCR_INI 参数。

```
./dmrman dcr_ini=/dmdata/dameng/dmdcr.ini
RMAN>RECOVER DATABASE '/opt/dmdbms/data/DAMENG_FOR_RESTORE/dm.ini' WITH
ARCHIVEDIR '+DMDATA/DSC0/arch','+DMDATA/DSC1/arch';
```

5） 数据库更新

```
RMAN>RECOVER DATABASE '/opt/dmdbms/data/DAMENG_FOR_RESTORE/dm.ini' UPDATE
```

DB_MAGIC

16.5 镜像环境下备份还原实例

16.5.1 配置文件说明

为了支持从老版本非镜像库备份还原至镜像版本库。首先，对老版本库备份操作完成后，需要使用新版 dmrman 工具导出备份集映射文件（mappedfile），然后在映射文件中指定配置参数镜像数（data_mirror）与条带化类型（data_striping），其他参数无需改动；还原时以通过指定映射文件方式进行还原恢复。

下表对备份集映射文件中的配置参数进行说明。除了 data_path 之外，mappedfile 中其他参数值均来自于备份源库。data_path 参数值由 DUMP 命令（DATABASE '<ini_path>' 或 TO '<system_dir>'指定）决定，如果未指定 DUMP 命令，则 data_path 参数值也来自于备份源库。更多的备份集映射文件说明请参考《DM8 备份与还原》。

表 16.1 mappedfile 配置参数

参数	含义
fil_id	文件 ID
ts_id	表空间 ID
ts_name	表空间名
data_path	表空间路径
mirror_path	DMASM 镜像环境专用。表示表空间创建时指定的数据文件镜像路径，和 ASM 文件镜像无关
data_mirror	DMASM 镜像环境专用。表示镜像类型。SYSTEM/MAIN/ROLL 表空间数据文件副本数，取值 1、2 或 3；缺省为 3。在单机或非镜像环境中，data_mirror 为 1
data_striping	DMASM 镜像环境专用。表示条带化类型。数据文件条带化粒度，取值 0、32、64、

	128、256。缺省为 32。单位 KB。在单机或非镜像环境中，data_striping 为 0

16.5.2 备份还原

镜像环境与非镜像环境下的备份还原过程一致，参考 16.4 非镜像环境下备份还原实例的备份文件。

1）生成备份集后需要使用新版 dmrman 工具导出备份集映射文件，映射文件内容参考 16.5.1 配置文件说明。

```
RMAN> DUMP BACKUPSET '/home/dm_bak/db_full_bak_for_dsc' DATABASE
'/opt/dmdbms/data/DAMENG_FOR_RESTORE/dm.ini' MAPPED FILE
'/opt/dmdbms/data/map_file_01.txt';
```

导出的 map_file_01.txt，老版本非镜像库中 data_mirror=1 与 data_striping=0。如下所示：

```
/***********************************************************/
/***    Delete the unnecessary modified groups                    **/
/***    Modify the data_path or mirror_path only in one group    **/
/***********************************************************/


/**=========================================================**/
/*[DAMENG_SYSTEM_FIL_0]*/
fil_id              = 0
ts_id               = 0
ts_name             = "SYSTEM"
data_path           = /opt/dmdbms/data/DAMENG_FOR_RESTORE/SYSTEM.DBF
mirror_path         =
```

```
data_mirror      = 1
data_striping    = 0

/**============================================================**/
/*[DAMENG_ROLL_FIL_0]*/
fil_id              = 0
ts_id               = 1
ts_name             = "ROLL"
data_path           = /opt/dmdbms/data/DAMENG_FOR_RESTORE/ROLL.DBF
mirror_path      =
data_mirror      = 1
data_striping    = 0

/**============================================================**/
/*[DAMENG_MAIN_FIL_0]*/
fil_id              = 0
ts_id               = 4
ts_name             = "MAIN"
data_path           =/opt/dmdbms/data/DAMENG_FOR_RESTORE/MAIN.DBF
mirror_path      =
data_mirror      = 1
data_striping    = 0

/**============================================================**/
```

```
/*[DAMENG_RLOG_FIL_4]*/
ts_id               = 2
ts_name             = "RLOG"
data_path           =
data_mirror         = 1
data_striping       = 0


/*************************** END ***************************/
```

2) 手动修改映射文件中的镜像和条带化类型参数（data_mirror 和 data_striping）。例如，修改为 data_mirror=2 和 data_striping=64。

修改后的 map_file_01.txt 如下：

```
/*************************************************************/
/***   Delete the unnecessary modified groups                ***/
/***   Modify the data_path or mirror_path only in one group ***/
/*************************************************************/


/**=========================================================**/
/*[DAMENG_SYSTEM_FIL_0]*/
fil_id              = 0
ts_id               = 0
ts_name             = "SYSTEM"
data_path           = /opt/dmdbms/data/DAMENG_FOR_RESTORE/SYSTEM.DBF
mirror_path         =
data_mirror         = 2
```

```
data_striping   = 64


/**============================================================**/
/*[DAMENG_ROLL_FIL_0]*/
fil_id              = 0
ts_id               = 1
ts_name             = "ROLL"
data_path           = /opt/dmdbms/data/DAMENG_FOR_RESTORE/ROLL.DBF
mirror_path         =
data_mirror         = 2
data_striping   = 64


/**============================================================**/
/*[DAMENG_MAIN_FIL_0]*/
fil_id              = 0
ts_id               = 4
ts_name             = "MAIN"
data_path           =/opt/dmdbms/data/DAMENG_FOR_RESTORE/MAIN.DBF
mirror_path         =
data_mirror         = 2
data_striping   = 64


/**============================================================**/
/*[DAMENG_RLOG_FIL_4]*/
```

```
ts_id               = 2

ts_name             = "RLOG"

data_path           =

data_mirror         = 2

data_striping       = 64



/*************************** END ***************************/
```

3）指定映射文件方式进行还原数据库。

```
RMAN> RESTORE DATABASE '/opt/dmdbms/data/DAMENG_FOR_RESTORE/dm.ini' FROM

BACKUPSET '/home/dm_bak/db_full_bak_for_dsc' MAPPED FILE

'/opt/dmdbms/data/map_file_01.txt';
```

4）恢复数据库。

```
RMAN> RECOVER DATABASE '/opt/dmdbms/data/DAMENG_FOR_RESTORE/dm.ini' FROM

BACKUPSET '/home/dm_bak/db_full_bak_for_dsc';
```

5）数据库更新。

```
RMAN> RECOVER DATABASE '/opt/dmdbms/data/DAMENG_FOR_RESTORE/dm.ini' UPDATE

DB_MAGIC;
```

16.6 使用说明

1. 配置远程归档时，必须同时配置本地归档；

2. DMDSC 集群环境中，备份还原涉及到的 trace 文件路径、DUMP 命令的映射文件路径、SHOW 命令的备份集信息输出文件路径都不支持 DMASM 类型文件；

3. 由于 DMDSC 集群中，各个节点可能存在 LSN 值相同的 REDO 日志，恢复过程中无法严格校验归档日志的完整性；因此，需要用户保证全局归档日志的完整性；

4. 如果由于节点故障等原因，导致归档日志不完整，则需要使用 DMRMAN 工具进行归档修复；

5. 在恢复过程中创建的数据文件，优先使用原始路径创建，如果创建失败，则会在 system_path 目录下创建。因此，在恢复结束后，需要检查一下是否有数据文件创建在本地磁盘上，如果有则需要用户手动执行 SQL，将这些文件重新存放到共享存储、或者 DMASM 文件系统中，确保数据文件可以被 DMDSC 集群中的所有节点访问；

6. 归档日志是恢复数据库的关键，建议将归档文件与数据文件分别保存到不同的磁盘上，防止归档文件和数据文件同时损坏，以降低数据无法修复的风险；

7. 如果需要访问 DMASM 文件系统，DMRMAN 必须设置 DCR_INI 参数，指定 DCR 的访问配置；

8. 还原操作指定 REUSE DMINI 选项时，会将备份集中的 DM.INI 参数更新还原节点上的 DM.INI 配置文件，DMDSC 集群中其他节点的 DM.INI 并不会更新，需要用户手动修改；

9. 若 INI 参数 DSC_TRX_CMT_LSN_SYNC 为零，事务提交时不同步 LSN，此时对于不同 DSC 节点无关联的事务，在还原恢复时日志重演顺序不保证和事务提交顺序相同。非完全恢复时（恢复到指定 LSN 或时间，或者缺失最新归档），数据库未恢复到最新状态，可能存在后提交的事务已重演完成，先提交的事务仍未重演的情况。

17 DMDSC 使用说明

DMDSC 集群针对同一份数据，提供了多个活动的数据库实例，因此具有负载均衡、高可靠性等特征。但与单机系统相比，DMDSC 具有更复杂的架构、更多的组件，因此对 DMDSC 集群的使用也提出了更高的要求。本章主要介绍 DMDSC 集群的使用说明。

17.1 统一组件版本

搭建 DMDSC 环境时，应注意 DMDSC 中各实例使用的 DM 服务器版本应一致，同时还应注意各实例所在主机的操作系统位数、大小端模式、时区及时间设置都应一致。

此外，DMDSC 集群包含 DMCSS、非镜像环境下的 DMASMSVR/DMASMAPI/DMSERVER、镜像环境下的 DMASMSVRM/DMASMAPIM/DMSERVER 等诸多组件。并且 DMASMSVR 和 DMASMAPI 之间，DMASMSVRM 和 DMASMAPIM 之间均需要用到共享内存进行数据交换。因此，搭建和使用 DMDSC 集群时，要特别注意各个组件的版本号是否一致。是否同为 32 位或 64 位。如果各组件版本号不一致，可能会导致系统异常，无法正常使用。

17.2 提升 DMDSC 性能

DMDSC 通过缓存交换和全局事务管理等机制，协调、管理多个节点的 CPU、内存等计算资源，以提升数据库管理系统的事务处理能力。数据和信息需要在节点间通过网络频繁地进行传输，如果多个数据库实例过多地访问、修改相同的数据页，就会导致缓存交换频繁，内部网络流量过高，形成 DMDSC 集群的性能瓶颈。为了提升 DMDSC 集群的整体性能，建议：

1.使用带宽更大、速度更快的网络设备，降低数据和信息在节点之间传递的从网络延迟，比如使用 InfiniBand 或者万兆的以太网。

2.根据业务逻辑，把不同类别的数据库请求分别部署到不同的节点，减少不同节点对相同数据页或者数据库对象的共享访问、和修改。

3.优化应用逻辑和每一句 SQL，DMDSC 集群并不是解决性能问题的万能方案，未经优化、糟糕的 SQL 查询可能会极大降低数据库的吞吐量。

4.充分分析应用系统特征，以及性能瓶颈在哪里。比如一些 IO 密集型的应用系统，性能瓶颈往往出现在存储的 IO 上，这种情况下单纯地增加 DMDSC 节点数并不会提高性能，反而可能由于节点数增加，缓存交换更加频繁，进一步降低系统性能。针对这种情况，正确的选择是增加磁盘，将 IO 分散到更多的磁盘；或者使用速度更快的固态盘来提升 IO 性能。

5.使用'ALTER TABLE XXX WITHOUT COUNTER'语句，关闭频繁插入、删除记录表的快速计数功能。DM8 默认开启表快速记录功能，系统动态维护表上的记录总数，并将记录数写入到表的控制页中；DMDSC 环境开启这个功能，并且多个节点并发向同一张表插入记录时，控制页的访问权限在节点间不断地进行回收和授权，控制页的内容也会不断地在节点间进行传递，进而影响 DMDSC 整体的并发性能。

17.3 心跳说明

DMDSC 集群中，DMCSS 依赖心跳机制来判断集群中的各个实例是否处于正常状态。DMDSC 集群支持磁盘心跳检测机制。

磁盘心跳的载体是共享存储上的 VOTE 磁盘（非镜像环境下）或 DCRV 磁盘（镜像环境下），DMCSS、DMASMSVR 或 DMASMSVRM、DMSERVER 启动后，每间隔 1 秒往 VOTE 磁盘或 DCRV 磁盘各自固定的偏移写入时间戳，DMCSS 定时读取 VOTE 磁盘或 DCRV 磁盘信息，根据时间戳变化情况来判断被监控对象是否活动。实例故障认定时间取决于配置参数 DCR_GRP_DSKCHK_CNT，一般建议 DCR_GRP_DSKCHK_CNT 至少配置为 60 秒以上，避免在系统极度繁忙情况下，由于操作系统调度原因导致误判实例故障。在规划 DMDSC 集群存储时，最好将磁盘（VOTE 或 DCRV）与 IO 密集的数据库文件和日志文件分开存放，

分别保存在不同的物理磁盘上，避免由于数据库 IO 影响心跳信息的写入和读取，导致极端情况下误判实例故障。

DMDSC 集群中磁盘心跳检测失败，DMCSS 会认为对应实例故障，启动故障处理流程。

17.4 重新格式化 DMASM

DMASM 文件系统格式是自描述的，DMASMSVR 启动时会从指定路径扫描块设备，根据块设备的头信息判读是否是 ASM 磁盘，进而获取 ASM 磁盘组元数据信息，初始化 DMASM 文件系统。所以要求一套硬件环境只能搭建一套 DMASM 文件系统，否则会产生冲突，会导致 dmasmsvr 启动失败。如果要重新初始化 DMASM 文件系统，需要将指定路径下的所有块设备头信息格式化一遍，避免新老环境 ASM 磁盘信息冲突。

如果出现磁盘 id 相同导致 dmasmsvr 启动失败的情况，在可以确定某一磁盘为无效磁盘的情况下，用 dmasmcmd 工具执行 create asmdisk disk_path name 的方式清理无效磁盘信息。

17.5 重新初始化 DMDSC 库

DMSERVER 发生异常的情况下，DMCSS 会根据配置的心跳时间来确认节点故障，并写入故障信息到 DCR Disk 中，然后执行对应的故障处理或故障恢复。

对于 DMSERVER 节点全部故障的情况，如果想直接重新初始化数据库，则需要手动清理 DCR Disk 中的故障信息（DCR Disk 没有重新初始化），避免 DMCSS 对新初始化的库再次执行故障处理。

注意手动清理的时机，需要在 DMCSS 写入故障信息到 DCR Disk 之后，否则手动清理完成后，DMCSS 再写入故障信息（DMSERVER 的心跳时间配置比较长时会出现这种情况），清理操作仍然起不到作用。

DMCSS 手动清理之前，可通过下面三种方式避免 DMCSS 在清理之后再次修改 DCR

Disk 中的故障信息：

1. 等待配置的心跳时间之后，确保 DMCSS 已写入故障信息之后，再执行手动清理。

2. 确定 DMSERVER 确实故障时，通过监视器的 ep halt 命令，通知 DMCSS 写入故障信息，然后再执行手动清理。

3. 直接退出 DMCSS，在手动清理完成之后再重启 DMCSS。

可使用以下两种方式对指定组的故障节点信息进行清理：

1. 通过 dmasmcmd 的 export 命令，将 DCR Disk 的配置信息导出，手动修改 DMSERVER 对应组中的 DCR_GRP_N_ERR_EP 值为 0，DCR_GRP_ERR_EP_ARR 内容为空，再通过 asmcmd 的 import 命令导入到 DCR Disk 中。

2. 通过 dmasmcmd 的 clear 命令，直接清理指定组的故障节点信息。

命令的具体用法请参考 10.6.1 DMASMCMD 的用法说明。

17.6 内部网络异常

DMDSC 环境多个 DMSERVER 之间的很多功能都通过 MAL 系统来传递控制命令或数据（事务管理、封锁管理等等，详见 4 DMDSC 关键技术）。如果网络故障导致 DMSERVER 间 MAL 通讯失败，出现 MAL 邮件丢失等情况，会导致 DMSERVER 一直收不到请求响应，整个系统卡住。

遇到这种情况，DMSERVER 会通过 VOTE 磁盘（非镜像环境下）或 DCRV 磁盘（镜像环境下）通知 DMCSS MAL 链路故障。如果此时节点实例都正常运行，则 DMCSS 的默认处理逻辑是主动 HALT 节点号大的节点，保留节点号小的节点，例如两节点集群包含 0 号和 1 号两个节点，则主动 HALT 1 号节点。用户也可配置 DMDCR_LINK_CHECK_IP 参数来指定一个第三方机器 IP，配置 DMDCR_LINK_CHECK_IP 参数后，如果出现网络故障，各个节点将分别尝试联通该第三方机器 IP，若联通失败则认定是网络故障的节点，此时主动 HALT 该节点。DMDCR_LINK_CHECK_IP 是 Linux 平台专用参数，关于

DMDCR_LINK_CHECK_IP 的配置方式以及使用场景的详细介绍请参考 9.2 DMDCR.INI。

由于 DMSERVER 的这种主动通知机制，DMDSC 系统可能会比 MAL_CONN_FAIL_INTERVAL 和 MAL_CHECK_INTERVAL 设置值更早发现内部网络故障，从而进行处理。

17.7 创建 DBLink

DMDSC 系统可支持 DBLink 的创建和访问。DMDSC 系统和单节点之间、DMDSC 和 DMDSC 系统之间都支持 DBLink 的创建，由于同构的 DBLink 之间依赖于 MAL 系统，只需要通过配置 DMMAL.INI 文件即可实现。将每个实例 mal 项都加入 DMMAL.INI 文件，并拷贝到每个实例目录下，即可创建 DBLink。

DMDSC 和单节点的 DBLink 配置，需在 DSC 节点的 DMMAL.INI 中加入单节点 mal 项，同时复制到单节点，打开单节点的 MAL 配置，全部实例重新启动后即可创建 DBLink。

举例如下：

```
[MAL_INST1]

    MAL_INST_NAME                = DSC01

    MAL_HOST                     = 10.0.2.101

    MAL_PORT                     = 7236

    MAL_INST_HOST                = 192.168.0.101

    MAL_INST_PORT                = 5336

    MAL_LINK_MAGIC               = 0

[MAL_INST2]

    MAL_INST_NAME                = DSC02

    MAL_HOST                     = 10.0.2.102
```

```
    MAL_PORT                = 7237
    MAL_INST_HOST           = 192.168.0.102
    MAL_INST_PORT           = 5336
    MAL_LINK_MAGIC          = 0
[MAL_INST3]    #单节点的配置项
    MAL_INST_NAME           = EP01
    MAL_HOST                = 10.0.2.103
    MAL_PORT                = 7238
    MAL_INST_HOST           = 192.168.0.103
    MAL_INST_PORT           = 5336
    MAL_LINK_MAGIC          = 0
```

两个 DMDSC 系统之间也类似，需在一套 DMDSC 的 DMMAL.INI 中加入另一套的 DMMAL.INI 内容，并拷贝到两套 DSC 系统的所有实例。

举例如下:

```
[MAL_INST1]
    MAL_INST_NAME           = DSC011
    MAL_HOST                = 10.0.2.101
    MAL_PORT                = 7236
    MAL_INST_HOST           = 192.168.0.101
    MAL_INST_PORT           = 5336
    MAL_LINK_MAGIC          = 0
[MAL_INST2]
    MAL_INST_NAME           = DSC012
```

```
    MAL_HOST                = 10.0.2.102
    MAL_PORT                = 7237
    MAL_INST_HOST           = 192.168.0.102
    MAL_INST_PORT           = 5337
    MAL_LINK_MAGIC          = 0

    #另一套 DSC 系统
[MAL_INST3]
    MAL_INST_NAME           = DSC021
    MAL_HOST                = 10.0.2.102
    MAL_PORT                = 7238
    MAL_INST_HOST           = 192.168.0.102
    MAL_INST_PORT           = 5338
    MAL_LINK_MAGIC          = 0
[MAL_INST4]
    MAL_INST_NAME           = DSC022
    MAL_HOST                = 10.0.2.101
    MAL_PORT                = 7239
    MAL_INST_HOST           = 192.168.0.101
    MAL_INST_PORT           = 5339
    MAL_LINK_MAGIC          = 0
```

某些应用场景下，DBLINK 需要跨网段访问数据库实例，我们只要根据实际情况，修改 MAL 配置参数，将处于相同网段实例的 MAL_LINK_MAGIC 配置为相同的值，不同网段实

例的 MAL_LINK_MAGIC 配置为不同的值，就可以实现跨网段访问。详情可参考《DM8 系统管理员手册》DMMAL.INI 配置章节。

17.8 节点硬件故障，如何启动 DMDSC 集群

正常的 DMDSC 集群启动流程，dmcss 会在监控到所有节点都启动后，再执行正常的数据库启动流程。当某个节点出现硬件故障无法启动时，DMDSC 集群无法自动启动。针对这种情况，dmcssm 监视器提供了 open force 命令，通知 dmcss 将未启动节点设置为故障状态，强制启动活动节点，以尽快恢复数据库服务。

17.9 MOUNT/OPEN 操作

登录 DSC 下任意节点都可以执行 MOUNT/OPEN 操作，仅在登录节点生效，如果需要所有节点都 MOUNT 或者 OPEN，需要登录所有节点执行命令。

17.10 块设备路径变化

基于 DMASM 的 DMDSC 在运行过程中，如果磁盘重新进行规划，可能会导致磁盘对应的块设备名称有变化，从而导致 DMASM 环境配置可能也需要进行改变。

DMASM 文件系统是自描述系统，会自动扫描/dev/raw/下的所有块设备，根据头信息来获取信息，和块设备名路径没有关系。只是 DCR 和 VOTE 磁盘的路径必须指定，若这两个路径没有变化，可以不用修改 DMASM 环境配置。

如果 DCR 和 VOTE 磁盘的路径发生了改变，则需要用 DMASMCMD 工具重新格式化 DCR 和 VOTE 磁盘，不会影响其他 ASM 磁盘，也不影响已经存在的 ASM 文件。格式化后需要对 DCR 和 VOTE 磁盘进行初始化：

- 如果保存了原始的 DMDCR_CFG.INI 文件，修改 DMDCR_CFG.INI 的 DCR_VTD_PATH 路径信息，再用 DMASMCMD 工具初始化就可以；

- 如果没有保存原始 DMDCR_CFG.INI 文件，可以用 DMASMCMD 工具先 export 出一份 DMDCR_CFG.INI 文件，再修改其中 DCR_VTD_PATH 路径信息，然后重新初始化。

17.11 滚动升级

DMDSC 可以确保在不中断数据库服务的情况下，实现滚动升级。

需要注意的是，V8.1.2.128 版本及更早的版本升级到 V8.1.4.48 版本，不支持滚动升级，只能正常退出所有进程，整体升级。V8.1.4.80 版本开始支持 V8.1.2.128 及更早的版本滚动升级。

滚动升级过程中，各节点轮流进行升级，最后升级主控 CSS 所在节点。将待升级的 DMDSC 节点单独退出并升级，活动节点继续对外提供服务。退出节点时，要求：先单独退出 DMSERVER；DMSERVER 单独退出完成后，再杀掉 DMCSS；最后杀掉 DMASMSVR。

滚动升级操作步骤如下：

以两节点的 DMDSC 举例说明，假设节点 2 为主控 CSS 所在节点。

第一步 退出节点 1、升级节点 1、再启动节点 1。此时活动节点 2 继续对外提供服务。

首先，退出本节点上的各服务（DMCSS、DMASMSVR 和 DMSERVER）。先单独退出 DMSERVER，可通过执行 SQL 语句“stop instance;”完成。DMSERVER 单独退出完成后，再杀掉 DMCSS，最后杀掉 DMASMSVR。



注意：DMDSC 滚动升级之前，要求库中不存在 pre-commit 状态的 xa 事务，如果存在，需要手动提交或回滚，否则无法使用 stop instance 正常退出节点。

其次，重新安装新版本服务器，或者直接将新的执行程序和动态库替换旧版本。

最后，再启动本节点上的各服务。依次启动 DMCSS、DMASMSVR 和 DMSERVER。

第二步 退出节点 2、升级节点 2、再启动节点 2。此时活动节点 1 继续对外提供服务。

节点 2 的升级步骤和节点 1 完全相同。至此，两节点的 DMDSC 滚动升级完成。



注意：**DMDSC 滚动升级期间，不允许在节点上执行修改动态 INI 参数、修改数据库状态或模式等操作。**

17.12 正常 HALT 现象

以下 DMDSC 情景中，节点出现 HALT 属于正常现象，符合预期。

情景一 节点 A 不是 OPEN 状态（MOUNT/SUSPEND），如果此时出现其他节点故障，且故障处理无法进行，那么节点 A 会主动 HALT。

情景二 系统长时间处于 SUSPEND 状态（例如：数据守护为自动切换模式，且没有配置确认监视器），日志刷盘执行不了，致使数据页 REMOTE READ 无法进行，并最终导致节点 HALT。

17.13 ASM 磁盘版本

DMASM 非镜像环境下，当 DM 针对 ASM 磁盘做了较大改进时，会升级 ASM 磁盘版本号（磁盘版本号增大），可以通过查询 V$ASMDISK 视图获取 ASM 磁盘版本号（镜像环境下查询 V$ASMDISK 获取的 ASM 磁盘版本号始终为 0）。

DMASM 非镜像环境下，DM 自 V8.1.3.129 版本起，将 ASM 磁盘版本号升级至 4100。对 ASM 磁盘做的主要改进为针对磁盘头信息进行偏移调整。ASM 磁盘需要在磁盘头部位置写入标识信息，老版本 ASM 磁盘会在 0 号偏移位置写入标识信息，新版本磁盘则往后偏移固定位置写入标识信息：普通数据磁盘往后偏移 32M 开始写入信息，DCR 和 VOTE 磁盘往后偏移 1M 开始写入信息。

17.14 表空间负载均衡

DMDSC 支持基于表空间的负载均衡，用户可使用 ALTER TABLESPACE 语句进行表空间负载均衡设置。

语法如下:

```
ALTER TABLESPACE <表空间名> <DSC 集群表空间负载均衡子句>;

<DSC 集群表空间负载均衡子句> ::= OPTIMIZE <DSC 集群节点号>
```

设置表空间负载均衡后，符合条件的查询语句会被自动重连至<DSC 集群节点号>指定的节点执行，从而实现负载均衡。当指定的<DSC 集群节点号>为非法节点号时，此表空间的优化节点失效。

表空间负载均衡设置语句仅当 INI 参数 DSC_TABLESPACE_BALANCE=1 时有效。

例 设置 BOOKSHOP 表空间的优化节点为 0 号节点。

```
SQL> ALTER TABLESPACE BOOKSHOP OPTIMIZE 0;
```

执行上述语句后，DSC_TABLESPACE_BALANCE=1 时 DMDSC 会将对 BOOKSHOP 表空间对象的查询语句连接至 0 号节点执行。

当 DSC_TABLESPACE_BALANCE=1 时，如果用户使用 JDBC 驱动连接 DMDSC 节点并且在连接串中设置了属性 reconnect(1)，则 JDBC 连接串中指定的服务名节点顺序必须与 dmdcr_cfg.ini 中的节点顺序一致。例如，若 dmdcr_cfg.ini 中包含 DSC01 和 DSC02 两个 DSC 节点信息，DSC01 节点信息在前，DSC02 节点信息在后，则 JDBC 连接串 jdbc:dm://test?test=(192.168.100.107:6636,192.168.100.110:6637)&reconnect(1)中 192.168.100.107:6636 必须为 DSC01 的 IP:PORT，192.168.100.110:6637 必须为 DSC02 的 IP:PORT，不能颠倒顺序。

17.15 暂停 DMDSC 库

kill -19 对 DMDSC 集群是高危动作，可能引发数据损坏，不建议对 DMDSC 库使用。对 DMDSC 库，kill -20 提供相同功能且更安全，建议使用 kill -20 代替 kill -19 暂停 DMDSC 库。

使用 kill -18 唤醒被暂停的 DMDSC 库，当发现被唤醒的节点已经不在 DSC 集群中，将

会主动 halt，保证数据安全。

18版本升级

达梦数据库产品注重向下兼容性，虽然版本不断地更新升级，但在绝大多数情况下，使用高版本的执行码启动低版本的数据库时，会自动执行一系列更新升级动作，对于 DMDSC，用户只需要保证集群中所有节点之前是正常退出的即可。

在数据库升降级操作前，建议先进行数据备份。

需要注意的是，DM 进行的两项较大的功能改进，使得 DMDSC 在跨这两个版本进行升级时需要进行额外的升级操作，下面进行详细的说明。

18.1 日志格式升级

从 V8.1.1.15 版本开始，DM 对 REDO 日志格式进行了升级，与老的日志格式不兼容，因此要求数据库在执行升级操作前，必须是使用之前版本的执行码执行正常退出后的库，以此来保证所有数据都已经刷盘，否则使用新版本的执行码启动数据库时，无法根据老的 REDO 日志对故障重启的数据库执行重做 REDO 日志、归档文件修复等动作，无法正常完成数据库升级。

18.1.1 版本说明

支持 DM7、DM8 老版本的库升级到 V8.1.1.15 或更高的数据库版本。

DM8 老版本的库正常退出后，允许直接启动升级到 V8.1.1.15 或者更高版本的 DM8，但必须要严格遵守 18.1.2 节中的升级步骤。

DM7 老版本的库，针对不同版本有一些升级限制，具体说明如下：

- V7.1.6.11 之后的版本（包括 V7.1.6.11）

允许直接启动升级到 V8.1.1.15 或者更高版本的 DM8，同样也需要严格遵守升级步骤。

- 低于 V7.1.6.11 的版本

需要先升级到 V7.6.0.183 或者更高版本的 DM7，对日志版本号进行升级，然后才允许升级到 V8.1.1.15 或者更高版本的 DM8。需要注意的是，V7.1.5.125 以及更低版本的库，如果打开了日志加密，则不再支持升级到 V8.1.1.15 或更高的数据库版本。

低于 V7.1.6.11 的版本需要升级两次，这两次版本升级的步骤是完全相同的，“升级到 V7.6.0.183 或者更高版本的 DM7”的步骤可以参考下文中描述的升级步骤，只是将升级步骤中的版本号替换为“V7.6.0.183 或者更高版本的 DM7”。

18.1.2 升级步骤

只有当 DMDSC 中所有节点都是 OK 状态时才允许升级。存在故障节点的情况下，OK 状态节点启动升级时会报错。由于 DSC 集群的特殊启动流程，对故障节点无法正常报错退出，因此需要用户保证升级前所有节点都处于 OK 状态并正常退出（包括 DMSERVER、DMASMSVR 和 DMCSS）。

由于升级后老的归档日志文件不再可用，因此必须在执行升级前先将归档日志文件全部从归档目录中移走，避免升级后再次降级时，误判归档日志文件不连续。

需要注意的是，DM7 和 DM8 的 DCR DISK 和 VOTE DISK 格式不兼容，如果 DM7 的库升级到 DM8，在执行升级之前，需要先重新初始化 DCR DISK 和 VOTE DISK 磁盘（注意，对于需要执行两次升级的 DM7 版本，第一次升级还是 DM7 内的版本升级，不需要执行这个操作）。

使用 DMASMCMD 工具，执行如下命令完成初始化，执行时需要根据实际环境修改路径：

DM7 执行码（升级前的版本）执行：

```
export dcrdisk 'F:\dmdsc\asmdisks\dmdcr.asm' to ' F:\dmdsc\data\dmdcr_cfg.ini'
```

DM8 执行码（V8.1.1.15 或者更高版本）执行：

```
create dcrdisk 'F:\dmdsc\asmdisks\dmdcr.asm' 'dcr'

create votedisk 'F:\dmdsc\asmdisks\dmvtdsk.asm' 'vtd'

init dcrdisk 'F:\dmdsc\asmdisks\dmdcr.asm' from 'F:\dmdsc\data\dmdcr_cfg.ini' identified

by 'DCRpsd_123'

init votedisk 'F:\dmdsc\asmdisks\dmvtdsk.asm' from 'F:\dmdsc\data\dmdcr_cfg.ini'
```

然后使用 V8.1.1.15 或者更高版本执行码依次启动 DMCSS、DMASMSVR 和 DMSERVER 即可，在启动过程中会自动完成系统表和动态视图等字典信息的升级。

DSC 集群可以正常启动到 OPEN 状态，就表示升级成功。升级成功的情况下，一定是可以查到 SYSOPENHISTORY 系统表的，可通过查询此系统表确认升级结果。

18.2 回滚管理段升级

DM 从 V8.1.1.101 版本开始支持回滚管理段，用于存放事务信息，在系统故障重启或者 DSC 故障处理时，通过扫描回滚管理段收集事务信息，以达到缩减事务收集时长的目的。

此版本增加了一个建库参数（PSEG_MGR_FLAG）表示是否仅使用回滚管理段记录事务信息，取值 0、1，缺省为 0。取值含义说明如下：

- 0：除了使用回滚管理段记录事务信息外，在事务的首个回滚页上也记录事务信息。
- 1：仅使用回滚管理段记录事务信息，事务的首个回滚页上不再记录。

老版本的库升级时，一律采用 PSEG_MGR_FLAG=0 的方式进行升级，老版本库在升级时会自动创建回滚管理段，升级成功后，不允许再使用老执行码启动升级后的库。

18.2.1 版本说明

具体的升级版本限制说明如下：

- 升级前的老版本库是 DM8，版本号等于或者高于 V8.1.1.15

  老版本库已支持升级后的日志格式，本次升级仅需要升级回滚管理段。

- 升级前的老版本库是 DM8，版本号低于 V8.1.1.15；或者升级前的老版本库是 DM7 老版本库不支持升级后的日志格式，本次升级需要先升级日志格式，再升级回滚管理段。

对这个版本范围的老版本库，升级前首先需要满足“18.1 日志格式升级”的各项升级条件，然后还要满足回滚管理段的升级条件才能升级。

18.2.2 升级步骤

回滚管理段的升级必须满足以下两个条件：

（1）升级前不存在故障节点，并且所有节点均正常退出。

（2）老版本库如果是新初始化库，则必须用老执行码正常启动过并正常退出，才允许升级。

如果老版本库已支持升级后的日志格式（版本号等于或者高于 V8.1.1.15），在满足这上述条件的前提下，直接使用 V8.1.1.101 或者更高版本的执行码按照 DMCSS、DMASMSVR、DMSERVER 的顺序启动老版本库到 OPEN 状态即可完成升级。

如果老库还未支持升级后的日志格式，则需要同时满足日志格式和回滚管理段的升级条件。具体执行升级时，要根据老版本库当前的版本号，按照“18.1 日志格式升级”中的步骤进行升级，只需要将其中的 V8.1.1.15 替换为 V8.1.1.101 即可，V8.1.1.101 可以同时完成日志格式和回滚管理段的升级。

18.3 HUGE 表空间升级

18.3.1 版本说明

从 V8.1.2.101 版本开始，DM 对 HUGE 表空间进行了升级改造。本节中低版本是指 V8.1.2.101 之前的库，高版本是指 V8.1.2.101 或更高的版本。

不使用 HUGE 表的用户可以不用升级 HUGE 表空间。需要特别注意的是，升级后将不

支持继续使用低版本库下所创建的 HUGE 表。若用户库中已有正在使用的 HUGE 表，建议谨慎升级。自定义混合表空间由用户可根据自己需要创建。本章升级对象为 MAIN 系统混合表空间。

高版本与低版本之间的差异如下:

- 用户自定义混合表空间

在低版本中，表空间分为普通表空间和 HUGE 表空间。

从高版本中，对 HUGE 表空间进行了升级改造，废除了 HUGE 表空间的概念，取而代之的是混合表空间的概念。在高版本中，表空间分为普通表空间和混合表空间。

使用<HUGE 路径子句>创建的表空间为混合表空间，未使用<HUGE 路径子句>创建的表空间即为普通表空间。普通表空间只能存储普通表（非 HUGE 表）；而混合表空间既可以存储普通表又可以存储 HUGE 表。HUGE 数据文件存储在<HUGE 路径子句>指定的路径中，普通（非 HUGE）数据文件存储在<数据文件子句>指定的路径中。HUGE 表对应的辅助表为普通表，因此，HUGE 辅助表数据存在<数据文件子句>指定的路径中。

若创建 HUGE 表使用自定义的混合表空间，则需要用户先创建一个混合表空间。自定义混合表空间创建方法参考《DM8_SQL 语言使用手册》CREATE TABLESPACE 语法。

例 创建 HUGE 表 T1，使用自定义的混合表空间 TS1。

```
CREATE TABLESPACE TS1  DATAFILE '+DMDATA/data/dameng/TS1.dbf' SIZE 128 WITH HUGE PATH
'+DMDATA/data/dameng/TS1/HUGE1';
CREATE HUGE TABLE T1(C1  INT, C2 INT)STORAGE(ON TS1);
```

- 系统 MAIN 表空间

在低版本中，普通表用户默认表空间 MIAN 为一个普通表空间。HUGE 表默认表空间 HMAIN 为一个 HUGE 表空间。

在高版本中，取消了 HMAIN 表空间。普通表和 HUGE 表用户默认表空间统一为 MAIN 表空间，MAIN 为一个混合表空间。

例　创建 HUGE 表 T2，缺省使用系统默认的 MAIN 混合表空间。

```
CREATE HUGE TABLE T2(C1　INT, C2 INT)；
```

18.3.2 升级步骤

有两种升级 MAIN 表空间的方式：联机升级和脱机升级。

- 联机方式升级

联机方式通过执行相关 SQL 语句完成。

若创建 HUGE 表使用默认的 MAIN 表空间，则需要升级 MAIN 表空间为混合表空间。参考《DM8_SQL 语言使用手册》使用 ALTER TABLESPACE ……<HUGE 路径子句>为 MAIN 表空间添加 HUGE 路径，即可升级为混合表空间。

例 为 MAIN 表空间增加 HUGE PATH。不可和其他表空间公用一个绝对路径。

```
ALTER TABLESPACE MAIN ADD HUGE PATH '+DMDATA/data/dameng/dsc3/TS1/HUGE1';
```

- 脱机方式升级

脱机方式通过直接修改控制文件的方式完成。

使用高版本的各软件均能够正常读取老库中的控制文件。因此，单节点库升级也可通过直接修改控制文件中的内容来实现。

升级步骤：

（1）正常退出节点，备份控制文件。

（2）使用高版本的 ctlcvt 工具，将控制文件转化为文本文件。

```
./dmctlcvt TYPE=1 SRC=/data/dmserver/dm.ctl DEST=/data/dmserver/dmctl.txt
```

（3）检查文本文件，升级 MAIN 普通表空间为混合表空间。在 MAIN 表空间的描述信息区域，在 MAIN 表空间数据文件描述的尾部，结束标志符号“#===”之前，追加 HUGE 路径（#huge path）的相关描述，并修改时间信息。HUGE 路径为 DM 系统目录下的 HMAIN 目录。此外，低版本中 HUGE 表空间描述“# HUGE table space”内容可保留也可删除，对

高版本无影响。

例 升级 MAIN 表空间。假定 HUGE 路径为+DMDATA/data/dameng/HMAIN。增加的 HUGE 路径为黑体字部分。

```
……
#============================================
# table space name
ts_name=MAIN
 ……
#--------------------------------------------
# file path
fil_path=+DMDATA/data/dameng/MAIN.DBF
……
#--------------------------------------------
# file path
#MAIN 表空间的所有数据文件，均位于 huge path 的描述之前
……
#--------------------------------------------
# huge path
huge_path=+DMDATA/data/dameng/HMAIN
# huge path create time
huge_create_time=DATETIME '2022-1-13 17:29:14'
# huge path modify time
huge_modify_time=DATETIME '2022-1-13 17:29:14'
#============================================
```

```
……
```

（4）通过高版本的 ctlcvt 工具，将文本文件重新转换回控制文件。

```
./dmctlcvt TYPE=2 SRC=/data/dmserver/dmctl.txt DEST=/data/dmserver/dm.ctl
```

（5）重启服务器。可通过 V$HUGE_TABLESPACE 检查是否包含 MAIN 表空间。成功查询到 MAIN 表空间的 HUGE PATH，即 HMAIN 目录，则升级完成。

```
SQL> SELECT NAME,PATHNAME FROM V$HUGE_TABLESPACE WHERE NAME='MAIN';

行号          NAME PATHNAME

---------- ---- --------------------------

1             MAIN +DMDATA/data/dameng/HMAIN
```

18.4 数据字典 SYSOBJECTS 升级

18.4.1 版本说明

从 V8.1.4.23 版本开始，对数据字典 SYSOBJECTS 中的 INFO5 字段的编码方式进行了优化。

用户使用高版本（V8.1.4.23 及以上版本）的执行码启动低版本的数据库时，系统自动对 SYSOBJECTS 的 INFO5 字段进行升级。

18.4.2 升级步骤

可以使用滚动升级的方式升级 DMDSC 集群，详细步骤请参考 17.11 滚动升级。

在滚动升级过程中，如果所有节点尚未全部升级完成，则不允许在已升级的节点上执行 DDL 语句。如果升级到的版本是 V8.1.4.23 至 V8.1.4.109 之间的版本（包含 V8.1.4.23 和 V8.1.4.109），则需要用户保证不在其上执行 DDL 语句；如果升级到的版本是 V8.1.4.109 以上的版本，则用户在其上执行 DDL 语句时将报错。待滚动升级完毕方可执行 DDL 语句。

19附录

19.1 DMASMAPI 接口

DMASMAPI 接口为未配置镜像的 DMASM 专用。

DMASMAPI 接口是连接 DMASMSVR 执行 DMASM 文件操作的接口。使用 DMASM 文件系统的应用程序（例如 DMSERVER）可通过 DMASMAPI 接口连接 DMASMSVR，进而进行各种 DMASM 文件操作。

19.1.1 DMASMAPI 接口介绍

DMASMAPI 提供的接口如下：

1. dmasm_sys_init

函数原型：

```
ASMRETURN
dmasm_sys_init(
    sdbyte*      err_desc,
    udint4*      err_len,
    udint4       char_code,
    udint4       lang_id
)
```

功能说明：

环境初始化接口。使用 ASMAPI 接口，必须第一个调用 dmasm_sys_init 接口。

参数说明：

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

char_code：输入参数，编码。0：PG_INVALID_CODE；1：PG_UTF8；2：PG_GBK；3：PG_BIG5；4：PG_ISO_8859_9；5：PG_EUC_JP；6：PG_EUC_KR；7：PG_KOI8R；8:PG_ISO_8859_1；9：PG_SQL_ASCII；10：PG_GB18030；11：PG_ISO_8859_11；12：PG_UTF16。-1 表示使用默认编码。

lang_id ：输入参数，语言。0：中文；1：英文；2：繁体中文。-1 表示使用默认语言。

2. dmasm_sys_deinit

函数原型：

```
void
dmasm_sys_deinit()
```

功能说明：

环境销毁接口。使用 ASMAPI 接口结束时，调用 dmasm_sys_deinit 销毁资源。

3. dmasm_alloc_con

函数原型：

```
ASMRETURN
dmasm_alloc_con(
    asmcon_handle*   con,
    sdbyte*          err_desc,
    udint4*          err_len
)
```

功能说明：

申请并初始化连接句柄。返回申请的连接句柄。

参数说明:

con：输入参数，连接句柄。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

4. dmasm_close_con

函数原型:

```
void
dmasm_close_con(
    asmcon_handle    con_in
)
```

功能说明:

关闭连接。关闭连接句柄，释放资源。

参数说明:

conn_in：输入参数，连接句柄。

5. dmasm_free_con

函数原型:

```
void
dmasm_free_con(
    asmcon_handle   con_in
)
```

功能说明:

释放连接句柄。

参数说明:

con_in：输入参数，连接句柄。

6. dmasm_connect

函数原型:

```
ASMRETURN
    dmasm_connect(
    asmcon_handle    con_in,
    sdbyte*          username,
    sdbyte*          password,
    sdbyte*          hostname,
    udint2           portnum,
    asmbool*         con_is_local,
    sdbyte*          err_desc,
    udint4*          err_len
)
```

功能说明:

登录接口。ASMSVR 允许本地连接和远程连接，但是 DMSERVER 仅允许本地连接，依据 con_is_local 判断是否远程连接。

参数说明:

con_in：输入参数，连接句柄。

username：输入参数，用户名。

password：输入参数，密码。

hostname：输入参数，主机 ip 或主机名。

portnum：输入参数，主机监听端口号。

con_is_local：输出参数，标识远程或本地登录。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

7. dmasm_get_n_group

函数原型：

```
ASMRETURN
dmasm_get_n_group(
    asmcon_handle    conn_in,
    udint2*          num,
    sdbyte*          err_desc,
    udint4*          err_len
)
```

功能说明：

获取 ASM Disk Group 个数。获取有多少个磁盘组。

参数说明：

conn_in：输入参数，连接句柄。

num：输出参数，磁盘组个数。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

8. dmasm_get_group_id_arr

函数原型:

```
ASMRETURN
dmasm_get_group_id_arr(
    asmcon_handle    con_in,
    udint2*          id_arr,
    udint4*          au_size_arr,
    udint4*          ex_size_arr,
    udint2           arr_size,
    udint2*          num,
    sdbyte*          err_desc,
    udint4*          err_len
)
```

功能说明:

获取 ASM Disk Group ID 数组。配合 dmasm_get_n_group 使用，获取所有 Disk Group ID。

参数说明:

conn_in：输入参数，连接句柄。

id_arr：输出参数，磁盘 ID 数组。

au_size_arr：输出参数，无实际意义。

ex_size_arr：输出参数，无实际意义。

arr_size：输入参数，数组最大长度。

num：输出参数，返回数组长度。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

9. dmasm_get_disk_id_arr_by_group

函数原型：

```
ASMRETURN
dmasm_get_disk_id_arr_by_group(
    asmcon_handle    con_in,
    udint2           group_id,
    udint2*          id_arr,
    udint2           arr_size,
    udint2*          n_disk,
    sdbyte*          err_desc,
    udint4*          err_len
)
```

功能说明：

获取磁盘组内磁盘 ID 数组。根据磁盘组 ID 获取磁盘组内包含的所有磁盘 ID。

参数说明：

conn_in：输入参数，连接句柄。

group_id：输入参数，磁盘组 ID。

id_arr：输出参数，磁盘 ID 数组。

arr_size：输入参数，数组最大长度。

n_disk：输出参数，返回数组长度。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

10. dmasm_get_disk_info

函数原型：

```
ASMRETURN
dmasm_get_disk_info(
    asmcon_handle       conn_in,
    udint2              group_id,
    udint4              disk_id,
    sdbyte*             path,
    udint2              path_buflen,
    sdbyte*             name,
    udint2              name_buflen,
    sdbyte*             frp_name,
    udint2              frp_buflen,
    sdbyte*             status,
    udint2              status_buflen,
    udint4*             size,
    udint4*             free_auno,
    sdbyte*             create_time,
    sdbyte*             modify_time,
    sdbyte*             uuid_str,
    sdbyte*             err_desc,
    udint4*             err_len
```

)

功能说明：

获取 ASM 磁盘详细信息。根据磁盘组 ID 和磁盘 ID 获取 ASM 磁盘详细信息。

参数说明：

conn_in：输入参数，连接句柄。

group_id：输入参数，磁盘组 ID。

disk_id：输入参数，磁盘 ID。

path：输出参数，磁盘路径。

path_buflen：输入参数，path 缓冲区长度。

name：输出参数，磁盘名称。

name_buflen：输入参数，name 缓冲区长度。

frp_name：输出参数，无实际意义。

frp_buflen：输入参数，无实际作用。

status：输出参数，无实际意义。

status_buflen：输入参数，无实际作用。

size：输出参数，磁盘大小，单位 MB。

free_auno：输出参数，最大 au 号。

create_time：输出参数，磁盘创建时间。

modify_time：输出参数，磁盘最近一次修改时间。

uuid_str：输出参数，磁盘唯一标识。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

11. dmasm_create_diskgroup

函数原型：

```
ASMRETURN

dmasm_create_diskgroup(

    asmcon_handle    conn_in,

    sdbyte*          group_name,

    sdbyte*          disk_path_in,

    sdbyte*          err_desc,

    udint4*          err_len

)
```

功能说明：

创建 ASM Disk Group。使用 disk_path 所指的 ASM 磁盘创建 ASM 磁盘组。

参数说明：

conn_in：输入参数，连接句柄。

group_name：输入参数，磁盘组名。

disk_path_in：输入参数，磁盘路径。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

12. dmasm_add_disk_to_diskgroup

函数原型：

```
ASMRETURN

dmasm_add_disk_to_diskgroup(
```

```
    asmcon_handle    conn_in,
    sdbyte*          group_name,
    sdbyte*          disk_path_in,
    sdbyte*          err_desc,
    udint4*          err_len
)
```

功能说明：

磁盘组增加磁盘。往磁盘组增加 ASM 磁盘。

参数说明：

conn_in：输入参数，连接句柄。

group_name：输入参数，磁盘组名。

disk_path_in：输入参数，磁盘路径。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

13. dmasm_drop_diskgroup_by_name

函数原型：

```
ASMRETURN
dmasm_drop_diskgroup_by_name(
    asmcon_handle    conn_in,
    sdbyte*          group_name,
    sdbyte*          err_desc,
    udint4*          err_len
)
```

功能说明：

删除磁盘组。ASM 文件系统不支持单独删除 ASM 磁盘，只能删除整个 ASM 磁盘组。任意文件打开的情况下，其所属的磁盘组不能被删除。

参数说明：

conn_in：输入参数，连接句柄。

group_name：输入参数，磁盘组名。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

14. dmasm_file_create

函数原型：

```
ASMRETURN
dmasm_file_create(
    asmcon_handle       conn_in,
    asmbool p_flag,
    sdbyte*             filepath_in,
    udint8    filesize,
    asm_fhandle_t*  fil_handle,
    asmbool             need_init,
    udint2              striping_size,
    udbyte              redundancy_type,
    sdbyte*             err_desc,
    udint4*             err_len
)
```

功能说明：

创建 ASM 文件。在 ASM 文件系统中创建 ASM 文件，如果文件父亲目录不存在，并且 p_flag 为 TRUE 的情况，会自动创建父目录，否则会报错。

参数说明：

conn_in：输入参数，连接句柄。

p_flag：输入参数，创建父目录标记。

filepath_in：输入参数，文件路径。

filesize：输入参数，文件大小，单位 Byte。

fil_handle：输出参数，文件句柄。

need_init：输入参数，是否需要初始化。

striping_size：输入参数，无实际作用。

redundancy_type：输入参数，无实际作用。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

15. dmasm_file_open

函数原型：

```
ASMRETURN
dmasm_file_open(
    asmcon_handle       conn_in,
    sdbyte*             filepath_in,
    asm_fhandle_t*      fhandle,
    sdbyte*             err_desc,
    udint4*             err_len
```

```
)
```

功能说明：

打开 ASM 文件，获取 ASM 文件句柄。已经打开的文件，不能被删除。

参数说明：

conn_in：输入参数，连接句柄。

filepath_in：输入参数，文件路径。

fhandle：输入参数，文件句柄。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

16. dmasm_file_trunc

函数原型：

```
ASMRETURN
dmasm_file_trunc(
    asmcon_handle    conn_in,
    asm_fhandle_t    fhandle,
    udint8           truncate_size,
    udint8*          real_size,
    sdbyte*          err_desc,
    udint4*          err_len
)
```

功能说明：

截断 ASM 文件。将 ASM 文件截断到 truncate_size，如果 truncate_size 小于文件大小，文件会被截断到 truncate_size；如果 truncate_size 大于文件大小，文件大小不

变，接口返回 EC_SUCCESS。

参数说明:

conn_in：输入参数，连接句柄。

fhandle：输入参数，文件句柄。

truncate_size：输入参数，截断后的大小，单位 Byte。

real_size：输出参数，执行后实际大小，单位 Byte。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

17. dmasm_file_extend

函数原型:

```
ASMRETURN
dmasm_file_extend(
    asmcon_handle    conn_in,
    asm_fhandle_t    fhandle,
    udint8           offset,
    udint8           extend_size,
    udint4           need_init,
    sdbyte*          err_desc,
    udint4*          err_len
)
```

功能说明:

扩展 ASM 文件。将文件从 offset 偏移处，扩展 extent_size 大小，最终实际大小为 offset+extent_size。如果 offset+extend_size 大于文件大小，文件会被扩展到

offset+extend_size 大小；如果 offset+extend_size 小于文件大小，直接返回成功。

参数说明：

conn_in：输入参数，连接句柄。

fhandle：输入参数，文件句柄。

offset：输入参数，起始偏移。

extend_size：输入参数，扩展大小。

need_init：输入参数，是否需要初始化。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

18. dmasm_file_close

函数原型：

```
void
dmasm_file_close(
    asmcon_handle    conn_in,
    asm_fhandle_t    fhandle
)
```

功能说明：

关闭 ASM 文件。关闭打开的 ASM 文件。

参数说明：

conn_in：输入参数，连接句柄。

fhandle：输入参数，文件句柄。

19. dmasm_file_delete

函数原型：

```
ASMRETURN
dmasm_file_delete(
    asmcon_handle    conn_in,
    sdbyte*          filepath_in,
    sdbyte*          err_desc,
    udint4*          err_len
)
```

功能说明：

删除 ASM 文件。删除 ASM 文件，正在被使用的 ASM 文件不能被删除。

参数说明：

conn_in：输入参数，连接句柄。

filepath_in：输入参数，文件路径。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

20. dmasm_file_read_by_offset

函数原型：

```
ASMRETURN
dmasm_file_read_by_offset(
    asmcon_handle   conn_in,
    asm_fhandle_t     fhandle,
```

```
    udint8              offset,
    sdbyte*             buffer,
    udint4              bytes_to_read,
    sdbyte*             err_desc,
    udint4*             err_len
)
```

功能说明：

从 ASM 文件读取数据。从 ASM 文件 offset 偏移读取 bytes_to_read 大小的内容到缓冲区 buffer，调用者保证缓冲区够用。因为块设备读写限制，offset、buffer、bytes_to_read 都必须能被 512 整除，否则会报错。

参数说明：

conn_in：输入参数，连接句柄。

fhandle：输入参数，文件句柄。

offset：输入参数，起始偏移。

buffer：输入参数，缓冲区。

bytes_to_read：输入参数，读取数据大小，单位 Byte。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

21. dmasm_file_read_by_offset_normal

函数原型：

```
ASMRETURN
dmasm_file_read_by_offset_normal(
    asmcon_handle   conn_in,
```

```
    asm_fhandle_t      fhandle,
    udint8             offset,
    sdbyte*            buffer,
    udint4             bytes_to_read,
    sdbyte*            err_desc,
    udint4*            err_len
)
```

功能说明:

从 ASM 文件读取数据。从 ASM 文件 offset 偏移读取 bytes_to_read 大小的内容到缓冲区 buffer，调用者保证缓冲区够用。该接口支持 offset,buffer,bytes_to_read 不是 512 整数倍，但是性能比 dmasm_file_read_by_offset 慢。

参数说明:

conn_in：输入参数，连接句柄。

fhandle：输入参数，文件句柄。

offset：输入参数，起始偏移。

buffer：输入参数，缓冲区。

bytes_to_read：输入参数，写取数据大小，单位 Byte。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

22. dmasm_file_write_by_offset

函数原型:

```
ASMRETURN
dmasm_file_write_by_offset(
```

```
    asmcon_handle    conn_in,
    asm_fhandle_t      fhandle,
    udint8             offset,
    sdbyte*            buffer,
    udint4             bytes_to_write,
    sdbyte*            err_desc,
    udint4*            err_len
)
```

功能说明:

将数据写入 ASM 文件。将缓冲区中的内容写入 ASM 文件，从 offset 偏移开始。因为块设备读写限制，offset，buffer 地址，bytes_to_write 都必须能被 512 整除。

参数说明:

conn_in：输入参数，连接句柄。

fhandle：输入参数，文件句柄。

offset：输入参数，起始偏移。

buffer：输入参数，缓冲区。

bytes_to_write：输入参数，写数据大小，单位 Byte。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

23. dmasm_file_write_by_offset_normal

函数原型:

```
ASMRETURN
dmasm_file_write_by_offset_normal(
```

```
    asmcon_handle    conn_in,
    asm_fhandle_t      fhandle,
    udint8     offset,
    sdbyte*                buffer,
    udint4                 bytes_to_write,
    sdbyte*                err_desc,
    udint4*                err_len
)
```

功能说明:

将数据写入 ASM 文件。将缓冲区中的内容写入 ASM 文件，从 offset 偏移开始。该接口支持 offset,buffer,bytes_to_write 不是 512 倍数，但是性能比 dmasm_file_write_by_offset 慢。

参数说明:

conn_in：输入参数，连接句柄。

fhandle：输入参数，文件句柄。

offset：输入参数，起始偏移。

buffer：输入参数，缓冲区。

bytes_to_write：输入参数，写数据大小，单位 Byte。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

24. dmasm_file_copy

函数原型:

```
ASMRETURN
```

```
dmasm_file_copy(
    asmcon_handle    conn_in,
    sdbyte*          source_in,
    sdbyte*          dest_in,
    asmbool          bOverwriteIfExists,
    sdbyte*          err_desc,
    udint4*          err_len
)
```

功能说明：

文件拷贝操作。支持 ASM 文件拷贝到 ASM 文件; ASM 文件拷贝到普通文件系统文件; 普通文件系统文件拷贝到 ASM 文件系统；不支持普通文件拷贝到普通文件。bOverwriteIfExists：0 或者 NULL 表示不覆盖，其他非 0 值表示覆盖。

参数说明：

conn_in：输入参数，连接句柄。

source_in：输入参数，源文件路径。必须是全路径，不能是相对路径。

dest_in：输入参数，目标文件路径。必须是全路径，不能是相对路径。

bOverwriteIfExists：输入参数，如果目标存在是否删除。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

25. dmasm_dir_create

函数原型：

```
ASMRETURN
dmasm_dir_create(
```

```
    asmcon_handle    conn_in,
    asmbool          p_flag,
    sdbyte*          fdir_in,
    sdbyte*          err_desc,
    udint4*          err_len
)
```

功能说明:

创建目录。ASM 文件系统创建目录，当 p_flag=TRUE 时会级联创建父目录，否则父目录不存在会报错。

参数说明:

conn_in：输入参数，连接句柄。

p_flag：输入参数，是否级联创建父目录。

fdir_in：输入参数，目录路径。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

26. dmasm_dir_delete

函数原型:

```
ASMRETURN
dmasm_dir_delete(
    asmcon_handle    conn_in,
    sdbyte*          fdir_in,
    sdbyte*          err_desc,
    udint4*          err_len
```

```
)
```

功能说明:

删除目录，以及目录下面所有的文件。

参数说明:

conn_in：输入参数，连接句柄。

fdir_in：输入参数，目录路径。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

27. dmasm_get_file_num_by_group

函数原型:

```
ASMRETURN
dmasm_get_file_num_by_group(
    asmcon_handle       conn_in,
    udint2              group_id,
    udint4*             num,
    sdbyte*             err_desc,
    udint4*             err_len
)
```

功能说明:

获取磁盘组内总的文件个数。根据磁盘组 ID 获取总的文件个数，包括文件和目录。

参数说明:

conn_in：输入参数，连接句柄。

group_id：输入参数，磁盘组的 ID 。

num　　：输出参数，文件的个数。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

28. dmasm_get_file_info

函数原型：

```
ASMRETURN
dmasm_get_file_info(
    asmcon_handle    conn_in,
    asm_fhandle_t    file_id,
    ASM_FILE_ATTR*   fattr_out,
    sdbyte*          err_desc,
    udint4*          err_len
)
```

功能说明：

获取 ASM 文件详细信息。

ASM_FILE_ATTR 包括：

type：目录标记，整型。1 表示文件，2 表示目录。

name：文件名称，字符型。

full_path：完整路径，字符型。

size：文件大小，整型，单位 Byte。目录忽略此字段。

c_time：创建时间，time_t 类型。

m_time：修改时间，time_t 类型。

group_id：所在磁盘组编号，整型。

disk_id：inode 项所在磁盘 ID，整型。

disk_auno：inode 项所在磁盘 AU 编号，整型。

offset：inode 项 AU 偏移，整型。

参数说明：

conn_in：输入参数，连接句柄。

file_id：输入参数，打开的文件句柄。

fattr_out：输出参数，文件属性结构。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

29. dmasm_get_group_info_by_name

函数原型：

```
ASMRETURN
dmasm_get_group_info_by_name(
    asmcon_handle    con_in,
    sdbyte*          group_name,
    udint2*          group_id,
    udint2*          status,
    udint2*          n_disk,
    udint4*          total_size,
    udint4*          free_size,
    sdbyte*          err_desc,
    udint4*          err_len
)
```

功能说明：

通过磁盘组名获取磁盘组详细信息。

参数说明：

con_in：输入参数，连接句柄。

group_name：输入参数，磁盘组名字。

group_id：输出参数，磁盘组 ID。

status：输出参数，磁盘组状态(1:正在创建中；2：正常的；3：正在删除中)。

n_disk：输出参数，磁盘数。

total_size：输出参数，磁盘组大小，单位 MB。

free_size：输出参数，磁盘组空闲大小，单位 MB。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

30. dmasm_get_group_info_by_id

函数原型：

```
ASMRETURN
dmasm_get_group_info_by_id(
    asmcon_handle    con_in,
    udint2           group_id,
    sdbyte*          group_name,
    udint2           name_buflen,
    udint2*          status,
    udint2*          n_disk,
    udint4*          total_size,
```

```
    udint4*          free_size,
    udint4*          au_size,
    byte*            n_copy,
    udint4*          redo_size,
    udint2*          rbl_stat,
    udint2*          rbl_pwr,
    sdbyte*          err_desc,
    udint4*          err_len
)
```

功能说明：

通过磁盘组 ID 获取磁盘组详细信息。

参数说明：

con_in：输入参数，连接句柄。

group_id：输入参数，磁盘组 ID。

group_name：输出参数，磁盘组名字。

name_buflen：输入参数，磁盘组的 buf 长度

status：输出参数，磁盘组状态(1:正在创建中；2：正常的；3：正在删除中)。

n_disk：输出参数，磁盘数。

total_size：输出参数，磁盘组大小，单位 MB。

free_size：输出参数，磁盘组空闲大小，单位 MB。

au_size：输出参数，无实际意义。

n_copy：输出参数，无实际意义。

redo_size：输出参数，无实际意义。

rbl_stat：输出参数，无实际意义。

rbl_pwr：输出参数，无实际意义。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

31. dmasm_dir_get_first

函数原型：

```
ASMRETURN
dmasm_dir_get_first(
    asmcon_handle       conn_in,
    sdbyte*             path_in,
    sdbyte*             suffix,
    asm_dhandle_t*      dir_handle_out,
    ASM_FILE_ATTR*      fattr_out,
    asmbool*            exist_flag,
    sdbyte*             err_desc,
    udint4*             err_len
)
```

功能说明：

获取目录下第一个文件信息。

参数说明：

conn_in：输入参数，连接句柄。

path_in：输入参数，目录路径。

suffix：输入参数，指定扩展名关键字，如：".log"。用于取出以指定关键字作为扩展名的文件。

dir_handle_out：输出参数，打开的目录句柄。

fattr_out：输出参数，文件属性结构。

exist_flag：输出参数，是否存在。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

32. dmasm_dir_get_next

函数原型：

```
ASMRETURN
dmasm_dir_get_next(
    asmcon_handle       conn_in,
    asm_dhandle_t       dir_handle,
    sdbyte*             path_in,
    sdbyte*             suffix,
    ASM_FILE_ATTR*      fattr_out,
    asmbool*            exist_flag,
    sdbyte*             err_desc,
    udint4*             err_len
)
```

功能说明：

获取目录下下一个文件信息，须配合函数 dmasm_dir_get_first 使用。

参数说明：

conn_in：输入参数，连接句柄。

dir_handle：输入参数，打开的目录句柄。

path_in：输入参数，目录路径，须与目录句柄 dir_handle 中打开的 path_in 保持一致。

suffix：输入参数，指定扩展名关键字，如：".log"。用于取出以指定关键字作为扩展名的文件。

fattr_out：输出参数，文件属性结构。

exist_flag：输出参数，是否存在。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

33. dmasm_dir_close

函数原型：

```
ASMRETURN
dmasm_dir_close(
    asmcon_handle       conn_in,
    asm_dhandle_t       dir_handle,
    sdbyte*             err_desc,
    udint4*             err_len
)
```

功能说明：

关闭目录。

参数说明：

con_in：输入参数，连接句柄。

dir_handle：输入参数，打开的目录句柄。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

34. dmasm_file_attributes_get

函数原型：

```
ASMRETURN
dmasm_file_attributes_get(
    asmcon_handle      conn_in,
    sdbyte*            path,
    ASM_FILE_ATTR*     fattr_out,
    sdbyte*            err_desc,
    udint4*            err_len
)
```

功能说明：

根据文件路径获取文件详细信息。

参数说明：

conn_in：输入参数，连接句柄。

path：输入参数，路径。

fattr_out：输出参数，文件属性结构。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

35. dmasm_file_is_exist

函数原型：

```
ASMRETURN
```

```
dmasm_file_is_exist(
    asmcon_handle    conn_in,
    sdbyte*          path,
    sdbyte*          err_desc,
    udint4*          err_len
)
```

功能说明：

判断在 ASM 文件系统中，以 path 为文件路径的文件是否存在。

参数说明：

conn_in：输入参数，连接句柄。

path：输入参数，文件路径。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

19.1.2 返回值说明

返回值分为三种类型：正数，0 和负数。0 表示正常；正数为警告信息；负数为错误信息，对应的错误码请参考 19.1.3 错误码汇编。

19.1.3 错误码汇编

DMASMAPI 的错误码值域从-11000 开始。具体可参考 V$ERR_INFO。

19.1.4 编程示例

下面举一个简单的例子，展示 DMASMAPI 的使用方法。若本示例在编译过程中出现实参过多或者变量未声明等问题，可查询相应版本的 DMASMAPI 接口使用手册，适当增减参

数或者将变量改为实际值。

```
#include <stdlib.h>

#include <string.h>

#include <stdio.h>

#include "asmapi2.h"



#define TRUE            1

#define FALSE           0



#define IS_SUCCESS(code)      ((sdint4)code >= 0)

#define RET_IF_ERR_EX(c, act) \

    { if (!IS_SUCCESS(c)) { {act;} return (c); }}

/*运行前修改file_name username passwd ip port

  镜像不支持远程写，示例程序需要和服务端在同一台机器

  LINUX下编译

  gcc -DLINUX -I/home/test/include test.c -L. -ldmasmapi -o demo

  -I:指定头文件目录，本例中需要指定asmapi2.h头文件所在目录

  -L(大写L):指定库搜索路径

  -l(小写L):指定库名

  -D:定义预处理宏*/



int

main(

    int                 argc,
```

```
    char*               argv[]
)
{
    int                 code = 0;
    char                err_desc[1024];
    int                 err_len;
    asm_fhandle_t    fil_handle;
    void*               conn;
    char*               wbuf            = "hello asm";
    char                rbuf[10];
    int                 len;
    int                 file_size = 20;          //单位字节
    int                 offset = 0;
    int                 strip_size   = 0;           //条带大小
    int                 redundancy_type = 1;     //镜像类型
    char*               username = "ASMSYS";
    char*               passwd = "DCRpsd_123";
    char*               file_name = "+DATA/test1.dta";
    char*               ip = "192.168.100.118";
    int                 port = 10060;
#ifdef WIN32
    /*windows环境需要添加附加依赖项ws2_32.lib*/
    WSADATA WsaData;
    if (SOCKET_ERROR == WSAStartup(0x0101, &WsaData))
```

```
    {
        fprintf(stderr, "WSAStartup Failed\n");
        exit(-1);
    }
#endif

    err_len         = sizeof(err_desc);
    len             = (int)strlen(wbuf);

    code = dmasm_sys_init(err_desc, &err_len, -1, 1);
    if (!IS_SUCCESS(code))
    {
        fprintf(stderr, "dmasm_sys_init error:[%s]\n", err_desc);
        return -1;
    }
/* 分配连接句柄 */
    code = dmasm_alloc_con(&conn, err_desc, &err_len);
    if (!IS_SUCCESS(code))
    {
        fprintf(stderr, "dmasm_alloc_con error:[%s]\n", err_desc);
        return -1;
    }

    /* 登录asmsvr，本地登录不校验密码，可以输入任意字符串，远程登录会校验用户名密码*/
```

```
    code = dmasm_connect(conn, username, passwd, ip, port, NULL, err_desc, &err_len);
    if (!IS_SUCCESS(code))
    {
        fprintf(stderr, "dmasm_connect error:[%s]\n", err_desc);
        return -1;
    }
/* 文件存在直接打开 */
    if (dmasm_file_is_exist(conn, file_name, err_desc, &err_len) == TRUE)
    {
        code = dmasm_file_open(conn, file_name, &fil_handle, err_desc, &err_len);
        RET_IF_ERR_EX(code, goto fun_end);
    }
    else
    {
        /* 在已有的磁盘组内创建asm文件，  */
        code = dmasm_file_create(conn, 1, file_name, file_size, &fil_handle, FALSE, strip_size,
redundancy_type, err_desc, &err_len);
        if (!IS_SUCCESS(code))
        {
            fprintf(stderr, "dmasm_file_create error code:%d [%s]\n", code, err_desc);
            goto fun_end;
        }
    }
```

```
    /* 写文件 */
    code = dmasm_file_write_by_offset_normal(conn, fil_handle, offset, wbuf, len, err_desc,
&err_len);
    if (!IS_SUCCESS(code))
    {
        fprintf(stderr, "dmasm_file_write_by_offset_normal error code:%d [%s]\n", code,
err_desc);
        goto fun_end;
    }

    /* 读文件 */
    code = dmasm_file_read_by_offset_normal(conn, fil_handle, offset, rbuf, len, err_desc,
&err_len);
    if (!IS_SUCCESS(code))
    {
        fprintf(stderr, "dmasm_file_write_by_offset_normal error code:%d [%s]\n", code,
err_desc);
        goto fun_end;
    }
    else
    {
        rbuf[len] = '\0';
        fprintf(stderr, "dmasm_file_write_by_offset_normal succ buf:%s\n", rbuf);
    }
```

```
fun_end:
    // 文件存在就关闭删除
    if (dmasm_file_is_exist(conn, file_name, err_desc, &err_len) == TRUE)
    {
        /* 关闭打开的文件句柄，否则删除会报错 */
        dmasm_file_close(conn, fil_handle);

        /* 删除文件 */
        code = dmasm_file_delete(conn, file_name, err_desc, &err_len);
        if (!IS_SUCCESS(code))
        {
            fprintf(stderr, "dmasm_file_delete error:[%s]\n", err_desc);
        }
    }

    dmasm_close_con(conn);
    dmasm_free_con(conn);
    dmasm_sys_deinit();
    return 0;
}
```

19.2 DMCSSM 接口

19.2.1 DLL 依赖库

监视器的 DLL 名称为 dmcssmon.dll（windows）和 libdmcssm.so（linux），使用监视器接口时必须加载，另外监视器 DLL 还依赖了其他的一些动态链接库，编程时需要将达梦 bin 目录的以下库拷贝到和 dmcssmon.dll 或 libdmcssm.so 相同目录下，也可以在加载动态链接库时直接指定 bin 目录，就不需要再拷贝库文件：

序号	WINDOWS	非 WINDOWS（LINUX 等）
1	dmcalc.dll	libdmcalc.so
2	dmcfg.dll	libdmcfg.so
3	dmcomm.dll	libdmcomm.so
4	dmcvt.dll	libdmcvt.so
5	dmcyt.dll	libdmcyt.so
6	dmdcr.dll	libdmdcr.so
7	dmelog.dll	libdmelog.so
8	dmmem.dll	libdmmem.so
9	dmmout.dll	libdmmout.so
10	dmos.dll	libdmos.so
11	dmsys.dll	libdmsys.so
12	dmutl.dll	libdmutl.so

19.2.2 返回值说明

监视器接口总体的返回值策略为：小于0表示执行失败，其他值表示执行成功。

对应的错误码请参考19.2.3错误码汇编，也可借助cssm_get_error_msg

_by_code接口获取错误码对应的错误描述信息。

19.2.3 错误码汇编

错误码	解释
-1	普通错误
-2	无效的句柄
-3	初始化日志信息失败
-4	初始化系统信息失败
-5	读取 DMCSSM.INI 文件失败（路径错误或配置错误或没有权限）
-6	无效的日志文件路径
-7	建立到所有 CSS 的连接失败，可能 CSS 都未启动，由用户决定是否忽略此错误码继续往下执行，如果不忽略此错误，认定初始化失败，则需要先销毁监视器环境，再释放操作句柄。
-8	非法的参数，参数为空或长度超长或值非法
-9	没有找到 code 对应的错误信息
-10	内存不足
-11	监视器已达到最大个数（同一套 DSC 集群最多允许同时启动 10 个监视器）
-1001	用户自定义异常或者未知错误
-1003	根据 ep_seqno 获取发送端口失败
-1004	到 dmcss 的连接断开
-1005	有消息正在发送中,您的命令当前不能执行
-1007	设置 CSS 的 MID 命令执行失败
-1009	监视器操作冲突
-1012	CSS 执行失败

-1017	通知 CSS 执行命令失败
-1018	获取有效的 CSS 通信端口失败，CSS 尚未启动或监视器配置错误
-1019	无效的组名或尚未收到消息
-1020	接收 CSS 消息超时
-1022	非法的 DCR 类型
-1023	只有主 CSS 能执行该操作
-1024	CSS 中没有找到指定类型的组信息
-1025	执行 OPEN FORCE 的组当前不是 SLAVE STARTUP 状态
-1026	无效的 EP 名称或尚未收到消息
-1027	未找到指定类型的组信息
-1028	检测到主 CSS 发生变化
-1029	当前不存在活动的 CSS
-1031	通知 CSS 打开监控失败
-1033	通知 CSS 关闭监控失败
-1035	命令执行失败
-1036	组当前没有活动 EP
-1037	当前存在有活动的 DB 节点，需要先退出对应的 DB 组，才允许退出 ASM 组
-1038	组中的 EP 已处于 ACTIVE 状态（只收集活动 CSS 对应的 EP），或者 CSS 配置有自动重启，请等待 CSS 自动检测重启
-1039	当前活动的 CSS 监控都已处于打开状态
-1040	当前活动的 CSS 监控都已处于关闭状态
-1041	获取 CSS 的通信端口失败
-1042	通知 CSS 执行 EP STARTUP 失败

-1045	CSS 中没有找到指定组的信息
-1046	CSS 对指定的组配置了自动重启，请等待 CSS 自动检测重启对应的 EP
-1047	EP 已经处于 ACTIVE 状态
-1048	ASM 实例未处于 ACTIVE 状态
-1049	ASM 组未处于 OPEN 状态
-1050	指定组中没有活动 EP
-1051	当前存在有活动的 DB 节点，需要先退出对应的 DB 组
-1052	CSS 监控处于关闭状态，不允许执行此命令
-1053	CSS 类型的组不允许执行此命令
-1057	获取主 CSS 通信端口失败，主 CSS 尚未选出或 CSS 未启动或者监视器配置错误
-1058	ASM 组没有活动节点或者活动节点不是 OPEN 状态，或者 ASM 组不是 OPEN 状态，不允许启动 DB 组
-1059	检测到主 CSS 发生变化，命令中断执行
-1065	关闭 CSS 监控失败
-1067	打开 CSS 监控失败
-1069	指定组不是 SLAVE STARTUP 状态，不允许执行 OPEN FORCE
-1070	CSS 监控处于打开状态，不允许手动执行此命令，请等待 CSS 自动处理
-1071	CSS 当前正在做故障处理或故障恢复
-1072	CSS 监控处于打开状态，不允许执行此命令
-1073	没有找到满足故障恢复条件的节点
-1074	CSS 当前正在做故障处理或故障恢复
-1075	没有找到满足故障处理条件的节点
-1076	需要先对 ASM 组做故障处理

-1077	CSS 对指定的组没有配置自动重启参数
-1078	监视器超时仍未等到组中的所有 EP 执行成功，命令执行失败
-1079	监视器超时仍未等到当前 EP 执行成功，命令执行失败
-1080	控制节点尚未选出，获取信息失败
-1081	无效的节点名或尚未收到消息
-1082	指定节点的组内序号非法
-1083	组中的 EP 不是活动状态
-1085	对应的 ASM 组中没有活动节点，无法执行故障处理
-1086	指定组中没有活动节点，无法执行故障处理
-1087	没有找到满足故障恢复条件的节点
-1088	需要先对 ASM 组做故障处理
-1089	监视器正在执行命令，请稍候重试
-1090	CSS 没有对组配置重启参数
-1091	非法的 EP 序号
-1092	对 EP 写入 HALT 命令成功，请等待处理结果
-1094	ASM 组中没有活动 EP，不允许执行故障处理
-1095	指定组中没有活动 EP，不允许执行故障处理
-1096	打开 CSS 对节点的自动拉起成功
-1097	关闭 CSS 对节点的自动拉起成功
-1098	组中没有非活动 EP
-1105	扩展节点失败，请查看日志获取详细信息
-1521	DSC 环境同一机器上的节点不允许配置相同的归档路径

19.2.4 C 接口说明

1. cssm_alloc_handle

函数原型:

```
CSSM_RETURN
cssm_alloc_handle(
    mhandle*          mhdle
);
```

功能说明:

分配监视器操作句柄，监视器的接口都通过此句柄调用执行。

参数说明:

mhdle：输出参数，分配到的监视器句柄。

返回值:

0：执行成功。

<0：执行失败。

2. cssm_get_error_msg_by_code

函数原型:

```
CSSM_RETURN
cssm_get_error_msg_by_code(
    mhandle           mhdle,
    mint4             code,
    mschar*           buf_msg,
    muint4            buf_len,
```

```
    muint4*          msg_len_out
);
```

功能说明:

获取输入 code 对应的错误描述信息，code 必须是小于 0 的值。

参数说明:

mhdle：输入参数，分配到的监视器句柄，注意如果是分配句柄或初始化环境时失败，允许 mhdle 是无效的，否则要求 mhdle 必须是有效句柄。

code：输入参数，需要获取错误信息的错误码 code，注意 code 必须是小于 0 的值。

buf_msg：输出参数，输出错误描述信息，注意 buf_msg 缓存长度建议大于 4096，避免输出信息被截断。

buf_len:输入参数，指定 buf_msg 可写入的最大长度。

msg_len_out：输出参数，buf_msg 缓存实际写入长度。

返回值:

0：执行成功。

<0：执行失败，-10 表示没有找到 code 对应的错误信息。

3. cssm_init

函数原型:

```
CSSM_RETURN
cssm_init(
    mhandle          mhdle,
    mschar*          ini_path,
    mschar*          log_path
);
```

功能说明:

初始化监视器环境。

参数说明:

mhdle: 输入参数，监视器操作句柄。

ini_path: 输入参数，指定 DMCSSM.INI 的绝对路径。

log_path: 输入参数，指定日志文件的存放路径，如果为 NULL 或空串，则将 DMCSSM.INI 中配置的 CSSM_LOG_PATH 作为日志文件路径，如果 DMCSSM.INI 中没有配置 CSSM_LOG_PATH，则将 DMCSSM.INI 的同级目录作为日志文件路径。

返回值:

0：执行成功。

<0: 执行失败，-7 表示监视器尝试建立到所有 CSS 的连接失败，可能 CSS 都未启动，允许忽略此错误继续执行其他操作，如果不忽略此错误，认定初始化失败，则需要先正常销毁监视器环境（调用 cssm_deinit 接口），再释放操作句柄（调用 cssm_free_handle 接口）。

4. cssm_msg_event_wait

函数原型:

```
void
cssm_msg_event_wait(
    mhandle          mhdle
);
```

功能说明:

等待消息事件。监视器接口命令执行过程中产生的中间输出消息，以及收到CSS自动处理的消息时都会将消息写入到缓存并通知消息事件，调用者只需要调用此接口等待即可，等

待事件发生后，可通过接口cssm_get_exec_msg去读取输出消息。

这种方式需要单独起一个线程来处理消息，以免消息被阻塞。

参数说明:

mhdle: 输入参数，监视器操作句柄。

返回值:

无

5. cssm_get_exec_msg

函数原型:

```
CSSM_RETURN
cssm_get_exec_msg(
    mhandle          mhdle,
    mschar*          buf_msg,
    muint4           buf_len,
    muint4*          msg_len_out,
    muint4*          get_flag
);
```

功能说明:

获取输出信息，和 cssm_msg_event_wait 配合使用。

参数说明:

mhdle:输入参数，监视器操作句柄。

buf_msg：输出参数，保存获取到的输出消息，注意 buf_msg 缓存长度需要大于4096，以免长度不够导致消息被截断。

buf_len:输入参数，指定 buf_msg 可写入的最大长度，建议 buf_msg 缓存长度及

buf_len 输入值大于 4096，避免消息被截断。

msg_len_out:输出参数，写消息成功后，输出实际写入的消息长度。

get_flag:输出参数，是否继续读取消息，TRUE 表示还有消息可读，FALSE 表示已全部读完。

返回值:

0：执行成功。

<0：执行失败。

6. cssm_get_msg_exit_flag

函数原型:

```
CSSM_RETURN
cssm_get_msg_exit_flag(
    mhandle          mhdle,
    mbool*           exit_flag
);
```

功能说明:

如果上层应用程序中有单独的消息线程，可通过调用此接口获取退出标记，在标记为 TRUE 时可正常退出线程。

参数说明:

mhdle：输入参数，监视器操作句柄。

exit_flag：输出参数，输出退出标记，1 表示可以正常退出，0 表示不能退出。

返回值:

0：执行成功。

<0：执行失败。

7. cssm_set_msg_exit_event

函数原型:

```
void
cssm_set_msg_exit_event(
    mhandle         mhdle
);
```

功能说明:

设置消息退出事件。如果上层应用程序中有单独的消息线程，在 cssm_get_msg_exit_flag 接口获取到退出标记为 TRUE 时，需要调用此接口设置退出事件。

参数说明:

mhdle: 输入参数，监视器操作句柄。

返回值:

无

8. cssm_msg_event_deinit

函数原型:

```
void
cssm_msg_event_deinit(
    mhandle         mhdle
);
```

功能说明:

通知并等待消息退出。使用等待事件方式获取监视器消息时，需要单独起一个线程，在

程序结束，退出线程时，需要调用此接口通知并等待消息线程退出。

消息线程相关的接口有 cssm_msg_event_wait、cssm_get_exec_msg、cssm_get_msg_exit_flag、cssm_set_msg_exit_event 和 cssm_msg_event_deinit，这几个接口需要配合使用。

参数说明：

mhdle：输入参数，监视器操作句柄。

返回值：

无

9. cssm_get_master_css_info

函数原型：

```
CSSM_RETURN
cssm_get_master_css_info(
    mhandle          mhdle,
    mbyte*           ep_seqno,
    mschar*          ep_name
);
```

功能说明：

获取主 CSS 节点的组内序号和节点名称。



注意：CSS 需要 5s 的选举时间来确定主 CSS，在 CSS 的选举时间内，此接口会执行失败，获取不到主 CSS 信息。

参数说明：

mhdle：输入参数，监视器操作句柄。

ep_seqno：输出参数，输出主 CSS 节点的组内序号。

ep_name：输出参数，输出主 CSS 节点的名称，注意缓存长度不能小于 65，避免长度溢出。

返回值：

0：执行成功。

<0：执行失败。

10. cssm_get_master_asm_info

函数原型：

```
CSSM_RETURN
cssm_get_master_asm_info(
    mhandle          mhdle,
    mbyte*           ep_seqno,
    mschar*          ep_name
);
```

功能说明：

获取主 ASM 节点的组内序号和节点名称。

参数说明：

mhdle：输入参数，监视器操作句柄。

ep_seqno：输出参数，输出主 ASM 节点的组内序号。

ep_name：输出参数，输出主 ASM 节点的名称，注意缓存长度不能小于 65，避免长度溢出。

返回值：

0：执行成功。

<0：执行失败。

11. cssm_get_master_db_info

函数原型:

```
CSSM_RETURN
cssm_get_master_db_info(
    mhandle          mhdle,
    mschar*          group_name,
    mbyte*           ep_seqno,
    mschar*          ep_name
);
```

功能说明:

获取主 DB 节点的组内序号和节点名称。

参数说明:

mhdle：输入参数，监视器操作句柄。

group_name：输入参数，需要获取主 DB 信息的组名，允许配置有多个 DB 组。

ep_seqno：输出参数，输出主 DB 节点的组内序号。

ep_name：输出参数，输出主 DB 节点的名称，注意缓存长度不能小于 65，避免长度溢出。

返回值:

0：执行成功。

<0：执行失败。

12. cssm_get_css_auto_flag

函数原型:

```
CSSM_RETURN
cssm_get_css_auto_flag(
    mhandle          mhdle,
    mschar*          css_name,
    mbool*           auto_flag
);
```

功能说明:

获取指定 CSS 的监控状态。

参数说明:

mhdle：输入参数，监视器操作句柄。

css_name：输入参数，需要获取监控状态的 CSS 名称。

auto_flag：输出参数，为 1 表示 CSS 监控处于打开状态，为 0 表示 CSS 监控处于关闭状态。

返回值:

0：执行成功。

<0：执行失败。

13. cssm_get_css_auto_info

函数原型:

```
CSSM_RETURN
cssm_get_css_auto_info(
    mhandle          mhdle,
    mschar*          css_name,
    mbyte*           auto_flag,
```

```
    mschar*         asm_name,
    mbyte*          asm_auto_restart,
    mschar*         db_name,
    mbyte*          db_auto_restart
);
```

功能说明:

获取指定 CSS 上的自动监控、自动拉起信息。

参数说明:

mhdle：输入参数，监视器操作句柄。

css_name：输入参数，需要获取信息的 CSS 名称。

auto_flag：输出参数，为 1 表示 CSS 监控处于打开状态，为 0 表示 CSS 监控处于关闭状态。

asm_name：输出参数，当前 css 可以控制自动拉起的 asm 节点名称。

asm_auto_restart：输出参数，asm_name 当前的自动拉起状态，为 1 表示自动拉起打开，为 0 表示自动拉起关闭，0xFE 表示 DMDCR.INI 中配置有自动拉起命令串，但自动拉起检测间隔为 0，为 0xFF 则表示 DMDCR.INI 中自动拉起参数配置错误。

db_name：输出参数，当前 CSS 可以控制自动拉起的 DB 节点名称。

db_auto_restart：输出参数，db_name 当前的自动拉起状态，为 1 表示自动拉起打开，为 0 表示自动拉起关闭，0xFE 表示 DMDCR.INI 中配置有自动拉起命令串，但自动拉起检测间隔为 0，为 0xFF 则表示 DMDCR.INI 中自动拉起参数配置错误。

返回值:

0：执行成功。

<0：执行失败。

14. cssm_set_group_auto_restart

函数原型:

```
CSSM_RETURN

cssm_set_group_auto_restart(

mhandle          mhdle,

mschar*          group_name,

mbyte            auto_restart_flag

);
```

功能说明:

设置指定组的自动拉起标记。

参数说明:

mhdle：输入参数，监视器操作句柄。

group_name：输入参数，输入需要修改自动拉起标记的组名。

auto_restart_flag：输入参数，输入需要修改的值，只能是 0 或 1，为 0 表示关闭指定组自动拉起功能，为 1 表示打开指定组自动拉起功能。

返回值:

0：执行成功。

<0：执行失败。

15. cssm_get_n_group

函数原型:

```
CSSM_RETURN

cssm_get_n_group(
```

```
    mhandle mhdle,
    muint4*         n_group,
    mbyte*          group_seqno,
    mschar**        group_name,
    mschar**        group_type
);
```

功能说明:

获取 DSC 集群所有的组信息。

参数说明:

mhdle：输入参数，监视器操作句柄。

n_group：输入输出参数，输入参数指定要获取的最多组个数，建议不小于 16，避免取不到完整信息，输出参数为实际获取的组个数。

group_seqno：输出参数，输出各组的序号，参数为 mbyte 数组类型，数组长度为 n_group 的输入值。

group_name：输出参数，输出各组的名称，参数为字符串指针数组类型，数组长度是 n_group 的输入值，每个指针元素指向的缓存长度不能小于 65，避免长度溢出。

group_type：输出参数，输出各组的类型，参数为字符串指针数组类型，数组长度是 n_group 的输入值，每个指针元素指向的缓存长度不能小于 65，避免长度溢出。

返回值:

0：执行成功。

<0：执行失败。

16. cssm_get_n_ep

函数原型:

```
CSSM_RETURN
cssm_get_n_ep(
    mhandle          mhdle,
    mschar*          group_name,
    muint4*          n_ep,
    mbyte*           ep_seqno,
    mschar**         ep_name
);
```

功能说明:

获取指定组中的节点信息。

参数说明:

mhdle：输入参数，监视器操作句柄。

group_name：输入参数，指定要获取节点信息的组名。

n_ep：输入输出参数，输入参数指定要获取的最多节点个数，建议不小于 16，避免取不到完整信息，输出参数为实际获取的节点个数。

ep_seqno：输出参数，输出各节点的组内序号，参数为 mbyte 数组类型，数组长度为 n_ep 的输入值。

ep_name：输出参数，输出各节点的名称，参数为字符串指针数组类型，数组长度是 n_ep 的输入值，每个指针元素指向的缓存长度不能小于 65，避免长度溢出。

返回值:

0：执行成功。

<0：执行失败。

17. cssm_get_group_info_by_name

函数原型:

```
CSSM_RETURN
cssm_get_group_info_by_name(
    mhandle          mhdle,
    mschar*          group_name,
    mbyte*           group_seqno,
    mschar*          group_type,
    mbyte*           n_ep,
    mbyte*           ep_seqno_arr,
    mschar**         ep_name_arr,
    mbyte*           n_ok_ep,
    mbyte*           ok_ep_arr,
    mbyte*           master_ep,
    mschar*          master_ep_name,
    mschar*          sta,
    mschar*          sub_sta,
    mbyte*           n_break_ep,
    mbyte*           break_ep_arr,
    mbyte*           recover_ep,
    mbyte*           crash_over
);
```

功能说明:

获取指定名称的组信息。

参数说明:

mhdle: 输入参数，监视器操作句柄。

group_name: 输入参数，指定要获取信息的组名。

group_seqno: 输出参数，输出组的序号。

group_type: 输出参数，输出组的类型，注意缓存长度不能小于 65，避免长度溢出。

n_ep: 输入输出参数，输入参数指定最多要获取的节点个数，建议输入值不小于 16，避免取不到完整信息，输出参数为实际获取的节点个数。

ep_seqno_arr: 输出参数，输出各节点的组内序号，是 mbyte 数组类型，数组长度为 n_ep 的输入值。

ep_name_arr: 输出参数，输出各节点的名称，是字符串指针数组类型，数组长度为 n_ep 的输入值，每个指针元素指向的缓存长度不能小于 65，避免长度溢出。

n_ok_ep: 输入输出参数，输入参数指定最多要获取的正常节点个数，建议输入值不小于 16，避免取不到完整信息，输出参数为实际获取的正常节点个数，该字段只对 ASM/DB 类型的组有效，CSS 类型的组输出为 0。

ok_ep_arr: 输出参数，输出组中正常节点的组内序号，是 mbyte 数组类型，数组长度是 n_ok_ep 的输入值，该字段只对 ASM/DB 类型的组有效。

master_ep: 输出参数，输出组中控制节点的组内序号。

master_ep_name: 输出参数，输出组中控制节点的名称，注意缓存长度不能小于 65，避免长度溢出。

sta: 输出参数，输出组的状态，该字段只对 ASM/DB 类型的组有效，CSS 类型的组输出为空串。

sub_sta: 输出参数，输出组的子状态，该字段只对 ASM/DB 类型的组有效，CSS 类型的组输出为空串。

n_break_ep：输出参数，输出组中正在执行故障处理的节点个数。

break_ep_arr：输出参数，输出组中正在执行故障处理的节点序号，该字段只对 ASM/DB 类型的组有效，CSS 类型的组输出为 0xFF。

recover_ep：输出参数，输出组中正在执行故障恢复的节点序号，该字段只对 ASM/DB 类型的组有效，CSS 类型的组输出为 0xFF。

crash_over：输出参数，表示故障处理是否结束。0 表示未结束，1 表示结束。

返回值：

0：执行成功。

<0：执行失败。

18. cssm_get_ep_info_by_name

函数原型：

```
CSSM_RETURN
cssm_get_ep_info_by_name(
    mhandle         mhdle,
    mschar*         group_name,
    mschar*         ep_name,
    mbyte*          css_seqno,
    mschar*         css_name,
    mschar*         css_time,
    mbyte*          ep_seqno,
    mschar*         work_mode,
    mschar*         inst_stat,
    mschar*         vtd_stat,
```

```
    mschar*         ok_stat,
    mschar*         is_active,
    mschar*         ep_guid,
    mschar*         ep_ts
);
```

功能说明:

获取指定名称的组信息。

参数说明:

mhdle: 输入参数，监视器操作句柄。

group_name: 输入参数，指定要获取信息的组名。

ep_name: 输入参数，指定要获取信息的节点名称。

css_seqno: 输出参数，输出取得节点信息的 CSS 序号。

css_name: 输出参数，输出取得节点信息的 CSS 名称，注意缓存长度不能小于 65，避免长度溢出。

css_time: 输出参数，输出取得节点信息的 CSS 当前时间，注意缓存长度不能小于 65，避免长度溢出。

ep_seqno: 输出参数，输出节点的组内序号。

work_mode: 输出参数，输出节点的工作模式，注意缓存长度不能小于 65，避免长度溢出。

inst_stat: 输出参数，输出节点的工作状态，注意缓存长度不能小于 65，避免长度溢出。

vtd_stat: 输出参数，输出节点在 VTD 文件所在磁盘（VOTE 磁盘（非镜像环境下）或 DCRV 磁盘（镜像环境下））中的状态，注意缓存长度不能小于 65，避免长度溢出。

ok_stat: 输出参数, 输出节点状态是否正常 (OK/ERROR), 注意缓存长度不能小于 6,

避免长度溢出。

is_active：输出参数，输出节点是否处于活动状态（TRUE/FALSE），注意缓存长度不能小于 6，避免长度溢出。

ep_guid：输出参数，输出节点的 guid 值。

ep_ts：输出参数，输出节点的时间戳值。

返回值：

0：执行成功。

<0：执行失败。

19. cssm_get_header_config_info

函数原型：

```
CSSM_RETURN
cssm_get_header_config_info(
    mhandle          mhdle,
    muint4*          dcr_n_group,
    mschar*          dcr_vtd_path,
    muint8*          dcr_oguid,
);
```

功能说明：

输出 DMDCR_CFG.INI 中配置的全局信息。

参数说明：

mhdle：输入参数，监视器操作句柄。

dcr_n_group：输出参数，输出配置的组个数。

dcr_vtd_path：输出参数，输出 VTD 文件所在路径，注意缓存长度不能小于 257，

避免长度溢出。

dcr_oguid：输出参数，输出配置的 OGUID 标识。

返回值：

0：执行成功。

<0：执行失败。

20. cssm_get_group_config_info

函数原型：

```
CSSM_RETURN
cssm_get_group_config_info(
    mhandle          mhdle,
    mschar*          dcr_grp_name,
    mschar*          dcr_grp_type,
    mbyte*           dcr_grp_n_ep,
    muint4*          dcr_grp_dskchk_cnt,
);
```

功能说明：

输出 DMDCR_CFG.INI 中配置的组信息。

参数说明：

mhdle：输入参数，监视器操作句柄。

dcr_grp_name：输入参数，输入要获取信息的组名。

dcr_grp_type：输出参数，输出配置的组类型，注意缓存长度不能小于 65，避免长度溢出。

dcr_grp_n_ep：输出参数，输出组中配置的节点个数。

dcr_grp_dskchk_cnt：输出参数，输出指定组配置的磁盘心跳容错时间。

返回值：

0：执行成功。

<0：执行失败。

21. cssm_get_ep_config_info

函数原型：

```
CSSM_RETURN
cssm_get_ep_config_info(
    mhandle          mhdle,
    mschar*          dcr_group_name,
    mschar*          dcr_ep_name,
    mbyte*           dcr_ep_seqno,
    mschar*          dcr_ep_host,
    muint4*          dcr_ep_port,
    muint4*          dcr_ep_shm_key,
    muint4*          dcr_ep_shm_size,
    mschar*          dcr_ep_asm_load_path
);
```

功能说明：

输出 DMDCR_CFG.INI 中配置的节点信息。

参数说明：

mhdle：输入参数，监视器操作句柄。

dcr_group_name：输入参数，输入节点所在的组名。

dcr_ep_name：输入参数，输入节点名称。

dcr_ep_seqno：输出参数，输出节点的组内序号，CSS/ASM 类型的节点为自动分配的值，DB 类型的节点如果没有显式配置 DCR_EP_SEQNO，也是自动分配的值，否则为手动配置的值。

dcr_ep_host：输出参数，输出节点的 IP 地址，对 CSS/ASM 类型的节点有效，表示登录 CSS/ASM 节点的 IP 地址，注意缓存长度不能小于 65，避免长度溢出。

dcr_ep_port：输出参数，输出节点的 TCP 监听端口，对 CSS/ASM 类型的节点有效，对应登录 CSS/ASM 的端口号。

dcr_ep_shm_key：输出参数，输出节点的共享内存标识，对 ASM 类型的节点有效。

dcr_ep_shm_size：输出参数，输出节点的共享内存大小，单位 MB，对 ASM 类型的节点有效。

dcr_ep_asm_load_path：输出参数，输出节点的 ASM 磁盘扫描路径，对 ASM 类型的节点有效，注意缓存长度不能小于 257，避免长度溢出。

返回值：

0：执行成功。

<0：执行失败。

22. cssm_get_monitor_info

函数原型：

```
CSSM_RETURN
cssm_get_monitor_info(
    mhandle          mhdle,
    mbyte*           css_seqno,
    mschar*          css_name,
```

```
    muint4*          n_mon,
    mschar**         conn_time_arr,
    muint8*          mid_arr,
    mschar**         mon_ip_arr
);
```

功能说明:

输出连接到主 CSS 上的所有监视器信息，如果主 CSS 故障或尚未选出，则任选一个 CSS 输出所有的连接信息。

参数说明:

mhdle：输入参数，监视器操作句柄。

css_seqno：输出参数，输出取得监视器连接信息的 CSS 序号。

css_name：输出参数，输出取得监视器连接信息的 CSS 名称，注意缓存长度不能小于 65，避免长度溢出。

n_mon：输入输出参数，输入值指定可获取的最多的监视器个数，建议不小于 10，避免取不到完整信息，输出值为实际获取到的监视器个数。

conn_time_arr：输出参数，参数类型为字符串指针数组类型，数组长度为 n_mon 的输入值，输出各监视器连接到 css_name 的时间，注意每个数组元素指向的缓存长度不能小于 65，避免长度溢出。

mid_arr：输出参数，参数类型为 muint8 数组类型，数组长度为 n_mon 的输入值，输出各监视器的 mid 值。

mon_ip_arr：参数类型为字符串指针数组类型，数组长度为 n_mon 的输入值，输出各监视器的 IP 地址，注意每个数组元素指向的缓存长度不能小于 65，避免长度溢出。

返回值:

0：执行成功。

<0：执行失败。

23. cssm_css_startup

函数原型：

```
CSSM_RETURN
cssm_css_startup(
    mhandle          mhdle
);
```

功能说明：

打开当前所有活动 CSS 的监控功能。

参数说明：

mhdle：输入参数，监视器操作句柄。

返回值：

0：执行成功。

<0：执行失败。

24. cssm_css_stop

函数原型：

```
CSSM_RETURN
cssm_css_stop(
    mhandle          mhdle
);
```

功能说明：

关闭当前所有活动 CSS 的监控功能。

CSS 监控被关闭后，只负责调整各组节点的 active 状态，除此之外不会做任何自动处理，也不会自动拉起对应的节点，如果在监控关闭后，通过手动方式（非 cssm_ep_startup 接口方式）启动 ASM 或 DB 组，各节点也无法正常启动到 OPEN 状态。

参数说明：

mhdle：输入参数，监视器操作句柄。

返回值：

0：执行成功。

<0：执行失败。

25. cssm_open_force

函数原型：

```
CSSM_RETURN
cssm_open_force(
    mhandle          mhdle,
    mschar*          group_name
);
```

功能说明：

强制 OPEN 指定的组。

使用场景：

在启动 ASM 或 DB 组时，如果某个节点故障一直无法启动，可借助此接口将 ASM 或 DB 组强制 OPEN。

接口会发送消息到主 CSS 执行，并且主 CSS 的监控需要处于打开状态，如果主 CSS 故障或尚未选出，则接口执行失败。

参数说明：

mhdle：输入参数，监视器操作句柄。

group_name：输入参数，指定要强制 OPEN 的组名。

返回值：

0：执行成功。

<0：执行失败。

26. cssm_ep_startup

函数原型：

```
CSSM_RETURN
cssm_ep_startup(
    mhandle          mhdle,
    mschar*          group_name
);
```

功能说明：

启动指定 ASM 或 DB 组的所有节点。

如果 CSS 已经配置了自动重启，并且 CSS 的监控处于打开状态，则接口不允许执行，需要等待 CSS 自动检测故障并执行重启操作。

每个 CSS 只负责重启和自己的 DMDCR.INI 中配置的 DMDCR_SEQNO 相同的 ASM 或 DB 节点，因此需要所有 CSS 都处于活动状态，否则只通知当前活动的 CSS 重启相对应的节点。


注意：

只有在 ASM 组正常启动到 OPEN 状态，并且所有活动的 ASM 节点都处于 OPEN 状态时，才允许启动 DB 组，否则执行 DB 组的启动操作会报错。

另外在接口执行时，如果 CSS 监控处于关闭状态，则会直接打开 CSS 监

控，否则 ASM 或 DB 组无法正常启动到 OPEN 状态。

参数说明：

mhdle：输入参数，监视器操作句柄。

group_name：输入参数，指定要启动的组名。

返回值：

0：执行成功。

<0：执行失败。

27. cssm_ep_stop

函数原型：

```
CSSM_RETURN
cssm_ep_stop(
    mhandle          mhdle,
    mschar*          group_name
);
```

功能说明：

退出指定的 ASM 或 DB 组，如果主 CSS 故障或尚未选出，则接口执行失败。


注意：

在退出 ASM 组时，需要保证 DB 组已经退出，否则会报错处理。

另外如果退出时，CSS 监控没有关闭，在退出完成后，达到 CSS 设置的重启间隔之后，CSS 仍然会执行自动拉起，如果要正常退出集群，不需要自动拉起，则退出之前需要调用 cssm_css_stop 接口停止 CSS 的监控功能。

参数说明：

mhdle：输入参数，监视器操作句柄。

group_name：输入参数，指定要退出的组名。

返回值：

0：执行成功。

<0：执行失败。

28. cssm_ep_break

函数原型：

```
CSSM_RETURN
cssm_ep_break(
    mhandle          mhdle,
    mschar*          group_name
);
```

功能说明：

此接口只允许在主 CSS 监控关闭的情况下使用。

ASM 组或 DB 组在正常运行时，如果某个节点出现故障，则需要调用此接口执行故障处理，将故障节点从组的 OK 节点数组中摘除。



注意：

在主 CSS 监控关闭的情况下，如果 ASM 组的某个节点故障，和故障节点有相同 DCR_SEQNO 的 DB 节点需要使用 cssm_ep_halt 接口执行强制退出，否则 DB 节点访问对应的 ASM 文件系统失败，也会自动 HALT。

在 ASM 和 DB 节点都出现故障的情况下，需要先对 ASM 组执行故障处理，再对 DB 组执行故障处理，否则会报错不允许执行。

参数说明：

mhdle：输入参数，监视器操作句柄。

group_name：输入参数，指定要执行故障处理的组名。

返回值：

0：执行成功。

<0：执行失败。

29. cssm_ep_recover

函数原型：

```
CSSM_RETURN
cssm_ep_recover(
    mhandle          mhdle,
    mschar*          group_name
);
```

功能说明：

此接口只允许在主 CSS 监控关闭的情况下使用。

执行过故障处理的节点重启成功后，可通过调用此接口将故障节点重加入指定的 ASM 或 DB 组，重新恢复组到 OPEN 状态。



注意：如果主 CSS 故障或尚未选出，则接口执行失败。另外只有在 ASM 组的故障节点重加入成功后，才允许重启对应的 DB 节点，并执行 DB 组的故障重加入。

参数说明：

mhdle：输入参数，监视器操作句柄。

group_name：输入参数，指定要执行故障恢复的组名。

返回值：

0：执行成功。

<0：执行失败。

30. cssm_ep_halt

函数原型:

```
CSSM_RETURN
cssm_ep_halt(
    mhandle          mhdle,
    mschar*          group_name,
    mschar*          ep_name
);
```

功能说明:

强制退出指定组的指定 EP。

此接口在 CSS 监控打开或关闭的情况下都允许使用，适用于下述场景:

a. 某个 ASM 或 DB 节点故障，CSS DCR_GRP_DSKCHK_CNT 配置值很大，在容错时间内，CSS 不会调整故障节点的 active 标记，一直是 TRUE，CSS 认为故障 EP 仍然处于活动状态，不会自动执行故障处理，并且不允许手动执行故障处理。

另外执行 cssm_ep_startup 或 cssm_ep_stop 接口时，会误认为故障 EP 仍然处于活动状态，导致执行结果与预期不符。

此时可以通过执行此接口，通知 CSS 再次 HALT 故障 EP，确认 EP 已经被 HALT 后，CSS 会及时调整 active 标记为 FALSE，在此之后，对自动/手动故障处理，启动/退出 EP 节点等操作都可以正常执行。

b. 需要强制 HALT 某个正在运行的 ASM 或 DB 节点，也可以通过此接口完成。

参数说明:

mhdle: 输入参数，监视器操作句柄。

group_name: 输入参数，指定要强制退出的节点所在的组名。

ep_name：输入参数，指定要强制退出的节点名称。

返回值：

0：执行成功。

<0：执行失败。

31. cssm_deinit

函数原型：

```
CSSM_RETURN
cssm_deinit(
    mhandle          mhdle
);
```

功能说明：

销毁监视器执行环境。

参数说明：

mhdle：输入参数，监视器操作句柄。

返回值：

0：执行成功。

<0：执行失败。

32. cssm_free_handle

函数原型：

```
CSSM_RETURN
cssm_free_handle(
    mhandle          mhdle
```

```
);
```

功能说明:

释放分配到的监视器句柄。

参数说明:

mhdle：输入参数，分配到的句柄。

返回值:

0：执行成功。

<0：执行失败。

19.2.5 JAVA 接口说明

1. cssm_init_handle

函数原型:

```
boolean

cssm_init_handle();
```

功能说明:

分配监视器操作句柄，监视器的接口都通过此句柄调用执行。

参数说明:

无

返回值:

true：分配成功，返回值为监视器操作句柄。

false：分配失败。

2. cssm_get_error_msg_by_code

函数原型：

```
CssMonMsg

cssm_get_error_msg_by_code(

    int                 code

);
```

功能说明：

获取输入 code 对应的错误描述信息，code 必须是小于 0 的值。

参数说明：

code：输入参数，需要获取错误信息的错误码 code，注意 code 必须是小于 0 的值。

返回值：

返回 CssMonMsg 对象，可通过对象成员 returnCode 获取执行结果。

0：执行成功，可通过 CssMonMsg 的其他对象成员获取输出消息。

<0：执行失败。

3. cssm_init_env

函数原型：

```
int

cssm_init_env(

    String          ini_path,

    String          log_path

);
```

功能说明：

初始化监视器环境。

参数说明:

ini_path: 输入参数，指定 DMCSSM.INI 的绝对路径。

log_path: 输入参数，指定日志文件的存放路径，如果为 NULL 或空串，则将 DMCSSM.INI 中配置的 CSSM_LOG_PATH 作为日志文件路径，如果 DMCSSM.INI 中没有配置 CSSM_LOG_PATH，则将 DMCSSM.INI 的同级目录作为日志文件路径。

返回值:

0：执行成功。

<0: 执行失败，-7 表示监视器尝试建立到所有 CSS 的连接失败，可能 CSS 都未启动，允许忽略此错误继续执行其他操作，如果不忽略此错误，认定初始化失败，则需要先正常销毁监视器环境（调用 cssm_deinit 接口），再释放操作句柄（调用 cssm_free_handle 接口）。

4. cssm_msg_event_wait

函数原型:

```
void

cssm_msg_event_wait();
```

功能说明:

等待消息事件。监视器接口命令执行过程中产生的中间输出消息，以及收到CSS自动处理的消息时都会将消息写入到缓存并通知消息事件，调用者只需要调用此接口等待即可，等待事件发生后，可通过接口cssm_get_exec_msg去读取输出消息。

这种方式需要单独起一个线程来处理消息，以免消息被阻塞。

参数说明:

无

返回值:

无

5. cssm_get_exec_msg

函数原型:

```
CssMonMsg

cssm_get_exec_msg();
```

功能说明:

获取输出信息，和 cssm_msg_event_wait 配合使用。

参数说明:

无

返回值:

返回 CssMonMsg 对象，可通过对象成员 returnCode 获取执行结果。

0：执行成功，可通过 CssMonMsg 的其他对象成员获取输出消息。

<0：执行失败。

6. cssm_get_msg_exit_flag

函数原型:

```
boolean

cssm_get_msg_exit_flag();
```

功能说明:

如果上层应用程序中有单独的消息线程，可通过调用此接口获取退出标记，在标记为 true 时可正常退出线程。

参数说明:

无

返回值:

true：允许线程退出。

false：不允许线程退出，需要继续等待获取输出消息。

7. cssm_set_msg_exit_event

函数原型:

```
void

cssm_set_msg_exit_event();
```

功能说明:

设置消息退出事件。如果上层应用程序中有单独的消息线程，在cssm_get_msg_exit_flag 接口获取到退出标记为 true 时，需要调用此接口设置退出事件。

参数说明:

无

返回值:

无

8. cssm_msg_event_deinit

函数原型:

```
void

cssm_msg_event_deinit();
```

功能说明:

通知并等待消息退出。使用等待事件方式获取监视器消息时，需要单独起一个线程，在

程序结束，退出线程时，需要调用此接口通知并等待消息线程退出。

消息线程相关的接口有 cssm_msg_event_wait、cssm_get_exec_msg、cssm_get_msg_exit_flag 、 cssm_set_msg_exit_event 和 cssm_msg_event_deinit，这几个接口需要配合使用。

参数说明:

无

返回值:

无

9. cssm_get_master_css_info

函数原型:

```
CssMonMasterEpInfo

cssm_get_master_css_info();
```

功能说明:

获取主 CSS 节点的组内序号和节点名称。



注意: **CSS 需要 5s 的选举时间来确定主 CSS，在 CSS 的选举时间内，此接口会执行失败，获取不到主 CSS 信息。**

参数说明:

无

返回值:

返回 CssMonMasterEpInfo 对象，可通过对象成员 returnCode 获取执行结果。

0：执行成功，可通过 CssMonMasterEpInfo 的其他对象成员获取主 CSS 信息。

<0：执行失败。

10. cssm_get_master_asm_info

函数原型:

```
CssMonMasterEpInfo

cssm_get_master_asm_info();
```

功能说明:

获取主 ASM 节点的组内序号和节点名称。

参数说明:

无

返回值:

返回 CssMonMasterEpInfo 对象，可通过对象成员 returnCode 获取执行结果。

0：执行成功，可通过 CssMonMasterEpInfo 的其他对象成员获取主 ASM 信息。

<0：执行失败。

11. cssm_get_master_db_info

函数原型:

```
CssMonMasterEpInfo

cssm_get_master_db_info(

    String                group_name

);
```

功能说明:

获取主 DB 节点的组内序号和节点名称。

参数说明:

group_name：输入参数，需要获取主 DB 信息的组名，允许配置有多个 DB 组。

返回值:

返回 CssMonMasterEpInfo 对象，可通过对象成员 returnCode 获取执行结果。

0：执行成功，可通过 CssMonMasterEpInfo 的其他对象成员获取主 DB 信息。

<0：执行失败。

12. cssm_get_css_auto_flag

函数原型:

```
boolean

cssm_get_css_auto_flag(

    String                  css_name

);
```

功能说明:

获取指定 CSS 的监控状态。

参数说明:

css_name：输入参数，需要获取监控状态的 CSS 名称。

返回值:

true：表示 CSS 监控处于打开状态。

false：表示 CSS 监控处于关闭状态。

13. cssm_get_css_auto_info

函数原型:

```
CssMonAutoInfo

cssm_get_css_auto_info(

    String                  css_name
```

```
);
```

功能说明:

获取指定 CSS 上的自动监控、自动拉起信息。

参数说明:

css_name：输入参数，需要获取信息的 CSS 名称。

返回值:

返回 CssMonAutoInfo 对象，可通过对象成员 returnCode 获取执行结果。

0：执行成功，可通过 CssMonAutoInfo 的其他对象成员获取相关信息。

<0：执行失败。

14. cssm_set_group_auto_restart

函数原型:

```
int
cssm_set_group_auto_restart(
    String          group_name,
    byte            auto_restart_flag
);
```

功能说明:

设置指定组的自动拉起标记。

参数说明:

group_name：输入参数，输入需要修改自动拉起标记的组名。

auto_restart_flag：输入参数，输入需要修改的值，只能是 0 或 1，为 0 表示关闭指定组自动拉起功能，为 1 表示打开指定组自动拉起功能。

返回值:

0：执行成功。

<0：执行失败。

15. cssm_get_n_group

函数原型:

```
CssMonGrpNameArray

cssm_get_n_group();
```

功能说明:

获取 DSC 集群所有的组信息。

参数说明:

无

返回值:

返回 CssMonGrpNameArray 对象，可通过对象成员 returnCode 获取执行结果。

0：执行成功，可通过 CssMonGrpNameArray 的其他对象成员获取组信息。

<0：执行失败。

16. cssm_get_n_ep

函数原型:

```
CssMonEpNameArray

cssm_get_n_ep(

    String                group_name

);
```

功能说明:

获取指定组中的节点信息。

参数说明:

group_name：输入参数，指定要获取节点信息的组名。

返回值:

返回 CssMonEpNameArray 对象，可通过对象成员 returnCode 获取执行结果。

0：执行成功，可通过 CssMonEpNameArray 的其他对象成员获取节点信息。

<0：执行失败。

17. cssm_get_group_info_by_name

函数原型:

```
CssMonGrpInfo

cssm_get_group_info_by_name(

    String                  group_name

);
```

功能说明:

获取指定名称的组信息。

参数说明:

group_name：输入参数，指定要获取信息的组名。

返回值:

返回 CssMonGrpInfo 对象，可通过对象成员 returnCode 获取执行结果。

0：执行成功，可通过 CssMonGrpInfo 的其他对象成员获取节点信息。

<0：执行失败。

18. cssm_get_ep_info_by_name

函数原型:

```
CssMonEpInfo

cssm_get_ep_info_by_name(

    String          group_name,

    String          ep_name

);
```

功能说明:

获取指定名称的组信息。

参数说明:

group_name：输入参数，指定要获取信息的组名。

ep_name：输入参数，指定要获取信息的节点名称。

返回值:

返回 CssMonEpInfo 对象，可通过对象成员 returnCode 获取执行结果。

0：执行成功，可通过 CssMonEpInfo 的其他对象成员获取节点信息。

<0：执行失败。

19. cssm_get_header_config_info

函数原型:

```
CssMonHeaderConfigInfo

cssm_get_header_config_info();
```

功能说明:

输出 DMDCR_CFG.INI 中配置的全局信息。

参数说明:

无

返回值:

返回 CssMonHeaderConfigInfo 对象，可通过对象成员 returnCode 获取执行结果。

0：执行成功，可通过 CssMonHeaderConfigInfo 的其他对象成员获取配置信息。

<0：执行失败。

20. cssm_get_group_config_info

函数原型：

```
CssMonGroupConfigInfo

cssm_get_group_config_info(

    String                dcr_group_name

);
```

功能说明：

输出 DMDCR_CFG.INI 中配置的组信息。

参数说明：

dcr_group_name：输入参数，输入要获取信息的组名。

返回值：

返回 CssMonGroupConfigInfo 对象，可通过对象成员 returnCode 获取执行结果。

0：执行成功，可通过 CssMonGroupConfigInfo 的其他对象成员获取配置信息。

<0：执行失败。

21. cssm_get_ep_config_info

函数原型：

```
CssMonEpConfigInfo

cssm_get_ep_config_info(

    String          dcr_group_name,
```

```
    String                  dcr_ep_name
);
```

功能说明:

输出 DMDCR_CFG.INI 中配置的节点信息。

参数说明:

dcr_group_name：输入参数，输入节点所在的组名。

dcr_ep_name：输入参数，输入节点名称。

返回值:

返回 CssMonEpConfigInfo 对象，可通过对象成员 returnCode 获取执行结果。

0：执行成功，可通过 CssMonEpConfigInfo 的其他对象成员获取配置信息。

<0：执行失败。

22. cssm_get_monitor_info

函数原型:

```
CssMonitorInfo
cssm_get_monitor_info();
```

功能说明:

输出连接到主 CSS 上的所有监视器信息，如果主 CSS 故障或尚未选出，则任选一个 CSS 输出所有的连接信息。

参数说明:

无

返回值:

返回 CssMonitorInfo 对象，可通过对象成员 returnCode 获取执行结果。

0：执行成功，可通过 CssMonitorInfo 的其他对象成员获取配置信息。

<0：执行失败。

23. cssm_open_force

函数原型：

```
int
cssm_open_force(
    String              group_name
);
```

功能说明：

强制 OPEN 指定的组。

使用场景：

在启动 ASM 或 DB 组时，如果某个节点故障一直无法启动，可借助此接口将 ASM 或 DB 组强制 OPEN。

接口会发送消息到主 CSS 执行，并且主 CSS 的监控需要处于打开状态，如果主 CSS 故障或尚未选出，则接口执行失败。

参数说明：

group_name：输入参数，指定要强制 OPEN 的组名。

返回值：

0：执行成功。

<0：执行失败。

24. cssm_ep_startup

函数原型：

```
int
```

```
cssm_ep_startup(
    String              group_name
);
```

功能说明:

启动指定 ASM 或 DB 组的所有节点。

如果 CSS 已经配置了自动重启，并且 CSS 的监控处于打开状态，则接口不允许执行，需要等待 CSS 自动检测故障并执行重启操作。

每个 CSS 只负责重启和自己的 DMDCR.INI 中配置的 DMDCR_SEQNO 相同的 ASM 或 DB 节点，因此需要所有 CSS 都处于活动状态，否则只通知当前活动的 CSS 重启相对应的节点。


注意:

只有在 ASM 组正常启动到 OPEN 状态，并且所有活动的 ASM 节点都处于 OPEN 状态时，才允许启动 DB 组，否则执行 DB 组的启动操作会报错。

另外在接口执行时，如果 CSS 监控处于关闭状态，则会直接打开 CSS 监控，否则 ASM 或 DB 组无法正常启动到 OPEN 状态。

参数说明:

group_name：输入参数，指定要启动的组名。

返回值:

0：执行成功。

<0：执行失败。

25. cssm_ep_stop

函数原型:

```
int
cssm_ep_stop(
```

```
    String              group_name
);
```

功能说明：

退出指定的 ASM 或 DB 组，如果主 CSS 故障或尚未选出，则接口执行失败。



在退出 ASM 组时，需要保证 DB 组已经退出，否则会报错处理。

另外如果退出时，CSS 监控没有关闭，在退出完成后，达到 CSS 设置的重启间隔之后，CSS 仍然会执行自动拉起，如果要正常退出集群，不需要自动拉起，则退出之前需要调用 cssm_css_stop 接口停止 CSS 的监控功能。

参数说明：

group_name：输入参数，指定要退出的组名。

返回值：

0：执行成功。

<0：执行失败。

26. cssm_ep_break

函数原型：

```
int
cssm_ep_break(
    String              group_name
);
```

功能说明：

此接口只允许在主 CSS 监控关闭的情况下使用。

ASM 组或 DB 组在正常运行时，如果某个节点出现故障，则需要调用此接口执行故障处理，将故障节点从组的 OK 节点数组中摘除。


注意：

在主 CSS 监控关闭的情况下，如果 ASM 组的某个节点故障，和故障节点有相同 DCR_SEQNO 的 DB 节点需要使用 cssm_ep_halt 接口执行强制退出，否则 DB 节点访问对应的 ASM 文件系统失败，也会自动 HALT。

在 ASM 和 DB 节点都出现故障的情况下，需要先对 ASM 组执行故障处理，再对 DB 组执行故障处理，否则会报错不允许执行。

参数说明：

group_name：输入参数，指定要执行故障处理的组名。

返回值：

0：执行成功。

<0：执行失败。

27. cssm_ep_recover

函数原型：

```
int
cssm_ep_recover(
    String          group_name
);
```

功能说明：

此接口只允许在主 CSS 监控关闭的情况下使用。

执行过故障处理的节点重启成功后，可通过调用此接口将故障节点重加入指定的 ASM 或 DB 组，重新恢复组到 OPEN 状态。


注意：

如果主 CSS 故障或尚未选出，则接口执行失败。另外只有在 ASM 组的故障节点重加入成功后，才允许重启对应的 DB 节点，并执行 DB 组的故障重加入。

参数说明:

group_name：输入参数，指定要执行故障恢复的组名。

返回值:

0：执行成功。

<0：执行失败。

28. cssm_ep_halt

函数原型:

```
int
cssm_ep_halt(
    String          group_name,
    String          ep_name
);
```

功能说明:

强制退出指定组的指定 EP。

此接口在 CSS 监控打开或关闭的情况下都允许使用，适用于下述场景：

a. 某个 ASM 或 DB 节点故障，CSS 的 DCR_GRP_DSKCHK_CNT 配置值很大，在容错时间内，CSS 不会调整故障节点的 active 标记，一直是 TRUE，CSS 认为故障 EP 仍然处于活动状态，不会自动执行故障处理，并且不允许手动执行故障处理。

另外执行 cssm_ep_startup 或 cssm_ep_stop 接口时，会误认为故障 EP 仍然处于活动状态，导致执行结果与预期不符。

此时可以通过执行此接口，通知 CSS 再次 HALT 故障 EP，确认 EP 已经被 HALT 后，CSS 会及时调整 active 标记为 FALSE，在此之后，对自动/手动故障处理，启动/退出 EP 节点等操作都可以正常执行。

b. 需要强制 HALT 某个正在运行的 ASM 或 DB 节点，也可以通过此接口完成。

参数说明:

group_name：输入参数，指定要强制退出的节点所在的组名。

ep_name：输入参数，指定要强制退出的节点名称。

返回值:

0：执行成功。

<0：执行失败。

29. cssm_deinit_env

函数原型:

```
int

cssm_deinit_env();
```

功能说明:

销毁监视器执行环境。

参数说明:

无

返回值:

0：执行成功。

<0：执行失败。

30. cssm_free_handle

函数原型:

```
int

cssm_free_handle();
```

功能说明:

释放分配到的监视器句柄。

参数说明:

无

返回值:

0：执行成功。

<0：执行失败。

19.2.6 C 编程示例

这里给出 C 程序的编程示例，运行环境为 VS2010，通过配置项目属性，使用 dmcssm_dll.lib 隐式加载 dll 的方式，注意此处仅为使用示例，实际使用时可以根据自己的需求再做调整。

```
#include "cssm_dll.h"

#include "stdio.h"

#include "stdlib.h"

#include "windows.h"

#include "process.h"



// 消息线程

DWORD

WINAPI

cssm_get_msg_thread(

mhandle              handle

)
```

```
{
    mschar          buf_msg[4097];
    muint4          msg_len_out;
    muint4          get_flag;
    mbool           exit_flag;
    CSSM_RETURN     code;

while (TRUE)
    {
        //等待消息事件
        cssm_msg_event_wait(handle);

        // 获取退出标记
        cssm_get_msg_exit_flag(handle, &exit_flag);
if (exit_flag == TRUE)
        {
    break;
        }

do
        {
            // 获取并打印消息
code = cssm_get_exec_msg(handle, buf_msg, 4097, &msg_len_out, &get_flag);
if (code >=0 && msg_len_out > 0)
```

```
                {
fprintf(stdout, "%s", buf_msg);
                }

            } while (get_flag); // 判断是否还有未读消息
        }

        // 设置退出事件
        cssm_set_msg_exit_event(handle);

return 0;
}

// 清理环境
void
cssm_clear_env(
mhandle        handle
)
{
        //通知并等待 dmmon_get_msg_thread 消息线程退出
        cssm_msg_event_deinit(handle);

        // 销毁监视器环境
        cssm_deinit(handle);
```

```
    // 释放句柄
    cssm_free_handle(handle);
}

int
main()
{
CSSM_RETURN       ret;
mhandle           handle;
HANDLE            thread_handle;
char              msg[4097];
int               msg_len;
muint4            n_group;
mbyte             group_seqno[16];
mschar*           group_name[16];
mschar*           group_type[16];
msint2            offset;
mschar            mem[4096];
msint2            i;

    //分配操作句柄
ret       = cssm_alloc_handle(&handle);
if (ret < 0)
```

```
    {
fprintf(stdout, "dmcssm alloc handle failed!\n");

        // 获取 ret 对应的错误描述信息
        cssm_get_error_msg_by_code(handle, ret, msg, 4097, &msg_len);
if (msg_len > 0)
        {
fprintf(stdout, "code:%d, error msg:%s", ret, msg);
        }
return ret;
    }
    // 创建消息线程
    thread_handle = (HANDLE)_beginthreadex(NULL, 0, cssm_get_msg_thread, handle, 0,
NULL);
if (thread_handle == NULL)
    {
ret = GetLastError();
fprintf(stderr, "_beginthreadex error! desc:%s, code:%d\n", strerror(ret), ret);

return ret;
    }
    //初始化监视器环境, 可以选择忽略-8 错误(建立到所有 CSS 连接失败, 可能当前 CSS 都还未启动)
ret       = cssm_init(handle, "D:\\dmcssm\\dmcssm.ini", "D:\\dmcssm\\log");
if (ret < 0 && ret != -8)
```

```
    {
fprintf(stdout, "dmcssm init failed!\n");
        // 获取 ret 对应的错误描述信息
        cssm_get_error_msg_by_code(handle, ret, msg, 4097, &msg_len);
if (msg_len > 0)
        {
fprintf(stdout, "code:%d, error msg:%s", ret, msg);
        }
        cssm_clear_env(handle);
return ret;
    }


offset      = 0;
for (i = 0; i < 16; i++)
    {
        group_name[i]    = mem + offset;
offset          += 129;


        group_type[i]    = mem + offset;
offset          += 65;
    }
    n_group      = 16;
ret          = cssm_get_n_group(handle, &n_group, group_seqno, group_name,
group_type);
```

```
if (ret < 0)
    {
fprintf(stdout, "dmcssm get group info failed!\n");
        //获取 ret 对应的错误描述信息
        cssm_get_error_msg_by_code(handle, ret, msg, 4097, &msg_len);
if (msg_len > 0)
        {
fprintf(stdout, "code:%d, error msg:%s", ret, msg);
        }
    }
else
    {
fprintf(stdout, "Get group info succeed, n_group: %d\n\n", n_group);
    }
    // 执行其他接口调用
    //...
    // 清理环境
cssm_clear_env(handle);
system("pause");

return 0;
}
```

19.2.7 JAVA 编程示例

这里给出JAVA程序的编程示例，运行环境为Eclipse，使用加载监视器jar包的方式，jar包名称为com.dameng.jni_7.0.0.jar，注意此处仅为使用示例，实际使用时可以根据自己的需求再做调整。

```
package com.dameng.test;

import com.dameng.cssm.*;

public class CssMonTest {

    public CssMonDLL monDll = null;

    public CssMonMsg monMsg = null;

    public Thread msgThread = null;

    public String ini_path = "D:\\dsc\\dmcssm.ini";

    public String log_path = "D:\\dsc\\log";

    //接收消息线程

    public void startThread()

    {

        msgThread = new Thread(new Runnable()

       {

            @Override

public void run()

            {

    while (true)

                {

        //等待消息事件
```

```
        monDll.cssm_msg_event_wait();
        //获取退出标记
        if (monDll.cssm_get_msg_exit_flag() == true)
            break;
do
                    {
        //获取并打印消息
        monMsg = monDll.cssm_get_exec_msg();
        if(monMsg.getReturnCode() < 0)
            break;
        if(monMsg.getMsg().length() > 0)
        {
            System.out.print(monMsg.getMsg());
        }
        } while (monMsg.getMsgFlag() == 1); //是否还有未读消息
              }
//设置退出标记
monDll.cssm_set_msg_exit_event();
          }
      });
        //启动线程
        msgThread.start();
}
//退出线程
```

```
public void endThread() throws InterruptedException
{
    if (msgThread != null && msgThread.isAlive())
    {
        //通知并等待消息线程退出
        monDll.cssm_msg_event_deinit();
    }
}
public void test() throws InterruptedException
{
    boolean  ret1 = false;
    int      ret2 = 0;
    CssMonMsg   retMsg = null;
    CssMonGrpNameArray grpArr = null;
    monDll = new CssMonDLL();
    //分配句柄
    ret1 = monDll.cssm_init_handle();
    if (ret1 == false)
    {
        System.out.println("dmcssm alloc handle failed!");
        return;
    }
    else
    {
```

```
            System.out.println("dmcssm alloc handle success!");
        }
        //启动消息接收线程
        startThread();
        //初始化监视器，可以选择忽略-7 错误（建立到所有 CSS 连接失败，可能当前 CSS 都还未启
动）
        ret2 = monDll.cssm_init_env(ini_path, log_path);
        if (ret2 < 0 && ret2 != -7)
        {
            System.out.println("dmcssm init failed!");
            retMsg = monDll.cssm_get_error_msg_by_code(ret2);
            if (retMsg.getReturnCode() >= 0)
            {
                System.out.println("code:" + ret2 + "," + retMsg.getMsg());
            }
            //初始化失败，退出线程，销毁句柄
            endThread();
            monDll.cssm_free_handle();
            return;
        }
        else
        {
            System.out.println("dmcssm init success!");
        }
```

```
            grpArr     = monDll.cssm_get_n_group();
            if(grpArr.getReturnCode() < 0)
            {
                System.out.println("get dmcssm group info failed!");
                retMsg = monDll.cssm_get_error_msg_by_code(grpArr.getReturnCode());
                if (retMsg.getReturnCode() >= 0)
                {
                    System.out.println("code:" + grpArr.getReturnCode() + "," +
retMsg.getMsg());
                }
            }
            else
            {
                System.out.println("dmcssm get group info success, n_group:" +
grpArr.getN_group());
            }
            //执行其他接口调用
           //...
            //退出消息线程
            endThread();
            //销毁监视器环境
            monDll.cssm_deinit_env();
            //销毁句柄
            monDll.cssm_free_handle();
```

```
            System.out.println("\ndmcssm deinit success!");
        }
        /**
         * @param args
         * @throws InterruptedException
         */
        public static void main(String[] args) throws InterruptedException {
            CssMonTest test = new CssMonTest();
            test.test();
        }
}
```

19.3 DMASMAPIM 接口

DMASMAPIM 接口为配置了镜像的 DMASM 专用。

镜像环境中的大部分 DMASMAPIM 接口和非镜像环境中的 DMASMAPI 接口功能保持不变，只是接口名称发生改变。非镜像环境中接口名称为 DMASM_XXX，镜像环境中接口名称为 DMASMM_XXX，均在 DMASM 后面增加一个 M。

19.3.1 DMASMAPIM 接口介绍

DMASMAPIM 提供的接口如下：

1. dmasmm_sys_init

函数原型：

```
ASMRETURN
dmasmm_sys_init(
```

```
    sdbyte*      err_desc,
    udint4*      err_len,
    udint4       char_code,
    udint4       lang_id
)
```

功能说明:

环境初始化接口。使用 ASMAPIM 接口，必须第一个调用 dmasmm_sys_init 接口。

参数说明:

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

char_code：输入参数，编码。0：PG_INVALID_CODE；1：PG_UTF8；2：PG_GBK；3：PG_BIG5；4：PG_ISO_8859_9；5：PG_EUC_JP；6：PG_EUC_KR；7：PG_KOI8R；8:PG_ISO_8859_1；9：PG_SQL_ASCII；10：PG_GB18030；11：PG_ISO_8859_11；12：PG_UTF16。-1 表示使用默认编码。

lang_id ：输入参数，语言。0：中文；1：英文；2：繁体中文。-1 表示使用默认语言。

2. dmasmm_sys_deinit

函数原型:

```
void
dmasmm_sys_deinit()
```

功能说明:

环境销毁接口。使用 ASMAPIM 接口结束时，调用 dmasmm_sys_deinit 销毁资源。

3. dmasmm_alloc_con

函数原型:

```
ASMRETURN
```

```
dmasmm_alloc_con(
    asmcon_handle*   con,
    sdbyte*          err_desc,
    udint4*          err_len
)
```

功能说明:

申请并初始化连接句柄。返回申请的连接句柄。

参数说明:

con：输入参数，连接句柄。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

4. dmasmm_close_con

函数原型:

```
void
dmasmm_close_con(
    asmcon_handle    con_in
)
```

功能说明:

关闭连接。关闭连接句柄，释放资源。

参数说明:

conn_in：输入参数，连接句柄。

5. dmasmm_free_con

函数原型:

```
void
```

```
dmasmm_free_con(
    asmcon_handle   con_in
)
```

功能说明:

释放连接句柄。

参数说明:

con_in：输入参数，连接句柄。

6. dmasmm_connect

函数原型:

```
ASMRETURN
    dmasmm_connect(
    asmcon_handle    con_in,
    sdbyte*          username,
    sdbyte*          password,
    sdbyte*          hostname,
    udint2           portnum,
    asmbool*         con_is_local,
    sdbyte*          err_desc,
    udint4*          err_len
)
```

功能说明:

登录接口。ASMSVRM 允许本地连接和远程连接，但是 DMSERVER 仅允许本地连接，依据 con_is_local 判断是否远程连接。

参数说明:

con_in：输入参数，连接句柄。

username：输入参数，用户名。

password：输入参数，密码。

hostname：输入参数，主机 ip 或主机名。

portnum：输入参数，主机监听端口号。

con_is_local：输出参数，标识远程或本地登录。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

7. dmasmm_get_n_group

函数原型：

```
ASMRETURN
dmasmm_get_n_group(
    asmcon_handle    conn_in,
    udint2*          num,
    sdbyte*          err_desc,
    udint4*          err_len
)
```

功能说明：

获取 ASM Disk Group 个数。获取有多少个磁盘组。

参数说明：

conn_in：输入参数，连接句柄。

num：输出参数，磁盘组个数。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

8. dmasmm_get_group_id_arr

函数原型：

```
ASMRETURN

dmasmm_get_group_id_arr(

    asmcon_handle    con_in,

    udint2*          id_arr,

    udint4*          au_size_arr,

    udint4*          ex_size_arr,

    udint2           arr_size,

    udint2*          num,

    sdbyte*          err_desc,

    udint4*          err_len

)
```

功能说明：

获取 ASM Disk Group ID 数组。配合 dmasmm_get_n_group 使用，获取所有 Disk Group ID。

参数说明：

conn_in：输入参数，连接句柄。

id_arr：输出参数，磁盘 ID 数组。

au_size_arr：输出参数，磁盘组 AU 大小数组。

ex_size_arr：输出参数，无实际意义。

arr_size：输入参数，数组最大长度。

num：输出参数，返回数组长度。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

9. dmasmm_get_disk_id_arr_by_group

函数原型：

```
ASMRETURN
dmasmm_get_disk_id_arr_by_group(
    asmcon_handle    con_in,
    udint2           group_id,
    udint2*          id_arr,
    udint2           arr_size,
    udint2*          n_disk,
    sdbyte*          err_desc,
    udint4*          err_len
)
```

功能说明：

获取磁盘组内磁盘 ID 数组。根据磁盘组 ID 获取磁盘组内包含的所有磁盘 ID。

参数说明：

conn_in：输入参数，连接句柄。

group_id：输入参数，磁盘组 ID。

id_arr：输出参数，磁盘 ID 数组。

arr_size：输入参数，数组最大长度。

n_disk：输出参数，返回数组长度。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

10. dmasmm_get_disk_info

函数原型:

```
ASMRETURN
dmasmm_get_disk_info(
        asmcon_handle          conn_in,
        udint2                 group_id,
        udint4                 disk_id,
        sdbyte*                path,
        udint2                 path_buflen,
        sdbyte*                name,
        udint2                 name_buflen,
        sdbyte*                frp_name,
        udint2                 frp_buflen,
        sdbyte*                status,
        udint2                 status_buflen,
        udint4*                size,
        udint4*                free_auno,
        sdbyte*                create_time,
        sdbyte*                modify_time,
        sdbyte*                uuid_str,
        sdbyte*                err_desc,
        udint4*                err_len
)
```

功能说明:

获取 ASM 磁盘详细信息。根据磁盘组 ID 和磁盘 ID 获取 ASM 磁盘详细信息。

参数说明:

conn_in：输入参数，连接句柄。

group_id：输入参数，磁盘组 ID。

disk_id：输入参数，磁盘 ID。

path：输出参数，磁盘路径。

path_buflen：输入参数，path 缓冲区长度。最长不能超过 256 字节。

name：输出参数，磁盘名称。

name_buflen：输入参数，name 缓冲区长度。最长不能超过 32 字节。

frp_name：输出参数，磁盘所属故障组名称。

frp_buflen：输入参数，frp_name 缓冲区长度。最长不能超过 32 字节。

status：输出参数，磁盘状态

status_buflen：输入参数，status 缓冲区长度。最长不能超过 32 字节。

size：输出参数，磁盘大小，单位 M。

free_auno：输出参数，最大 AU 号。

create_time：输出参数，磁盘创建时间。

modify_time：输出参数，磁盘最近一次修改时间。

uuid_str：输出参数，磁盘唯一标识。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。最长不能超过 32 字节。

11. dmasmm_drop_diskgroup_by_name

函数原型:

```
ASMRETURN

dmasmm_drop_diskgroup_by_name(
```

```
    asmcon_handle     conn_in,
    sdbyte*           group_name,
    sdbyte*           err_desc,
    udint4*           err_len
)
```

功能说明：

删除磁盘组。

参数说明：

conn_in：输入参数，连接句柄。

group_name：输入参数，磁盘组名。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

12. dmasmm_file_create

函数原型：

```
ASMRETURN
dmasmm_file_create(
    asmcon_handle       conn_in,
    asmbool             p_flag,
    sdbyte*             filepath_in,
    udint8              filesize,
    asm_fhandle_t*      fil_handle,
    asmbool             need_init,
    udint2              striping_size,
    udbyte              redundancy_type,
```

```
    sdbyte*             err_desc,
    udint4*             err_len
)
```

功能说明:

创建 ASM 文件。在 ASM 文件系统中创建 ASM 文件，如果文件父亲目录不存在，并且 p_flag 为 TRUE 的情况，会自动创建父目录，否则会报错。

参数说明:

conn_in：输入参数，连接句柄。

p_flag：输入参数，创建父目录标记。

filepath_in：输入参数，文件路径。

filesize：输入参数，文件大小，单位 Byte。

fil_handle：输出参数，文件句柄。

need_init：输入参数，是否需要初始化。

striping_size：输入参数，条带化粒度。

redundancy_type：输入参数，镜像类型。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

13. dmasmm_file_open

函数原型:

```
ASMRETURN
dmasmm_file_open(
    asmcon_handle       conn_in,
    sdbyte*             filepath_in,
    asm_fhandle_t*      fhandle,
```

```
    sdbyte*          err_desc,
    udint4*          err_len
)
```

功能说明:

打开 ASM 文件，获取 ASM 文件句柄。

DMASM 文件可以重复打开，但是建议用户在使用过程中，尽量避免反复打开同一个 DMASM 文件。如果用户反复打开同一个 DMASM 文件，并且没有及时关闭文件，有可能会降低 DMASM 文件的访问效率。

DMASM 文件句柄不保证全局唯一，只保证连接级别的唯一性，一个连接重复打开同一个 DMASM 文件，会返回不同的文件句柄；不同连接打开同一个 DMASM 文件，有可能返回相同的文件句柄。

参数说明:

conn_in：输入参数，连接句柄。

filepath_in：输入参数，文件路径。

fhandle：输入参数，文件句柄。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

14. dmasmm_file_trunc

函数原型:

```
ASMRETURN
dmasmm_file_trunc(
    asmcon_handle    conn_in,
    asm_fhandle_t    fhandle,
    udint8           truncate_size,
```

```
    udint8*             real_size,
    sdbyte*             err_desc,
    udint4*             err_len
)
```

功能说明：

截断 ASM 文件。将 ASM 文件截断到 truncate_size，如果 truncate_size 小于文件大小，文件会被截断到 truncate_size；如果 truncate_size 大于文件大小，文件大小不变，接口返回 EC_SUCCESS。

参数说明：

conn_in：输入参数，连接句柄。

fhandle：输入参数，文件句柄。

truncate_size：输入参数，截断后的大小，单位 Byte。

real_size：输出参数，执行后实际大小，单位 Byte。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

15. dmasmm_file_extend

函数原型：

```
ASMRETURN
dmasmm_file_extend(
    asmcon_handle    conn_in,
    asm_fhandle_t    fhandle,
    udint8           offset,
    udint8           extend_size,
    udint4           need_init,
```

```
    sdbyte*          err_desc,
    udint4*          err_len
)
```

功能说明:

扩展 ASM 文件。将文件从 offset 偏移处，扩展 extend_size 大小，最终实际大小为 offset+extend_size。如果 offset+extend_size 大于文件大小，文件会被扩展到 offset+extend_size 大小；如果 offset+extend_size 小于文件大小，直接返回成功。

参数说明:

conn_in：输入参数，连接句柄。

fhandle：输入参数，文件句柄。

offset：输入参数，起始偏移。

extend_size：输入参数，扩展大小。

need_init：输入参数，是否需要初始化。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

16. dmasmm_file_close

函数原型:

```
void
dmasmm_file_close(
    asmcon_handle    conn_in,
    asm_fhandle_t    fhandle
)
```

功能说明:

关闭 ASM 文件。关闭打开的 ASM 文件

参数说明：

conn_in：输入参数，连接句柄。

fhandle：输入参数，文件句柄。

17. dmasmm_file_delete

函数原型：

```
ASMRETURN
dmasmm_file_delete(
    asmcon_handle    conn_in,
    sdbyte*          filepath_in,
    sdbyte*          err_desc,
    udint4*          err_len
)
```

功能说明：

删除 ASM 文件。正在被使用的 ASM 文件不能被删除。

参数说明：

conn_in：输入参数，连接句柄。

filepath_in：输入参数，文件路径。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

18. dmasmm_file_read_by_offset

函数原型：

```
ASMRETURN
dmasmm_file_read_by_offset(
    asmcon_handle   conn_in,
```

```
    asm_fhandle_t    fhandle,
    udint8           offset,
    sdbyte*          buffer,
    udint4           bytes_to_read,
    sdbyte*          err_desc,
    udint4*          err_len
)
```

功能说明：

从 ASM 文件读取数据。从 ASM 文件 offset 偏移读取 bytes_to_read 大小的内容到缓冲区 buffer，调用者保证缓冲区够用。因为块设备读写限制，offset，buffer，bytes_to_read 都必须能被 512 整除，否则会报错。

参数说明：

conn_in：输入参数，连接句柄。

fhandle：输入参数，文件句柄。

offset：输入参数，起始偏移。

buffer：输入参数，缓冲区。

bytes_to_read：输入参数，读取数据大小，单位 Byte。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

19. dmasmm_file_read_by_offset_normal

函数原型：

```
ASMRETURN
dmasmm_file_read_by_offset_normal(
    asmcon_handle    conn_in,
```

```
    asm_fhandle_t     fhandle,
    udint8            offset,
    sdbyte*           buffer,
    udint4            bytes_to_read,
    sdbyte*           err_desc,
    udint4*           err_len
)
```

功能说明:

从 ASM 文件读取数据。从 ASM 文件 offset 偏移读取 bytes_to_read 大小的内容到缓冲区 buffer，调用者保证缓冲区够用。该接口支持 offset、buffer、bytes_to_read 不是 512 整数倍，但是性能比 dmasmm_file_read_by_offset 慢。

参数说明:

conn_in：输入参数，连接句柄。

fhandle：输入参数，文件句柄。

offset：输入参数，起始偏移。

buffer：输入参数，缓冲区。

bytes_to_read：输入参数，写取数据大小，单位 Byte。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

20. dmasmm_file_write_by_offset

函数原型:

```
ASMRETURN
dmasmm_file_write_by_offset(
    asmcon_handle   conn_in,
```

```
    asm_fhandle_t    fhandle,
    udint8           offset,
    sdbyte*          buffer,
    udint4           bytes_to_write,
    sdbyte*          err_desc,
    udint4*          err_len
)
```

功能说明:

将数据写入 ASM 文件。将缓冲区中的内容写入 ASM 文件，从 offset 偏移开始。因为块设备读写限制，offset，buffer 地址，bytes_to_write 都必须能被 512 整除。

参数说明:

conn_in：输入参数，连接句柄。

fhandle：输入参数，文件句柄。

offset：输入参数，起始偏移。

buffer：输入参数，缓冲区。

bytes_to_write：输入参数，写数据大小，单位 Byte。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

21. dmasmm_file_write_by_offset_normal

函数原型:

```
ASMRETURN
dmasmm_file_write_by_offset_normal(
    asmcon_handle    conn_in,
    asm_fhandle_t    fhandle,
```

```
    udint8          offset,
    sdbyte*         buffer,
    udint4          bytes_to_write,
    sdbyte*         err_desc,
    udint4*         err_len
)
```

功能说明:

将数据写入 ASM 文件。将缓冲区中的内容写入 ASM 文件，从 offset 偏移开始。该接口支持 offset、buffer、bytes_to_write 不是 512 倍数，但是性能比 dmasmm_file_write_by_offset 慢。

参数说明:

conn_in：输入参数，连接句柄。

fhandle：输入参数，文件句柄。

offset：输入参数，起始偏移。

buffer：输入参数，缓冲区。

bytes_to_write：输入参数，写数据大小，单位 Byte。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

22. dmasmm_file_copy

函数原型:

```
ASMRETURN
dmasmm_file_copy(
    asmcon_handle   conn_in,
    sdbyte*         source_in,
```

```
    sdbyte*         dest_in,
    asmbool         bOverwriteIfExists,
    sdbyte*         err_desc,
    udint4*         err_len
)
```

功能说明：

文件拷贝操作。支持 ASM 文件拷贝到 ASM 文件; ASM 文件拷贝到普通文件系统文件; 普通文件系统文件拷贝到 ASM 文件系统；不支持普通文件拷贝到普通文件。bOverwriteIfExists：0 或者 NULL 表示不覆盖，其他非 0 值表示覆盖。

参数说明：

conn_in：输入参数，连接句柄。

source_in：输入参数，源文件路径。必须是全路径，不能是相对路径。

dest_in：输入参数，目标文件路径。必须是全路径，不能是相对路径。

bOverwriteIfExists：输入参数，如果目标存在是否删除。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

23. dmasmm_dir_create

函数原型：

```
ASMRETURN
dmasmm_dir_create(
    asmcon_handle   conn_in,
    asmbool         p_flag,
    sdbyte*         fdir_in,
    sdbyte*         err_desc,
```

```
    udint4*             err_len

)
```

功能说明:

创建目录。ASM 文件系统创建目录，当 p_flag=TRUE 时会级联创建父目录，否则父目录不存在会报错。

参数说明:

conn_in：输入参数，连接句柄。

p_flag：输入参数，是否级联创建父目录。

fdir_in：输入参数，目录路径。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

24. dmasmm_dir_delete

函数原型:

```
ASMRETURN

dmasmm_dir_delete(

    asmcon_handle   conn_in,

    sdbyte*         fdir_in,

    sdbyte*         err_desc,

    udint4*         err_len

)
```

功能说明:

删除目录，以及目录下面所有的文件。

参数说明:

conn_in：输入参数，连接句柄。

fdir_in：输入参数，目录路径。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

25. dmasmm_get_file_num_by_group

函数原型：

```
ASMRETURN

dmasmm_get_file_num_by_group(

    asmcon_handle       conn_in,

    udint2              group_id,

    udint4*             num,

    sdbyte*             err_desc,

    udint4*             err_len

)
```

功能说明：

获取磁盘组内总的文件个数。根据磁盘组 ID 获取总的文件个数，包括文件和目录。

参数说明：

conn_in：输入参数，连接句柄。

group_id：输入参数，磁盘组的 ID 。

num ：输出参数，文件的个数。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

26. dmasmm_get_file_info

函数原型：

```
ASMRETURN
```

```
dmasmm_get_file_info(
    asmcon_handle   conn_in,
    asm_fhandle_t   file_id,
    ASM3_FILE_ATTR*  fattr_out,
    sdbyte*         err_desc,
    udint4*         err_len
)
```

功能说明：

获取 ASM 文件详细信息。

ASM3_FILE_ATTR 包括：

type：目录标记，整型。1 表示文件，2 表示目录。

name：文件名称，字符型。

full_path：完整路径，字符型。

size：文件大小，整型，单位 Byte。目录忽略此字段。

c_time：创建时间，time_t 类型。

m_time：修改时间，time_t 类型。

group_id：所在磁盘组编号，整型。

disk_id：inode 项所在磁盘 ID，整型。

disk_auno：inode 项所在磁盘 AU 编号，整型。

offset：inode 项 AU 偏移，整型。

fil_id：文件句柄，asm_fhandle_t 类型。

strip:文件条带化粒度，整型。

n_copy:文件镜像类型，即副本数，整型。

参数说明：

conn_in：输入参数，连接句柄。

file_id：输入参数，打开的文件句柄。

fattr_out：输出参数，文件属性结构。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

27. dmasmm_get_group_info_by_name

函数原型：

```
ASMRETURN
dmasmm_get_group_info_by_name(
    asmcon_handle   con_in,
    sdbyte*         group_name,
    udint2*         group_id,
    udint2*         status,
    udint2*         n_disk,
    udint4*         total_size,
    udint4*         free_size,
    sdbyte*         err_desc,
    udint4*         err_len
)
```

功能说明：

通过磁盘组名获取磁盘组详细信息。

参数说明：

con_in：输入参数，连接句柄。

group_name：输入参数，磁盘组名字。

group_id：输出参数，磁盘组 ID。

status：输出参数，磁盘组状态。1：正在创建中；2：正常的；3：正在删除中。

n_disk：输出参数，磁盘数。

total_size：输出参数，磁盘组大小，单位 AU。

free_size：输出参数，磁盘组空闲大小，单位 AU。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

28. dmasmm_get_group_info_by_id

函数原型：

```
ASMRETURN
dmasmm_get_group_info_by_id(
    asmcon_handle    con_in,
    udint2           group_id,
    sdbyte*          group_name,
    udint2           name_buflen,
    udint2*          status,
    udint2*          n_disk,
    udint4*          total_size,
    udint4*          free_size,
    udint4*          au_size,
    byte*            n_copy,
    udint4*          redo_size,
    udint2*          rbl_stat,
    udint2*          rbl_pwr,
```

```
    sdbyte*             err_desc,
    udint4*             err_len
)
```

功能说明:

通过磁盘组 ID 获取磁盘组详细信息。

参数说明:

con_in：输入参数，连接句柄。

group_id：输入参数，磁盘组 ID。

group_name：输出参数，磁盘组名字。

name_buflen：输入参数，磁盘组的 buf 长度。

status：输出参数，磁盘组状态。1：正在创建中；2：正常的；3：正在删除中。

n_disk：输出参数，磁盘数。

total_size：输出参数，磁盘组大小，单位 AU。

free_size：输出参数，磁盘组空闲大小，单位 AU。

au_size：输出参数，磁盘组 AU 大小，单位 Byte。

n_copy：输出参数，磁盘组镜像类型。

redo_size：输出参数，磁盘组中 redo 日志文件大小。

rbl_stat：输出参数，磁盘组重平衡状态。0：未启用；1：已启用。

rbl_pwr：输出参数，磁盘组重平衡并行度。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

29. dmasmm_dir_get_first

函数原型:

```
ASMRETURN
```

```
dmasmm_dir_get_first(
    asmcon_handle       conn_in,
    sdbyte*             path_in,
    sdbyte*             suffix,
    asm_dhandle_t*      dir_handle_out,
    ASM3_FILE_ATTR*     fattr_out,
    asmbool*            exist_flag,
    sdbyte*             err_desc,
    udint4*             err_len
)
```

功能说明：

获取目录下第一个文件信息。

参数说明：

conn_in：输入参数，连接句柄。

path_in：输入参数，目录路径。

suffix：输入参数，指定扩展名关键字，如：".log"。用于取出以指定关键字作为扩展名的文件。

dir_handle_out：输出参数，打开的目录句柄。

fattr_out：输出参数，文件属性结构。

exist_flag：输出参数，是否存在。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

30. dmasmm_dir_get_next

函数原型：

```
ASMRETURN

dmasmm_dir_get_next(

    asmcon_handle       conn_in,

    asm_dhandle_t       dir_handle,

    sdbyte*             path_in,

    sdbyte*             suffix,

    ASM3_FILE_ATTR*     fattr_out,

    asmbool*            exist_flag,

    sdbyte*             err_desc,

    udint4*             err_len

)
```

功能说明:

获取目录下下一个文件信息，须配合函数 dmasmm_dir_get_first 使用。

参数说明:

conn_in：输入参数，连接句柄。

dir_handle：输入参数，打开的目录句柄。

path_in：输入参数，目录路径，须与目录句柄 dir_handle 中打开的 path_in 保持一致。

suffix：输入参数，指定扩展名关键字，如：".log"。用于取出以指定关键字作为扩展名的文件。

fattr_out：输出参数，文件属性结构。

exist_flag：输出参数，是否存在。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

31. dmasmm_dir_close

函数原型：

```
ASMRETURN

dmasmm_dir_close(

    asmcon_handle       conn_in,

    asm_dhandle_t       dir_handle,

    sdbyte*             err_desc,

    udint4*             err_len

)
```

功能说明：

关闭目录。

参数说明：

con_in：输入参数，连接句柄。

dir_handle：输入参数，打开的目录句柄。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

32. dmasmm_file_attributes_get

函数原型：

```
ASMRETURN

dmasmm_file_attributes_get(

    asmcon_handle       conn_in,

    sdbyte*             path,

    ASM3_FILE_ATTR*     fattr_out,

    sdbyte*             err_desc,
```

```
    udint4*                 err_len

)
```

功能说明:

根据文件路径获取文件详细信息。

参数说明:

conn_in：输入参数，连接句柄。

path：输入参数，路径。

fattr_out：输出参数，文件属性结构。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

33. dmasmm_file_is_exist

函数原型:

```
ASMRETURN

dmasmm_file_is_exist(

    asmcon_handle    conn_in,

    sdbyte*          path,

    sdbyte*          err_desc,

    udint4*          err_len

)
```

功能说明:

判断 ASM 文件系统中，以 path 为文件路径的文件是否存在。

参数说明:

conn_in：输入参数，连接句柄。

path：输入参数，文件路径。

err_desc：输出参数，错误描述信息。

err_len：输入输出参数，错误描述信息长度。

19.3.2 返回值说明

返回值分为三种类型：正数，0 和负数。0 表示正常；正数为警告信息；负数为错误信息，对应的错误码请参考 19.3.3 错误码汇编。

19.3.3 错误码汇编

DMASMAPIM 和 DMASMAPI 共用一套错误码。错误码值域从-11000 开始。具体可参考 V$ERR_INFO。

19.3.4 编程示例

下面举一个简单的例子，展示 DMASMAPIM 的使用方法。若本示例在编译过程中出现实参过多或者变量未声明等问题，可查询相应版本的 DMASMAPIM 接口使用手册，适当增减参数或者将变量改为实际值。

```
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
#include "asmapi3.h"


#define TRUE            1
#define FALSE           0


#define IS_SUCCESS(code)      ((sdint4)code >= 0)
```

```
#define RET_IF_ERR_EX(c, act) \
    { if (!IS_SUCCESS(c)) { {act;} return (c); }}


/*运行前修改file_name username passwd ip port
  镜像不支持远程写，示例程序需要和服务端在同一台机器
  LINUX下编译
  gcc -DLINUX -I/home/test/include test.c -L. -ldmasmapim -o demo
  -I:指定头文件目录，本例中需要指定asmapi3.h头文件所在目录
  -L(大写L):指定库搜索路径
  -l(小写L):指定库名
  -D:定义预处理宏*/
int
main(
    int                 argc,
    char*               argv[]
)
{
    int                 code = 0;
    char                err_desc[1024];
    int                 err_len;
    asm_fhandle_t     fil_handle;
    void*               conn;
    char*               wbuf            = "hello asm";
    char                rbuf[10];
```

```
    int                 len;
    int                 file_size = 20;          //单位字节
    int                 offset = 0;
    int                 strip_size   = 0;        //条带大小
    int                 redundancy_type = 1;   //镜像类型
    char*               username = "ASMSYS";
    char*               passwd = "DCRpsd_123";
    char*               file_name = "+DATA/test1.dta";
    char*               ip = "192.168.1.118";
    int                 port = 10060;


#ifdef WIN32
    /*windows环境需要添加附加依赖项ws2_32.lib*/
    WSADATA WsaData;
    if (SOCKET_ERROR == WSAStartup(0x0101, &WsaData))
    {
        fprintf(stderr, "WSAStartup Failed\n");
        exit(-1);
    }
#endif


    err_len             = sizeof(err_desc);
    len                 = (int)strlen(wbuf);
```

```
code = dmasmm_sys_init(err_desc, &err_len, -1, 1);
if (!IS_SUCCESS(code))
{
    fprintf(stderr, "dmasmm_sys_init error:[%s]\n", err_desc);
    return -1;
}
/* 分配连接句柄 */
code = dmasmm_alloc_con(&conn, err_desc, &err_len);
if (!IS_SUCCESS(code))
{
    fprintf(stderr, "dmasmm_alloc_con error:[%s]\n", err_desc);
    return -1;
}

/* 登录asmsvr，本地登录不校验密码，可以输入任意字符串，远程登录会校验用户名密码*/
code = dmasmm_connect(conn, username, passwd, ip, port, NULL, err_desc, &err_len);
if (!IS_SUCCESS(code))
{
    fprintf(stderr, "dmasmm_connect error:[%s]\n", err_desc);
    return -1;
}

/* 文件存在直接打开 */
if (dmasmm_file_is_exist(conn, file_name, err_desc, &err_len) == TRUE)
```

```
    {
        code = dmasmm_file_open(conn, file_name, &fil_handle, err_desc, &err_len);
        RET_IF_ERR_EX(code, goto fun_end);
    }
    else
    {
        /* 在已有的磁盘组内创建asm文件，  */
        code = dmasmm_file_create(conn, 1, file_name, file_size, &fil_handle, FALSE,
strip_size, redundancy_type, err_desc, &err_len);
        if (!IS_SUCCESS(code))
        {
            fprintf(stderr, "dmasmm_file_create error code:%d [%s]\n", code, err_desc);
            goto fun_end;
        }
    }
    /* 写文件 */
    code = dmasmm_file_write_by_offset_normal(conn, fil_handle, offset, wbuf, len,
err_desc, &err_len);
    if (!IS_SUCCESS(code))
    {
        fprintf(stderr, "dmasmm_file_write_by_offset_normal error code:%d [%s]\n", code,
err_desc);
        goto fun_end;
    }
```

```
    /* 读文件 */
    code = dmasmm_file_read_by_offset_normal(conn, fil_handle, offset, rbuf, len,
err_desc, &err_len);
    if (!IS_SUCCESS(code))
    {
        fprintf(stderr, "dmasmm_file_write_by_offset_normal error code:%d [%s]\n", code,
err_desc);
        goto fun_end;
    }
    else
    {
        rbuf[len] = '\0';
        fprintf(stderr, "dmasmm_file_write_by_offset_normal succ buf:%s\n", rbuf);
    }

fun_end:
    // 文件存在就关闭删除
    if (dmasmm_file_is_exist(conn, file_name, err_desc, &err_len) == TRUE)
    {
        /* 关闭打开的文件句柄，否则删除会报错 */
        dmasmm_file_close(conn, fil_handle);

        /* 删除文件 */
```

```
        code = dmasmm_file_delete(conn, file_name, err_desc, &err_len);
        if (!IS_SUCCESS(code))
        {
            fprintf(stderr, "dmasmm_file_delete error:[%s]\n", err_desc);
        }
    }

    dmasmm_close_con(conn);
    dmasmm_free_con(conn);
    dmasmm_sys_deinit();
    return 0;
}
```

DM 技术支持

如果您在安装或使用 DM 及其相应产品时出现了问题，请首先访问我们的 Web 站点 http://www.dameng.com/。在此站点我们收集整理了安装使用过程中一些常见问题的解决办法，相信会对您有所帮助。

您也可以通过以下途径与我们联系，我们的技术支持工程师会为您提供服务。

武汉达梦数据库股份有限公司

地址：武汉市东湖新技术开发区甲铺岭街 39 号达梦中国数据库产业基地

电话：(+86)027-87588000

传真：(+86)027-87588810

武汉达梦数据技术有限公司

地址：武汉市东湖新技术开发区甲铺岭街 39 号达梦中国数据库产业基地 13-14 层

电话：(+86)027-87588000

传真：(+86)027-87588810

北京达梦数据库技术有限公司

地址：北京市海淀区中关村南大街 2 号数码大厦 B 座 1003

电话：(+86)010-51727900

传真：(+86)010-51727983

上海达梦数据库有限公司

地址：上海市张江高科技园区亮秀路 112 号 A 座 401

电话：(+86)021-50111338

四川蜀天梦图数据科技有限公司

地址：成都市天府新区湖畔西路 99 号 B7 栋（天府英才中心）6 层

电话：(+86)028-61458838

达梦数据技术（江苏）有限公司

地址：苏州吴中经济开发区越溪街道吴中大道 1431 号苏州（太湖）软件产业园智慧谷园区 1601 室 03-04 号

电话：(+86)0512-65285955

传真：(+86)0512-65286955

重庆达梦大数据有限公司

地址：重庆市沙坪坝区重庆师范大学（沙坪坝校区）第二教学楼 1002

电话：(+86)023-68165682

达梦数据技术（深圳）有限公司

地址：深圳市龙岗区园山街道荷坳社区龙岗大道 8288 号大运软件小镇 17 栋 101

技术服务：

咨询热线：400-991-6599

技术支持：dmtech@dameng.com

商务咨询：400@dameng.com

手册版本

版本号：V8

发版日期：2026 二季度版-26876



武汉达梦数据库股份有限公司
官方网站：www.dameng.com

达梦数据版权所有，侵权必究©